

Anotações sobre Machine Learning

Igor Ruys Cartucho

SUMÁRIO

Capítulo 1:	Introdução	6
Capítulo 2:	Estimação Pontual	7
2.1	Avaliação de Estimadores	7
2.2	Estimação Funcional em <i>Machine Learning</i>	9
2.3	Dilema Viés-Variância em <i>Machine Learning</i>	9
2.4	Maximum Likelihood Estimation (MLE)	14
2.5	Maximum A Posteriori (MAP) Estimation	19
Capítulo 3:	<i>K-Nearest Neighbors</i>	20
3.1	Implementações do Método	20
3.1.1	Força bruta	20
3.1.2	K-D Tree	22
3.1.3	<i>Ball Tree</i>	25
3.1.4	Variações do KNN	27
3.2	Maldição da Dimensionalidade	28
3.3	Métricas de Distância	30
Capítulo 4:	Regressão Linear	33
4.1	Diferentes Interpretações da Regressão Linear	34
4.1.1	Ponto de Vista de Otimização	34
4.1.2	Ponto de Vista de Álgebra Linear	35
4.1.3	Ponto de Vista Probabilístico	37
4.2	Regressão Polinomial	38
4.3	Forma Geral da Regressão Linear	39
4.4	Condições de Gauss-Markov	39
Capítulo 5:	Regressão Logística	40
5.1	Mínimos Quadrados Para Classificação	41
5.2	Modelando a Probabilidade A Priori	44
5.2.1	Caso $K = 2$	44
5.2.2	Caso $K > 2$	45
5.2.3	Aplicação para distribuições conhecidas	45
5.3	Construção da Regressão Logística	48
5.4	Discussão sobre a Função Custo	51
5.5	Conclusões	52
Capítulo 6:	Support Vector Machine	53

6.1 Hiperplano de separação Ótimo	53
6.2 Classificador de Vetor de Suporte	57
6.3 Máquina de Vetor de Suporte.....	60
6.3.1 SVM como Método de Penalização	62
6.3.2 Classificação Multiclasse	63
6.4 SVM para Regressão.....	63
6.5 Aspectos Práticos	66
Capítulo 7: Árvore de Decisão	68
7.1 Árvore de Classificação	68
7.2 Árvore de Regressão	72
7.3 Regularização	73
Capítulo 8: Boosting.....	75
8.1 AdaBoost	75
8.1.1 Descrição do Algoritmo para Classificação	75
8.1.2 Discussão sobre a <i>loss function</i>	81
8.1.3 Adaboost para Regressão.....	83
8.2 Gradient Boosting	84
8.2.1 Exemplo com a <i>loss erro quadrático</i>	85
8.2.2 Derivação Matemática	85
8.3 XGBoost.....	87
8.3.1 Função Custo	87
8.3.2 Construção das Árvores	90
8.3.3 XGBoost é um modelo de <i>Gradient Boosting</i> ?.....	91
8.3.4 Exemplos com Funções Loss Específicas	92
Capítulo 9: Rede Neural.....	94
9.1 Estrutura da Rede Neural	94
9.2 Algoritmo de Backpropagation	97
9.3 Escolhendo a Função Custo e as Não-Linearidades	101
9.3.1 Observando a Camada de Saída.....	101
9.3.2 Observando as Camadas Escondidas	102
9.4 Regularização	103
Capítulo 10: Pré-Processamento de Dados	104
10.1 Valores Faltantes.....	104
Capítulo 11: Seleção de Variáveis.....	105
11.1 Baixa Variância	106
11.2 Informação Mútua	106

11.2.1 Maximização da Informação Mútua (MIM)	107
11.2.2 <i>Mutual Information Feature Selection</i> (MIFS)	108
11.2.3 <i>Minimum Redundancy Maximum Relevance</i> (MRMR)	108
11.2.4 <i>Joint Mutual Information</i> (JMI)	108
11.2.5 Comparação entre Métodos	109
Capítulo 12: Principal Component Analysis	110
12.1 Formulação da Máxima Variância	111
12.2 Detalhes Importantes	112
12.3 Relação com a Decomposição SVD	113
Capítulo 13: Avaliação de Desempenho e Seleção de Modelo	114
13.1 Métricas para Classificação Binária	114
13.1.1 Acurácia	114
13.1.2 True Positive Rate (TPR) ou Sensibilidade ou <i>Recall</i>	114
13.1.3 False Positive Rate (FPR)	114
13.1.4 Precisão	115
13.1.5 F1-score	115
13.1.6 F-score	116
13.1.7 Curva ROC (<i>Receiver Operating Characteristic</i>)	116
13.1.8 Curva <i>Precision-Recall</i>	117
13.1.9 <i>Trade-off</i> precisão-recall	118
13.2 Métricas para Classificação Multiclasse	119
13.2.1 Acurácia	119
13.2.2 Acurácia Balanceada	119
13.2.3 Métricas Macro	120
13.2.4 Métricas Macro Ponderadas (<i>weighted</i>)	120
13.2.5 Métricas Micro	121
13.3 Métricas para Regressão	121
13.3.1 <i>Mean Squared Error</i> (MSE)	121
13.3.2 <i>Root Mean Squared Error</i> (RMSE)	121
13.3.3 <i>Mean Absolute Error</i> (MAE)	122
13.3.4 <i>Mean Absolute Percentage Error</i> (MAPE)	122
13.3.5 Coeficiente de Correlação de Pearson (PCC)	122
13.3.6 Coeficiente de Determinação (<i>R</i>²)	122
13.4 Métricas para Agrupamento	124
13.4.1 Inércia	124
13.4.2 Coeficiente de Silhueta	125

13.4.3 Score de Silhueta	125
13.4.4 <i>Akaike Information Criterion</i> (AIC) e <i>Bayesian Information Criterion</i> (BIC)	127
13.4.5 Índice de Calinski-Harabasz.....	128
13.5 Métodos de Validação.....	129
Capítulo 14:	130
Capítulo 15: Apêndices	131
Capítulo 16: Referências.....	157

Capítulo 1: INTRODUÇÃO

- Tipos de aprendizado (supervisionado, não-supervisionado, semi-supervisionado, federado, por reforço).
- Teoria da decisão (olhar [1, p. 348]).
- Tipos de modelos (paramétrico x não-paramétrico; generativo x discriminativo)
- Estimacão de uma função (minimização do valor esperado de uma certa *loss*).
- Função *loss* x função custo.
- Lista de principais *losses*? Ou fica para um apêndice?
- *No free lunch theorem*

Capítulo 2: ESTIMAÇÃO PONTUAL

Um **estimador pontual** [1] pode ser definido como uma função $\hat{\theta} = W(\mathbf{X})$ de uma amostra representada pelo vetor aleatório $\mathbf{X}$, ou seja, qualquer estatística pode ser considerada um estimador pontual (média amostral, desvio padrão amostral etc).

De maneira geral, o estimador pontual é uma função definida para estimar uma quantidade de interesse θ qualquer, como por exemplo, um parâmetro de um modelo ou um valor futuro de alguma variável aleatória de interesse. Portanto, é interessante reforçar que $\hat{\theta}$ é uma variável aleatória, já que é uma função que depende de $\mathbf{X}$, e θ é um valor fixo.

Uma grandeza importante quando tratamos de estimadores pontuais é o **viés**, que indica quanto um estimador pontual desvia na média da quantidade que se deseja estimar. Matematicamente, isso é equivalente a

$$\text{viés}[\hat{\theta}] = \mathbb{E}[\hat{\theta}] - \theta. \quad (2.1)$$

Quando $\text{viés}[\hat{\theta}] = 0$, dizemos que o estimador pontual é **não enviesado**.

2.1 AVALIAÇÃO DE ESTIMADORES

Tendo definido o viés, podemos definir métricas de qualidade para avaliar estimadores pontuais. Uma delas é o **mean squared error (MSE)**, que é dado por

$$\text{MSE} = \mathbb{E}[(\hat{\theta} - \theta)^2]. \quad (2.2)$$

Curiosamente, a expressão do MSE pode ser reescrita como

$$\begin{aligned} \text{MSE} &= \mathbb{V}\text{ar}[\hat{\theta} - \theta] + (\mathbb{E}[\hat{\theta} - \theta])^2 = \mathbb{V}\text{ar}[\hat{\theta}] + (\mathbb{E}[\hat{\theta}] - \theta)^2 \\ \text{MSE} &= \mathbb{V}\text{ar}[\hat{\theta}] + (\text{viés}[\hat{\theta}])^2. \end{aligned} \quad (2.3)$$

Com isso, vemos que o MSE consegue incorporar dois aspectos:

- variância, quantificando a dispersão/variabilidade do estimador;
- viés quadrático, quantificando a tendência sistemática do estimador.

O efeito dessas componentes no estimador pontual é ilustrado na Figura 2-1.

Apesar de o MSE trazer informações úteis sobre um estimador, ele pode não ser suficiente para tomar a decisão de qual o melhor estimador pontual de θ . Na verdade, o melhor $\hat{\theta}$ depende de vários fatores que dependem das especificidades do problema e exigências específicas. Alguns dilemas envolvidos na escolha do melhor estimador envolvendo o MSE são:

- **o estimador com menor MSE pode não ser o que tem menor viés, o que pode ser um limitante dependendo da aplicação** [1, pp. 331-332]. Um exemplo disso são os estimadores $S = \frac{1}{N-1} \sum_{n=1}^N (\bar{X} - X_n)^2$ e $\hat{\sigma}_{\text{MLE}}^2 = \frac{1}{N} \sum_{n=1}^N (\bar{X} - X_n)^2$ para variância de uma v.a. X com distribuição normal. S é não enviesado, porém $\mathbb{V}\text{ar}[S^2] > \mathbb{V}\text{ar}[\hat{\sigma}_{\text{MLE}}^2]$

e $\text{MSE}[S^2] > \text{MSE}[\hat{\sigma}_{\text{MLE}}^2]$. Portanto, nesse caso, a escolha depende do que é preferível para o problema: um estimador que erra consistentemente, mas com menor variância, ou um estimador sem viés, mas com maior variância e maior MSE;

- **MSE de determinado estimador pode ser maior ou menor que de outro estimador dependendo da faixa de valores de θ considerado** [1, pp. 331-332]. Muitas vezes, o MSE é uma função de θ e, para estimadores diferentes, os gráficos de MSE em função de θ podem até se cruzar, mostrando que para diferentes faixas, um estimador pode ser melhor que outro usando o MSE como critério;
- **MSE de determinado estimador pode ser maior ou menor que de outro estimador dependendo do tamanho N da amostra** [1, p. 332]. Como $\text{MSE} = \text{MSE}[W(X)]$, MSE depende da amostra X . Nesse sentido, a explicação é a mesma da do ponto anterior.

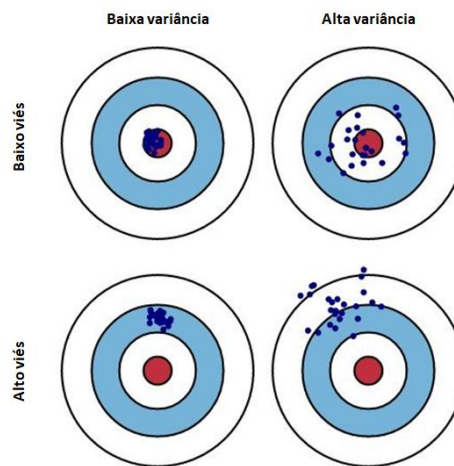


Figura 2-1: Viés e variância.

Portanto, fica claro que não é tão simples escolher um melhor estimador apenas observando o MSE. Uma maneira de tornar essa tarefa um pouco mais fácil é limitar a classe de estimadores, o que, popularmente, é feito ao considerar apenas os estimadores não enviesados. Assim, um estimador é considerado **uniform minimum variance unbiased estimator (UMVUE)** ou **best unbiased estimator**, quando ele é o estimador com menor variância dentre todos os estimadores não enviesados. Formalmente, o UMVUE é o estimador $\hat{\theta}^*$ tal que $\mathbb{E}[\hat{\theta}^*] = \theta$ e $\text{Var}[\hat{\theta}^*] \leq \text{Var}[\hat{\theta}], \forall \hat{\theta} : \mathbb{E}[\hat{\theta}] = \theta$. Por consequência, esse estimador também apresenta o menor MSE, pois, nesse caso, $\text{MSE}[\hat{\theta}] = \text{Var}[\hat{\theta}]$.

Em geral, um empecilho associado ao conceito de UMVUE está no fato de que muitas vezes não conseguimos garantir que testamos todos os estimadores não enviesados possíveis. Assim, muitas vezes, mesmo tendo encontrado um estimador com menor variância dentro de uma lista de estimadores não enviesados selecionados, não conseguimos garantir que testamos todos os estimadores não enviesados.

Um artifício utilizado nessa tarefa é a **desigualdade de Cramér-Rao**, que estabelece um valor mínimo para o valor da variância de um estimador. Assim, se descobrirmos que $\text{Var}[\hat{\theta}]$ é igual a esse valor mínimo, teremos a garantia de que o estimador é UMVUE. Para mais detalhes, consultar [1, p. 335].

2.2 ESTIMAÇÃO FUNCIONAL EM *MACHINE LEARNING*

Quando falamos de estimadores pontuais anteriormente, focamos principalmente em funções que se propunham a estimar uma quantidade a partir de uma amostra aleatória $\mathbf{X} = (X_1, \dots, X_n)$, como por exemplo, o estimador média amostral mostrado abaixo:

$$\hat{\theta}_{\mathbf{X}} = W(\mathbf{X}) = \frac{\sum_{n=1}^N X_n}{N}.$$

Aqui, o subscrito $\mathbf{X}$ em $\hat{\theta}_{\mathbf{X}}$ foi incluído apenas para reforçar que o estimador está associado à amostra $\mathbf{X}$. Vale notar que $\hat{\theta}_{\mathbf{X}}$ é uma função apenas dessa amostra.

Quando passamos ao campo de *machine learning*, estamos interessados, na maior parte das vezes, em estimar não apenas um valor a partir da amostra, mas sim, uma função, representando um modelo preditivo. Por exemplo, se considerarmos uma amostra contendo tamanhos de casas e seus respectivos valores, podemos estar interessados em estimar uma função linear que se ajuste a esses dados. Nesse caso, nossa amostra seria o conjunto de dados $\mathcal{T}$, que poderia ser modelado, não mais como um vetor aleatório, mas como uma matriz aleatória, composta pelos vetores aleatórios Θ , com os valores das casas, por exemplo, e $\mathbf{T}$, com os preços das casas. Isso nos daria, portanto, um **estimador funcional** $\hat{f}_{\mathcal{T}}(x) = W(\mathcal{T}, x)$, que é uma função aleatória, pois depende de $\mathcal{T}$, mas assume um valor diferente para cada x , que é determinístico.

Uma forma diferente de ver esse estimador funcional é interpretá-lo como uma coleção de estimadores pontuais. Isso porque, para cada valor de x fixado, temos um estimador pontual, que é função unicamente de $\mathcal{T}$. Por exemplo, se fixarmos $x = 100$, teríamos o estimador pontual $\hat{\theta}_{\mathcal{T},100} = \hat{f}_{\mathcal{T}}(100)$. Para deixarmos ainda mais concreto, se estivermos falando de uma regressão por mínimos quadrados, teríamos

$$\hat{\theta}_{\mathcal{T},100} = 100 a(\mathcal{T}) + b(\mathcal{T}) = 100 \frac{\sum_{n=1}^N (\theta_i - \bar{\Theta})(T_i - \bar{T})}{\sum_{n=1}^N (\theta_i - \bar{\Theta})^2} + \bar{T} - a\bar{\Theta},$$

em que a barra em cima do vetor aleatório indica a média amostral.

Logo, vemos que temos de fato um estimador pontual, pois $\hat{\theta}_{\mathcal{T},100}$ é uma função que depende apenas da amostra aleatória $\mathcal{T}$. Lembrando que $\hat{\theta}_{\mathcal{T},100}$ também é uma variável aleatória e, para cada amostra observada, teremos um valor observado diferente de $\hat{\theta}_{\mathcal{T},100}$. Portanto, podemos ver o estimador funcional como uma família de estimadores pontuais $\{\hat{\theta}_{\mathcal{T},x}\}_{x \in \mathcal{X}} = \{\hat{\theta}_{\mathcal{T},x_0}, \hat{\theta}_{\mathcal{T},x_1}, \hat{\theta}_{\mathcal{T},x_2} \dots\}$.

2.3 DILEMA VIÉS-VARIÂNCIA EM *MACHINE LEARNING*

Depois de termos visto que modelos de *machine learning* também podem ser visto como uma família de estimadores pontuais, podemos explorar questões relacionadas a viés e variância desses modelos.

A decomposição do MSE nessas duas grandezas, ilustrada na eq. (2.3), se aplica igualmente no caso de modelos de *machine learning* na forma $\hat{f}_{\mathcal{T}}(x)$, com a diferença que agora, o MSE é função da variável determinística x , ou seja,

$$\text{MSE}(x) = \text{Var}[\hat{f}_{\mathcal{T}}(x)] + (\text{viés}[\hat{f}_{\mathcal{T}}(x)])^2.$$

Agora, consideremos que a relação entre T e X é dada por

$$T = g(X) + \varepsilon,$$

em que ε é uma flutuação aleatória independente de X tal que $\mathbb{E}[\varepsilon] = 0$ [2, p. 28]. Com isso, estamos considerando a hipótese de que existe uma relação bem definida entre T e X , mas que é perturbada por uma flutuação aleatória. Isso quer dizer que, mesmo que X tenha um valor observado x_1 , não teremos sempre um mesmo valor observado $y_1 = g(x_1)$ correspondente, pois a $g(x_1)$ sempre será acrescentada uma quantidade aleatória e independente de x_1 . Assim, mesmo conhecendo o valor observado x_1 , nunca teremos a certeza exata do valor de $Y = y$ correspondente.

Levando em conta a hipótese acima e voltando na eq.(2.2), vemos algo interessante:

$$\begin{aligned}\text{MSE}[\hat{g}_T(x)] &= \mathbb{E}[(\hat{g}_T(x) - T)^2] \\ &= \mathbb{E}[(\hat{g}_T(x) - g(X) - \varepsilon)^2] \\ &= \text{Var}[\hat{g}_T(x) - g(X) - \varepsilon] + (\mathbb{E}[\hat{g}_T(x) - g(X) - \varepsilon])^2 \\ &= \text{Var}[\hat{g}_T(x)] + \text{Var}[\varepsilon] + (\mathbb{E}[\hat{g}_T(x)] - g(X))^2 \\ &= \sigma_\varepsilon^2 + \text{Var}[\hat{g}_T(x)] + (\text{viés}[\hat{g}_T(x)])^2\end{aligned}$$

O termo σ_ε^2 surgiu na expressão do MSE, justamente porque agora não estamos falando em estimar um valor escalar, mas sim um valor aleatório que não pode ser determinado deterministicamente apenas conhecendo o valor de x . Esse termo não depende da escolha do modelo e está associado unicamente ao fenômeno que se deseja descrever, descrito pela relação entre T e X . Portanto, não é possível reduzir o valor de σ_ε^2 pela escolha do estimador/modelo. Por esse motivo, esse termo é chamado de **erro irreduzível** e o MSE está limitado inferiormente por ele.

A avaliação de viés e variância de modelos pode ser confuso, pois em estatística, esses termos são usados com frequência, principalmente para descrever variáveis aleatórias. No contexto de avaliação de modelos preditivos, esses termos são usados para descrever o comportamento de um modelo no que diz respeito a sua sensibilidade ao conjunto de treinamento utilizado para construí-lo. Modelos com alta variância e baixo viés, tendem a sofrer mudanças mais drásticas até para pequenas mudanças no conjunto de treinamento. Por outro lado, um modelo com alto viés e baixa variância tendem a ser menos adaptáveis, o que faz com que mudanças no conjunto de treinamento não gerem modelos muito diferentes.

A variância e o viés estão bastante relacionados com a complexidade do modelo. Quanto maior a complexidade, mais adaptável é o modelo e mais ele se ajusta ao conjunto de treinamento, chegando ao ponto de, para modelos muito complexos, se adaptar até às menores estruturas dos dados, como flutuações aleatórias, o que costuma ser indesejável. Assim, quanto mais complexo um modelo, maior sua variância. Por outro lado, modelos menos complexos tendem a ser construídos com hipóteses e suposições muito fortes acerca do fenômeno sendo modelado, o que o torna menos flexível e menos adaptável ao conjunto de treinamento. Modelos desse tipo tendem a ter alto viés e baixa variância.

Essa relação entre a complexidade e a variância e o viés podem ser mais bem entendidas com ajuda de um exemplo. Considere que desejamos criar um modelo preditivo $\hat{f}(X)$ para descrever um fenômeno, representado pelas variáveis aleatórias X e Y de entrada e *target*, respectivamente, que apresentam distribuição conjunta $p_{X,Y}(x, y)$. Considere também que o modelo preditivo é construído a partir de um conjunto de treinamento $\mathcal{T} = \{(x_n, y_n)\}_{n=1}^N$, formado a partir de N amostragens conjuntas de (X, Y) . Nesse problema específico, podemos considerar que $Y = f(X) + \epsilon$, em que $f(X) = \sin(2\pi X)$ e ϵ representam flutuações aleatórias gaussianas. O modelo preditivo é dado por uma regressão linear usando 24 funções base Gaussianas, com regularização *ridge* controlada pelo coeficiente λ para ajustar a complexidade do modelo.

Amostragem a partir de $p_{X,Y}(x, y)$

Para entender intuitivamente como é feita uma amostragem a partir de uma distribuição conjunta, considere o seguinte exemplo. Em determinado bairro, o preço das casas, representado por Y , é dependente do tamanho delas, representado por X . Nesse exemplo, $X \sim \mathcal{N}(100, 20^2)$ (sendo as unidades em m^2) e $Y = 2000X + \epsilon$ reais, em que $\epsilon \sim \mathcal{N}(0, 5000^2)$ (sendo as unidades em reais) representa uma flutuação aleatória. Seja, $g(\cdot; \mu, \sigma^2)$ a p.d.f. de uma distribuição $\mathcal{N}(\mu, \sigma^2)$. Com isso, temos:

- $p_X(x) = g(x; 100, 20^2)$;
- $p_Y(y) = g(y; 200\,000, 40\,000^2 + 5000^2)$;
- $p_{Y|X}(y|x) = 2000x + g(y; 0, 5000^2) = g(y; 2000x, 5000^2)$;
- $p_{X,Y}(x, y) = p_{Y|X}(y|x)p_X(x) = g(y; 2000x, 5000^2) \times g(x; 100\,m^2, 20^2)$.

A Figura 2-2 mostra alguns valores amostrados das variáveis X e Y , junto com a curva de nível da função de densidade conjunta delas.

Quando falamos em amostrar valores de X e Y em conjunto, estamos falando em amostrar de uma distribuição com densidade $p_{X,Y}(x, y)$. Obviamente, na prática, não conhecemos previamente $p_{X,Y}(x, y)$. Logo, coletamos manualmente os valores de x_n e y_n , registrando o tamanho de uma casa desse bairro e o seu preço.

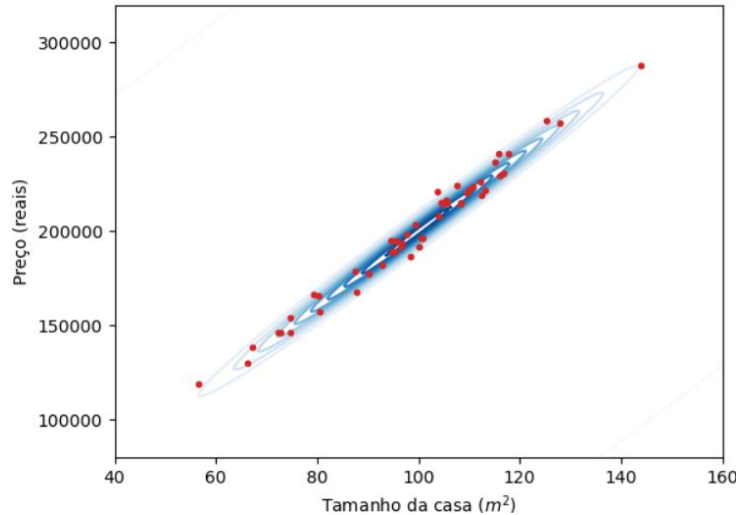


Figura 2-2: Algumas amostras (em vermelho) de (X, Y) do exemplo do preço das casas. As curvas de nível da p.d.f. conjunta $p_{X,Y}(x, y)$ também são mostradas (em azul).

Dadas essas premissas, geramos 100 conjuntos de treinamento diferentes amostrando pontos de $p_{X,Y}(x, y)$, com $N = 25$. Com isso, geramos 100 modelos diferentes $\hat{f}_{\mathcal{T}_i}(X)$, cada um gerado a partir de um conjunto de treinamento $\mathcal{T}_i$ diferente. Fazemos exatamente esse mesmo procedimento para 3 valores de λ diferentes. Os resultados são mostrados na Figura 2-3.

Primeiramente, podemos perceber que, para o modelo menos complexo (maior valor de λ), os modelos não se adaptam muito bem à forma da senoide dada por f , além de serem todos muito próximos uns dos outros. Modelos desse tipo apresentam viés grande e baixa variância. Por outro lado, à medida que aumentamos a complexidade do modelo (diminuindo o valor de λ), percebemos uma grande variabilidade das curvas de regressão, o que indica que mudanças no conjunto de treinamento têm alto impacto na forma do modelo. Modelos desse tipo apresentam alta variância e baixo viés.

Outro ponto interessante de se reparar é que, no lado direito, em que é mostrada a curva de média dos modelos, podemos ver que, quanto mais complexos são os modelos, melhor se torna a curva de média. Esse é o princípio que está na base dos métodos de *ensemble*, que serão detalhados em capítulos posteriores.

No entanto, em geral, tanto modelos com alto viés, quanto modelos com alta variância tendem a ser indesejáveis, pois ambos tendem a aumentar o erro do modelo. Para entender isso, precisamos primeiramente definir alguns conceitos.

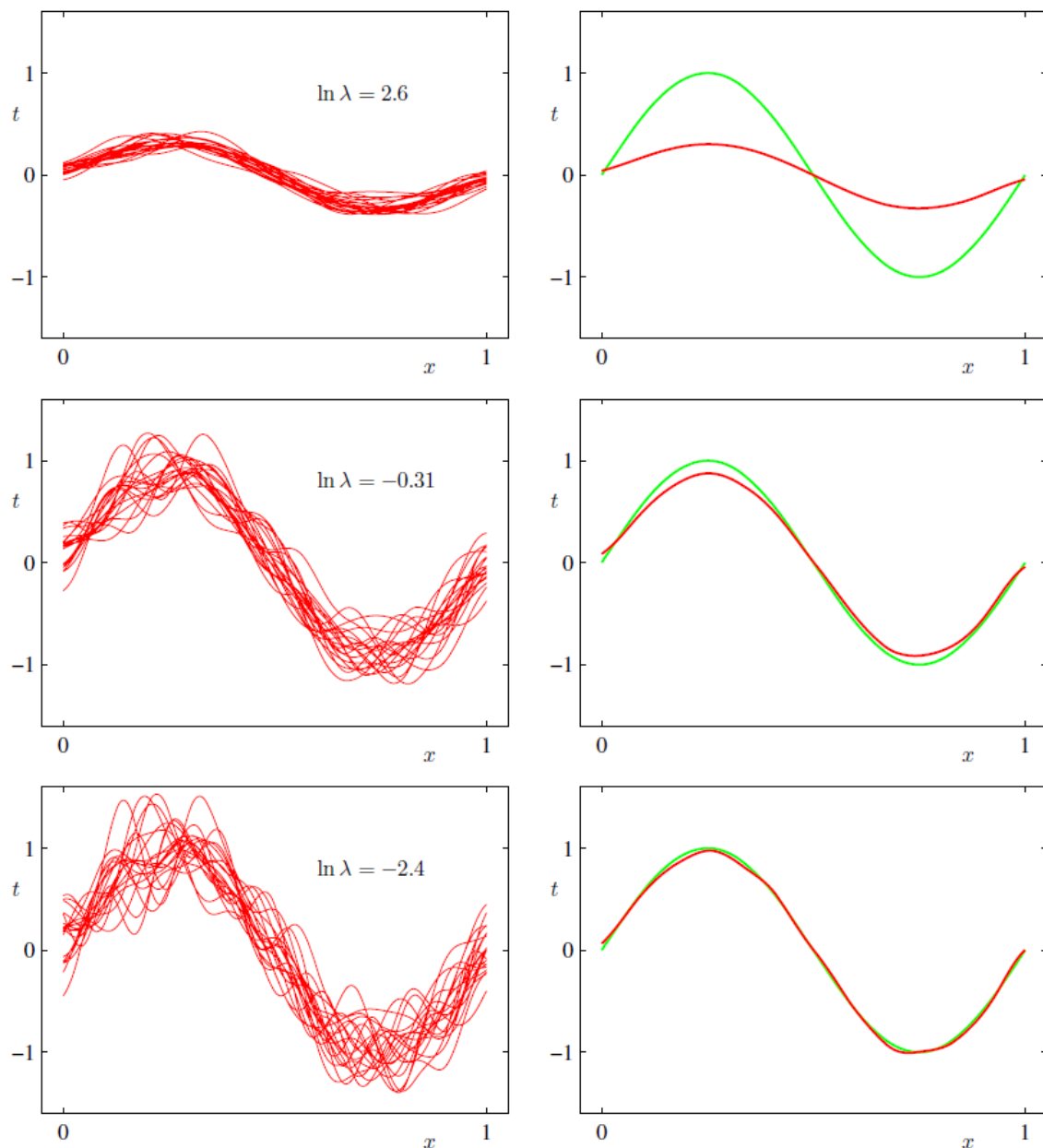


Figura 2-3: Ilustração da relação entre a complexidade do modelo e o viés e a variância. São usados 100 conjuntos de treinamento para cada valor de λ , cada conjunto com tamanho $N = 25$. Os gráficos do lado esquerdo mostram os 100 modelos treinados, cada linha representando um valor de λ diferente. Já os gráficos do lado direito, mostram a função de referência $f(X)$ em verde e, em vermelho, a média dos modelos para o valor de λ dessa linha. Essa imagem foi retirada de [3].

O primeiro deles é o chamado de **erro de teste** ou **erro de generalização**, que é o erro de predição obtido por meio de dados de teste independentes, que seguem a distribuição conjunta $p_{X,Y}(x, y)$. O erro de generalização é dado por

$$\text{Err}_{\mathcal{T}} = \mathbb{E} \left[L \left(Y, \hat{f}(X) \right) | \mathcal{T} \right],$$

em que L é uma função *loss* que quantifica o erro entre Y e $\hat{f}(X)$.

O que essa expressão está dizendo é que, dado um conjunto de treinamento $\mathcal{T}$ fixo utilizado para construir $\hat{f}$, é calculado o valor esperado de $L \left(Y, \hat{f}(X) \right)$ sobre as diferentes observações (x_n, y_n) das variáveis aleatórias X e Y , que representam a amostra de teste. Observar que, como estamos calculando o valor esperado, estamos considerando as infinitas combinações de (x_n, y_n) para os dados de teste, uma vez que o cálculo de $\mathbb{E}$ envolve uma integração (possivelmente uma soma) ao longo de todas as realizações possíveis de X e Y .

Ao construirmos o estimador $\hat{f}$ usando diferentes conjuntos de teste, obtemos também diferentes valores de $\text{Err}_{\mathcal{T}}$, pois o estimador será diferente para cada $\mathcal{T}$ escolhido. Logo, se $\mathcal{T}_1 \neq \mathcal{T}_2$, em geral, $\text{Err}_{\mathcal{T}_1} \neq \text{Err}_{\mathcal{T}_2}$. Para termos uma noção do erro médio sobre diferentes conjuntos de teste, calculamos o valor esperado do erro de generalização com relação a $\mathcal{T}$, o que dá origem ao **erro de teste esperado**:

$$\text{Err} = \mathbb{E}[\text{Err}_{\mathcal{T}}] = \mathbb{E}_{\mathcal{T}} \left[\mathbb{E} \left[L \left(Y, \hat{f}(X) \right) | \mathcal{T} \right] \right] = \mathbb{E} \left[L \left(Y, \hat{f}(X) \right) \right],$$

em que o subscrito $\mathcal{T}$ em $\mathbb{E}_{\mathcal{T}}$ foi utilizado apenas para deixar explícito que o valor esperado mais externo nessa expressão é calculado sobre os diferentes conjuntos de teste.

Além dessas métricas de erro apresentadas, também existe o **erro de treinamento**, calculado a partir da média amostral da amostra de treinamento:

$$\overline{\text{err}} = \frac{1}{N} \sum_{n=1}^N L \left(y_n, \hat{f}(x_n) \right).$$

Além disso, também podemos calcular o erro de treinamento esperado $\mathbb{E}[\overline{\text{err}}]$, calculando o valor esperado sobre os diferentes conjuntos de treinamento.

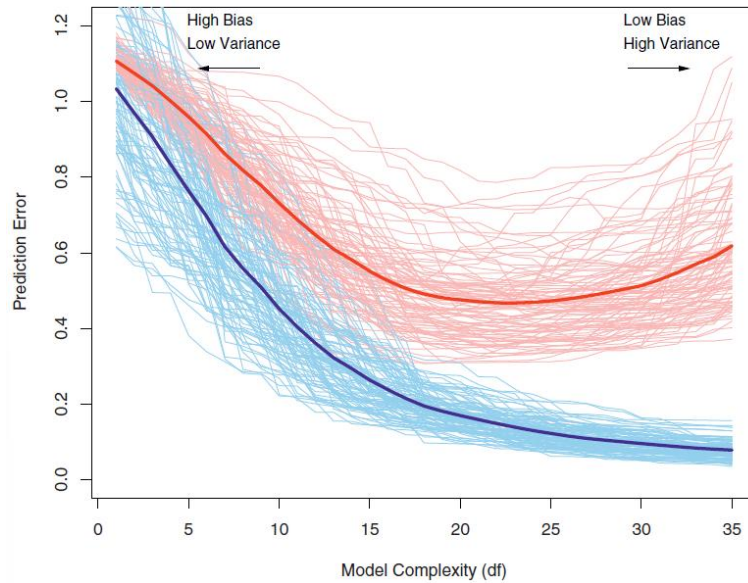


Figura 2-4: Comportamento dos erros de teste e treinamento em função da complexidade do modelo. As curvas de cor azul-claro representam o erro de treinamento $\overline{err}_T$ e as curvas de cor vermelho-claro representam o erro de teste Err_T , para 100 diferentes conjuntos de treinamento T (para cada cor), todos de tamanho 50. Já as curvas azul-escura e vermelho-escura representam Err e $\mathbb{E}[\overline{err}_T]$, respectivamente. Essa imagem foi retirada de [2].

A Figura 2-4 ilustra como se comportam esses erros de acordo com a complexidade do modelo. Com isso, conseguimos perceber que, à medida que o modelo fica mais complexo e, consequentemente, com maior variância, seu erro de treinamento diminui, mas seu erro de teste aumenta. Além disso, à medida que o modelo fica menos complexo e, consequentemente, com maior viés, seu erro de treinamento aumenta, assim como seu erro de teste. Portanto, podemos perceber que existe uma faixa de complexidade ideal para o modelo, em que ele não apresenta muito viés nem muita variância e seu erro de teste é menor.

2.4 MAXIMUM LIKELIHOOD ESTIMATION (MLE)

Para entendermos os métodos desta seção, precisamos lembrar do Teorema de Bayes:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)},$$

em que

$P(\theta)$: **Probabilidade a priori** de a distribuição que gerou os dados D tenham θ como parâmetro.

$P(D)$: Probabilidade de os dados D terem sido gerados. O símbolo D resume o acontecimento de um conjunto de eventos. Se estivermos considerando um experimento em que uma moeda é lançada N vezes, teremos uma variável aleatória que assume valor 0 (coroa) e 1 (cara). D poderia representar, por exemplo, o conjunto de 3 lançamentos, ou seja, $(X_1 = 1) \cap (X_2 = 1) \cap (X_3 = 0)$. Assim, $P(D)$ representaria a probabilidade de acontecer essa sequência de resultados.

$P(\theta|D)$: **Probabilidade a posteriori**, ou seja, é a probabilidade de que seja θ o parâmetro da distribuição que gera os dados, sabendo que os dados D foram gerados a partir dessa distribuição.

$P(D|\theta)$: **Função de verossimilhança** (*likelihood*), que é a probabilidade de os dados D terem sido gerados por uma distribuição com parâmetro θ dado. Normalmente, em problemas de estimação, temos os dados e tentamos determinar θ . Dependendo do contexto, ela também costuma ser definida como [4, p. 122]

$$\mathcal{L}_N(\theta) = \prod_{n=1}^N P(X_i; \theta),$$

em que N é o número de amostra contidas nos dados. Lembrando que, $\prod_{n=1}^N P(X_i; \theta) = P(\cap_{n=1}^N X_i; \theta) = P(D; \theta)$. Logo, é só uma questão de como interpretar θ : como uma informação a priori ($P(D|\theta)$) ou como um parâmetro da lei de probabilidade ($P(D; \theta)$).

Em geral, quando falamos de *machine learning*, as probabilidades a priori e a posteriori e a verossimilhança são sempre representadas por essas probabilidades mostradas acima. No entanto, em geral, em qualquer outro problema de estatística, as probabilidades a priori e a posteriori vão depender do ponto de vista. Em dado exemplo, podemos considerar $P(A)$ como sendo a probabilidade a priori e $P(A|B)$ a probabilidade posteriori, mas se invertermos o ponto de vista, podemos também considerar que a $P(B)$ e a $P(B|A)$ são as probabilidades a priori e a posteriori, respectivamente.

Estimamos os parâmetros somente com base nas evidências (dados). Para isso, maximizamos a função de verossimilhança da distribuição, isto é,

$$\theta_{MLE} = \arg \max_{\theta} P(D|\theta).$$

Conhecendo-se a expressão de $P(D|\theta)$, torna-se fácil resolver esse problema. Isso nem sempre é possível, pois nem sempre conhecemos a distribuição geradora dos dados $X_i \in D$. No caso de lançamentos de moedas, como veremos no exemplo abaixo, podemos supor que os dados tem distribuição de Bernoulli com parâmetro p . Mesmo nos casos em que não conhecemos a distribuição dos dados, ainda é possível realizar hipóteses e assumir que eles seguem alguma distribuição conhecida (Bernoulli, Gaussiana...).

Lembrar que, usualmente, quando nos referimos à expressão das distribuições usamos uma notação como $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$, sem deixar explícitos os parâmetros. No entanto, é só uma questão de notação e poderíamos escrever $f(x; \mu, \sigma^2)$, $f(x|\mu, \sigma^2)$ ou $\mathcal{L}(\mu, \sigma^2)$. Cada notação interpreta os parâmetros de uma maneira diferente, como já foi discutido no início do capítulo.

É importante perceber que toda probabilidade pode ser interpretada como uma probabilidade condicional, por mais que a informação a posteriori não esteja explicitamente indicada na expressão matemática. Assim, de maneira resumida, quando escrevíamos apenas $P(x)$ nas aulas de probabilidade e estatística, era só uma maneira resumida de escrever $P(x|K)$, em que K é algum conhecimento a posteriori (os parâmetros da distribuição, por exemplo, que estavam implícitos). Em [5, p. 59], é feita uma ótima discussão sobre isso.

Para finalizar, é necessário prestar atenção que, quando utilizamos a notação $P(D)$, estamos assumindo que não temos nenhuma outra informação a priori, nem mesmo a distribuição dos dados e, por isso, não podemos utilizar as expressões conhecidas das funções de distribuição de probabilidade.

Exemplo 1

Consideramos uma moeda que foi lançada 5 vezes, dando como resultado $D = \{C, K, C, C, C\}$. Queremos determinar a distribuição de probabilidades que representa os resultados dessa moeda. Podemos dizer que é uma distribuição de Bernoulli com parâmetro p (probabilidade de sucesso, que será representado pelo resultado “Cara”).

Usando o critério de MLE, desejamos que a *likelihood* $P(D|p)$, seja a maior possível, ou seja, a probabilidade de que os dados D tenham vindo de uma distribuição de Bernoulli com parâmetro p dado. Se considerarmos que os lançamentos são independentes, temos

$$P(D|p) = P(C|p)P(K|p)P(C|p)P(C|p)P(C|p)$$

Como, $P(K|p) = 1 - p$ e $P(C|p) = p$, temos

$$P(D|p) = p^4(1 - p) = -p^5 + p^4$$

Para maximizar a expressão acima, fazemos

$$\frac{\partial P(D|p_{MLE})}{\partial p_{MLE}} = 0$$

$$-5p_{MLE}^4 + 4p_{MLE}^3 = 0$$

$$p_{MLE}^3(-5p_{MLE} + 4) = 0$$

A única solução coerente para esse problema é a solução para

$$-5p_{MLE} + 4 = 0$$

$$p_{MLE} = 4/5 = 0,8.$$

Com isso, a partir das evidências, chegamos à conclusão de que a moeda é viciada. No entanto, podemos argumentar que os dados não são representativos e que precisaríamos de mais dados para chegar a um resultado mais acurado. De toda forma, o método nos dá um mecanismo para estimar a probabilidade p de sair “Cara”.

Se tivéssemos um número genérico n de “Caras” e m de “Coroas”, com $N = n + m$, teríamos uma expressão do tipo

$$P(D|p) = p^n(1 - p)^m.$$

Essa expressão é muito difícil de derivar. Um possível artifício que podemos utilizar é aplicar o logaritmo, o que nos daria

$$\ell_N(p) = \log P(D|p) = n \log p + m \log(1 - p),$$

ficamos com uma soma em vez de um produto, que é muito mais simples de derivar.

À primeira vista isso pode parecer estranho, mas, se repararmos, o ponto de mínimo de $\ell_N(p)$ também é o ponto de mínimo de $P(D|p)$. O termo $\ell_N(p)$ é chamado de **log likelihood**.

Assim, prosseguimos fazendo

$$\frac{\partial}{\partial p} \ell_N(p) = 0$$

$$\frac{n}{p} - \frac{m}{1-p} = 0$$

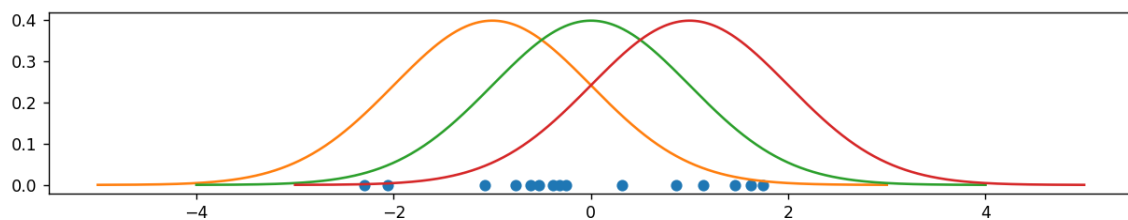
$$\frac{n - np - mp}{p(1-p)} = 0$$

$$p_{MLE} = \frac{n}{N}, \text{ para } p \in (0,1),$$

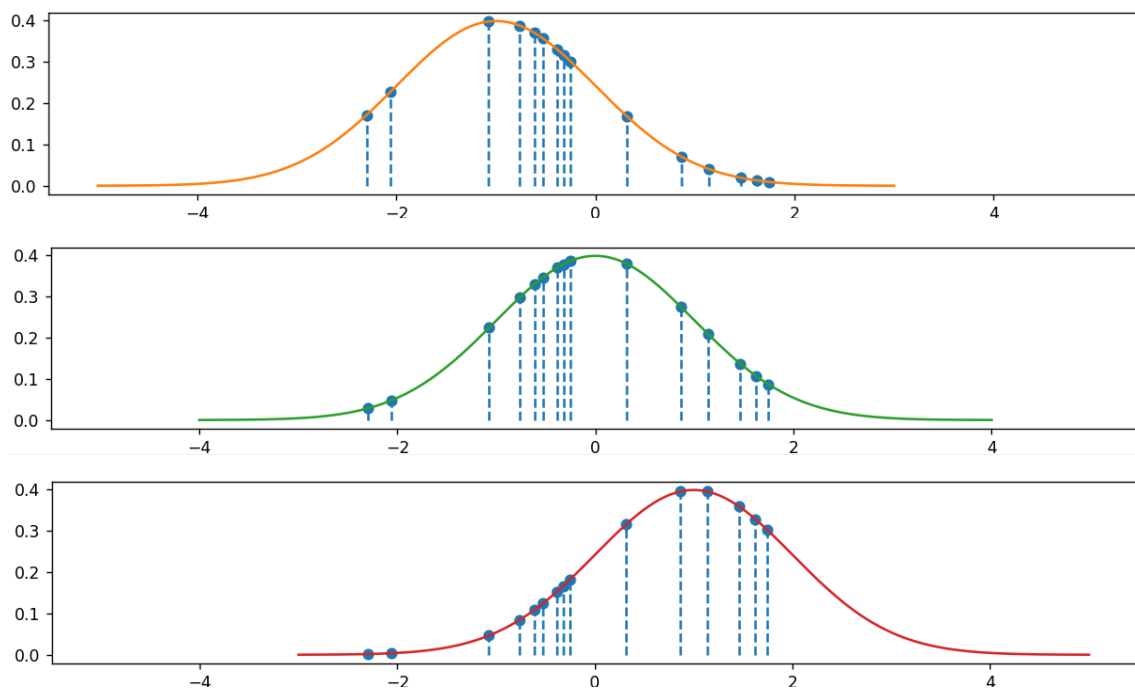
que é justamente a proporção de “Caras” no experimento.

Exemplo 2

Vamos considerar que agora temos alguns dados D e sabemos que eles seguem uma distribuição Gaussiana, mas com parâmetros desconhecidos μ e σ . Nosso trabalho, novamente, é encontrar o valor desses parâmetros ao tentar maximizar a *likelihood* $f(D|\theta)$, com $\theta = (\mu, \sigma)$ (aqui estamos usando f no lugar de P por se tratar de uma distribuição contínua e não mais de uma distribuição discreta). Intuitivamente, estamos tentando encontrar a curva representativa da distribuição Gaussiana que se encaixa melhor nos dados. Na imagem abaixo, observamos os dados em azul e três exemplos de gaussianas que poderiam ter gerado esses dados.



Abaixo, as três Gaussianas são representadas em gráficos separados e os pontos são projetados sobre as curvas para facilitar a análise.



A partir dessa análise visual, podemos ver que as Gaussianas em laranja e verde são as que mais parecem ter gerado os dados em azul.

Intuitivamente, é isso que fazemos: variamos os parâmetros até encontrar a curva que compreende os dados de maneira mais “coerente”, ou seja, encontramos a curva mais provável de ter gerado esses dados. Isso é o equivalente a encontrar os parâmetros que dão o maior valor da *likelihood* $f(D|\theta)$.

Como temos N dados supostamente gerados independentemente, a *likelihood* pode ser escrita como

$$f(D|\theta) = \prod_{n=1}^N f(x_n|\theta).$$

Assim, queremos que esse produto seja o maior possível. Falando de maneira simplificada e visual, queremos encontrar θ tal que os pontos estejam arranjados em sua maioria mais próximos do pico da gaussiana do que das caudas, pois assim, o produto será o maior possível.

Partindo para a análise matemática, teremos um sistema de duas equações: uma considerando a derivada com relação a μ e outra com relação a σ , como mostrado abaixo:

$$\begin{cases} \frac{\partial f(D|\theta)}{\partial \mu} = 0 \\ \frac{\partial f(D|\theta)}{\partial \sigma} = 0 \end{cases}$$

O produtório de $f(D|\theta)$ torna as derivadas um pouco inconvenientes. Por isso, adotamos um procedimento similar ao do exemplo anterior e tentamos minimizar a *log likelihood*.

Com isso, teremos

$$\begin{cases} \frac{\partial \log f(D|\theta)}{\partial \mu} = 0 \\ \frac{\partial \log f(D|\theta)}{\partial \sigma} = 0 \end{cases}$$

Por sorte, a primeira equação desse sistema é independente de σ , o que facilita as contas.

Lembrando que, $f(x|\theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$, a primeira equação fica

$$\frac{\partial \log f(D|\theta)}{\partial \mu} = \frac{\partial}{\partial \mu} \left[- \sum_{n=1}^N \log \frac{1}{\sqrt{2\pi\sigma^2}} \frac{(x_n - \mu)^2}{2\sigma^2} \right] = 0$$

$$\frac{\partial}{\partial \mu} \left[\sum_{n=1}^N (x_n - \mu)^2 \right] = 0$$

$$-2 \sum_{n=1}^N (x_n - \mu) = 2N\mu - 2 \sum_{n=1}^N x_n = 0$$

$$\mu_{MLE} = \frac{1}{N} \sum_{n=1}^N x_n$$

Ao desenvolver a segunda equação, encontramos

$$\sigma_{MLE} = \frac{1}{N} \sum_{n=1}^N (x_n - \mu_{MLE})^2$$

Portanto, para encontrar o valor dos parâmetros da distribuição gaussiana mais provável de ter gerado os dados, pelo critério MLE, basta substituir os valores de x na equação acima.

2.5 MAXIMUM A POSTERIORI (MAP) ESTIMATION

Estimamos os parâmetros com base nas evidências (dados), mas também em conhecimento a posteriori. Com isso, o que fazemos é calcular

$$\theta_{MAP} = \arg \max_{\theta} P(\theta|D).$$

Diferentemente do caso MLE, não temos uma expressão direta para $P(\theta|D)$, pois, normalmente, não conhecemos uma expressão para a distribuição de probabilidade do parâmetro θ . Por isso, nos utilizamos do teorema de Bayes e reescrevemos essa equação como

$$\theta_{MAP} = \arg \max_{\theta} \frac{P(D|\theta)P(\theta)}{P(D)} = \arg \max_{\theta} P(D|\theta)P(\theta).$$

Capítulo 3: K-NEAREST NEIGHBORS

O *K-nearest neighbors* (KNN) é um método que pode ser utilizado para classificação e regressão, baseado no princípio de que a saída do método depende dos K vizinhos mais próximos do ponto para o qual se deseja realizar uma predição. Por isso ele é considerado um método baseado em distância. Existem diferentes métricas que podem ser utilizadas para determinar a distância entre dois pontos, como será apresentado ainda neste capítulo, porém a mais comum é a distância Euclidiana.

3.1 IMPLEMENTAÇÕES DO MÉTODO

Nesta seção, são apresentados diferentes algoritmos para resolver o problema de encontrar os K vizinhos mais próximos de um determinado ponto de teste. É importante mencionar que, apesar de serem algoritmos diferentes, eles sempre terão como resultado os mesmos conjuntos de K vizinhos. A diferença entre eles está apenas na forma como é feita a busca por esses K vizinhos mais próximos e, consequentemente, na complexidade de tempo deles.

3.1.1 Força bruta

É a implementação mais comum de ser vista do KNN. Para entender seu funcionamento, comecemos pelo problema de classificação. Para um dado valor de K e um certo ponto $\mathbf{z}$ que se deseja classificar, o algoritmo consiste em comparar a distância $D(\mathbf{x}_i, \mathbf{z})$ (podendo ser a distância Euclidiana, por exemplo, como veremos adiante) entre o ponto $\mathbf{z}$ e cada um dos pontos $\mathbf{x}_i$, que possuem classes conhecidas. Em seguida, separamos os K pontos $\mathbf{x}_i$ mais próximos de $D(\mathbf{x}_i, \mathbf{z})$ e fazemos a contagem de quantos desses pontos pertencem a cada classe, como uma espécie de votação. A classe mais votada (com mais pontos) é a classe utilizada para classificar o ponto $\mathbf{x}_i$. Essa maneira de classificar o ponto $\mathbf{z}$ é chamada de *hard labeling*, em oposição ao *soft labeling*, que será apresentado adiante. Em caso de empate entre classes, normalmente, a decisão é feita arbitrariamente. Particularmente, no scikit-learn, o classificador KNeighborsClassifier resolve esse empate escolhendo a classe que aparece primeiro no conjunto de dados [6]. Para evitar esse tipo de problema, é recomendável que K não seja um múltiplo do número de classes [7, p. 316].

Outra maneira de classificar o ponto $\mathbf{z}$, é por meio do *soft labeling* [8], que, em vez de atribuir a classe majoritária ao ponto $\mathbf{z}$, associa probabilidades a cada classe. Dessa forma, para cada classe C_j , é calculada a probabilidade de $\mathbf{z}$ pertencer a essa classe:

$$\mathbb{P}(C_j|\mathbf{z}) = \frac{k_j}{K},$$

em que k_j é o número de vizinhos pertencentes à classe C_j .

É interessante notar que, não é necessário um treinamento propriamente dito para utilizar o KNN. Isso porque esse método não aprende parâmetros para classificar novas instâncias. Basta dispor de um conjunto de dados anotados e com isso, já é possível classificar novas instâncias. Assim, para funcionar, ele precisa memorizar todas as instâncias de “treinamento”. Por isso, podemos dizer que o KNN é um método que não apresenta um modelo [2, p. 463]. Outros termos também são usados para caracterizar esse tipo de “aprendizado” do KNN, como *instance based learning* e *non-generalizing learning* [6].

Apesar de, até o momento, só termos tratado o caso da classificação, o KNN também pode ser utilizado para realizar regressões. O funcionamento é muito similar, com a diferença que, em vez de realizar uma classificação com base na classe majoritária dos vizinhos, atribuímos um valor contínuo a $\mathbf{z}$, calculado a partir da média do *target* dos K vizinhos, ou seja,

$$t_{\mathbf{z}} = \frac{1}{K} \sum_{i=1}^K t_{\mathbf{x}_i}.$$

Na Figura 3-1 podemos ver um exemplo de classificação e na Figura 3-2 podemos ver um exemplo de regressão usando o método KNN.

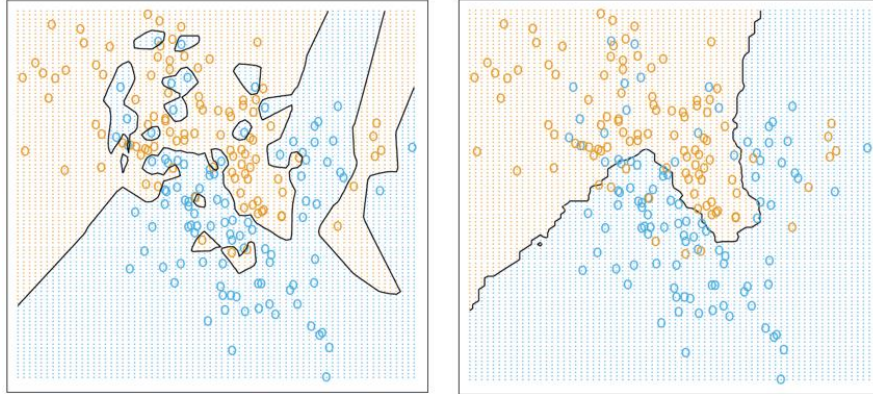


Figura 3-1: Fronteiras de decisão encontrada usando KNN para $K = 1$ (esquerda) e $K = 15$ (direita).

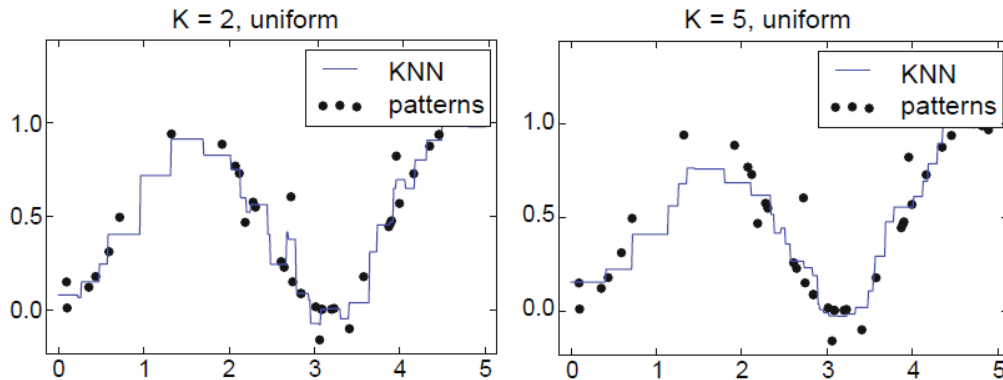


Figura 3-2: Regressão realizado com KNN para dois valores de K .

Do ponto de vista de complexidade, como esse algoritmo não exige treinamento, só faz sentido analisar a complexidade de tempo da inferência diretamente. Nesse caso, consideremos que, para um vizinho, ou seja, $K = 1$, deveremos iterar ao longo dos N pontos de dimensão d e calcular suas respectivas distâncias para um ponto $\mathbf{z}$. Cada cálculo de distância (considerando a distância Euclidiana) envolve resolver uma expressão do

tipo $\sqrt{\sum_{j=1}^d (x_j - z_j)^2}$. Logo, nesse caso, teremos uma complexidade $O(N \times d)$. No entanto, para um número K de vizinhos, precisamos repetir todo esse processo K vezes, calculando a cada iteração o k -ésimo vizinho mais próximo. Nesse caso, a complexidade total do algoritmo é de $O(K \times N \times d)$. Portanto, vemos que para espaços de dimensão elevada ou para grandes quantidades de amostra, esse algoritmo pode ser ineficiente.

Com relação à escolha do valor de K , o valor ótimo para esse hiperparâmetro varia dependendo do problema. No entanto, quanto menor o valor de K , maior será a tendência de o método capturar padrões mais sutis do conjunto de dados e possivelmente ruído caso K seja muito pequeno, o que pode levar o método a sofrer *overfitting* durante a predição de novos dados. No caso limite em que $K = 1$, o método tem 100% de acurácia no conjunto de dados inicial, mas pode apresentar erros elevados para novos dados.

Por outro lado, quanto maior o valor de K menos sensível é o método a detalhes mais finos da distribuição dos dados iniciais. Se o valor de K for excessivamente grande, o método tem maior chance de sofrer *underfitting* e também acabamos por desviar da suposição elementar do método, de que entradas (pontos) semelhantes/próximas possuem saídas semelhantes. Isso porque ao aumentar excessivamente o valor de K , acabamos levando em consideração muitos outros pontos que muitas vezes não estarão perto do ponto de interesse, o que é equivalente a ter uma vizinhança muito grande. No limite em que $K = N$, a solução do problema é trivial, pois qualquer ponto terá todos os outros pontos do conjunto inicial como vizinhos. O problema, então, se resume a sempre classificar (no caso de um problema de classificação) novos pontos como sendo da classe majoritária, o que eleva o erro de classificação. Na Figura 3-1 e na Figura 3-2 é possível ver como se comporta o KNN para diferentes valores de K .

Assim, podemos dizer que:

- ao aumentar o valor de K , diminuimos a complexidade do método, aumentamos seu viés e, para valores excessivamente elevados de K , aumentamos a chance de *underfitting*;
- ao diminuir o valor de K , aumentamos a complexidade do método, aumentamos sua variância e, para valores excessivamente baixos de K , aumentamos a chance de *overfitting*.

3.1.2 K-D Tree

Outra maneira de se encontrar os K vizinhos mais próximos de um ponto z se dá por meio do algoritmo K-D tree [9]. Ele surge como uma alternativa ao algoritmo de força bruta, que precisa checar a distância do ponto z para cada um dos pontos x_i , o que pode ser muito custoso computacionalmente, principalmente se o espaço tiver um número grande de dimensões ou grande quantidade de dados. O K-D tree particiona o espaço previamente de maneira que, no geral, não é necessário visitar todos os pontos para encontrar o vizinho mais próximo de um novo ponto (porém, no pior caso, pode acontecer de ser necessário visitar todos os pontos) [10]. Uma observação interessante é que, o nome K-D tree vem originalmente de K-dimensional tree. Logo, esse “K” indica a dimensão dos dados com os quais estamos trabalhando e não o número de vizinhos.

Primeiramente, é necessário construir uma árvore binária em que cada nó representa um ponto do conjunto inicial de dados x_i . Para entender como é feita a decisão de qual ponto representará qual nó da árvore, começamos a montar a árvore pela raiz. Escolhemos arbitrariamente uma das dimensões do espaço e encontramos a mediana dos pontos referente a essa dimensão. O ponto que representa a mediana nessa dimensão é o ponto escolhido para representar o nó raiz. Por exemplo, se tivermos pontos $x_1 = (1 \ 1 \ 1)$, $x_2 = (2 \ 1 \ 1)$ e $x_3 = (3 \ 1 \ 1)$, ao escolhermos o eixo referente à primeira dimensão, veremos que o ponto que representa a mediana (com valor 2) nessa dimensão é x_2 e, por isso, ele seria o ponto representante do nó raiz. Ao escolher o ponto do nó raiz, segmentamos o espaço em

duas regiões, por meio de um hiperplano ortogonal ao eixo da dimensão escolhida e passando pelo ponto escolhido. A Figura 3-3 ilustra essa primeira etapa para o caso de duas dimensões.

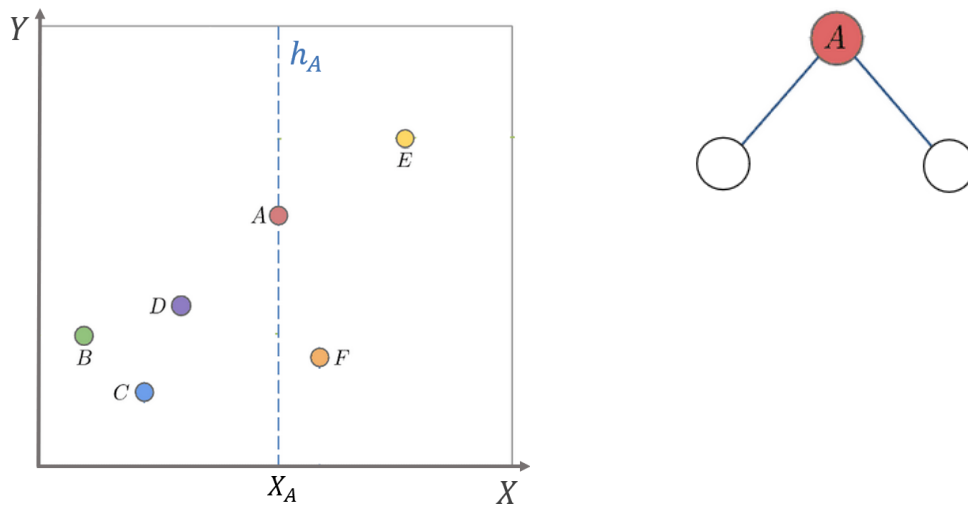


Figura 3-3: Primeira etapa da construção da K-D tree.

Após essa primeira etapa, o nó raiz é dividido em dois nós filhos, mas ainda não sabemos a quais pontos eles estão associados. Então, repetimos o procedimento descrito anteriormente, só que dessa vez, trocamos a dimensão que iremos considerar. No caso da Figura 3-3, olhamos primeiro para o eixo X , então agora olharemos para o eixo Y . Claramente, na porção do espaço à esquerda do hiperplano h_A , o ponto que representa a mediana em Y é o B . Do lado direito, a escolha é arbitrária, então escolhemos o ponto E . Com isso, dividimos cada uma das regiões que tínhamos anteriormente em duas, como mostra a Figura 3-4.

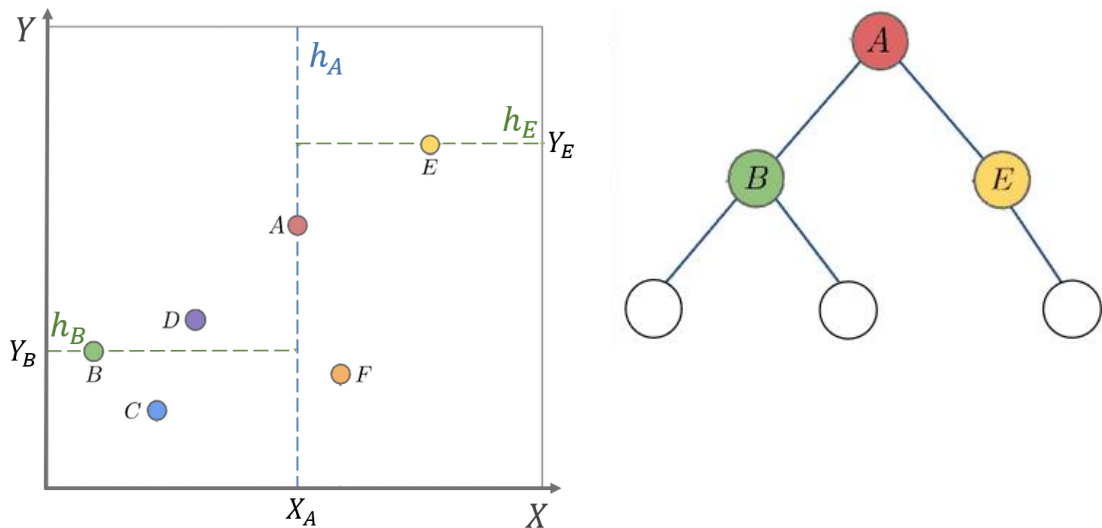


Figura 3-4: Segunda etapa da construção da K-D tree.

Esse procedimento é repetido analogamente para os demais nós da árvore, sempre alternando os eixos considerados e traçando de uma vez todos os hiperplanos correspondentes a nós de uma mesma profundidade da árvore. O resultado da terceira e última etapa do exemplo em duas dimensões é mostrado na Figura 3-5.

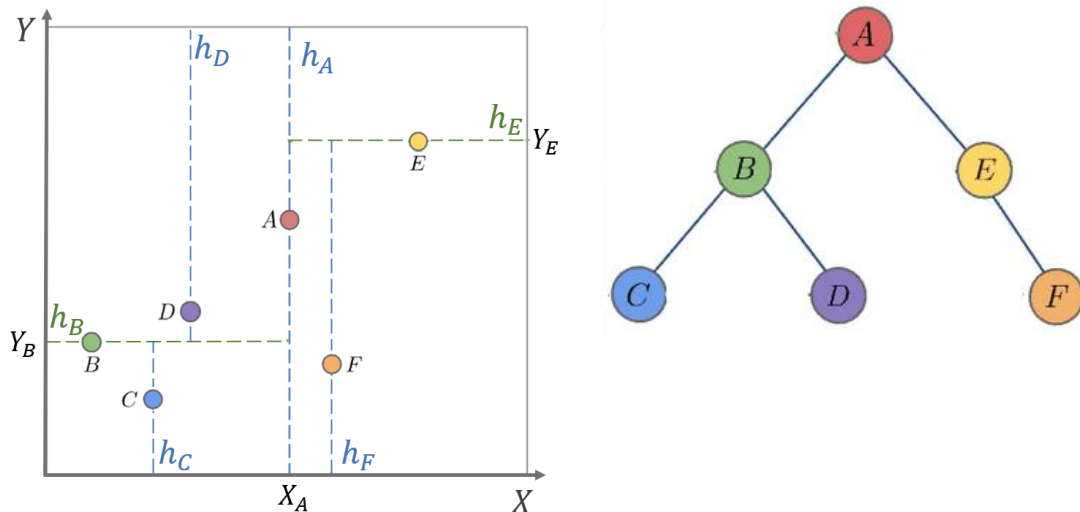


Figura 3-5: Última etapa da construção da K-D tree.

Construída a árvore, podemos utilizá-la para encontrar o vizinho mais próximo de um novo ponto z . Para isso, começando na raiz, percorremos a árvore descendo para o nó esquerdo sempre que o ponto estiver à esquerda do hiperplano vertical ou abaixo do hiperplano horizontal e descendo para o nó da direita caso contrário, até chegar a uma folha. Por exemplo, considere o ponto z , da Figura 3-6. Ao percorrermos a árvore, fazemos o seguinte caminho: da raiz para B, pois z está à esquerda de h_A ; de B para C, pois z está abaixo de h_B . No entanto, o ponto C não é o vizinho mais próximo. Repare que, mesmo z estando na partição abaixo de h_B e à esquerda de h_A , podem existir pontos próximos desses hiperplanos que estão mais próximos. De fato, é o que acontece. Reparar que, apesar de o ponto D estar localizado em outra partição, ele está mais próximo de z do que C. Para resolver esse problema, o algoritmo exige que voltemos em outros nós da árvore para conferir se não existe outro vizinho mais próximo.

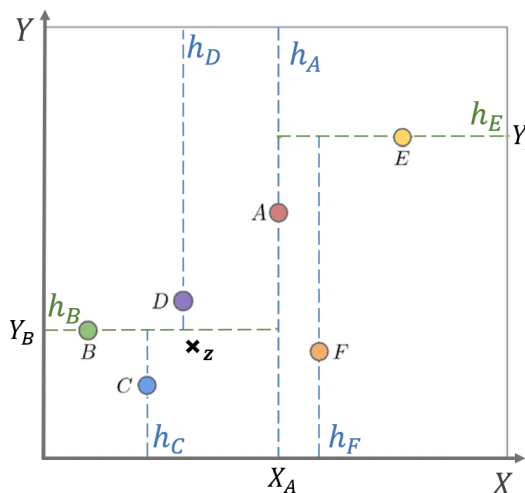


Figura 3-6: Procura pelo vizinho mais próximo de z .

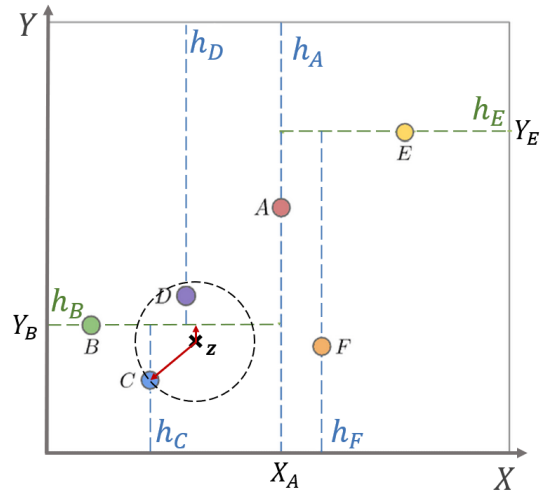


Figura 3-7: Checagem de interseção com outros hiperplanos de separação

Primeiramente, após a primeira descida da raiz até um nó folha, armazenamos o valor da distância do ponto z até o ponto C, ou seja, $D(C, z)$, que é o ponto mais próximo no momento. Depois, subimos um nó na árvore, indo para o nó B, como $D(B, z) > D(C, z)$, mantemos C como vizinho mais próximo. Como $D(C, z) > D(z, h_B)$ (Figura 3-7), é possível que haja pontos acima desse hiperplano que deveríamos considerar. Por isso, é necessário descer para a

subárvore da direita, onde está o nó D. Como $D(C, \mathbf{z}) > D(D, \mathbf{z})$, então o novo vizinho mais próximo é D.

Para decidir se vale a pena continuar percorrendo a árvore e subir para o nó raiz, verificamos a distância de $\mathbf{z}$ para h_A . Como $(\mathbf{z}, h_A) > D(\mathbf{z}, D)$, é impossível que haja pontos à direita de h_A que estejam mais próximos de $\mathbf{z}$ do que D. Então, encerramos o algoritmo e o vizinho mais próximo de $\mathbf{z}$ é o ponto D. Com isso, vemos que foi possível encontrar o vizinho mais próximo sem necessariamente ter que calcular a distância de $\mathbf{z}$ para todos os pontos do conjunto de dados.

É interessante notar que, em algumas implementações desse algoritmo, é possível estabelecer uma profundidade máxima que a árvore pode chegar ou o número máximo de pontos que se pode ter em uma folha. Nesse caso, ao chegar ao nó folha, o algoritmo precisa comparar a distância do ponto $\mathbf{z}$ para todos os pontos abrangidos por essa folha e encontrar o vizinho mais próximo dentro dessa sub-região. Essa forma alternativa de executar o K-D *tree* é motivada por casos em que a árvore pode se tornar muito grande, o que pode ocupar muito espaço na memória se o algoritmo tiver que considerar um ponto por folha [11].

Ao analisarmos a complexidade desse algoritmo, considerando um caso genérico em que temos K vizinhos, notamos que, para construir a árvore, temos uma complexidade $O(d \times N \times \log N)$ na média [12], sendo N o número total de pontos (dependendo do algoritmo utilizado para calcular a mediana, a complexidade pode aumentar). Por outro lado, dado que a árvore já foi construída, a complexidade para encontrar o vizinho mais próximo de um ponto $\mathbf{z}$ é $O(K \times \log N)$ em média. Com isso, vemos que a implementação por K-D *tree*, na média, tem uma complexidade menor do que por força bruta.

Comparado com o método de força bruta, a procura de vizinhos mais próximos pela K-D *tree* é mais rápido, durante a inferência, na maior parte dos casos – a complexidade é de $O(K \times d \times N)$ para a força bruta e $O(K \times \log N)$ para a K-D *tree*.

Outro ponto interessante a ser observado é que, para espaços de dimensão muito elevada, o algoritmo K-D *tree* perde eficiência e

3.1.3 *Ball Tree*

O *ball tree* [13] é outro algoritmo de árvore que pode ser utilizado na tarefa de encontrar o vizinho mais próximo. Ele é uma alternativa ao K-D *tree* quando o número de dimensões do problema é muito grande. Os dois métodos se baseiam no mesmo princípio de segmentar o espaço dos dados e separá-los em subgrupos, representados por cada nó da árvore, porém, em vez de criar partições com fronteiras ortogonais aos eixos, o *ball tree* divide o espaço criando regiões circulares.

Para entender a construção dessa árvore, considere o conjunto de dados mostrado na Figura 3-8. Primeiramente, calculamos a média $\mathbf{m}_0$ dos pontos do conjunto de dados e encontramos o ponto mais distante dessa média, que está a uma distância D_0 de $\mathbf{m}_0$. Em seguida, traçamos um círculo com centro em $\mathbf{m}_0$ e raio D_0 (Figura 3-9).

Após isso, selecionamos aleatoriamente um ponto $\mathbf{x}_0$ do conjunto de dados e encontramos o ponto $\mathbf{x}_1$ mais distante dele. Após isso, encontramos o ponto $\mathbf{x}_2$ mais distante de $\mathbf{x}_1$ e traçamos um segmento de reta auxiliar entre eles (Figura 3-10). Em seguida, realizamos a projeção de todos os pontos sobre esse segmento auxiliar e calculamos a mediana dessas projeções (Figura 3-11). Separamos os dados em dois grupos: dados cujas projeções estão

antes da mediana (grupo G_a) e depois dela (grupo G_d). Para cada um desses grupos, calculamos a média dos pontos (Figura 3-12). Em cada grupo, encontramos o ponto mais distante da média e traçamos um círculo centrado na média e com raio igual à distância entre a média e o ponto mais distante (Figura 3-13). Para cada grupo, repetimos todos os procedimentos realizados na bola maior (escolha do ponto aleatório, traçar segmento de reta entre esse ponto até ponto mais distante, projeção nesse segmento etc). Repetimos esses procedimento, gerando novos círculos menores até que cada um contenha apenas um ponto.

A árvore resultante é tal que cada nó representa um desses círculos, guardando a localização do centro e seu raio. Assim, o nó raiz representa o primeiro círculo, que engloba todos os pontos, os dois nós filhos representam os dois círculos gerados a partir do primeiro e assim por diante até chegar a uma folha, que representa um círculo contendo um único ponto.

Construída a árvore, ela pode ser utilizada para encontrar o vizinho mais próximo de um novo ponto z . O processo é análogo ao processo de procurar o vizinho mais próximo da *K-D tree*. Começamos na raiz e, a cada nó, precisamos decidir se descemos para o nó filho da direita ou da esquerda. Se o centro do círculo do filho esquerdo for mais próximo do que o do direito, seguimos para o filho esquerdo. Caso contrário, seguimos para o filho direito. Ao chegar a um nó folha, temos um candidato a vizinho mais próximo, que será o ponto representado por essa folha. No entanto, assim como acontece na *K-D tree*, não é possível ter certeza de que esse ponto da folha é o vizinho mais próximo, pois podem existir pontos mais próximos fora do círculo atual que não foram considerados. Para isso ser possível, é necessário que a distância de z para a borda de qualquer outro círculo (de qualquer nó e de qualquer profundidade) seja menor do que a distância de z para o ponto mais próximo no nó folha. Matematicamente, há a possibilidade de existir um vizinho mais próximo em um outro círculo C_2 se $D(z, x_1) > D(z, c_2) - r_2$, em que c_2 e r_2 são o centro e o raio de C_2 , respectivamente, e x_1 o vizinho mais próximo até o momento. $D(z, c_2) - r_2$ representa a distância de z até a margem de C_2 .

Com relação à complexidade de tempo, a construção da árvore tem uma complexidade $O(d \times N \times \log N)$ e o processo de encontrar os K vizinhos mais próximos, dado que a árvore já está construída, tem complexidade $O(K \times \log N)$ [14]. Apesar de apresentar a mesma complexidade de tempo que o algoritmo *K-D tree*, para espaços de maior dimensão, o *ball tree* tende a ser mais rápido que o *K-D tree*, devido a um problema relacionado à maldição da dimensionalidade, conforme será explicado adiante [15].

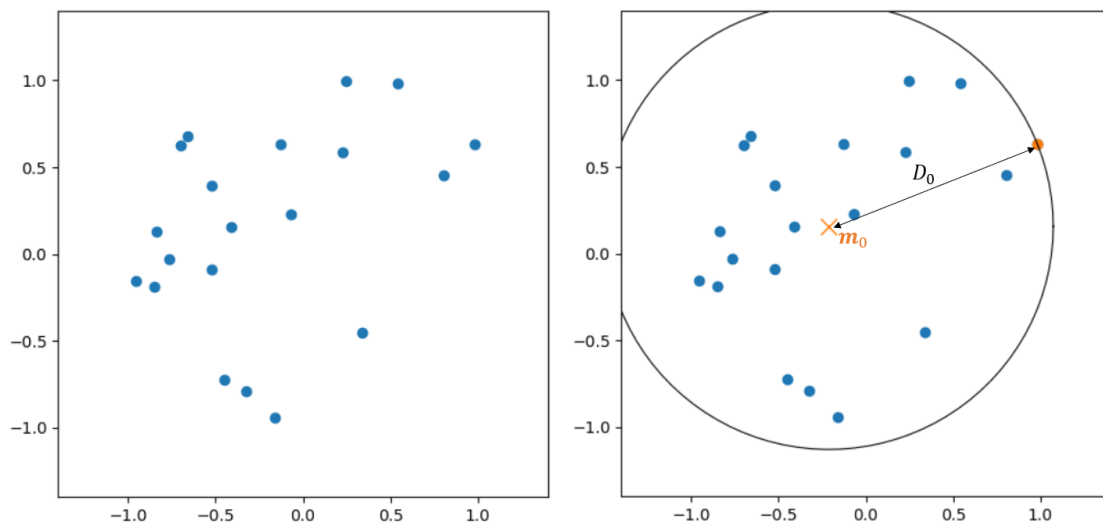


Figura 3-8: Conjunto de dados de exemplo para construção da ball tree.

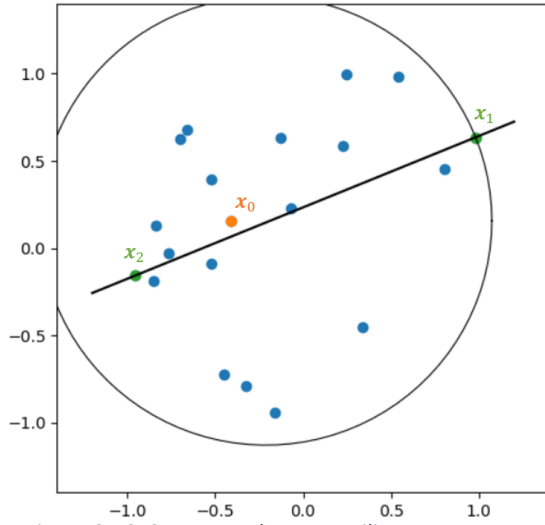


Figura 3-10: Segmento de reta auxiliar entre pontos mais distantes.

Figura 3-9: Primeiro círculo da árvore, representando o nó raiz.

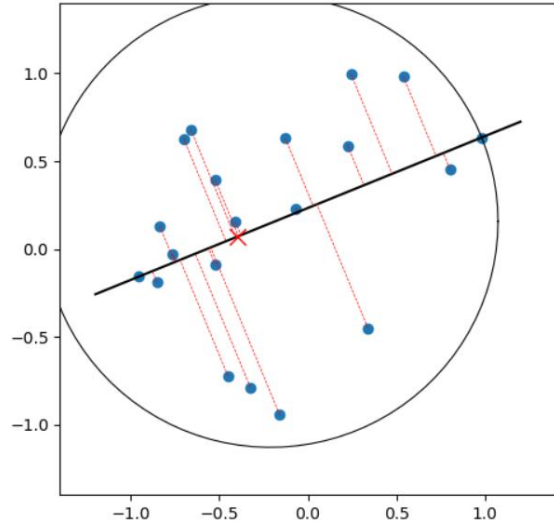


Figura 3-11: Mediana das projeções representada por um X vermelho.

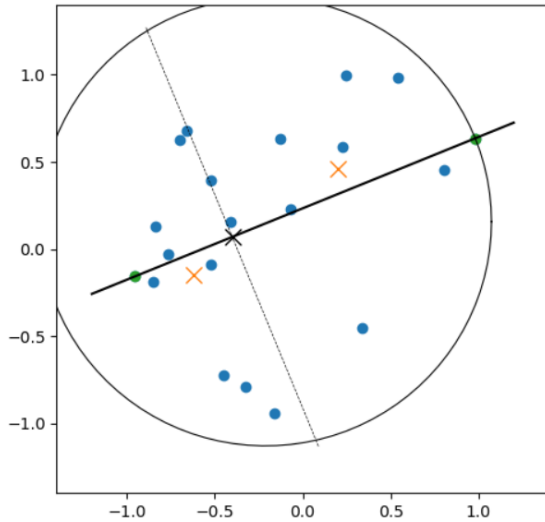


Figura 3-12: Médias dos grupos G_a e G_d representadas em amarelo.

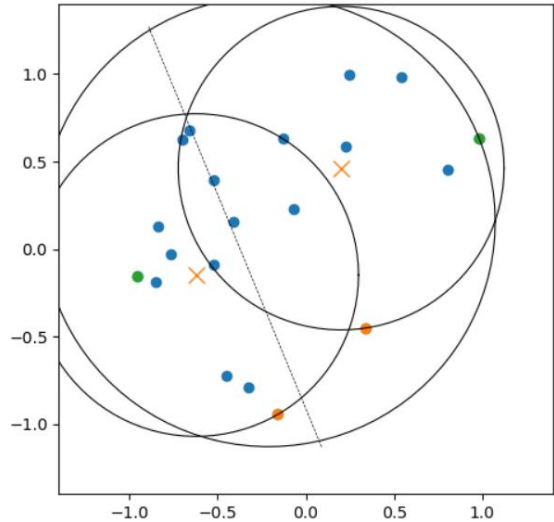


Figura 3-13: Círculos centrados na média, passando pelo ponto mais distante de cada grupo.

3.1.4 Variações do KNN

Uma variação ao método tradicional do KNN consiste em ponderar a importância dos vizinhos considerados de acordo com suas respectivas distâncias para um determinado ponto de teste $\mathbf{z}$. Normalmente, nesse caso, são atribuídos pesos proporcionais ao inverso da distância de cada ponto até $\mathbf{z}$. No caso da classificação, para encontrar a saída do modelo, podemos realizar a média ponderada dos K pontos mais próximos de $\mathbf{z}$, de forma que o peso associado a cada vizinho $\mathbf{x}_k$ é dado por $\frac{1/D(\mathbf{z}, \mathbf{x}_k)}{\sum_{i=1}^K 1/D(\mathbf{z}, \mathbf{x}_i)}$. No caso da classificação, podemos fazer de maneira

análoga, mas para cada classe, consideramos um score $\sum_{\mathbf{x} \in C_j} \frac{1/D(\mathbf{z}, \mathbf{x})}{\sum_{i=1}^K 1/D(\mathbf{z}, \mathbf{x}_i)}$ para cada classe C_i , fazendo com que a predição seja $\arg \max_{C_j} \sum_{\mathbf{x} \in C_j} \frac{1/D(\mathbf{z}, \mathbf{x})}{\sum_{i=1}^K 1/D(\mathbf{z}, \mathbf{x}_i)}$. Dessa forma, é possível fazer com que a contribuição de pontos mais próximos no resultado da predição seja maior do que a de pontos mais distantes.

Outro método com filosofia similar ao do KNN é o *Radius Nearest Neighbors* [6], que em vez de encontrar um número fixo de vizinhos de um determinado ponto de teste $\mathbf{z}$, encontra todos os vizinhos dentro de uma hipersfera centrada em $\mathbf{z}$ e de raio fixo. Quando o problema conta com dados de densidade muito desuniforme, é preferível utilizar esse método do que o tradicional KNN. Isso porque nesse tipo de problema, nas áreas mais densas do conjunto de dados, os K vizinhos mais próximos de $\mathbf{z}$ tendem a estar a uma distância menor e constante dele, mas em regiões mais esparsas desse mesmo conjunto de dados, os K vizinhos mais próximos podem incluir pontos muito distantes de $\mathbf{z}$, que são pouco semelhantes a ele. Ao fixar um valor de raio em volta de $\mathbf{z}$, garantimos que só haverá vizinhos até uma determinada distância.

3.2 MALDIÇÃO DA DIMENSIONALIDADE

Os algoritmos de KNN são baseados na suposição de que pontos similares possuem *labels* similares, o que faz com que eles sejam especialmente afetados pela maldição da dimensionalidade. Isso acontece porque, para um mesmo número N de observações sorteadas aleatória e uniformemente, a distância entre essas amostras tende a aumentar à medida que o número de dimensões cresce.

Para entender como isso afeta os algoritmos de KNN, imaginemos que os dados estão restritos a um espaço representado por um hipercubo unitário, isto é, $[0,1]^d$ e que estamos considerando os $K = 10$ vizinhos mais próximos de um certo ponto de teste. Consideremos também que ℓ é o tamanho da aresta do menor hipercubo que engloba todos os K vizinhos mais próximos do ponto de teste (Figura 3-14). A aresta desse hipercubo pode ser escrita como $\ell \approx \left(\frac{K}{N}\right)^{1/d}$ [16]. Utilizando essa equação e, lembrando que $N \geq K$, notamos que, à medida que aumentamos d , a aresta ℓ também aumenta (Tabela 3-1) e, consequentemente, maior é a proporção do espaço necessária para o hipercubo de aresta ℓ englobar os K vizinhos.

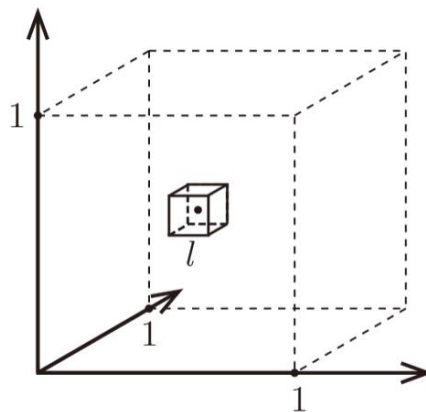


Figura 3-14: Hipercubo de aresta 1 representando o espaço $[0,1]^d$ e o hipercubo de aresta ℓ .

d	ℓ
2	0,1
3	0,215
10	0,63
100	0,955

1000	0,9954
------	--------

Tabela 3-1: Variação de ℓ à medida que aumentamos a dimensão d do espaço.

Outro gráfico que ilustra esse fenômeno é o mostrado na Figura 3-15. Com isso vemos que, por mais que o espaço seja restrito a um hipercubo unitário, as distâncias dentro dele crescem à medida que acrescentamos uma nova dimensão. É importante lembrar que a maior distância dentro de um hipercubo unitário é dada por $\sqrt{d}$.

Por isso, como para valores muito elevados de d há uma tendência de não haver pontos muito próximos, a hipótese de que pontos próximos possuem a mesma *label* perde o sentido. No entanto, mesmo que o espaço tenha dimensão muito elevada, caso os dados estejam concentrados em um subespaço de menor dimensão, ainda pode ser possível aplicar algoritmos de KNN. Um exemplo que ilustra essa situação é mostrado na Figura 3-16.

Esse problema da esparsidade dos dados em dimensões maiores poderia ser resolvido ao aumentar o número N de amostras. Para saber a quantidade de amostras ideal, poderíamos usar como critério o tamanho do menor hipercubo que engloba os K vizinhos e estabelecer $\ell = 0,1$. Com isso, como $\ell \approx \left(\frac{K}{N}\right)^{1/d}$, poderíamos escrever $N = \frac{K}{\ell^d} = 10^d K$. Logo, vemos que o número de amostras necessário cresce exponencialmente, o que inviabiliza a coleta de novas amostras para valores elevados de d .

Esse problema relacionado à maldição da dimensionalidade não é exclusivo de um único algoritmo de KNN. As implementações por força bruta, K-D *tree* e *ball tree* estão sujeitas a ele. No entanto, a K-D *tree* ainda enfrenta um problema adicional relacionado à perda de eficiência no que se refere ao aumento da dimensão do espaço.

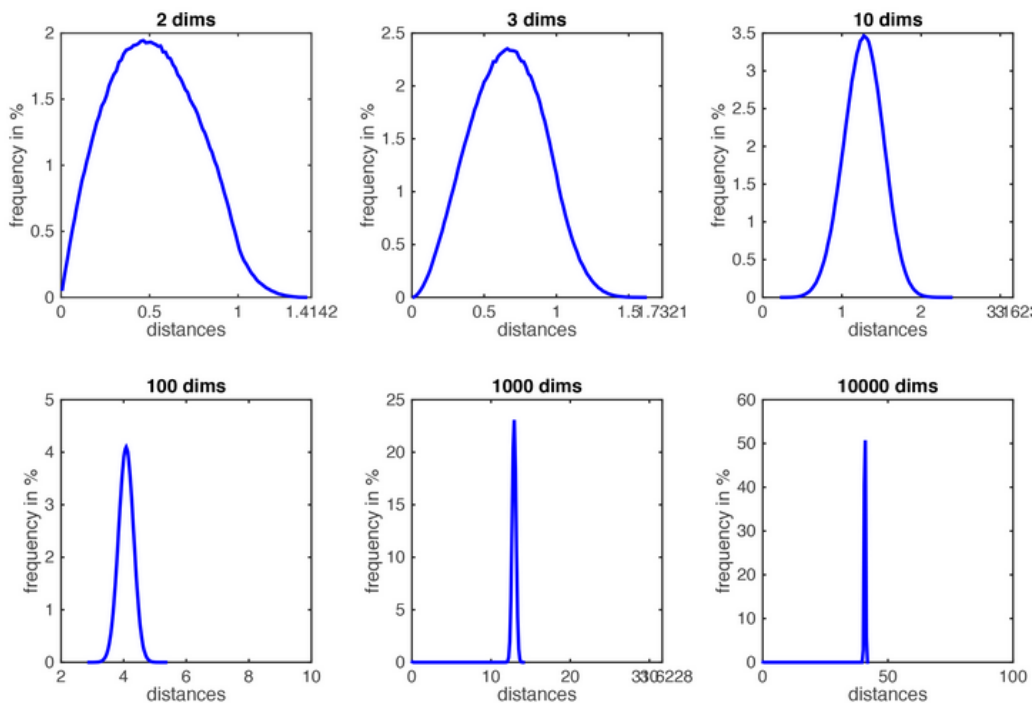


Figura 3-15: Histogramas das distâncias entre pares de pontos para espaços de diferentes dimensões.

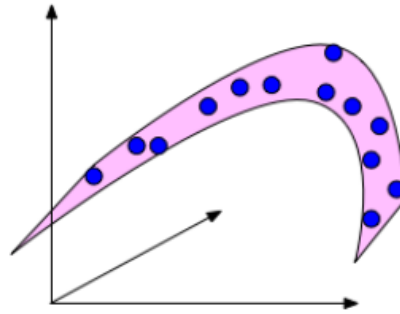


Figura 3-16: Subespaço de dimensão 2 em um espaço de 3 dimensões.

Como visto anteriormente, o algoritmo *K-D tree* se baseia em criar hiperplanos de separação ortogonais aos eixos para segmentar o espaço em partições. À medida que o algoritmo é executado, é necessário conferir se a distância entre o ponto mais próximo dentro de uma partição D_{nn} é maior do que a distância do ponto teste até um dos hiperplanos D_h . Sendo maior, é necessário fazer mais cálculos de distância, procurando por pontos mais próximos em outras partições. Considerando ainda a situação em que o espaço é um hipercubo unitário, conforme aumentamos d , a maior distância possível dentro desse hipercubo ($\sqrt{d}$) aumenta, porém, as distâncias ao longo dos eixos serão sempre limitadas ao intervalo $[0, 1]$, já que as arestas do hipercubo são unitárias. Com isso, à medida que d aumenta, as distâncias entre os pontos e os hiperplanos se mantêm, mas as distâncias entre pares de ponto aumentam (Figura 3-17), o que tende a aumentar o número de casos em que $D_h < D_{nn}$. Isso aumenta o número de cálculos de distância necessários, o que torna o algoritmo *K-D tree* mais lento.

O algoritmo *ball tree*, por outro lado, não sofre com esse problema, pois as partições criadas são circulares e não são dependentes dos eixos, apenas das distâncias entre os pontos. Por isso, em espaços com número muito elevado de dimensões, o algoritmo *ball tree* apresenta uma melhor performance comparado com o *K-D tree*.

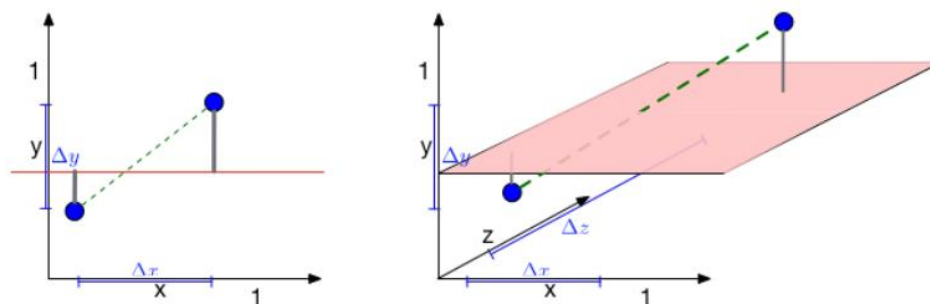


Figura 3-17: Efeito do aumento de dimensão nas distâncias de um espaço.

3.3 MÉTRICAS DE DISTÂNCIA

Algoritmos de KNN são baseados nas distâncias entre pontos. Normalmente, quando falamos em distância, pensamos primeiramente na distância Euclidiana, que é a que faz mais parte do nosso dia a dia. No entanto, existem outras métricas que podem ser utilizadas para medir distância, que podem ser mais adequadas do que a Euclidiana dependendo da situação.

Uma métrica bastante conhecida é a **métrica de Minkowski**. Para um espaço de dimensão q , podemos escrevê-la como

$$\|x - y\|_p = \left(\sum_{i=1}^q |x_i - y_i|^p \right)^{1/p}.$$

Como podemos ver, ela possui uma equação geral, que varia conforme mudamos o valor de p . Para alguns valores específicos de p , essa equação pode receber um nome especial como mostrado abaixo:

Distância de Manhattan ($p = 1$)	$\ x - y\ _1 = \sum_{i=1}^q x_i - y_i $
Distância Euclidiana ($p = 2$)	$\ x - y\ _2 = \sqrt{\sum_{i=1}^q (x_i - y_i)^2}$

A Figura 3-18 compara as distâncias de Manhattan e Euclidiana.

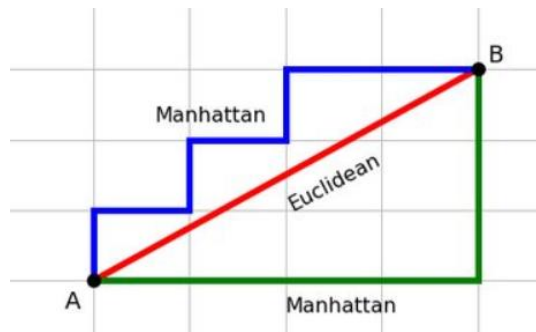


Figura 3-18: Comparação entre as distâncias de Manhattan e Euclidiana.

Como podemos ver, para o cálculo da distância de Manhattan, desconsideramos a possibilidade de realizar caminhos na diagonal entre A e B, apenas caminhos paralelos aos eixos. De uma maneira mais prática, considerando as arestas de cada quadrado/retângulo de *grid* como um passo, a distância de Manhattan é equivalente ao número mínimo de passos para ir de um ponto a outro. Ela encontra maior utilidade quando tratamos de problemas em que faz sentido considerarmos a possibilidade de deslocamento apenas em direções paralelas aos eixos. Um exemplo, é quando o problema envolve *features* representando dimensões espaciais de uma cidade, em que não se pode atravessar um quarteirão.

Outra métrica que pode ser utilizada é a **distância de Hamming**, que é dada por

$$D_H(x, y) = \sum_{i=1}^p (x_i + y_i \bmod 2),$$

com x e y sendo vetores binários. Em outras palavras, a distância de Hamming calcula o número total de valores diferentes entre duas sequências de bits. Por exemplo, se $x = (1 \ 0 \ 1)$ e $y = (1 \ 1 \ 1)$, então, $D_H = 2$, pois o primeiro e o último bits de x e y são diferentes. Pode fazer sentido utilizar essa métrica quando temos *features* binárias. Nesse caso, teríamos um espaço de *features* similar ao mostrado na Figura 3-19.

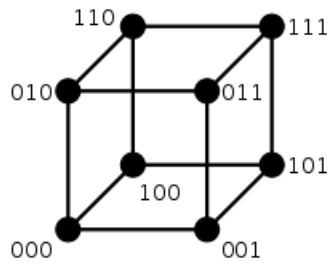


Figura 3-19: Espaço de Hamming mostrando coordenadas binárias.

Por fim, outra métrica conhecida é a **distância de Chebyshev**. Ela é dada por $\max_i |x_i - y_i|$, em que $\mathbf{x} = (x_1 \ x_2 \ \dots)$ e $\mathbf{y} = (y_1 \ y_2 \ \dots)$ são dois pontos. De maneira intuitiva, se considerarmos um jogo de xadrez podemos dizer que todas as casas que o rei pode acessar em um turno estão a uma distância de Chebyshev de uma unidade. Além disso, ainda nesse exemplo, a distância de Chebyshev do rei para qualquer casa do tabuleiro é igual ao número de turnos necessários para essa peça chegar à tal casa Figura 3-20.

	a	b	c	d	e	f	g	h	
8	5	4	3	2	2	2	2	2	8
7	5	4	3	2	1	1	1	2	7
6	5	4	3	2	1		1	2	6
5	5	4	3	2	1	1	1	2	5
4	5	4	3	2	2	2	2	2	4
3	5	4	3	3	3	3	3	3	3
2	5	4	4	4	4	4	4	4	2
1	5	5	5	5	5	5	5	5	1
	a	b	c	d	e	f	g	h	

Figura 3-20: Distância de Chebyshev de cada casa de um tabuleiro de xadrez para o rei.

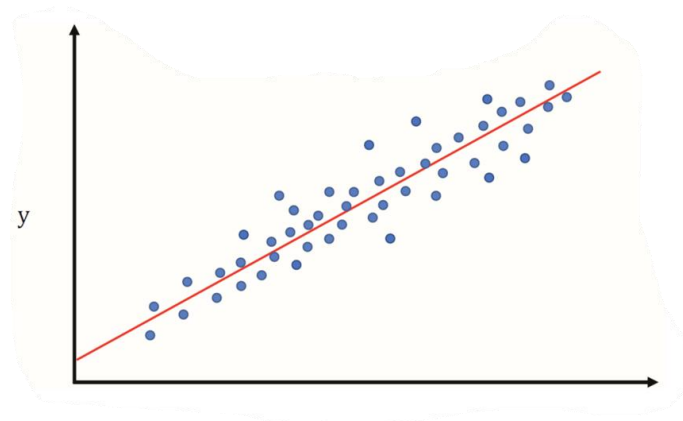
Capítulo 4: REGRESSÃO LINEAR

O problema de regressão linear consiste em criar um modelo que consiga relacionar variáveis $\{x_m\}_{m=1}^M$ com uma saída y de referência por meio de uma equação linear da seguinte forma:

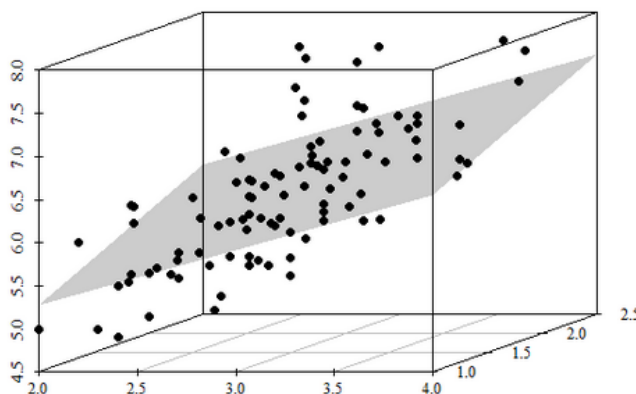
$$y = \theta_0 + \sum_{m=1}^M \theta_m x_m = \boldsymbol{\theta}^T \mathbf{x},$$

em que $\boldsymbol{\theta} = [\theta_M \ \theta_{M-1} \ \dots \ \theta_0]^T$, $\mathbf{x} = [x^T \ 1]^T$ e $\mathbf{x} = [x_M \ x_{M-1} \ \dots \ x_1]^T$. O termo θ_0 é conhecido como *bias* ou *intercept*.

No caso em que temos apenas uma variável x , o problema consiste em encontrar a reta descrita pela equação $y = \boldsymbol{\theta}^T \mathbf{x} = \theta_1 x_1 + \theta_0$, que aproximar se ajustar aos pontos (x, t) , em que t é o valor de referência da variável x . Assim, o objetivo é que, com essa equação, consigamos fazer com que y da melhor maneira possível os valores de referência t , como ilustra a figura abaixo.



Quando temos um problema em que há duas variáveis de entrada, ou seja, $\mathbf{x} \in \mathbb{R}^2$, o problema é parecido, mas, para representá-lo visualmente, precisamos de um espaço de três dimensões, como mostrado na figura abaixo.



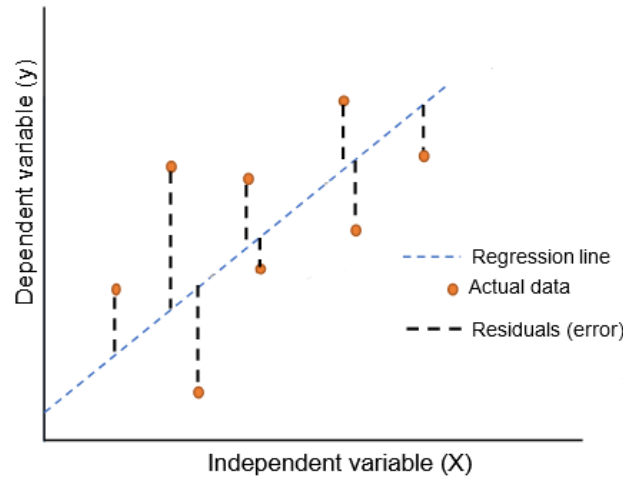
Com isso, podemos perceber que o problema da regressão linear consiste em encontrar o **hiperplano** que melhor descreve a relação entre as variáveis x_m e a saída y . O caso da reta e do plano mostrados acima são apenas casos particulares do hiperplano em que os vetores $\mathbf{x}$ pertencem ao $\mathbb{R}^1$ e ao $\mathbb{R}^2$, respectivamente.

Agora, como determinar os valores de θ que definem esse hiperplano? A seguir, são mostradas diferentes interpretações desse problema que levam a mesma solução.

4.1 DIFERENTES INTERPRETAÇÕES DA REGRESSÃO LINEAR

4.1.1 Ponto de Vista de Otimização

Uma maneira de resolver esse problema é encontrar o θ tal que a diferença entre os valores preditos y_n e o valores de referência t_n sejam os menores possíveis, levando em consideração todos os pares $(x_n, t_n)_{n=1}^N$ existentes no conjunto de dados. Essa diferença é comumente chamada de **erro** ou **resíduo**. No caso em que $x_n \in \mathbb{R}$ temos algo similar ao mostrado na imagem abaixo:



Assim, queremos que a soma de todos os resíduos em módulo seja a menor possível, ou seja, queremos algo como $\min_{\theta} \sum_{n=1}^N |y_n - t_n|$, em que $y_n = f(\theta)$. No entanto, se levarmos em consideração que para utilizar métodos tradicionais de minimização teríamos que derivar essa expressão, percebemos que talvez a **loss** $|y_n - t_n|$ não seja a melhor escolha.

Uma alternativa seria utilizar como **loss** o erro quadrático $(y_n - t_n)^2$ que é facilmente derivável e não altera o objetivo final, pois minimizar a soma dos erros quadráticos é equivalente a minimizar o módulo dos resíduos (o ponto de ótimo será o mesmo).

Com isso, adotamos como **função custo** a ser minimizada a função descrita abaixo:

$$J(\theta) = \frac{1}{2} \sum_{n=1}^N (y_n - t_n)^2.$$

O termo $\frac{1}{2}$, que ainda não tinha sido considerado, surge apenas para simplificar futuras derivações, em que o 2 advindo do expoente cancela com o termo $\frac{1}{2}$. Novamente, isso não altera o resultado final, pois

$$\theta = \arg \min_{\theta} \sum_{n=1}^N (y_n - t_n)^2 = \arg \min_{\theta} \frac{1}{2} \sum_{n=1}^N (y_n - t_n)^2.$$

Para encontrar valor de θ^* que minimiza a função custo, derivamos e igualamos a zero:

$$\frac{\partial J(\theta)}{\partial \theta} = \nabla J(\theta) = \sum_{n=1}^N \frac{\partial}{\partial \theta} \frac{\theta^T x_n}{\widehat{y}_n} \left(\frac{\theta^T x_n}{\widehat{y}_n} - t_n \right) = \sum_{n=1}^N x_n^T (\theta^T x_n - t_n) = \mathbf{0}$$

$$\sum_{n=1}^N x_n^T \theta^T x_n = \sum_{n=1}^N x_n^T t_n$$

Com isso, vemos que o θ ótimo é aquele que satisfaz a equação acima.

Na prática, uma forma de se resolver esse problema é utilizar o algoritmo do gradiente descendente, em que escolhemos um θ inicial aleatório para ser nosso ponto de partida e, a cada iteração, movemos esse ponto na direção de máxima decida, ou seja, a cada iteração, fazemos

$$\theta \leftarrow \theta - \alpha \nabla J(\theta),$$

em que α é a taxa de aprendizado e controla o tamanho do passo que será dado na direção de maior descida.

Apesar de podermos aplicar o método do gradiente descendente, há um outro método muito mais direto e que resolve o problema da regressão linear em uma iteração apenas, como veremos a seguir.

Observação: *Loss function x Cost function*

Loss function

Função utilizada para medir o erro entre o valor de referência t e o valor predito y . Nesta seção, foram vistos dois exemplos: o erro absoluto $|y - t|$ e o erro quadrático $(y - t)^2$.

Cost function

Função $J(\theta)$ utilizada para medir o custo total por se adotar $\theta = \theta'$ como parâmetro, considerando todas as N observações do conjunto de dados D . Muitas vezes, algumas pessoas usam de maneira intercambiável os termos *loss function* e *cost function*, apesar de serem diferentes.

Exemplo:

$$\overbrace{\frac{1}{2} \sum_{n=1}^N (y_n - t_n)^2}^{\text{cost}}$$

loss

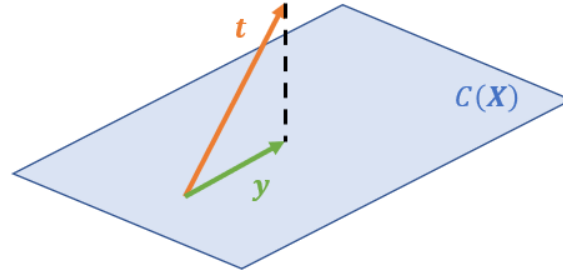
4.1.2 Ponto de Vista de Álgebra Linear

O nosso problema de relacionar variáveis x_m a uma saída y pode ser representado de maneira vetorial por meio da seguinte equação

$$y = X\theta,$$

em que $y = [y_1 \quad \cdots \quad y_N]^T$ e $X = \begin{bmatrix} x_{1M} & \cdots & x_{11} & 1 \\ x_{2M} & \cdots & x_{21} & 1 \\ \vdots & \ddots & \ddots & \vdots \\ x_{NM} & \cdots & x_{N1} & 1 \end{bmatrix}$.

Idealmente, desejaríamos que $X\theta = t$, em que $t = [t_1 \quad \cdots \quad t_N]^T$, porém, normalmente, essa equação não tem solução, pois $t \notin C(X)$, ou seja, t não pertence ao espaço coluna de X . Uma possível solução para esse problema é encontrar um vetor $y \in C(X)$ que está mais próximo de t do que qualquer outro vetor, ou seja, queremos $\theta^* = \arg \min_{\theta} \|X\theta - t\|$. Evidentemente, esse vetor $y = X\theta$ precisa ser necessariamente a projeção de t em $C(X)$, como ilustra a figura abaixo.



Com auxílio dessa figura, reparamos que $y - t = X\theta^* - t$ é ortogonal a $C(X)$, isto é, $X\theta^* - t \in C(X)^\perp$. Como consequência disso, temos que o produto interno de $X\theta^* - t$ por qualquer vetor da coluna de X é nulo, ou seja, $X^T(X\theta^* - t) = 0$. Logo, chegamos à equação

$$X^T X \theta^* = X^T t,$$

que possui uma solução, diferentemente de $X\theta = t$. Se $X^T X$ for invertível (que normalmente é, a não ser que $X\theta = t$ admita várias soluções, o que não é o caso no nosso problema), podemos escrever

$$\theta^* = (X^T X)^{-1} X^T t.$$

Assim, conseguimos resolver o problema da regressão linear com uma única equação em um único passo.

Bônus:

Se reparamos, o que desejamos é $\theta^* = \arg \min_{\theta} \|X\theta - t\|$, que é equivalente a

$$\theta^* = \arg \min_{\theta} \|X\theta - t\|^2 = \arg \min_{\theta} \sum_{n=1}^N (y_n - t_n)^2 = \arg \min_{\theta} \frac{1}{2} \sum_{n=1}^N (y_n - t_n)^2 = \arg \min_{\theta} J(\theta),$$

em que $J(\theta)$ é a função custo encontrada na Seção 4.1.1.

Além disso, essa equação poderia ter sido resolvida mais facilmente na forma matricial, fazendo

$$\nabla_{\theta} \|X\theta - t\|^2 = \nabla_{\theta} [(X\theta - t)^T (X\theta - t)] = (X\theta - t)^T X + (X\theta - t)^T X = 0$$

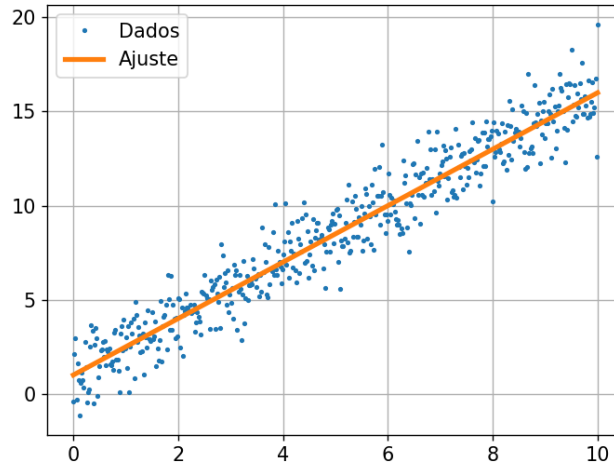
$$2(X\theta - t)^T X = 0$$

$$X^T (X\theta - t) = 0$$

$$\boldsymbol{\theta}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t}.$$

4.1.3 Ponto de Vista Probabilístico

Também é possível chegar ao mesmo resultado adotando uma abordagem probabilística. Para isso, assumimos que a equação que define a geração dos pontos é dada por $t = \boldsymbol{\theta}^T \mathbf{x} + \eta$, ou seja, é um hiperplano definido por $y = \boldsymbol{\theta}^T \mathbf{x}$ mais uma perturbação η . Na imagem abaixo, a reta em laranja representa o hiperplano $y = \boldsymbol{\theta}^T \mathbf{x}$ e os pontos em azuis são gerados por $t = \boldsymbol{\theta}^T \mathbf{x} + \eta$. O que gostaríamos de obter é a curva laranja, que é totalmente definida pelo parâmetro $\boldsymbol{\theta}$, porém, só conhecemos os dados $D = (\mathbf{x}_n, t)_{n=1}^N$, não conhecemos os valores de y_n correspondentes a $\mathbf{x}_n$.



Uma possível suposição é de que $\eta \sim \mathcal{N}(0, \sigma^2)$, o que faz com que $t \sim \mathcal{N}(y(\boldsymbol{\theta}), \sigma^2)$.

Assim, podemos aplicar a estimação de máxima verossimilhança (MLE) para determinar o parâmetro $\boldsymbol{\theta}$.

Para começar, sabemos que

$$p(t | y(\boldsymbol{\theta}), \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(x - y(\boldsymbol{\theta}, \mathbf{x}))^2}{2\sigma^2} \right].$$

Se considerarmos todos os pares $(\mathbf{x}_n, t_n)$ de D e considerarmos que eles são gerados de maneira independente, teremos

$$\mathcal{L}_N(\boldsymbol{\theta}) = p(\mathbf{t} | \mathbf{y}(\boldsymbol{\theta}), \sigma^2) = \prod_{n=1}^N p(t | y(\boldsymbol{\theta}, \mathbf{x}_n), \sigma^2)$$

Seguindo um procedimento similar ao visto na Seção 2.4, aplicamos a função log (base e) para encontrar a log verossimilhança:

$$\ell_N(\boldsymbol{\theta}) = \sum_{n=1}^N \log p(t | y(\boldsymbol{\theta}, \mathbf{x}_n), \sigma^2) = N \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \sum_{n=1}^N \frac{(x_n - y(\boldsymbol{\theta}, \mathbf{x}_n))^2}{2\sigma^2}$$

Com isso, passamos para a etapa de maximizar a log verossimilhança, o que é equivalente a minimizar o negativo da log verossimilhança. Normalmente, é preferível esse ponto de vista de “minimização” em vez de “maximização”, pois, eventualmente, podemos considerar o

negativo da log verossimilhança como uma função custo, que normalmente é uma função que visamos normalizar:

$$J(\boldsymbol{\theta}) = -\ell_N(\boldsymbol{\theta}).$$

Portanto, ao minimizarmos essa função custo obtemos

$$\begin{aligned}\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) &= \min_{\boldsymbol{\theta}} -N \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) + \sum_{n=1}^N \frac{(x_n - y(\boldsymbol{\theta}, \mathbf{x}_n))^2}{2\sigma^2} \\ &= \min_{\boldsymbol{\theta}} \frac{1}{2} \sum_{n=1}^N (x_n - \boldsymbol{\theta}^T \mathbf{x}_n)^2,\end{aligned}$$

que é justamente a conclusão que tínhamos chegado na Seção 4.1.1 e que é equivalente a minimizar a função custo

$$J(\boldsymbol{\theta}) = \frac{1}{2} \sum_{n=1}^N (x_n - \boldsymbol{\theta}^T \mathbf{x}_n)^2.$$

4.2 REGRESSÃO POLINOMIAL

É uma modalidade da regressão linear em que temos uma variável de entrada x e a substituímos por variáveis $\phi_m(x) = x^m$ que são potências de x . Em geral, funções $\phi_m(x)$ substituem as variáveis antigas são chamadas de **funções de base**. Além disso, de forma geral, podemos chamar essas novas variáveis formadas pelas funções base de **atributos** ou **features**. Dependendo da interpretação, também podemos chamar as variáveis x_m de *features*, mas geralmente, o que temos nesse caso, na verdade, são funções base identidade na forma $\phi_m(x) = x_m$.

Normalmente, escolhemos essa abordagem quando não conseguimos modelar os dados com uma equação na forma

$$y = \theta_0 + \boldsymbol{\theta}^T \mathbf{x},$$

e precisamos recorrer a uma modelagem não linear da relação entre y e x .

Assim, utilizamos um modelo na forma

$$y = \boldsymbol{\theta}^T \boldsymbol{\phi} = \sum_{m=0}^M \theta_m \phi_m = \sum_{m=0}^M \theta_m x^m,$$

em que M é o grau do polinômio e $\boldsymbol{\phi} = [x^M \quad x^{M-1} \quad \dots \quad 1]^T$.

Por mais que existam termos de ordem maior do que 1 e que a relação entre y e x seja não linear, ainda temos uma regressão linear, pois a relação entre y e os termos $\boldsymbol{\theta}$ é linear. Se considerarmos todos os dados, independentemente da relação que cada *feature* mantém entre elas, ainda temos uma equação linear $\mathbf{v}(\mathbf{w}) = \mathbf{A}\mathbf{w} + \mathbf{b}$, em que $\mathbf{A}$ corresponde à matriz

de dados $\boldsymbol{\Phi} = \begin{bmatrix} \phi_{1M} & \dots & \phi_{11} & 1 \\ \phi_{2M} & \dots & \phi_{21} & 1 \\ \vdots & \ddots & \ddots & \vdots \\ \phi_{NM} & \dots & \phi_{N1} & 1 \end{bmatrix}$, a variável independente $\mathbf{w}$ corresponde a $\boldsymbol{\theta}$ e $\mathbf{b} = \mathbf{0}$.

Por mais que essa seja a explicação oficial do porquê de essa regressão ainda ser linear, ainda acho mais lógico ela ser linear pelo fato de y ser linear com relação às *features* ϕ_m e podermos escrever uma equação linear $y = \theta^T \phi$. Fazendo uma comparação, nas seções anteriores, tínhamos um vetor de variáveis de entrada $\mathbf{x} = [x_M \ x_{M-1} \ \dots \ x_1]^T$ e uma equação similar: $y = \theta^T \mathbf{x}$. A princípio, não sabemos se as variáveis x_m são independentes entre si ou se elas têm algum tipo de relação entre elas. É possível que $x_2 = x_1^2$ (x_1 sendo a largura de um terreno quadrado e x^2 sendo a área desse terreno, por exemplo), que é o caso em que teríamos algo que lembra uma regressão polinomial e, mesmo assim, ainda temos uma regressão linear.

4.3 FORMA GERAL DA REGRESSÃO LINEAR

Percebemos ao longo deste capítulo que a regressão linear é um problema cujo objetivo é encontrar uma equação que relacione de maneira linear uma variável de saída y com variáveis $\phi_m(x)$ (que podem ser dependentes entre si ou não), por meio de uma equação na forma

$$y = \theta^T \phi,$$

sendo que o parâmetro ótimo é dado por

$$\theta^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t}.$$

O caso da regressão polinomial é apenas um caso particular, em que forçamos a dependência entre as variáveis ϕ_m . No início do capítulo tínhamos outro caso particular, só que não forçamos nenhuma relação entre as variáveis de entrada que nos foram dadas. Tínhamos $\phi_m = x_m$.

4.4 CONDIÇÕES DE GAUSS-MARKOV

- **Linearidade:** relação entre variáveis independentes e a variável independente é linear.
- **Exogeneidade:** valor esperado dos resíduos ϵ não está correlacionado com as variáveis independentes e é nulo, ou seja, $\mathbb{E}[\epsilon|\mathbf{X}] = 0$.
- **Homoscedasticidade:** matriz de covariância dos resíduos ϵ é igual a $\mathbb{E}[\epsilon\epsilon^T] = \sigma^2 I$. Isso implica em $\text{Var}[\epsilon|\mathbf{X}] = \sigma^2$ (a variância dos resíduos é constante e, consequentemente, independente das variáveis independentes) e $\text{Cov}(\epsilon_i, \epsilon_j) = 0 \ \forall i \neq j$ (inexistência de autocorrelação).
- **Não multicolinearidade perfeita:** as variáveis independentes devem ser linearmente independentes entre si. Pode existir correlação entre elas, mas ela não pode ser perfeita. Isso porque deve ser possível realizar a inversão de $\mathbf{X}^T \mathbf{X}$.

Respeitando essas condições, garantimos que o estimador dos parâmetros dos mínimos quadrados é BLUE (*Best Linear Unbiased Estimator*). Se os resíduos seguirem distribuição normal, então o estimador também é de máxima verossimilhança.

Capítulo 5: REGRESSÃO LOGÍSTICA

O modelo de regressão logística, apesar do nome, não é um modelo de regressão, mas sim, de classificação. Ele faz parte de um conjunto de **modelos discriminativos**, que associam a cada classe do problema uma **função discriminante** $\delta_k(\mathbf{x})$ [2, p. 101], de forma que, associamos o exemplo $\mathbf{x}$ à classe k se $\delta_k(\mathbf{x}) > \delta_i(\mathbf{x}) \forall k \neq i$. Mais especificamente, na regressão logística, essa função discriminante é a probabilidade a posteriori $P(\omega_i|\mathbf{x})$. Alguns exemplos de modelos discriminantes são: regressão logística, SVM, árvore de decisão.

Em oposição, existem os **modelos generativos**, que modelam a probabilidade a priori $P(\omega_i)$ e a verossimilhança $p(\mathbf{x}|\omega_i)$ para só depois encontrar a probabilidade a posteriori $P(\omega_i|\mathbf{x})$ por meio do teorema de Bayes.

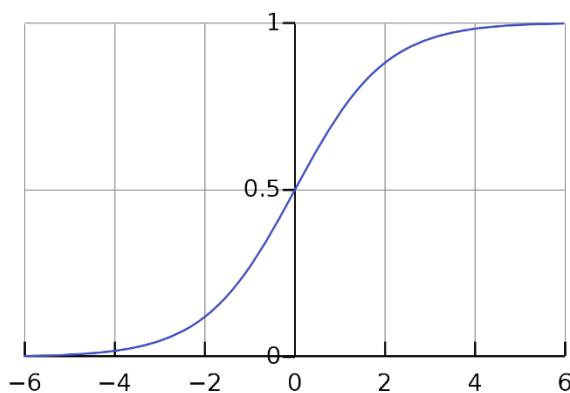
É interessante ressaltar que nos dois grupos de modelos, busca-se determinar $P(\omega_i|\mathbf{x})$ justamente porque, pela teoria de decisão, uma determinada observação $\mathbf{x}$ deverá ser classificada como classe ω_i se $P(\omega_i|\mathbf{x}) > P(\omega_j|\mathbf{x}) \forall i \neq j$, ou seja, atribuímos $\mathbf{x}$ à classe cuja probabilidade a posteriori for a maior dentre todas as classes.

No modelo da regressão logística, a probabilidade a posteriori é modelada por

$$P(\omega_i|\Phi) = \sigma(\theta^T \Phi),$$

em que $\Phi = [\phi_0 \ \phi_1 \ \dots \ \phi_M]^T$, $\phi_m = \phi_m(x_m)$ é uma função base de x e $\sigma(\cdot)$ é a função sigmoide logística, dada por

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$



Com isso, atribuímos a observação $\mathbf{x}$, correspondente às *features* Φ , à classe ω_i com maior $P(\omega_i|\Phi)$.

Para chegar a esse modelo, existe todo um desenvolvimento que será abordado a seguir.

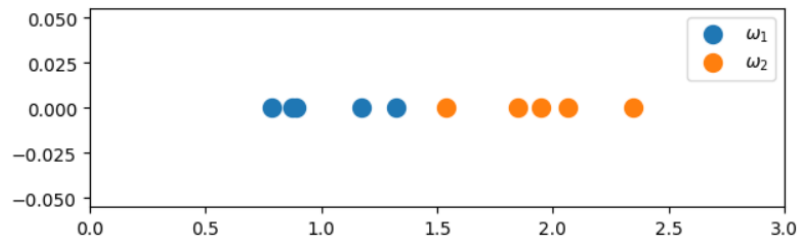
Antes de começarmos, é importante ressaltar que, como o problema é de classificação, o *target* t não assume mais valores decimais. Caso o problema tenha um número K de classes igual a 2, consideramos $t \in \{0,1\}$, em que cada valor de t indica uma classe diferente. Caso $K > 2$, adotamos uma codificação *one-hot* para t , em que temos um vetor de tamanho K cujas entradas são todas nulas, exceto na posição i , que vale 1, indicando que o exemplo em questão pertence à classe ω_i . Um ponto pertencente à classe ω_3 , por exemplo, teria um vetor de *target* da seguinte forma:

$$\mathbf{t} = [0 \ 0 \ 1]^T.$$

Passemos, agora, ao desenvolvimento que leva à criação do modelo da regressão logística.

5.1 MÍNIMOS QUADRADOS PARA CLASSIFICAÇÃO

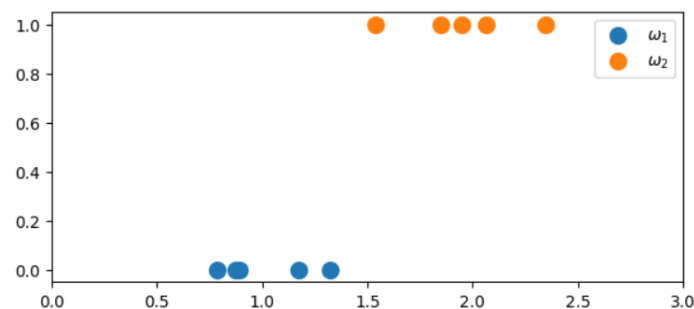
Consideremos um problema de classificação em que temos conjuntos de dados que pertencem a duas classes distintas, como ilustra a imagem abaixo:



Como criar um modelo matemático para, dados novos pontos, conseguir classificá-los em classe ω_1 ou ω_2 ? Se conhecêssemos bem as distribuições que geraram esses pontos, poderíamos propor um valor de x a partir do qual passaríamos classificar os pontos como classe ω_2 . Por exemplo, se soubéssemos que $X_1 \sim \mathcal{N}(1, 0,1)$ e $X_2 \sim \mathcal{N}(2, 0,1)$ são as variáveis aleatórias que geram essas duas classes de pontos, poderíamos propor uma fronteira de decisão passando em $x = 1,5$, na metade do caminho entre as médias das duas Gaussianas.

No entanto, em quase todos os casos, não temos essa informação e precisamos encontrar um modelo para classificar os pontos mesmo sem saber as funções de distribuição geradoras.

Uma possibilidade mais simples seria utilizar o método dos mínimos quadrados. Mas como fazer isso se temos pontos alinhados em uma única dimensão, com mesmo valor de y ? Podemos codificar a informação presente nas cores em número, por exemplo, fazendo $y = 0$ para a classe ω_1 (azul) e $y = 1$ para a classe ω_2 (laranja), o que nos daria a seguinte representação:



Com isso, fica um pouco menos obscura a ideia de utilizar mínimos quadrados para encontrar uma função $y(x) = \theta^T x$ que se ajuste a esses dados. O ideal seria que essa função $y(x)$ assumisse o valor zero quando x pertencesse a ω_1 e assumisse o valor 1 caso contrário. No entanto, só conseguiríamos encontrar uma reta $y(x) = \theta^T x$ desse tipo somente se todos os pontos azuis estivessem concentrados em um mesmo valor x_1 e todos os pontos laranjas estivessem concentrados em um mesmo valor de x_2 . Claramente isso não acontece e o que temos, na realidade, é um modelo em que y pode assumir qualquer valor real, como mostrado na Figura 5-1.

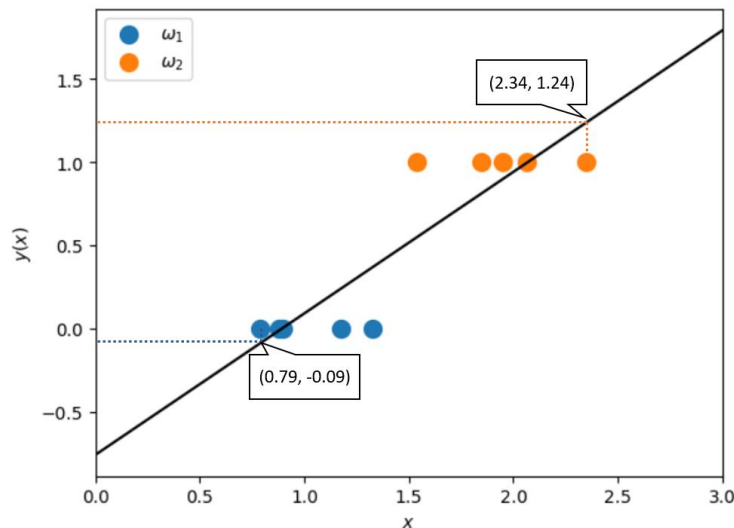


Figura 5-1: Ajuste de uma reta aos pontos das classes azul e laranja.

Uma solução para isso é adotarmos a regra de que, para $y < 0,5$, associamos x a ω_1 e associamos a ω_2 caso contrário. Com essa regra bastante intuitiva, separamos perfeitamente os pontos da imagem acima.

E no caso em que temos 3 classes, por exemplo, como mostra a Figura 5-2?

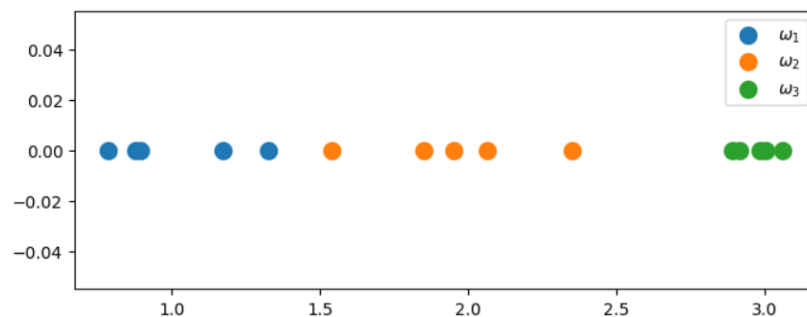


Figura 5-2: Problema multiclasse.

Nesse caso, existem diferentes abordagens. Em [3, p. 183], Bishop menciona algumas delas:

- Um contra um, em que se adota uma função discriminante na forma $y_{ij}(\mathbf{x}) = \boldsymbol{\theta}^T \mathbf{x}$ para cada par de classes (ω_i, ω_j) . Se $y_{ij}(\mathbf{x}) > 0$, classificamos como ω_i e, caso contrário, classificamos como ω_j . Essa abordagem gera uma região de indecisão, formada por pontos que são atribuídos a mais de uma classe, ou seja, quando para o mesmo ponto $\mathbf{x}$, existe mais de um $y_{ij}(\mathbf{x}) > 0$ (Figura 5-3).
Observar que as fronteiras de decisão da figura são curvas de nível do hiperplano $y_{ij}(\mathbf{x}) = \boldsymbol{\theta}^T \mathbf{x}$, e são encontradas fazendo $y_{ij}(\mathbf{x}) = 0$, que é o limiar entre uma classe e outra. Assim, em um exemplo em que tivéssemos $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, $y_{12}(x_1, x_2) = x_2 \theta_2 + x_1 \theta_1 + \theta_0$ é a nossa função discriminante, que é um plano no espaço tridimensional (pode ser reescrito como $x_2 \theta_2 + x_1 \theta_1 + \theta_0 + y_{12} = 0$ para facilitar a visualização) e $x_2 \theta_2 + x_1 \theta_1 + \theta_0 = 0$ é uma reta (que pode ser reescrita como $x_2 = x_1 \frac{\theta_1}{\theta_2} + \frac{\theta_0}{\theta_2}$ para facilitar a visualização), contida no mesmo plano onde se encontram os pontos $\mathbf{x}$.
- Um contra o resto, em que temos funções discriminante $y_i(\mathbf{x}) = \boldsymbol{\theta}^T \mathbf{x}$ e, se $y_i(\mathbf{x}) > 0$, dizemos que $\mathbf{x}$ pertence à classe ω_i . Caso contrário, dizemos que $\mathbf{x}$ não é da classe ω_i .

Essa abordagem gera uma região de indecisão, formada por pontos que não foram classificados como nenhuma classe, ou seja, pontos em que $y_k(\mathbf{x}) < 0 \forall k$ (Figura 5-4). Aqui, as fronteiras de decisão da imagem também são curvas de nível da função discriminante.

- Abordagem em que temos uma função $y_i(\mathbf{x}) = \theta^T \mathbf{x}$ para cada classe e atribuímos $\mathbf{x}$ à classe ω_k que tiver maior valor de $y_k(\mathbf{x})$. Nesse caso, não são criadas regiões de indecisão, pois, excetuando as interseções entre os hiperplanos discriminantes, sempre haverá $y_k(\mathbf{x}) > y_j(\mathbf{x}) \forall j \neq k$.

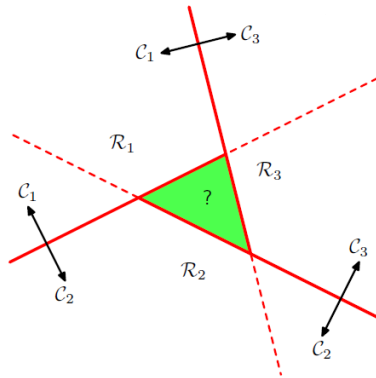


Figura 5-3: Um contra um.

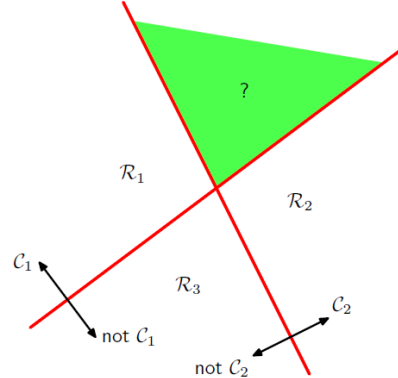


Figura 5-4: Um contra o resto.

Dessa forma, a abordagem que parece mais razoável para nosso problema é a terceira, já que não são criadas regiões de indecisão. Nela, poderíamos utilizar o método dos mínimos quadrados para encontrar as funções discriminantes $y(\mathbf{x})$. No entanto, as fronteiras de decisão são mais difíceis de encontrar, quando comparamos com os dois outros métodos, principalmente no caso multiclasse, pois elas são definidas pelas regiões em que cada curva tem seu valor de $y(\mathbf{x})$ maior do que o de todas as outras classes.

Uma propriedade interessante dessa abordagem é que $\sum_{k=1}^K y_k(\mathbf{x}) = 1$ [3, p. 185], o que é muito similar ao que temos em medidas de probabilidade. Apesar disso, $y_k(\mathbf{x})$ não pode ser considerado uma medida de probabilidade, pois $y_k(\mathbf{x})$ não está restrito apenas ao intervalo $[0, 1]$.

Portanto, aplicando a terceira abordagem ao problema multiclasse ilustrado na Figura 5-2, obtemos uma reta para cada classe, como ilustrado na Figura 5-5. Observamos que, para modelar cada $y_k(\mathbf{x})$, precisamos a cada instante considerar a classe ω_k com $t = 1$ e todas as outras com $t = 0$.

Percebemos que, para $K > 2$, o modelo dos mínimos quadrados para classificação começa a apresentar alguns problemas. A classe ω_2 , por exemplo, apresenta uma curva que atribui valores baixos e mais ou menos próximos de $y_k(\mathbf{x})$ para todos os pontos, mesmo que eles pertençam a outras classes. Isso faz com que os pontos de ω_2 sejam classificados incorretamente como pertencentes às outras classes, cujos $y_k(\mathbf{x})$ são maiores que $y_2(\mathbf{x})$.

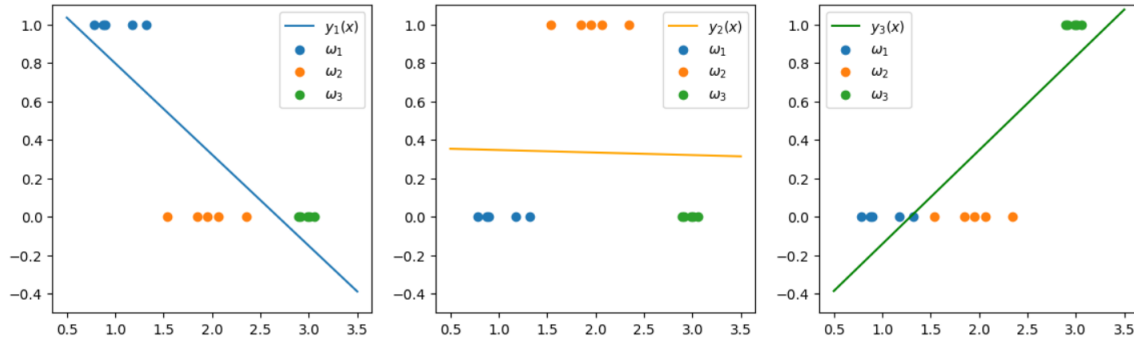


Figura 5-5: Abordagem em que se tem uma regressão linear para cada classe (terceira abordagem).

Por fim, podemos resumir esse problema em uma única equação vetorial:

$$\mathbf{y}(\mathbf{x}) = \mathbf{T}^T (\mathbf{X}^\dagger)^T \mathbf{x},$$

$$\text{em que } \mathbf{y}(\mathbf{x}) = [y_1(\mathbf{x}) \quad \dots \quad y_K(\mathbf{x})]^T, \mathbf{T} = \begin{bmatrix} - & \mathbf{t}_1^T & - \\ & \vdots & \\ - & \mathbf{t}_N^T & - \end{bmatrix}_{N \times K}, \mathbf{X}^\dagger = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \text{ e}$$

$$\mathbf{X} = \begin{bmatrix} - & \mathbf{x}_1^T & - \\ & \vdots & \\ - & \mathbf{x}_N^T & - \end{bmatrix}_{N \times K}. \text{ Lembrando que } \mathbf{t}_k \text{ é o vetor de } target \text{ na codificação } one-hot \text{ e } \mathbf{x} \text{ é o}$$

vetor $\mathbf{x}$ aumentado, ou seja, $\mathbf{x} = [1 \quad \mathbf{x}^T]^T$.

5.2 MODELANDO A PROBABILIDADE A PRIORI

Na seção anterior, vimos que o modelo dos mínimos quadrados pode ser utilizado em problema de classificação. Por outro lado, vimos também que ele não é muito adequado por alguns fatores: não transmitir uma noção de probabilidade (valor não estão restritos a intervalos $[0, 1]$), dificuldades para classificação com $K > 2$.

Por isso, nesta seção, estudamos uma forma de escrever uma equação para probabilidade a posteriori que possa auxiliar na construção de um modelo para classificação.

5.2.1 Caso $K = 2$

Podemos utilizar o Teorema de Bayes para modelar a probabilidade a posteriori da seguinte maneira:

$$P(\omega_1|\mathbf{x}) = \frac{p(\mathbf{x}|\omega_1)P(\omega_1)}{p(\mathbf{x}|\omega_1)P(\omega_1) + p(\mathbf{x}|\omega_2)P(\omega_2)}$$

Com algumas manipulações, podemos reescrever essa equação da seguinte maneira:

$$P(\omega_1|\mathbf{x}) = \frac{1}{1 + \frac{p(\mathbf{x}|\omega_2)P(\omega_2)}{p(\mathbf{x}|\omega_1)P(\omega_1)}} = \frac{1}{1 + e^{-\ln \frac{p(\mathbf{x}|\omega_1)P(\omega_1)}{p(\mathbf{x}|\omega_2)P(\omega_2)}}} = \frac{1}{1 + e^{-\ln \frac{P(\omega_1|\mathbf{x})}{P(\omega_2|\mathbf{x})}}},$$

que é equivalente a

$$P(\omega_1|\mathbf{x}) = \sigma\left(\ln \frac{P(\omega_1|\mathbf{x})}{P(\omega_2|\mathbf{x})}\right) = \sigma\left(\ln \frac{P(\omega_1|\mathbf{x})}{1 - P(\omega_1|\mathbf{x})}\right).$$

Por curiosidade, percebemos que $P(\omega_1|\mathbf{x})$ pode ser escrita com uma composição de funções dela mesma, ou seja,

$$P(\omega_1|\mathbf{x}) = \sigma \circ f(P(\omega_1|\mathbf{x})).$$

Ora, se $x = f \circ g(x)$, então $f = g^{-1}$. Assim, notamos que a inversa da função sigmoide é dada por

$$f(x) = \ln\left(\frac{x}{1-x}\right),$$

que é conhecida como **função logit**.

5.2.2 Caso $K > 2$

Para mais de duas classes, podemos escrever a probabilidade a posteriori da classe ω_i como

$$P(\omega_i|\mathbf{x}) = \frac{p(\mathbf{x}|\omega_i)P(\omega_i)}{\sum_{k=1}^K p(\mathbf{x}|\omega_k)P(\omega_k)} = \sigma\left(\left[\ln(p(\mathbf{x}|\omega_k)P(\omega_k))\right]_{k=1}^K\right)_i,$$

em que $\left[\ln(p(\mathbf{x}|\omega_k)P(\omega_k))\right]_{k=1}^K = \left[\ln(p(\mathbf{x}|\omega_1)P(\omega_1)) \quad \cdots \quad \ln(p(\mathbf{x}|\omega_K)P(\omega_K))\right]^T$.

A função

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}},$$

com $\mathbf{z} = [z_1 \quad \cdots \quad z_K]^T$, é conhecida como **softmax**. Observar que é necessário dar como entrada tanto $\mathbf{z}$ quanto o índice i . Ela é o caso geral da função sigmoide.

5.2.3 Aplicação para distribuições conhecidas

Se formos aplicar as equações desenvolvidas acima em casos em que conhecemos a *likelihood*, como no caso da distribuição normal, exponencial, discretas binárias, encontramos probabilidades a priori com o seguinte formato

$$K = 2:$$

$$P(\omega_k|\mathbf{x}) = \sigma(\boldsymbol{\theta}_k^T \mathbf{x})$$

$$K > 2:$$

$$P(\omega_i|\mathbf{x}) = \sigma\left(\left[\boldsymbol{\theta}_k^T \mathbf{x}\right]_{k=1}^K\right)_i$$

A expressão para $\boldsymbol{\theta}_k$ varia de acordo com a distribuição [3, pp. 198-203].

Com isso, percebemos que, para várias distribuições bastante comuns, temos uma probabilidade a posteriori que é dada pela *softmax* (o que, por generalização, inclui a

sigmoide) de uma função linear. Isso faz com que as fronteiras de decisão entre essas classes sejam hiperplanos.

Tomemos por exemplo o caso em que sabemos que os dados pertencem às classes ω_1 e ω_2 e que são gerados por distribuições gaussianas com parâmetros específicos para cada classe. Isso quer dizer que conhecemos as *likelihoods* $p(\mathbf{x}|\omega_1)$ e $p(\mathbf{x}|\omega_2)$, com $\mathbf{x} \in \mathbb{R}^2$.

Para o caso em que as duas distribuições apresentam a mesma matriz de covariância Σ temos probabilidade a posteriori como mostrado acima: $P(\omega_k|\mathbf{x}) = \sigma(\boldsymbol{\theta}_k^T \mathbf{x})$. Para encontrar a fronteira de decisão, segundo a teoria de decisão, precisamos determinar:

$$P(\omega_1|\mathbf{x}) = P(\omega_2|\mathbf{x})$$

$$\sigma(\boldsymbol{\theta}_1^T \mathbf{x}) = \sigma(\boldsymbol{\theta}_2^T \mathbf{x})$$

$$\boldsymbol{\theta}_1^T \mathbf{x} = \boldsymbol{\theta}_2^T \mathbf{x}$$

$$(\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2)^T \mathbf{x} = 0$$

$$(\theta_{1_2} - \theta_{2_2})x_2 + (\theta_{1_1} - \theta_{2_1})x_1 + (\theta_{1_0} - \theta_{2_0}) = 0$$

que é a equação de uma reta no $\mathbb{R}^2$ e de um plano no $\mathbb{R}^3$.

Um erro que poderia ser cometido é achar que o plano que correspondente a essa reta do $\mathbb{R}^2$ no $\mathbb{R}^3$ é a função $f(\mathbf{x}) = (\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2)^T \mathbf{x}$. No entanto, não podemos afirmar isso, pois a equação dessa reta não surgiu a partir de uma curva de nível de uma função $z = f(\mathbf{x})$, fazendo $f(\mathbf{x}) = 0$, mas sim, da equação $P(\omega_1|\mathbf{x}) = P(\omega_2|\mathbf{x})$.

Então, não é possível fazer o caminho inverso $(\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2)^T \mathbf{x} = 0 \rightarrow z = f(\mathbf{x}) = (\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2)^T \mathbf{x}$, pois não conhecemos o comportamento da variável z em outros valores diferentes de $z = 0$.

Por que não poderia ser $z = f(\mathbf{x}) = \frac{1}{3}(\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2)^T \mathbf{x}$, ou escrito de maneira equivalente, $(\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2)^T \mathbf{x} - 3z = 0$, que é igual a equação mostrada acima em $z = 0$?

No $\mathbb{R}^3$, temos sempre 3 variáveis: x_1 , x_2 e z . Nossa equação tem a forma $ax_1 + bx_2 + d = 0$. O z também está representado nessa equação, mas de maneira implícita: $ax_1 + bx_2 + 0z + d = 0$. Isso nos indica que, x_1 e x_2 só possuem dependência entre eles e o valor de z não influencia nos valores que essas duas variáveis podem assumir. Então, temos um plano paralelo ao eixo z , pois, independentemente do valor de z escolhido, a relação entre x_1 e x_2 é sempre a mesma: $x_1 = \frac{d}{a} - \frac{b}{a}x_2$.

Para encerrar essa discussão, isso é só uma questão de ponto de vista. Se estamos trabalhando com representações 2D desse problema, representaremos a fronteira de decisão como uma reta no $\mathbb{R}^2$. Se estamos trabalhando com representações em 3D, representaremos nossa fronteira de decisão como um plano no $\mathbb{R}^3$.

A Figura 5-7 e a Figura 5-7 ilustram as curvas desse exemplo de diferentes pontos de vista. Na curva da probabilidade a posteriori, pode parecer que ela representa uma função de densidade de probabilidade (contínua) de $\mathbf{x}$. Para que isso fosse verdade, precisaríamos que

$$\int_{\mathbf{x} \in \mathbb{R}^2} P(\omega_1|\mathbf{x}) d\mathbf{x} = 1,$$

o que é um absurdo, já que $\lim_{(x_1, x_2) \rightarrow (\infty, \infty)} \sigma(\theta_1^T \mathbf{x}) = 1$ e, por isso, $\int_{\mathbf{x} \in \mathbb{R}^2} P(\omega_1 | \mathbf{x}) d\mathbf{x} = \int_{\mathbf{x} \in \mathbb{R}^2} \sigma(\theta_1^T \mathbf{x}) d\mathbf{x}$ não converge. Lembrando que, como $\theta_1^T \mathbf{x}$ é linear, $\sigma(\theta_1^T \mathbf{x})$ tem a mesma forma da sigmoide, porém possivelmente deslocada e distorcida ao longo do eixo x , no caso unidimensional, ou ao longo do plano formado por pelas componentes de $\mathbf{x}$, no caso multidimensional.

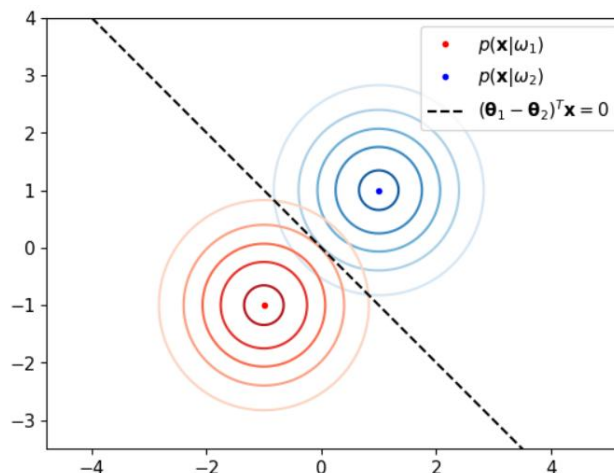


Figura 5-6: Curvas de nível das likelihoods.

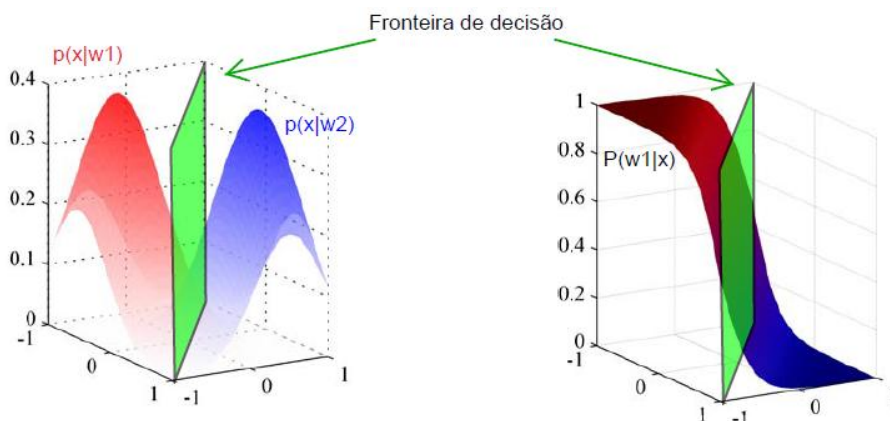


Figura 5-7: Representação das likelihoods (à esquerda) e da função de probabilidade a posteriori (à direita). Nessas figuras, está representada a fronteira de decisão $(\theta_1 - \theta_2)^T \mathbf{x} = 0$, só que aqui, representada no $\mathbb{R}^3$.

Em outras palavras, a aba vermelha do gráfico à direita na Figura 5-7 se estende até o infinito. Isso faz sentido, já que, quanto mais além do plano da fronteira de decisão maior é a chance do ponto $\mathbf{x}$ pertencer à classe ω_1 . Logo, esse gráfico não representa o gráfico de uma função de distribuição de probabilidade.

No entanto, isso não quer dizer $P(\omega | \mathbf{x})$ não seja uma p.d.f. Ela é, mas não em função de $\mathbf{x}$ e sim de ω . Mais especificamente, ela é uma função de massa, pois ela é discreta. Repare que $\sum_{k=1}^K P(\omega_k | \mathbf{x}) = 1$.

Uma última observação, é que podemos perceber perfeitamente o formato 3D da sigmoide na figura. Vemos também que a região em que a probabilidade de ser a classe ω_1 é 0,5 (chances iguais de ser ω_1 e ω_2) é justamente uma reta ao longo da qual passa o plano representando a fronteira de decisão.

Para finalizar este exemplo, é importante lembrar que a expressão de θ depende dos parâmetros das duas Gaussianas, ou seja, Σ , μ_1 e μ_2 . Se por acaso só soubermos que as duas classes geram dados segundo uma distribuição Gaussiana, mas não conhecermos o valor desses parâmetros (o que é comum na realidade), podemos estimá-los por meio da estimação de máxima verossimilhança utilizando os dados x . Com isso, determinamos os valores numéricos de θ caso os parâmetros não sejam conhecidos.

5.3 CONSTRUÇÃO DA REGRESSÃO LOGÍSTICA

Na seção anterior, vimos que, para alguns casos de *likelihood*, podemos escrever a probabilidade a posteriori como a função sigmoide/softmax de uma função linear. Assim, conhecendo a *likelihood* dos dados gerados por cada classe, conseguimos escrever uma equação para $P(\omega_k|x)$ com auxílio do teorema de Bayes.

No entanto, na grande maioria das vezes, os dados não seguem exatamente as distribuições especiais da seção anterior e, normalmente, não conhecemos a distribuição deles. Com isso, seríamos obrigados a estimar suas distribuições de probabilidade, o que é uma abordagem completamente válida e é a marca dos modelos generativos, mas também pode gerar resultados ruins se a estimação não for adequada.

Uma alternativa seria, em vez de desenvolvermos um método em que precisemos estimar a verossimilhança para só assim calcularmos a posteriori, desenvolver um modelo discriminativo. É daí que surge a ideia da regressão logística.

O modelo da regressão logística consiste em generalizar a probabilidade a posteriori por meio da expressão

$$P(\omega_i|\Phi) = \sigma \left([\theta_k^T \Phi]_{k=1}^K \right)_i,$$

o que parece bastante natural, visto que várias distribuições têm expressões analíticas exatas para a posteriori dessa mesma forma.

Assim, mesmo que não conheçamos a distribuição dos dados, aplicamos a mesma expressão que vimos na seção anterior. A diferença é que antes tínhamos uma expressão exata para $P(\omega_k|x)$ e, agora, temos algo estimado. Se antes tínhamos expressões analíticas exatas para θ calculadas a partir da expressão da verossimilhança conhecida/estimada e com auxílio do teorema de Bayes, agora, calcularemos a expressão da probabilidade a posteriori com auxílio da estimação de máxima verossimilhança sem conhecer a *likelihood*.

Observar que, aqui, substituímos o vetor x de entradas por um vetor genérico Φ de *features*, formado por funções base não lineares $\phi_m(x_m)$. Essas funções são importantes para separar dados que não podem ser separados linearmente, com fronteiras de decisão lineares. A Figura 5-8 ilustra um exemplo em que foram utilizadas funções base gaussianas com centros marcados nas cruzes verdes. Com isso percebemos que com a escolha de funções base corretas, conseguimos separar os dados (no espaço original das entradas) usando uma fronteira de decisão circular. Observar também que no espaço das *features*, a fronteira de decisão é linear, da forma como desenvolvemos ao longo desta seção.

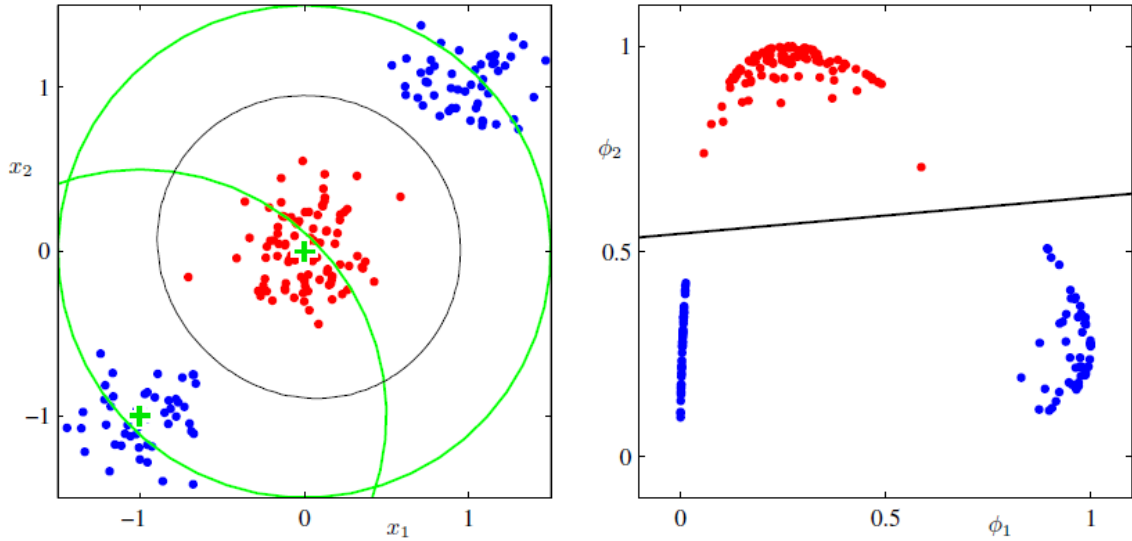


Figura 5-8: À esquerda, dados no espaço original das entradas x . À direita, a transformação desses dados para o espaço das features com funções base ϕ .

Dada a expressão da regressão logística, precisamos encontrar o parâmetro θ . Para isso, nos basearemos no princípio da estimação de máxima verossimilhança.

Precisaremos ficar atentos para não nos confundirmos. No início desta seção, foi sugerido usar a MLE para calcular os parâmetros a distribuição dos dados quando supúnhamos que eles eram gerados por uma distribuição específica conhecida (gaussiana). Nesse caso, tentávamos maximizar $p(x|\Sigma_i, \mu_i)$ para cada classe ω_i , pois, no final, por meio do teorema de Bayes, conseguíamos escrever a expressão de θ em função desses parâmetros (Σ_i, μ_i) específicos de cada classe.

Aqui, estimaremos o parâmetro θ diretamente por meio da MLE, sem tentar escrevê-lo em função de outros parâmetros, maximizando $P(t|\theta)$. É importante lembrar que $P(t|\phi, \theta) \equiv P(t|\theta)$, mas omitimos o ϕ para simplificar a notação. Se considerarmos que as amostras são independentes, teremos a seguinte expressão para a verossimilhança:

$$P(t|\theta) = \prod_{n=1}^N P(t_n|\theta). \quad (5.1)$$

No caso genérico em que temos K classes, podemos escrever

$$P(t_n|\theta_i) = P(\omega_i|\phi_n, \theta_i)$$

$$P(t_n|\theta_i) = \sum_{k=1}^K \mathbb{1}_{\{\phi_n \in \omega_k\}} P(\omega_k|\phi_n, \theta_k) \quad (5.2)$$

Reparar que esse somatório tem apenas uma parcela não nula, que corresponde à probabilidade de ϕ_n pertencer à classe a qual ele realmente pertence.

Deixamos explícito nas expressões acima que ϕ_n é dado porque senão não ficaria claro que essa probabilidade está associada apenas a esse exemplo de índice n . Além disso, reparar que manter o ϕ_n como uma informação dada é apenas uma notação. Poderíamos ter uma variável aleatória que codificasse isso. Por exemplo, poderíamos ter uma variável aleatória Ω_n que representa o evento “classe a qual pertence o exemplo ϕ_n ” e poderíamos expressar as

probabilidades acima como $P(\Omega_n = \omega_i; \theta)$. Nesse último caso, também mostramos que não precisamos deixar o θ como uma informação dada, mais sim, como um parâmetro, o que é marcado explicitamente pelo ponto e vírgula.

Por mais que possa parecer confuso, a expressão que estabelecemos inicialmente para a posteriori, $P(\omega_i|\Phi) = \sigma(\theta^T \Phi_n)$ (no caso binário), é equivalente a $P(\omega_i|\Phi_n, \theta)$. Isso porque na expressão $P(\omega_i|\Phi)$, temos o termo θ explicitamente. Logo, ele tem que ser um parâmetro dado, por mais que não apareça explicitamente na notação $P(\omega_i|\Phi)$. Lembrando que, dizer que θ é dado, não significa dizer que conhecemos os valores de suas componentes, pois ele pode ser “dado” e ainda assim ser uma variável desconhecida. O problema está na nomenclatura “dado”, que nesse caso, é o equivalente a dizer que estamos calculando a probabilidade do exemplo Φ_n pertencer à classe ω_i condicionado a θ . Assim, fica mais claro que não se trata de probabilidades em que se tem valores conhecidos dados, mas sim, de probabilidades condicionadas a uma variável. Portanto, se temos $\sigma(\theta^T \Phi_n)$, obviamente essa expressão está condicionada a θ , por mais que o vetor de parâmetros não se encontre explícito na expressão da probabilidade.

Depois dessa discussão, vemos que podemos reescrever a eq. (5.2) como

$$P(t_n|\theta_i) = \sum_{k=1}^K \mathbb{1}_{\{\Phi_n \in \omega_k\}} \sigma \left([\theta_j^T \Phi]_{j=1}^K \right)_k \quad (5.3)$$

Agora, voltemos ao problema de maximizar a verossimilhança da eq. (5.1). Por simplicidade, como de costume, transformamos essa maximização na minimização do negativo da log verossimilhança:

$$-\log P(t|\theta_i) = -\sum_{n=1}^N \log P(t_n|\theta_i) \quad (5.4)$$

Substituindo a eq. (5.3) na eq. (5.4), a função custo, dada pelo negativo da log verossimilhança, pode ser escrita como:

$$J(\Theta) = -\sum_{n=1}^N \log \left(\sum_{k=1}^K \mathbb{1}_{\{\Phi_n \in \omega_k\}} \sigma \left([\theta_j^T \Phi]_{j=1}^K \right)_k \right)$$

$$J(\Theta) = -\sum_{n=1}^N \sum_{k=1}^K \mathbb{1}_{\{\Phi_n \in \omega_k\}} \log \sigma \left([\theta_j^T \Phi]_{j=1}^K \right)_k \quad (5.5)$$

Observar que só foi possível “passar” o log para dentro do somatório mais interno, pois esse somatório possui apenas uma parcela não nula. Além disso, observar que a função custo é função de uma matriz Θ de vetores θ_j parâmetros. Isso porque, para cada classe k , temos um vetor de parâmetros θ_k correspondente.

Por fim, para encontrar os parâmetros θ aplicamos o método *Iterative Reweighted Least Squares (IRLS)*, que é baseado no método do Newton e consiste em atualizar o valor de θ a cada iteração da seguinte maneira:

$$\theta_{i+1} = \theta_i - H^{-1} \nabla J(\theta),$$

em que $\mathbf{H} = \nabla \nabla J(\boldsymbol{\theta})$ [3, p. 207].

5.4 DISCUSSÃO SOBRE A FUNÇÃO CUSTO

A função custo da eq. **Erro! Fonte de referência não encontrada.** tem uma forma especial. Isso fica evidente quando reescrevemos sua equação. Para mostrar isso, realizaremos as seguintes substituições:

- $p_k = \mathbb{1}_{\{x \in \omega_k\}}$;
- $q_k = \sigma(\boldsymbol{\theta}^T \boldsymbol{\phi}_n)$.

Com isso, a função custo pode ser escrita como

$$J(\boldsymbol{\theta}) = - \sum_{n=1}^N \sum_{k=1}^K p_k \log q_k. \quad (5.6)$$

Se interpretarmos p_k como sendo a p.d.f. correspondente a $\boldsymbol{\phi}_n \in \omega_k$ sabendo a qual classe $\boldsymbol{\phi}_n$ realmente pertence, ou seja, a p.m.f. ótima desse fenômeno, e q_k como sendo a p.d.f. de outra distribuição não ótima que tenta modelar o fenômeno $\boldsymbol{\phi}_n \in \omega_k$ podemos reescrever a eq. (5.6) como

$$J(\boldsymbol{\theta}) = - \sum_{n=1}^N H_c(\boldsymbol{\phi}_n), \quad (5.7)$$

em que

$$H_c = \sum_{k=1}^K p_k \log q_k$$

representa a **entropia cruzada** (para mais detalhes, checar seção B.1.6 do apêndice). Portanto, vemos que a função *loss* utilizada no modelo da regressão logística é a entropia cruzada.

Tentando ver esse problema de uma forma mais intuitiva, o que estamos fazendo é minimizar a entropia cruzada, que quantifica a dissimilaridade entre as distribuições p_k e q_k (lembrando que minimizar a entropia cruzada é equivalente a minimizar a divergência KL, conforme Anexo B:). Então, como $q_k(\mathbf{x}) = \mathbb{1}_{\{x \in \omega_k\}}$, queremos fazer com que a probabilidade p_k se aproxime o máximo possível de $\mathbb{1}_{\{x \in \omega_k\}}$.

Se considerarmos o caso específico de um problema de classificação binária, em que $t_n \in \{0,1\}$, a função custo é dada por

$$J(\boldsymbol{\theta}_1) = - \sum_{n=1}^N t_n \log \sigma(\boldsymbol{\theta}_1^T \boldsymbol{\phi}_n) + (1 - t_n) \log (1 - \sigma(\boldsymbol{\theta}_1^T \boldsymbol{\phi}_n)).$$

A expressão dentro desse somatório é uma função *loss* que recebe o nome de **entropia cruzada binária**.

5.5 CONCLUSÕES

Ao final de todo o desenvolvimento para se chegar ao modelo da regressão logística, percebemos que existe uma relação entre ele e o modelo dos mínimos quadrados para classificação.

No método dos mínimos quadrados para classificação, um dos pontos negativos era o fato de não termos funções $y_i(\mathbf{x})$ que possam ser interpretadas como probabilidades, pois seus valores não se limitavam ao intervalo $[0,1]$. Na regressão logística, é como se pegássemos o modelo dos mínimos quadrados que tínhamos desenvolvido e o empacotássemos na função logística sigmoide, obrigando, assim, que os valores fiquem limitados ao intervalo $[0,1]$.

Obviamente, não é possível encontrar $\boldsymbol{\theta}$ com a equação dos mínimos quadrados $\boldsymbol{\theta}^* = \mathbf{T}^T(\mathbf{X}^\dagger)^T$ e substituir esse valor em $\sigma(\boldsymbol{\theta}^T \boldsymbol{\phi})$, pois, só foi possível chegar a essa equação porque não havia inicialmente a não linearidade da função sigmoide.

Por fim, é interessante reparar que, mesmo para os casos em que a distribuição dos dados pertence a uma daquelas distribuições especiais descritas na Seção 5.2.3, ainda é preferível usar o modelo da regressão logística em vez de usar as expressões analíticas para cada um desses casos particulares. Isso porque, na regressão logística, precisamos estimar consideravelmente menos parâmetros comparado com os métodos que se baseiam no teorema de Bayes. Por exemplo, se, num problema de classificação binária, tivermos vetores de *features* de dimensão M e nossos dados seguirem uma distribuição Gaussiana, então, na regressão logística, teremos apenas M parâmetros para estimar. Já utilizando o modelo baseado no teorema de Bayes teríamos que estimar $2M$ parâmetros para as médias $\boldsymbol{\mu}$ e $M(M + 1)/2$ parâmetros para a matriz de covariância $\boldsymbol{\Sigma}$ [3, p. 205] [7, p. 319].

Capítulo 6: SUPPORT VECTOR MACHINE

6.1 HIPERPLANO DE SEPARAÇÃO ÓTIMO

Um conceito importante para entender o modelo SVM é o **hiperplano de separação ótimo**, que separa duas classes linearmente separáveis e maximiza a distância entre os pontos mais próximos de classes diferentes [2]. Esse conceito é ilustrado na Figura 6-1. A distância do hiperplano de separação para o ponto mais próximo de cada classe é chamada de **margem** e é representado pela letra M .

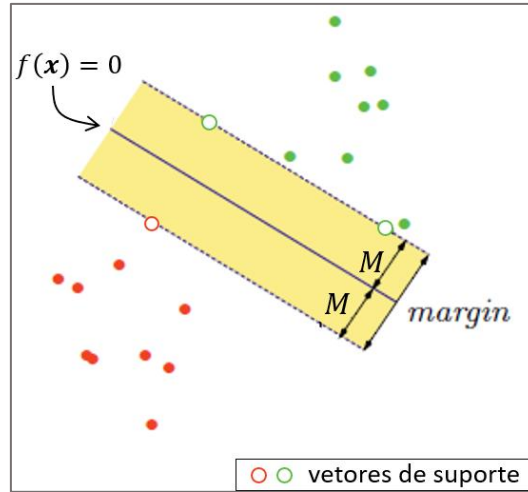


Figura 6-1: Exemplo de classificador de vetor suporte.

Um hiperplano de separação pode ser definido como $L = \{x : f(x) = w_0 + \mathbf{w}^T x = 0\}$, como mostra a Figura 6-2. Ele consegue separar duas classes de dados: os dados que estão acima do hiperplano, para os quais $f(x) > 0$, e os dados que estão abaixo do hiperplano, para os quais $f(x) < 0$. Assim, podemos usar o sinal de $f(x)$ para classificar os dados.

Uma métrica importante de se conhecer é a distância de um ponto x qualquer para esse hiperplano. Primeiramente, façamos algumas considerações.

Se $x_1, x_2 \in L$, então $\mathbf{w}^T(x_1 - x_2) = 0$. Como $(x_1 - x_2) \parallel L$, concluímos que $\mathbf{w} \perp L$.

Seja $x_0 \in L$. A distância de um ponto x para L é dada por

$$r = \frac{\mathbf{w}^T}{\|\mathbf{w}\|} (x - x_0) = \frac{1}{\|\mathbf{w}\|} (\mathbf{w}^T x - \mathbf{w}^T x_0)$$

Como $w_0 + \mathbf{w}^T x_0 = 0 \Rightarrow -\mathbf{w}^T x_0 = w_0$, temos que

$$\begin{aligned} r &= \frac{1}{\|\mathbf{w}\|} (\mathbf{w}^T x + w_0) \\ r &= \frac{f(x)}{\|\mathbf{w}\|} \end{aligned} \tag{6.1}$$

Observar que r pode ser positivo ou negativo, dependendo do sinal de $f(x)$.

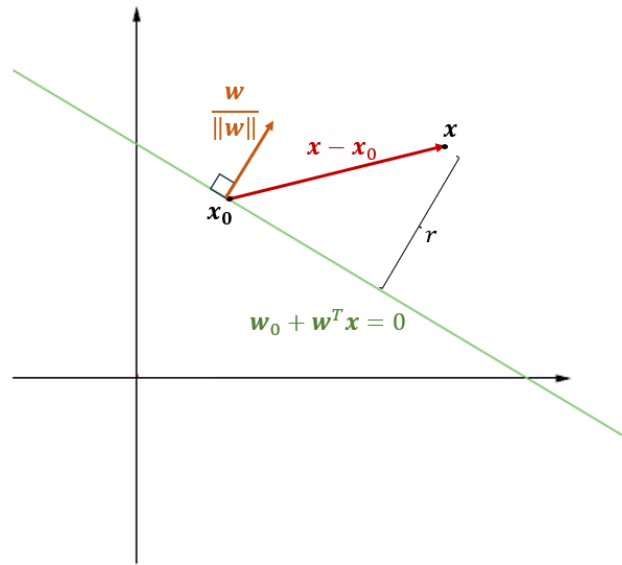


Figura 6-2: Distância de um hiperplano para um ponto.

Tendo uma expressão para a distância entre um hiperplano e um ponto qualquer, podemos passar ao problema do hiperplano de separação ótimo.

Consideremos o conjunto de dados (x_i, y_i) com $i = 1, \dots, N$ e $y_i \in \{-1, 1\}$. Consideremos também que é possível usar um hiperplano de separação $L = \{x : f(x) = 0\}$ para separar os dados, de forma que $f(x) > 0$ quando $y_i = 1$ e $f(x) < 0$ quando $y_i = -1$. O problema de encontrar o hiperplano de separação ótimo, como definido no início do capítulo, pode ser escrito matematicamente como uma maximização da margem M (para a qual ainda não temos uma expressão definida), sujeita a algumas restrições. De maneira simples, estabeleceremos como restrição a exigência de que todos os pontos devem estar fora da margem, ou seja, a distância de um ponto para o hiperplano L deve ser maior ou igual a M . No entanto, só estamos interessados nos pontos que foram corretamente classificados, para os quais $y_i f(x_i) > 0$ e cuja distância absoluta para L é dada por $|r| = y_i \frac{f(x_i)}{\|w\|} = \frac{y_i(w_0 + w^T x_i)}{\|w\|}$.

Matematicamente, escrevemos esse problema como

$$\begin{aligned} & \max_{w, w_0} \quad M \\ & \text{sujeito a} \quad \frac{y_i(w_0 + w^T x_i)}{\|w\|} \geq M, \quad i = 1, \dots, N \end{aligned}$$

Como $M > 0$, todos os pontos incorretamente classificados são automaticamente deixados de fora.

Observar que multiplicar w e w_0 (os únicos termos que temos a liberdade de variar nesse problema de otimização) por um fator α , ou seja, $w \leftarrow \alpha w$ e $w_0 \leftarrow \alpha w_0$, não altera em nada o problema acima:

$$\frac{y_i(\alpha w_0 + \alpha w^T x_i)}{\|\alpha w\|} = \frac{y_i(w_0 + w^T x_i)}{\|w\|} \geq M$$

Assim, se $w = w'$ e $w_0 = w'_0$ maximizam o problema acima, então $w = \alpha w'$ e $w_0 = \alpha w'_0$ também o maximizam. Temos essa liberdade para variar essas variáveis porque temos uma única restrição, mas duas variáveis.

Com isso, podemos escolher um α tal que $\alpha\|\mathbf{w}\| = \frac{1}{M}$. Portanto, fazemos $w_0 \leftarrow \alpha w'_0$ e $\mathbf{w} \leftarrow \alpha \mathbf{w}'$, com $\alpha = \frac{1}{M\|\mathbf{w}'\|}$ e encontramos:

$$y_i(\alpha w'_0 + \alpha \mathbf{w}'^T \mathbf{x}_i) \geq 1$$

Por simplicidade, podemos definir $w''_0 = \alpha w'_0$ e $\mathbf{w}'' = \alpha \mathbf{w}'$. Além disso, como $M = \frac{1}{\alpha\|\mathbf{w}'\|} = \frac{1}{\|\mathbf{w}''\|}$, o problema de otimização pode ser escrito como:

$$\begin{aligned} \max_{\mathbf{w}, w_0} \quad & \frac{1}{\|\mathbf{w}''\|} \\ \text{sujeito a} \quad & y_i(w''_0 + \mathbf{w}''^T \mathbf{x}_i) \geq 1, \quad i = 1, \dots, N \end{aligned}$$

Para manter a notação anterior, $w''_0 \leftarrow w_0$ e $\mathbf{w}'' \leftarrow \mathbf{w}$. Além disso, maximizar $\frac{1}{\|\mathbf{w}\|}$ é equivalente a minimizar $\|\mathbf{w}\|$, que é equivalente a minimizar $\frac{1}{2}\|\mathbf{w}\|^2$. Portanto, o problema de otimização se resume a

$$\begin{aligned} \min_{\mathbf{w}, w_0} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{sujeito a} \quad & y_i \overbrace{(w_0 + \mathbf{w}^T \mathbf{x}_i)}^{f(\mathbf{x}_i)} \geq 1, \quad i = 1, \dots, N. \end{aligned} \tag{6.2}$$

A escolha de minimizar $\frac{1}{2}\|\mathbf{w}\|^2$ em vez de $\|\mathbf{w}\|$ ficará mais evidente adiante.

A função de Lagrange $L(\mathbf{w}, w_0, \boldsymbol{\lambda})$ e a função de Lagrange dual $g(\boldsymbol{\lambda})$ desse problema são, respectivamente:

$$\begin{aligned} L(\mathbf{w}, w_0, \boldsymbol{\lambda}) &= \frac{1}{2}\|\mathbf{w}\|^2 + \sum_{i=1}^N \lambda_i [-y_i(w_0 + \mathbf{w}^T \mathbf{x}_i) + 1] \\ g(\boldsymbol{\lambda}) &= \inf_{\mathbf{w}, w_0} L(\mathbf{w}, w_0, \boldsymbol{\lambda}) \end{aligned}$$

Para encontrar $\inf_{\mathbf{w}, w_0} L(\mathbf{w}, w_0, \boldsymbol{\lambda})$, fazemos

$$\frac{\partial L}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w}^T - \sum_{i=1}^N \lambda_i y_i \mathbf{x}_i^T = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^N \lambda_i y_i \mathbf{x}_i \tag{6.3}$$

$$\frac{\partial L}{\partial w_0} = 0 \Rightarrow \sum_{i=1}^N \lambda_i y_i = 0 \tag{6.4}$$

Lembrando que $\|\mathbf{w}\|^2 = \mathbf{w}^T \mathbf{w}$. Assim, fica evidente que, anteriormente, foi escolhido minimizar $\frac{1}{2}\|\mathbf{w}\|^2$ porque o quadrado torna a derivação mais simples.

O passo anterior só é possível quando há dualidade forte, ou seja, quando o gap de dualidade é zero e temos $\inf_{\mathbf{w}, w_0} L(\mathbf{w}, w_0, \boldsymbol{\lambda}) = L(\mathbf{w}^*, w_0^*, \boldsymbol{\lambda})$, em que $\mathbf{w}^*$ e w_0^* são os parâmetros ótimos, que minimizam a função objetivo. Como o problema primal é convexo (função objetivo quadrática e restrição linear), basta que as condições de KKT sejam satisfeitas para que possamos afirmar que há dualidade forte. As condições de KKT são as seguintes:

1. $\mathbf{w} = \sum_{i=1}^N \lambda_i y_i \mathbf{x}_i$
2. $\sum_{i=1}^N \lambda_i y_i = 0$
3. $y_i (\mathbf{w}_0'' + \mathbf{w}''^T \mathbf{x}_i) - 1 \leq 0$
4. $\lambda_i [y_i (\mathbf{w}_0 + \mathbf{w}^T \mathbf{x}_i) - 1] = 0$
5. $\lambda_i \geq 0$

Se garantirmos que todas elas sejam satisfeitas, o que é trivial, podemos seguir assumindo dualidade forte.

A expressão da função de Lagrange dual fica:

$$\begin{aligned}
 g(\lambda) &= \frac{1}{2} \|\mathbf{w}\|^2 - \overbrace{\sum_{i=1}^N \lambda_i y_i}^{\text{eq. (6.4)}} - \sum_{i=1}^N \lambda_i y_i \overbrace{\mathbf{w}^T \mathbf{x}_i}^{\text{eq. (6.3)}} + \sum_{i=1}^N \lambda_i \\
 g(\lambda) &= \frac{1}{2} \left(\sum_{i=1}^N \lambda_i y_i \mathbf{x}_i^T \right) \left(\sum_{i=1}^N \lambda_i y_i \mathbf{x}_i \right) - \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j y_i y_j \mathbf{x}_j^T \mathbf{x}_i + \sum_{i=1}^N \lambda_i \\
 g(\lambda) &= \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j y_i y_j \mathbf{x}_j^T \mathbf{x}_i - \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j y_i y_j \mathbf{x}_j^T \mathbf{x}_i + \sum_{i=1}^N \lambda_i \\
 g(\lambda) &= -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j y_i y_j \mathbf{x}_j^T \mathbf{x}_i + \sum_{i=1}^N \lambda_i, \tag{6.5}
 \end{aligned}$$

para $\lambda \geq 0$, em que $\geq$ é o símbolo de maior ou igual elemento a elemento.

Com isso, o problema de otimização se resume a maximizar a função de Lagrange dual acima (comumente resolvida pelo computador) respeitando as condições de KKT.

Tendo encontrados os valores ótimos de $\mathbf{w}$ e w_0 , o hiperplano de separação ótimo produz uma função $\hat{f}(\mathbf{x}) = w_0 + \mathbf{w}\mathbf{x}$, que pode ser usada para classificar novos dados. Se $\hat{f}(\mathbf{x})$ for positiva, temos uma classe e, se for negativa, temos outra.

É interessante observar uma das condições KKT, dada pela *complementary slackness*, mostrada a seguir:

$$\lambda_i [y_i (\mathbf{w}_0 + \mathbf{w}^T \mathbf{x}_i) - 1] = \lambda_i [y_i f(\mathbf{x}_i) - 1] = 0$$

Para essa condição ser respeitada, precisamos ter $\lambda_i = 0$ ou $y_i f(\mathbf{x}_i) = 1$. Isso nos deixa com dois cenários interessantes:

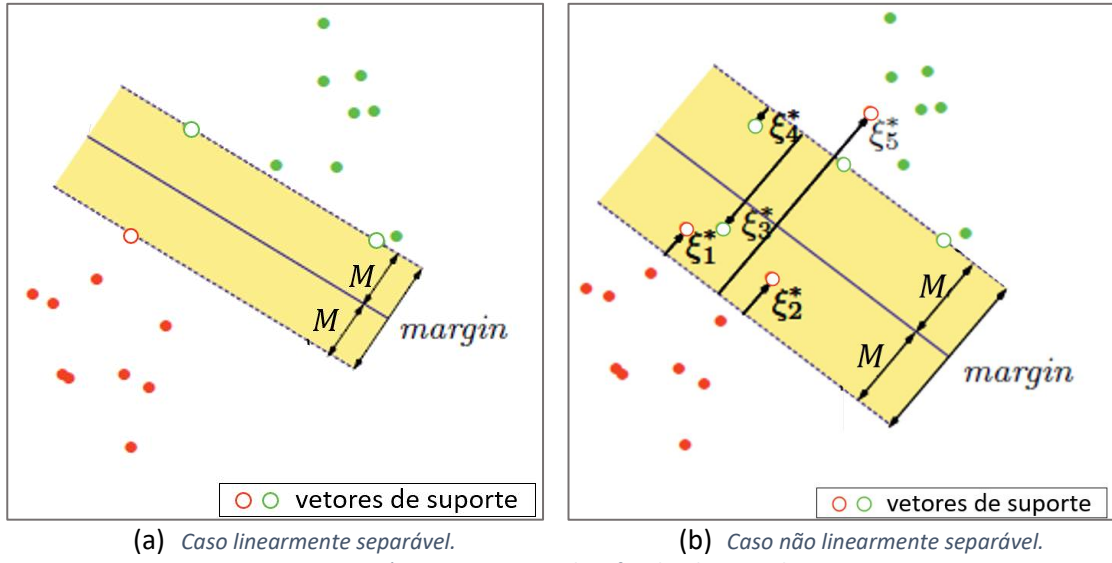
- $\lambda_i > 0$, o que implica $y_i f(\mathbf{x}_i) = 1$, isto é, $\mathbf{x}_i$ está na borda da margem, ou seja, em cima da linha pontilhada da Figura 6-1 (reparar que, em (6.2), $y_i f(\mathbf{x}_i) \geq 1$, havendo igualdade somente quando $\mathbf{x}_i$ está na margem).
- $y_i f(\mathbf{x}_i) > 1$, isto é, $\mathbf{x}_i$ não está na margem e $\lambda_i = 0$, o que zera o somatório da eq. (6.5) fazendo com que esse ponto não contribua para a solução do problema.

Portanto, reparamos que apenas os pontos que estão na margem contribuem para a determinação do plano de separação ótimo, já que $\mathbf{w}$ é uma combinação linear desses pontos (eq. (6.3)). Esses pontos são chamados de **vetores de suporte** (conferir Figura 6-1).

6.2 CLASSIFICADOR DE VETOR DE SUPORTE

Quando temos dados que não são linearmente separáveis, ou seja, que estão sobrepostos, o desenvolvimento acima não é válido. Por isso, refaremos todo esse desenvolvimento, mas de maneira mais geral.

No caso em questão, não conseguimos separar completamente todos os pontos com um hiperplano. É necessário aceitar que, em cada lado do hiperplano, haverá pontos das duas classes (observar a Figura 6-3). Além disso, haverá ainda pontos que estão do lado certo, mas que estão dentro da margem, ou seja, estão a uma distância do hiperplano menor que M . Essa abordagem, muitas vezes, é chamada de *soft margin classification*, em oposição ao método apresentado anteriormente, que não permitia que pontos estivessem dentro da margem ou do lado errado do hiperplano, chamado muitas vezes de *hard margin classification* [17].



Quando temos um ponto x_i dentro da margem ou do lado errado do hiperplano, dizemos que temos uma *violação*. Para levar em conta a importância das violações, usamos *variáveis de folga* $\xi_i, i = 1, \dots, N$, para indicar a distância, normalizada por M , que x_i está da borda da margem da classe a qual ele pertence (observando a Figura 6-3b isso fica mais claro). A cada ponto x_i , estando em violação ou não, é associada uma variável de folga ξ_i . Assim, escreveremos a restrição do nosso problema como

$$y_i \frac{(x_i^T \mathbf{w} + w_0)}{\|\mathbf{w}\|} \geq M(1 - \xi_i), \quad (6.6)$$

em que $\xi_i \geq 0$ e $\sum_{i=1}^N \xi_i \leq \text{constante}$. Isso nos deixa com alguns casos diferentes:

- $0 < \xi_i < 1$, indica que o ponto está do lado correto do hiperplano, mas dentro da margem;
- $\xi_i = 0$, indica que o ponto está do lado certo do hiperplano e está fora da margem, independentemente de sua distância para a margem;
- $\xi_i \geq 1$ indica que o ponto está do lado errado do hiperplano ou exatamente em cima do hiperplano, no caso em que $\xi_i = 1$.

Com isso, vemos que ξ_i indica a distância de um ponto para a margem certa de maneira relativa. Talvez uma forma mais natural de escrever essa restrição fosse $y_i \frac{(x_i^T \mathbf{w} + w_0)}{\|\mathbf{w}\|} \geq M - \xi_i$, pois, nesse caso, ξ_i está de fato indicando a distância absoluta para a margem. No entanto, essa forma de escrever a restrição não facilita o resto do desenvolvimento e, por isso, a restrição adotada será na forma da eq. (6.6).

Logo, seguindo a lógica adotada na construção do hiperplano de separação ótimo, o problema pode ser escrito como

$$\begin{aligned} & \text{minimizar}_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 \\ & \text{sujeito a} \quad y_i (\mathbf{x}_i^T \mathbf{w} + w_0) \geq 1 - \xi_i \\ & \quad \xi_i \geq 0 \\ & \quad \sum \xi_i \leq \text{constante} \end{aligned}$$

Uma forma mais conveniente de reescrever esse problema é a seguinte:

$$\begin{aligned} & \text{minimizar}_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \\ & \text{sujeito a} \quad y_i (\mathbf{x}_i^T \mathbf{w} + w_0) \geq 1 - \xi_i \\ & \quad \xi_i \geq 0 \end{aligned} \tag{6.7}$$

Dessa forma, temos um parâmetro “custo” C na função objetivo que assume um papel similar ao da constante que limita $\sum \xi_i$. Quando $C = \infty$, temos o caso separável.

Como o problema de otimização é convexo, podemos seguir passos análogos aos que foram adotados anteriormente para resolver esse problema. A função de Lagrange é dada por:

$$L(\mathbf{w}, w_0, \xi, \lambda, \mu) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i + \sum_{i=1}^N \lambda_i [1 - y_i (\mathbf{x}_i^T \mathbf{w} + w_0) - \xi_i] - \sum_{i=1}^N \mu_i \xi_i.$$

Como a função de Lagrange dual é $g(\lambda, \mu) = \inf_{\mathbf{w}, w_0, C} L(\mathbf{w}, w_0, \xi, \lambda, \mu)$, derivamos e igualamos a zero:

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^N \lambda_i y_i \mathbf{x}_i = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^N \lambda_i y_i \mathbf{x}_i \tag{6.8}$$

$$\frac{\partial L}{\partial w_0} = - \sum_{i=1}^N \lambda_i y_i = 0 \Rightarrow \sum_{i=1}^N \lambda_i y_i = 0$$

$$\frac{\partial L}{\partial \xi_i} = C - \lambda_i - \mu_i = 0 \Rightarrow \lambda_i = C - \mu_i \text{ ou } \mu_i = C - \lambda_i \tag{6.9}$$

Usando as equações acima, encontramos a seguinte expressão para função de Lagrange dual:

$$\begin{aligned} g(\lambda, \mu) = & \frac{1}{2} \left(\sum_{i=1}^N \lambda_i y_i \mathbf{x}_i^T \right) \left(\sum_{i=1}^N \lambda_i y_i \mathbf{x}_i \right) + C \sum_{i=1}^N \xi_i + \sum_{i=1}^N \lambda_i - \sum_{i=1}^N \lambda_i y_i \mathbf{x}_i^T \sum_{j=1}^N \lambda_j y_j \mathbf{x}_j \\ & - \sum_{i=1}^N \xi_i (C - \mu_i) - \sum_{i=1}^N (C - \lambda_i) \xi_i \end{aligned}$$

$$g(\lambda, \mu) = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j + \sum_{i=1}^N \lambda_i - \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$g(\lambda, \mu) = \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

Lembrando que é preciso considerar as condições de KKT para podermos resolver esse problema, que são dadas por:

$$1. \text{ Restrição 1: } y_i(\mathbf{x}_i^T \mathbf{w} + w_0) \geq 1 - \xi_i \text{ ou } y_i(\mathbf{x}_i^T \mathbf{w} + w_0) - (1 - \xi_i) \geq 0 \quad (6.10)$$

$$2. \text{ Restrição 2: } \xi_i \geq 0 \quad (6.11)$$

$$3. \text{ Condição do Lagrangiano 1: } \lambda_i \geq 0 \quad (6.12)$$

$$4. \text{ Condição do Lagrangiano 2: } \mu_i \geq 0 \quad (6.13)$$

$$5. \text{ Complementary slackness 1: } \lambda_i [y_i(\mathbf{x}_i^T \mathbf{w} + w_0) - (1 - \xi_i)] = 0 \quad (6.14)$$

$$6. \text{ Complementary slackness 2: } \mu_i \xi_i = 0 \quad (6.15)$$

$$7. \text{ Gradiente nulo 1: } \mathbf{w} = \sum_{i=1}^N \lambda_i y_i \mathbf{x}_i \quad (6.16)$$

$$8. \text{ Gradiente nulo 2: } \sum_{i=1}^N \lambda_i y_i = 0 \quad (6.17)$$

$$9. \text{ Gradiente nulo 3: } C - \lambda_i - \mu_i = 0 \Rightarrow \lambda_i = C - \mu_i \text{ ou } \mu_i = C - \lambda_i \quad (6.18)$$

A condição dada pela eq. (6.14), pode ser escrita como $\lambda_i [y_i f(\mathbf{x}_i) - (1 - \xi_i)] = 0$. A partir dessa equação e condição dada pela eq.(6.16), vemos que, assim como no caso linearmente separável, $\mathbf{w}$ é uma combinação linear de apenas alguns pontos $\mathbf{x}_i$, que são chamados de **vetores de suporte**. Esses pontos são todos aquelas que estão dentro da margem ou sobre uma de suas bordas ou que estão do lado errado do hiperplano (conferir Figura 6-3b).

Portanto, os únicos pontos $\mathbf{x}_i$ que não são vetores de suporte são aqueles tal que $y_i \frac{f(\mathbf{x}_i)}{\|\mathbf{w}\|} > M$.

Isso pode ser percebido ao observarmos as diferentes possibilidades para $\mathbf{x}_i$:

- Quando $\xi_i = 0$ (pontos do lado correto do hiperplano em cima da borda da margem ou fora da margem), temos uma situação parecida com o caso linearmente separável:
 - Quando $y_i f(\mathbf{x}_i) > 1$, ou seja, $\mathbf{x}_i$ está do lado certo do hiperplano, mas fora da margem, $\lambda_i = 0$ e, portanto, esse ponto não é contabilizado na eq. (6.16).
 - Quando $\lambda_i > 0$, temos $y_i f(\mathbf{x}_i) = 1$, ou seja, o ponto está exatamente em cima da borda da margem. Como $\lambda_i > 0$, o ponto é contabilizado.
 - Lembrar que uma das condições de KKT é que $y_i f(\mathbf{x}_i) \geq 1 - \xi = 1$, então, não haverá o caso $y_i f(\mathbf{x}_i) < 1$.

É interessante reparar que, nesse caso, pela eq. (6.18), $0 \leq \lambda_i \leq C$.

- Quando $\xi_i > 0$ (pontos do lado correto do hiperplano dentro da margem ou pontos do lado incorreto do hiperplano), $y_i f(\mathbf{x}_i) = 1 - \xi_i$. Logo, $\lambda_i \geq 0$ e todos os pontos são contabilizados na eq. (6.16).

Isso acontece porque $y_i f(\mathbf{x}_i) = r \|\mathbf{w}\| = \frac{r}{M}$ representa a distância normalizada de um ponto $\mathbf{x}_i$ para o hiperplano, sendo que ela será positiva quando o ponto estiver do lado certo do hiperplano e negativa caso contrário. Quando $\xi_i = 0$, isso não é necessariamente verdade, pois, esse caso inclui tanto os pontos que estão em cima da borda da margem, em que de fato $y_i f(\mathbf{x}_i) = 1 - \xi_i = 1$, mas também inclui os pontos corretamente classificados que estão fora da margem, em que $y_i f(\mathbf{x}_i) > 1 - \xi_i = 1$. Dois exemplos ajudam a entender isso:

- Se $0 < \xi_i < 1$, o ponto está do lado correto do hiperplano, mas dentro da margem, e $1 - \xi_i > 0$. Se, por exemplo, $\xi_i = 0,4$, sua distância normalizada da margem é dada por $\xi_i = 0,4$ e sua distância normalizada para o hiperplano é dada por $\frac{r}{M} = y_i f(\mathbf{x}_i) = 1 - \xi_i = 0,6$.
- Se $\xi_i \geq 1$, o ponto está do lado errado do hiperplano e $1 - \xi_i \leq 0$. Se, por exemplo, $\xi_i = 3$, sua distância normalizada da margem é dada por $\xi_i = 3$ e sua distância normalizada para o hiperplano é $\frac{r}{M} = y_i f(\mathbf{x}_i) = -(\xi_i - 1) = 1 - \xi_i = -2$. Lembrando que o sinal de menos em $-(\xi_i - 1)$ é usado porque $y_i f(\mathbf{x}_i)$ é negativo quando o ponto está do lado errado do hiperplano.

É interessante reparar que, nesse caso, pelas eqs. (6.15) e (6.18), $\lambda_i = C$.

Depois de $\mathbf{w}$ e w_0 terem sido determinados (eq. (6.16)), podemos escrever a equação do hiperplano como

$$f(\mathbf{x}_i) = \mathbf{x}_i^T \mathbf{w} + w_0 = \mathbf{x}_i^T \sum_{j=1}^N \lambda_j y_j \mathbf{x}_j + w_0 = \sum_{j=1}^N \lambda_j y_j \mathbf{x}_i^T \mathbf{x}_j + w_0. \quad (6.19)$$

Tendo a expressão para f , o ponto $\mathbf{x}_i$ é classificado com base no sinal de $f(\mathbf{x}_i)$.

C é um parâmetro de *tunning*. Quanto maior o valor de C , menor é a soma dos valores de ξ_i (olhar eq. (6.7)), o que diminui a quantidade de violações permitidas. Para isso acontecer, a margem precisa diminuir, pois isso garante que pontos do lado correto do hiperplano fiquem fora da margem e deixem de ser considerados violações. Os pontos do lado incorreto do hiperplano já são considerados violações independentemente do tamanho da margem, o que muda é apenas o peso dessa violação (ξ_i). De forma análoga, quanto menor for C , maior é a soma dos valores de ξ_i , o que permite que mais violações aconteçam. Logo, a margem precisa ser maior. A Figura 6-4 ilustra essas situações.

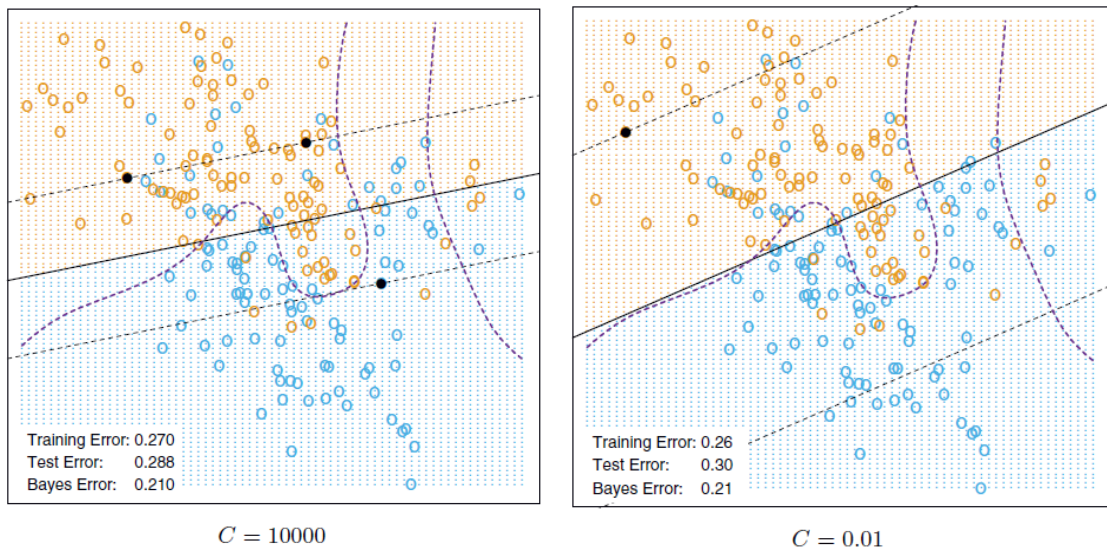


Figura 6-4: Margens do classificador de vetor de suporte para diferentes valores de C .

6.3 MÁQUINA DE VETOR DE SUPORTE

Como vimos acima, o classificador de vetor de suporte encontra fronteiras lineares de decisão no espaço das entradas $\mathbf{x}$. No entanto, é possível tornar esse procedimento mais flexível

aumentando esse espaço usando funções base, criando assim, um espaço de *features*. Quando aplicamos o método dessa maneira, encontramos uma fronteira de decisão linear no espaço das *features*, porém não-linear no espaço das entradas.

A utilização do método do classificador de vetor de suporte com a possibilidade de trabalhar em um outro espaço de dimensão maior (possivelmente infinito) dá origem ao modelo *support vector machine* (SVM).

O procedimento para encontrar a fronteira de decisão desse modelo é, basicamente, o mesmo que foi desenvolvido na Seção 6.2, bastando apenas substituir as entradas $\mathbf{x}$ pelas suas funções de base $\phi(\mathbf{x})$ correspondentes. Então, com base na eq. (6.19), podemos escrever

$$f(\mathbf{x}_i) = \phi(\mathbf{x}_i)^T \mathbf{w} + w_0 = \sum_{j=1}^N \lambda_j y_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle_{\mathbb{H}} + w_0. \quad (6.20)$$

É importante dizer que $\phi(\mathbf{x})$ é uma aplicação definida como

$$\begin{aligned} \phi: \mathbb{R}^d &\rightarrow \mathbb{H} \\ \mathbf{x} &\mapsto \phi(\mathbf{x}) \end{aligned}$$

em quem d é a dimensão dos dados e $\mathbb{H}$ é um *reproducing kernel Hilbert Space* (RKHS) de funções reais. $\phi(\mathbf{x}) = [\phi_1(\mathbf{x}), \phi_2(\mathbf{x}), \dots]^T$ é chamada de *feature map* e pode ter dimensão finita ou infinita. Para mais detalhes sobre o RKHS, consultar o Apêndice.

Observar que, na eq. (6.20), o produto interno que existia originalmente na eq. (6.19), $\mathbf{x}_i^T \mathbf{x}_j = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$, produto interno do espaço Euclidiano, foi substituído pelo produto interno dos *features maps* $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle_{\mathbb{H}}$. O subscrito $\mathbb{H}$ serve para indicar que o produto interno entre essas duas aplicações não é mais um produto interno do espaço Euclidiano, pois essa aplicação pertence ao espaço de Hilbert, que é um espaço de produto interno $\langle \cdot, \cdot \rangle_{\mathbb{H}}$.

Todo RKHS apresenta uma função kernel $\kappa(\cdot, \cdot)$ que o define. Além disso, nesse espaço, temos que $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \langle \phi(\mathbf{x}_1), \phi(\mathbf{x}_2) \rangle$. Logo, podemos reescrever a eq. (6.20) como

$$f(\mathbf{x}_i) = \sum_{j=1}^N \lambda_j y_j \kappa(\mathbf{x}_i, \mathbf{x}_j) + w_0. \quad (6.21)$$

Com isso, vemos que não é necessário conhecer as *features* $\phi_k(\mathbf{x})$ definidas por $\phi(\mathbf{x})$ para determinar a fronteira de decisão. Basta conhecer a função kernel, que normalmente, é uma escolha de *design* do modelo SVM. Escolhas populares para o kernel são:

- Kernel polinomial: $\kappa(\mathbf{x}_1, \mathbf{x}_2) = (1 + \langle \mathbf{x}_1, \mathbf{x}_2 \rangle)^d$
- Kernel Gaussiano: $\kappa(\mathbf{x}_1, \mathbf{x}_2) = e^{-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|_2^2}$
- Kernel sigmoide: $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \tanh(\gamma \langle \mathbf{x}_1, \mathbf{x}_2 \rangle + c)$

em que $\langle \cdot, \cdot \rangle$ é o produto interno do espaço Euclidiano.

É importante notar que, na grande maioria dos casos, não conhecemos $\phi(\mathbf{x}_i)$ e, por isso, não podemos avaliar o valor de f em um dado ponto de teste $\mathbf{x}$ como $f(\mathbf{x}) = \phi(\mathbf{x})^T \mathbf{w} + w_0$. É necessário utilizar a eq. (6.21), o que aumenta a complexidade de tempo para avaliar um novo ponto $\mathbf{x}$ quando comparado com o caso em temos um *kernel* linear (quando não

transformamos as variáveis de entrada em *features*). No caso de *kernel* linear, de fato, podemos utilizar a expressão $f(\mathbf{x}) = \phi(\mathbf{x})^T \mathbf{w} + w_0 = \mathbf{x}^T \mathbf{w} + w_0$.

6.3.1 SVM como Método de Penalização

O problema de minimização do modelo SVM é similar ao do classificador de vetor de suporte, definido na eq. (6.7), porém substituindo $\mathbf{x}$ por $\phi(\mathbf{x})$ quando necessário. Assim, omitindo as restrições, o problema de minimização consiste em

$$\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i. \quad (6.22)$$

Se repararmos, podemos escrever a variável de folga como

$$\xi_i = 1 - y_i r_M(\mathbf{x}_i),$$

$\forall \mathbf{x}_i \in \{\mathbf{x}_i : y_i r_M(\mathbf{x}_i) < 1\}$, em que $r_M(\mathbf{x}_i) = \frac{f(\mathbf{x}_i)}{\|\mathbf{w}\|} \frac{1}{M} = \frac{\phi(\mathbf{x}_i)^T \mathbf{w} + w_0}{\|\mathbf{w}\|} \frac{1}{M}$ é a distância com sinal normalizada com relação a M . Só que, como $M = \frac{1}{\|\mathbf{w}\|}$, temos que $r_M(\mathbf{x}_i) = f(\mathbf{x}_i)$. Logo,

$$\xi_i = 1 - y_i f(\mathbf{x}_i),$$

Reparar que $\{\mathbf{x}_i : y_i f(\mathbf{x}_i) < 1\}$ engloba todos os pontos, com exceção daqueles que estão do lado certo do hiperplano e fora da margem.

Logo, podemos reescrever o problema de minimização da eq. (6.22) da seguinte forma:

$$\begin{aligned} & \min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N [1 - y_i f(\mathbf{x}_i)] \mathbb{1}_A(\mathbf{x}_i) \\ & \equiv \min_{\mathbf{w}, w_0} \frac{1}{2C} \|\mathbf{w}\|^2 + \sum_{i=1}^N [1 - y_i f(\mathbf{x}_i)] \mathbb{1}_A(\mathbf{x}_i) \\ & \equiv \min_{\mathbf{w}, w_0} \overbrace{\frac{\lambda}{2} \|\mathbf{w}\|^2}^{\text{penalidade}} + \sum_{i=1}^N \overbrace{[1 - y_i f(\mathbf{x}_i)] \mathbb{1}_A(\mathbf{x}_i)}^{\text{loss}} \end{aligned}$$

em que $\lambda = 1/C$ e $A = \{\mathbf{x}_i : y_i f(\mathbf{x}_i) < 1\}$.

Com isso, vemos que o problema de minimização do modelo SVM tem a forma *loss* + penalidade, típica de um problema de otimização com regularização.

A função de *loss* $\ell(y, f) = [1 - yf(\mathbf{x})] \mathbb{1}_A(\mathbf{x})$ é chamada de *hinge loss* (*loss* articulação), devido ao seu formato, como mostrado na Figura 6-5. Nessa imagem, outras funções de *loss* são mostradas para efeito de comparação. Para mais detalhes, consultar [2].

Com isso, vemos que a *hinge loss* atribui perda nula para pontos que não representam uma violação ($y_i f(\mathbf{x}_i) \geq 1$) e perda linear para os pontos que representam uma violação.

Um detalhe importante para se atentar é que, só encontramos a expressão da *hinge loss* na modelagem do SVM por causa de algumas suposições feitas, como $M = \frac{1}{\|\mathbf{w}\|}$ e $y_i f(\mathbf{x}_i) \geq M(1 - \xi_i)$. Se outras escolhas tivessem sido feitas, obteríamos outra *loss*.

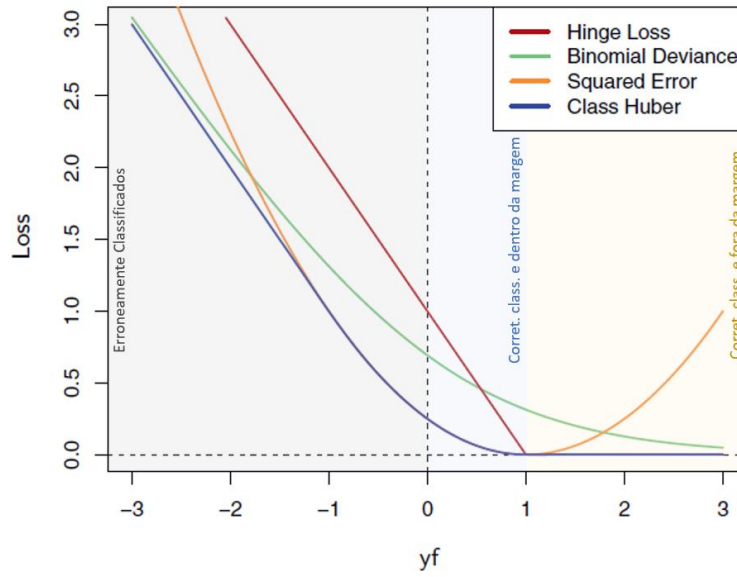


Figura 6-5: Comparação de diferentes funções de loss. Adaptação de imagem presente em [2].

6.3.2 Classificação Multiclasse

O mais comum é adotar a abordagem *one-vs-the-rest*. Para corrigir o problema das regiões de ambiguidade, diferentes técnicas podem ser adotadas, como mostrado em [3, pp. 338-339].

Notar que, no modelo da regressão logística, esse problema era facilmente resolvido ao atribuir uma função discriminante $\delta_k(\mathbf{x}) = \mathbb{P}(\mathbf{x} \in \omega_k | \mathbf{x})$ para cada uma das K funções e estabelecer a regra de que $\mathbf{x} \in \omega_i$ se $\delta_i(\mathbf{x}) > \delta_j(\mathbf{x}) \forall i \neq j$. Lembrando que a variável aleatória nesse caso é ω_k , que é o conjunto dos pontos pertencentes à classe k . Nesse caso, não existirá regiões de ambiguidade.

No entanto, no SVM, isso não é possível, pois não temos uma função discriminante que representa a probabilidade a posteriori de o ponto pertencer à classe k .

6.4 SVM PARA REGRESSÃO

Nas seções anteriores, tínhamos dados que pertenciam a duas classes diferentes e utilizávamos um hiperplano $f(\mathbf{x}) = 0$ para separá-los, considerando que existe uma margem ao redor desse hiperplano.

Para o caso da regressão, é necessário adaptar o método, já que o nosso problema não é mais de separação dos dados. A adaptação que pode ser feita é considerar que, agora, $f(\mathbf{x}) = \mathbf{x}^T \mathbf{w} + w_0$ representará uma curva de regressão que tentará aproximar a função geradora dos dados $\mathcal{G}(\mathbf{x}) = y$. Para isso, consideraremos que os dados podem estar exatamente em cima da curva $f(\mathbf{x})$ ou dentro de uma margem de espessura ϵ ao redor da curva, como mostra a Figura 6-6.

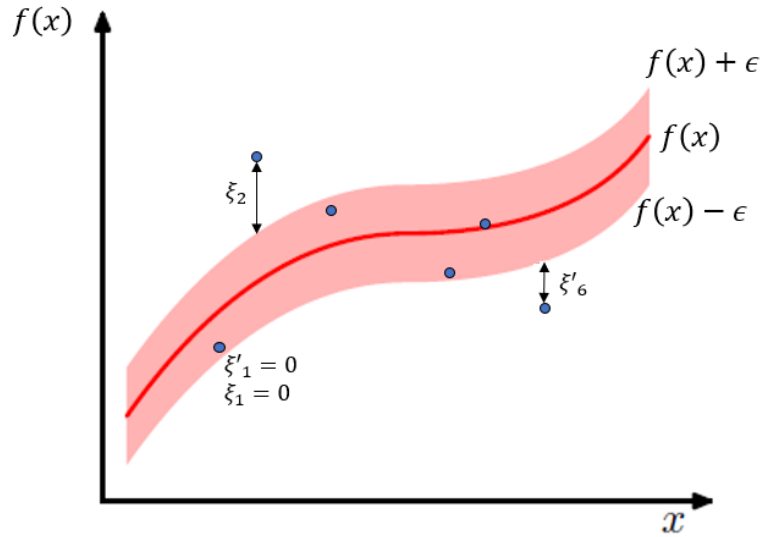


Figura 6-6: Modelo de SVM para regressão.

Podemos escrever esse problema no formato $\sum \text{loss} + \text{regularização}$, sujeito à condição de que os pontos devem estar dentro do “tubo- ϵ ” que envolve $f(x)$. Com isso, o método acima se resume a

$$\begin{aligned} \min_{\mathbf{w}, \mathbf{w}_0} \quad & C \sum_{i=1}^N V_{\epsilon}(y_i - f(x_i)) + \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{sujeito a} \quad & f(x_i) + \epsilon \geq y_i \\ & f(x_i) - \epsilon \leq y_i \end{aligned}$$

em que V_{ϵ} é a função de erro ϵ -insensível, dada por

$$V_{\epsilon}(r) = \begin{cases} 0, & |r| < \epsilon \\ |r| - \epsilon, & \text{c. c.} \end{cases}$$

o que faz com que desprezemos todo erro $|r|$ abaixo de ϵ , ou seja, desprezamos os erros para pontos dentro do tubo- ϵ . Ela é mostrada na Figura 6-7.

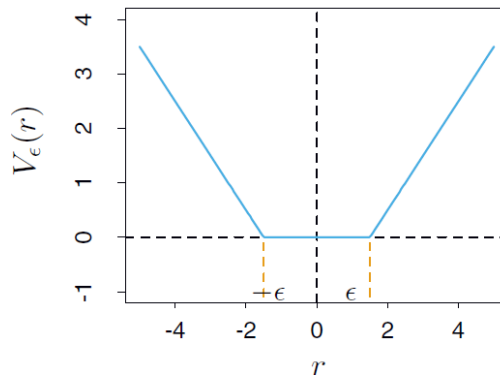


Figura 6-7: Função de erro ϵ -insensível.

No entanto, nem sempre vamos conseguir fazer com que todos os pontos se encontrem dentro do tubo- ϵ . Por isso, assim como no caso da classificação, introduzimos variáveis de folga ξ_i para permitir a contabilização de pontos que não estejam dentro do tubo- ϵ . No entanto, na regressão, precisaremos de variáveis $\xi_i \geq 0$ para os pontos acima da curva $f(x)$ e

variáveis $\xi'_i \geq 0$ para pontos abaixo da curva $f(\mathbf{x})$. Essas variáveis são definidas como $\xi_i = \{y_i - [f(\mathbf{x}_i) + \epsilon]\} \mathbb{1}_{\{y_i > f(\mathbf{x}_i) + \epsilon\}}(\mathbf{x}_i)$ e $\xi'_i = \{[f(\mathbf{x}_i) - \epsilon] - y_i\} \mathbb{1}_{\{y_i < f(\mathbf{x}_i) + \epsilon\}}(\mathbf{x}_i)$.

Incluindo essas variáveis no problema, temos um novo problema de otimização, dado por

$$\begin{aligned} \min_{\mathbf{w}, w_0} \quad & C \sum_{i=1}^N (\xi_i + \xi'_i) + \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{sujeito a} \quad & f(\mathbf{x}_i) + \epsilon + \xi_i \geq y_i \\ & f(\mathbf{x}_i) - \epsilon - \xi'_i \leq y_i \\ & \xi_i \geq 0 \\ & \xi'_i \geq 0 \end{aligned}$$

Se repararmos, temos algo muito parecido com o que tínhamos no problema (6.7). Com isso, vemos que é preferível ter ξ_i e ξ'_i menores possíveis, pois ξ_i e ξ'_i grandes levam a um custo maior.

Com isso, o Lagrangiano pode ser escrito como

$$\begin{aligned} L(\mathbf{w}, w_0, \xi, \xi') = & C \sum_{i=1}^N (\xi_i + \xi'_i) + \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \lambda_i [f(\mathbf{x}_i) + \epsilon - y_i] - \sum_{i=1}^N \lambda'_i [-f(\mathbf{x}_i) + \epsilon + y_i] \\ & - \sum_{i=1}^N (\mu_i \xi_i + \mu'_i \xi'_i) \end{aligned}$$

em que $\lambda_i \geq 0$, $\lambda'_i \geq 0$, $\mu_i \geq 0$ e $\mu'_i \geq 0$ são os multiplicadores de Lagrange.

A função dual de Lagrange é dada por

$$\begin{aligned} g(\lambda, \lambda') = \inf_{\mathbf{w}, w_0} L(\mathbf{w}, w_0, \xi, \xi') &= L(\mathbf{w}^*, w_0^*, \xi^*, \xi'^*) \\ &= -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N (\lambda_i - \lambda'_i)(\lambda_j - \lambda'_j) \langle \mathbf{x}_i, \mathbf{x}_j \rangle - \epsilon \sum_{i=1}^N (\lambda_i + \lambda'_i) + \sum_{i=1}^N (\lambda_i + \lambda'_i) y_i \end{aligned}$$

com

$$\mathbf{w}^* = \sum_{i=1}^N (\lambda'_i - \lambda_i) \mathbf{x}_i.$$

Para terminar de resolver o problema de otimização, basta maximizar $g(\lambda, \lambda')$ sujeito às condições de KKT. Duas dessas condições, advindas da propriedade *complementary slackness*, são interessantes para serem analisadas, como mostradas abaixo:

$$\lambda_i [\epsilon + f(\mathbf{x}_i) + \xi_i - y_i] = 0 \quad (6.23)$$

$$\lambda'_i [\epsilon - f(\mathbf{x}_i) + y_i + \xi'_i] = 0. \quad (6.24)$$

A partir da eq. (6.23), vemos que $\lambda_i \neq 0$ somente se $[\epsilon + f(\mathbf{x}_i) + \xi_i - y_i] = A = 0$, ou seja, se o ponto estiver fora do tubo- ϵ ou exatamente em sua borda. Isso porque

- se o ponto está fora do tubo- ϵ , então $y_i = f(\mathbf{x}_i) + \epsilon + \xi_i$ e $A = 0$;

- se o ponto está na borda do tubo- ϵ , então $\xi_i = 0$, $y_i = f(x_i) + \epsilon$ e $A = 0$.

No caso em que o ponto está dentro da margem $A = \epsilon + f(x_i) + \xi_i - y_i = \epsilon + f(x_i) - y_i > 0$, pois $y_i < \epsilon + f(x_i)$.

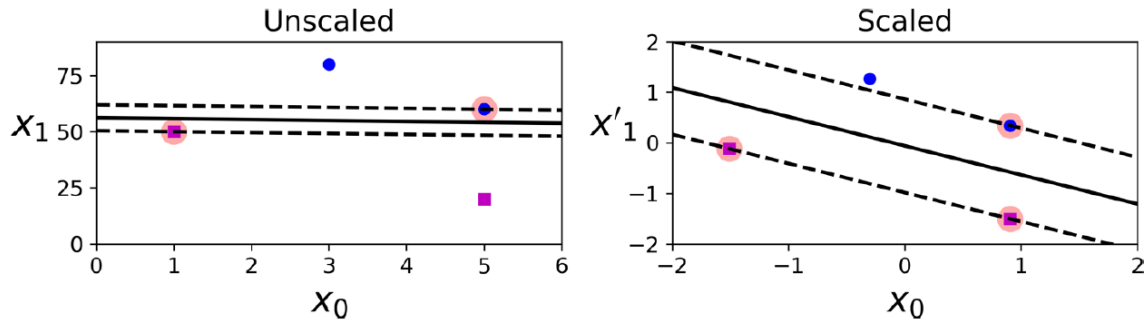
O raciocínio é análogo para λ'_i na eq. (6.24). Com isso, vemos que os únicos pontos x_i que contribuem para w^* são aqueles que estão fora do tubo- ϵ ou exatamente em cima de sua borda.

Assim, como foi feito no caso da classificação, todo esse desenvolvimento pode ser generalizado para o caso em que estamos trabalhando no espaço das *features*. Basta substituir as entradas x_i pelas *features* ϕ_i e o produto interno Euclidiano $\langle x_i, x'_i \rangle$ pela função kernel $\kappa(x_i, x'_i)$.

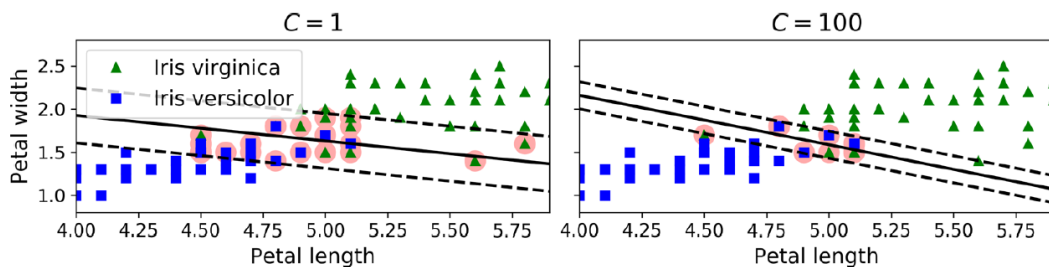
6.5 ASPECTOS PRÁTICOS

- A complexidade de treinamento de um modelo de SVM varia de acordo com a função kernel sendo utilizada. Para o SVM com kernel linear, a complexidade é quase linear, ou seja, se aproxima de $\mathcal{O}(m \times N)$, em que N é o número de exemplos e m é o número de *features* [18]. Para SVM mais genéricos, com outras possibilidades de kernel, a complexidade está entre $\mathcal{O}(N^2 \times m)$ e $\mathcal{O}(N^3 \times m)$ [17]. Ao usar o SVM pela biblioteca scikit-learn, é preferível usar a classe `LinearSVC` para treinar modelos de SVM com kernel linear, pois ela é otimizada para isso e garante a complexidade $\mathcal{O}(m \times N)$. A classe `SVC` do scikit-learn também permite utilizar kernel linear, porém ela não é otimizada para isso, por isso, a complexidade fica entre quadrática e cúbica, como já apresentado.
- É um modelo que não escala bem com os dados, pois sua complexidade é algo entre quadrático e cúbico (com exceção da implementação otimizada para kernel linear, que tem complexidade linear).
- Consegue lidar naturalmente bem com problemas com dimensão elevada e pouca quantidade de dados. Isso acontece porque o SVM *vanilla* já possui regularização, que é evidente pelo termo $\|w\|^2$ da eq. (6.22). Essa regularização, como já foi visto, surge pelo fato de estarmos impondo restrições ao problema: a existência de uma margem, que serve como referência para identificar pontos em violação, que devem ser penalizados. Como essa margem que desejamos minimizar é inversamente proporcional a $\|w\|$, estamos, consequentemente, impondo uma restrição ao vetor de parâmetros também.
De um ponto de vista mais intuitivo, o fato de o SVM não levar em conta todos os pontos do conjunto de treinamento para a determinação do hiperplano de separação já impede que o modelo se adapte excessivamente aos dados. O SVM leva em consideração apenas os vetores de suporte.
- É sensível à escala, pois é um modelo que se baseia em cálculos de distância dos pontos até hiperplanos.
- É vulnerável a *outliers* [19], pois pontos do lado errado do hiperplano são vetores de suporte e influenciam a determinação dos parâmetros do hiperplano de separação. Além disso, quanto mais distante do hiperplano um ponto do lado errado do hiperplano está, mais ele é penalizado, já que é utilizada a *hinge loss*.
- O SVM utilizando *soft margin* tem menor variância que o SVM utilizando *hard margin*. Se considerarmos casos em que os dados são linearmente separáveis, em alguns

desses casos pode acontecer de o uso do SVM com *hard margin* gerar uma margem muito pequena pela forma como os dados estão organizados, o que faz com que o modelo tenda a gerar *overfitting*. Por outro, um modelo com *soft margin* tende a aceitar mais erros de classificação no conjunto de treinamento, mas tende também a gerar um modelo generaliza melhor o fenômeno sendo modelado.



- Baixos valores de C aumentam a regularização do modelo, deixando a margem menos estreita.



- Poderíamos aplicar um classificador de vetor de suporte *soft* para separar dados não-linearmente separáveis. No entanto, precisaríamos criar novas *features* a partir das variáveis de entrada, de forma a encontrar uma fronteira de decisão adequada (e não linear). Criar *features* polinomiais geralmente é uma boa opção. Para polinômios de grau pequeno, isso pode até ser viável, mas para polinômios de grau elevado, isso se torna inviável. A partir disso, podemos ver a vantagem do SVM sobre métodos lineares que utilizam transformações para gerar novas *features*: não é preciso criar novas *features*, apenas escolher a função *kernel* adequada.
- As funções *kernel* mais comumente usadas são a linear e a *Gaussian Radial Basis Function* (RBF). Como regra de ouro para escolher a função *kernel* na prática, pode-se começar com a linear (dando preferência para o LinearSVC, em vez do SVC do scikit-learn, pela eficiência computacional), especialmente se o conjunto de dados for muito grande. Caso o conjunto de dados não seja tão grande, pode-se testar em seguida a *Gaussian RBF*.

Capítulo 7: ÁRVORE DE DECISÃO

A árvore de decisão é um modelo de aprendizado supervisionado que funciona dividindo o espaço dos atributos em regiões cuboides, cujas arestas são ortogonais aos eixos, e treinando um modelo simples (como uma constante) a cada uma dessas regiões. Existem diferentes métodos de construção de uma árvore de decisão, como o C4.5, C5.0 e o CART (*Classification and Regression Tree*), mas o mais popular é o último [3] e, por isso, é nele que o resto do capítulo se baseia.

O modelo da árvore de decisão pode ser aplicado tanto para problemas de classificação (nesse caso, ela recebe o nome de árvore de classificação) quanto de regressão (nesse caso, ela recebe o nome de árvore de regressão).

Uma das vantagens da árvore de decisão é que ela pode ser representada na forma de uma árvore binária, como mostra a Figura 7-1, o que torna sua visualização e interpretação mais simples. O primeiro nó é chamado de **raiz**, os nós das extremidades da árvore, de onde não partem outros nós, são chamados de **folhas** e os nós restantes são chamados de **nós internos**. Da raiz e dos nós internos sempre partem duas ramificações, que dão origem ao nó filho esquerdo e direito. A **profundidade** de um nó é a distância entre esse nó e a raiz da árvore, ou seja, é o número de passos que se dá para chegar na raiz a partir do nó em questão.

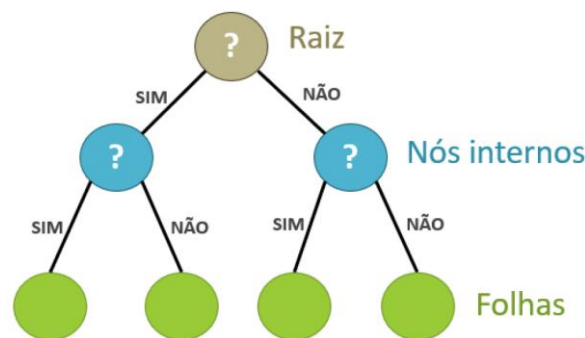


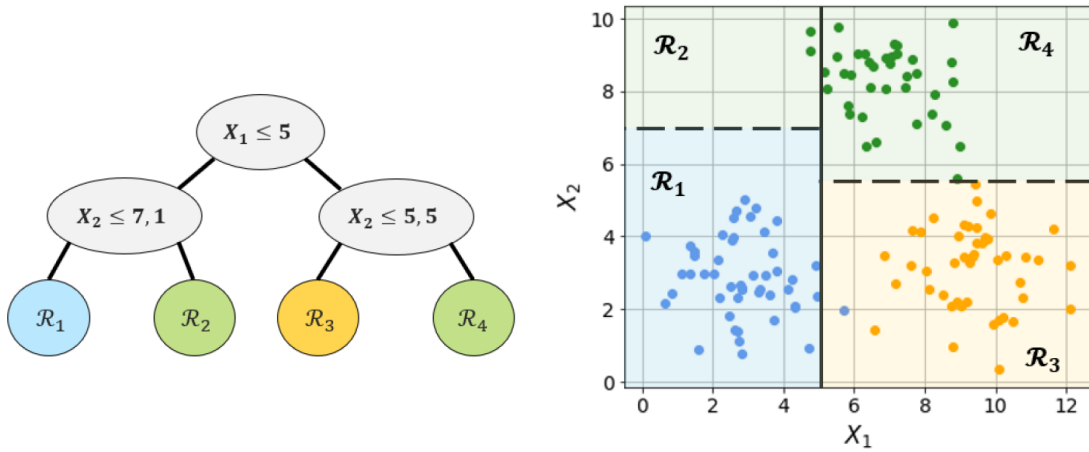
Figura 7-1: Diagrama de uma árvore de decisão.

A cada um dos nós internos e à raiz é associada uma condição do tipo $X_j < t$, em que X_j é um atributo e t uma constante. Considere certa instância que tem x_i como entrada. À medida que essa instância percorre a árvore, começando na raiz, toda vez que ela satisfizer a condição de um nó, ele deve descer um nível tomando o caminho da esquerda, caso contrário, ele deve tomar o caminho da direita. O percurso acaba quando se chega a uma folha que apresenta o resultado da classificação/regressão.

7.1 ÁRVORE DE CLASSIFICAÇÃO

Nas árvores de classificação, a cada folha, associamos uma classe do problema. A Figura 7-2a ilustra o exemplo de uma árvore desse tipo que foi construída com base em um conjunto de dados de treinamento com atributos X_1 e X_2 e que apresenta 3 classes (representadas pelas cores azul, verde e amarelo). Com auxílio da Figura 7-2b, é possível perceber que, a cada bifurcação dessa árvore, o espaço dos atributos é dividido em uma região $X_j < s$ e outra $X_j > s$, associadas aos nós filhos esquerdo e direito, respectivamente. A classificação que um dado exemplo x_i recebe ao chegar a um nó folha de índice m depende, portanto, da região $\mathcal{R}_m$ que

esse nó representa. Assim, se um certo exemplo estiver localizado na região $\mathcal{R}_3$, ele será classificado como amarelo.



(a) Representação da árvore de decisão.

(b) Dados divididos por região.

Figura 7-2: Exemplo de uma árvore de decisão aplicada a um conjunto de dados de dois atributos e três classes.

Considere um conjunto de dados de treinamento com N exemplos, com entradas x_i e saídas y_i , em que $i \in \{1, 2, \dots, N\}$. Para a construção de uma árvore de decisão com base nesses dados, o algoritmo parte do nó raiz. Como, inicialmente, a árvore só apresenta um único nó, ou seja, não apresenta bifurcações, o espaço dos atributos ainda não foi segmentado e o nó raiz representa todo o espaço dos atributos, que contém todos os dados de treinamento.

Primeiramente, o algoritmo precisa encontrar o par (X_j, s) que divide os dados em duas regiões da melhor forma possível. Para isso, é necessário definir uma **medida de impureza** e encontrar os valores de X_j e s que minimizam a soma ponderada das impurezas das duas regiões resultantes da divisão do nó raiz. Após ter encontrado o melhor par (X_j, s) , o nó raiz é dividido em duas regiões, uma para cada nó filho, e esse processo é repetido para cada novo nó. Considerando uma árvore T qualquer (ainda em processo de construção), um nó terminal m (associado à região $\mathcal{R}_m$), os eventuais futuros nós filhos esquerdo m_E e direito m_D desse nó m e uma métrica de impureza $Q_m(T)$ (isso ficará mais claro adiante), podemos escrever matematicamente o processo de encontrar o melhor par (X_j, s) como

$$(X_j, s) = \arg \min_{X_j, s} \frac{N_{m_E} Q_{m_E}(T) + N_{m_D} Q_{m_D}(T)}{N_m}, \quad (7.1)$$

em que N_{m_E} e N_{m_D} correspondem ao número de instâncias de treinamento pertencentes às regiões $\mathcal{R}_{m_E}$ e $\mathcal{R}_{m_D}$ após uma eventual divisão do nó pai m e $N_m = N_{m_E} + N_{m_D}$ é o total de instâncias na região $\mathcal{R}_m$ do nó pai. Portanto, vemos que a busca pelo melhor par (X_j, s) consiste em minimizar a impureza média ponderada da divisão do nó pai (lembrando que, como N_m não depende de X_j nem de s , ele pode ser desconsiderado da minimização).

É importante mencionar que, encontrar o particionamento que minimize a impureza de toda a árvore é praticamente impossível computacionalmente. Por isso, esse processo é feito de maneira gulosa, ou seja, os nós são divididos da raiz até as folhas e, dado que um nó já foi dividido, o par (X_j, s) desse nó não pode ser reconsiderado. Além disso, para cada variável de particionamento X_j , o valor de s é encontrado ao avaliar os diferentes valores dessa variável no conjunto de treinamento.

Por fim, depois de a árvore de classificação já ter sido construída, é necessário atribuir uma classe a cada folha. Essa atribuição é feita levando em consideração a proporção $\hat{p}_{mk}$ de instâncias de treinamento pertencente à classe k presentes na região $\mathcal{R}_m$ associada à folha m . A categoria atribuída à folha é aquela que está em maior proporção, ou seja, a categoria da folha m é dada por $k(m) = \arg \max_k \hat{p}_{mk}$. Na Figura 7-2b, por exemplo, a folha associada à região $\mathcal{R}_3$ contém 48 exemplos, sendo 47 amarelos (97,9 %), e 1 azul (2,1 %). Portanto, essa folha é associada à categoria amarela, pois é a categoria com maior frequência nesse nó.

Retornando à eq. (7.1), diferentes métricas $Q_m(T)$ podem ser utilizadas. Algumas delas são listadas abaixo:

Índice de Gini

$$Q_m(T) = \sum_{k \neq k'} \hat{p}_{mk} \hat{p}_{mk'} = 1 - \sum_{k=1}^K \hat{p}_{mk}^2$$

A igualdade acima vem do fato que $\sum_{k \neq k'} \hat{p}_{mk} \hat{p}_{mk'} = \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk}) = \sum_{k=1}^K \hat{p}_{mk} - \sum_{k=1}^K \hat{p}_{mk}^2 = 1 - \sum_{k=1}^K \hat{p}_{mk}^2$.

Entropia

$$Q_m(T) = - \sum_{k=1}^K \hat{p}_{mk} \log(\hat{p}_{mk})$$

Alguns autores chamam essa métrica de **entropia cruzada** ou *deviance*. Em [20], encontra-se uma discussão interessante sobre o assunto.

Taxa de erro de classificação

$$Q_m(T) = \frac{1}{N_m} \sum_{n|x_n \in \mathcal{R}_m} \mathbb{1}_{\{t \neq k(m)\}}(t_n) = 1 - \hat{p}_{mk(m)}$$

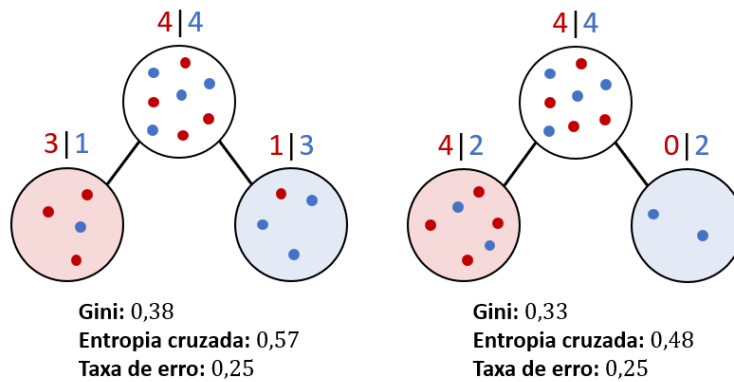


Figura 7-3: Valores da média ponderada da impureza (como na eq. (7.1)) nos nós filhos para as três métricas de impureza apresentada.

Quando todas as instâncias de um nó pertencem a uma mesma classe, esse nó é considerado **puro**. Nesse caso, as três métricas de impureza apresentadas são nulas. Por outro lado, para outros casos, essas métricas têm comportamentos diferentes. O índice de Gini e a entropia

cruzada tendem a apresentar valores menores quanto mais puros forem os nós, o que não é necessariamente verdade para a taxa de erro. Isso pode ser visto a partir do exemplo mostrado na Figura 7-3, em que são mostradas duas possibilidades de divisão de um certo nó. Nos dois casos, a taxa de erro é igual, porém, vemos que as médias ponderadas que usam o índice de Gini e a entropia cruzada são menores no segundo caso, em que a divisão gera nós mais puros. Portanto, como divisões que geram nós mais puros tendem a ser preferíveis [2], o índice de Gini e a entropia cruzada são mais comumente usadas na eq. (7.1) para encontrar os pares (X_j, s) . Além disso, essas duas métricas são diferenciáveis (como fica evidente pela Figura 7-4, que compara as três métricas de impureza apresentadas), o que facilita o processo de otimização numérica. Apesar disso, a taxa de erro de classificação é a métrica mais utilizada em um processo que será visto a diante neste capítulo, chamado de poda por custo-complexidade (*cost-complexity pruning*).

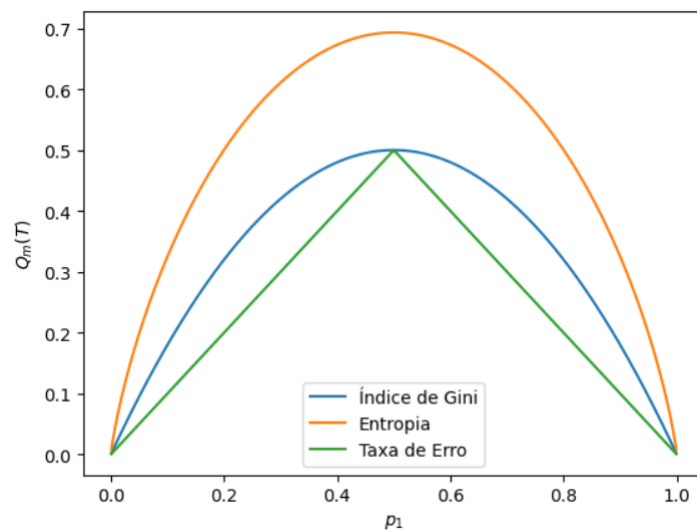


Figura 7-4: Comparação entre diferentes métricas de impureza no caso em que temos duas classes e proporções p_1 e p_2 para as classes 1 e 2.

Complexidade de treinamento de uma árvore: $O(n \log n)$

Observação: Maximização do Ganho de Informação

Algumas pessoas dizem a árvore de decisão maximiza o ganho de informação. Por mais que pareça estranho, a afirmação não está errada. Como apresentado na Seção B.1.7, ganho de informação é sinônimo de informação mútua. Se considerarmos um determinado nó, Y é a variável que representa a classe de um dado pertencente a esse nó. Ao ser dividido em dois nós filhos, cada dado será direcionado para a direita ou para a esquerda com base no par (X_j, s) encontrado da seguinte forma:

$$(X_j, s) = \arg \min_{X_j, s} \frac{N_{m_E} Q_{m_D}(T) + N_{m_D} Q_{m_D}(T)}{N_m}.$$

Se a métrica de impureza da árvore for a entropia então teremos

$$(X_j, s) = \arg \min_{X_j, s} \frac{N_{m_E} H(Y_e) + N_{m_D} H(Y_d)}{N_m},$$

em que Y_e e Y_d são variáveis aleatórias que representam a classe de cada dado no nó filho da esquerda e da direita, respectivamente, e $H(\cdot)$ é a entropia. Se considerarmos ainda que Z é uma variável aleatória que indica se um dado presente no nó pai foi direcionado para o nó filho da esquerda ($Z = 0$) ou da direita ($Z = 1$), podemos dizer que $\frac{N_{m_E}}{N_m} = \mathbb{P}(Z = 0)$ e $\frac{N_{m_D}}{N_m} = \mathbb{P}(Z = 1)$. Além disso, a partir da definição de entropia, podemos escrever

$$H(Y_e) = - \sum_{y_e} \mathbb{P}(Y_e = y_e) \log \mathbb{P}(Y_e = y_e) = - \sum_y \mathbb{P}(Y = y|Z = 0) \log \mathbb{P}(Y = y|Z = 0)$$

e

$$H(Y_d) = - \sum_{y_d} \mathbb{P}(Y_d = y_d) \log \mathbb{P}(Y_d = y_d) = - \sum_y \mathbb{P}(Y = y|Z = 1) \log \mathbb{P}(Y = y|Z = 1).$$

Portanto, para encontrarmos (X_j, s) , o que estamos minimizando é

$$\begin{aligned} & \arg \min_{X_j, s} \left[-\mathbb{P}(Z = 0) \sum_y \mathbb{P}(Y = y|Z = 0) \log \mathbb{P}(Y = y|Z = 0) \right. \\ & \quad \left. - \mathbb{P}(Z = 1) \sum_y \mathbb{P}(Y = y|Z = 1) \log \mathbb{P}(Y = y|Z = 1) \right] \\ & \Rightarrow \arg \min_{X_j, s} \left[- \sum_z \sum_y \mathbb{P}(Z = z) \mathbb{P}(Y = y|Z = z) \log \mathbb{P}(Y = y|Z = z) \right] \\ & \Rightarrow \arg \min_{X_j, s} \left[- \sum_z \sum_y \mathbb{P}(Z = z, Y = y) \log \mathbb{P}(Y = y|Z = z) \right] \\ & \Rightarrow \arg \min_{X_j, s} H(Y|Z). \end{aligned}$$

Perceba que, o que muda ao testarmos diferentes pares (X_j, s) é a distribuição de Z , ou seja, os dados mandados para a esquerda ($Z = 0$) e os mandados para a direita ($Z = 1$).

Logo, o que estamos fazendo é minimizar a entropia condicional $H(Y|Z)$. Como a distribuição de Y não varia com (X_j, s) , temos que

$$\arg \min_{X_j, s} H(Y|Z) \equiv \arg \max_{X_j, s} -H(Y|Z) \equiv \arg \max_{X_j, s} H(Y) - H(Y|Z) \equiv \arg \max_{X_j, s} I(Y; Z).$$

Assim, vemos que de fato estamos maximizando o ganho de informação ou a informação mútua entre Y e Z .

7.2 ÁRVORE DE REGRESSÃO

Em uma árvore de regressão, a forma como o modelo realiza a predição segue uma lógica similar a da árvore de classificação. Na última, a cada nó folha, era atribuída uma constante k representando uma das classes do problema. No caso da regressão, atribuímos uma constante

c_m que representará o valor da predição para qualquer ponto pertencente à região R_m do nó folha m . Para realizar a escolha do valor de c_m , podemos utilizar como critério a minimização do MSE dentro do nó em questão. Ao realizarmos a minimização dessa expressão com respeito a c_m , vemos que o valor ótimo para essa constante deve ser a média amostral dos valores de target, ou seja,

$$\hat{c}_m = \frac{1}{N_m} \sum_{n|x_n \in R_m} t_n.$$

O treinamento de uma árvore de regressão também segue uma lógica muito parecida com a da árvore de classificação, porém, como a natureza do problema é diferente é necessário fazer outra escolha para $Q_m(T)$. Como a saída do modelo não é mais categórica, não podemos utilizar as métricas de impureza apresentadas anteriormente. Uma opção muito comum é utilizar o MSE, como mostrado abaixo:

$$Q_m(T) = \frac{1}{N_m} \sum_{n|x_n \in R_m} [t_n - f(x_n)]^2,$$

em que $f(x_n)$ é o valor predito pela árvore para a entrada x_n .

Com a exceção dessas diferenças, todo o funcionamento da árvore de regressão é similar ao da de classificação. À medida que vamos construindo a árvore, também utilizamos a eq. (7.1). Além disso, também podemos utilizar as mesmas estratégias de poda para controlar a complexidade da árvore.

A Figura 7-3 ilustra exemplos de resultados de uma regressão usando uma árvore de decisão para um problema de apenas uma variável. Verificar que, quanto maior o valor do hiperparâmetros `max_depth`, mais ajustada aos dados é a curva de regressão.

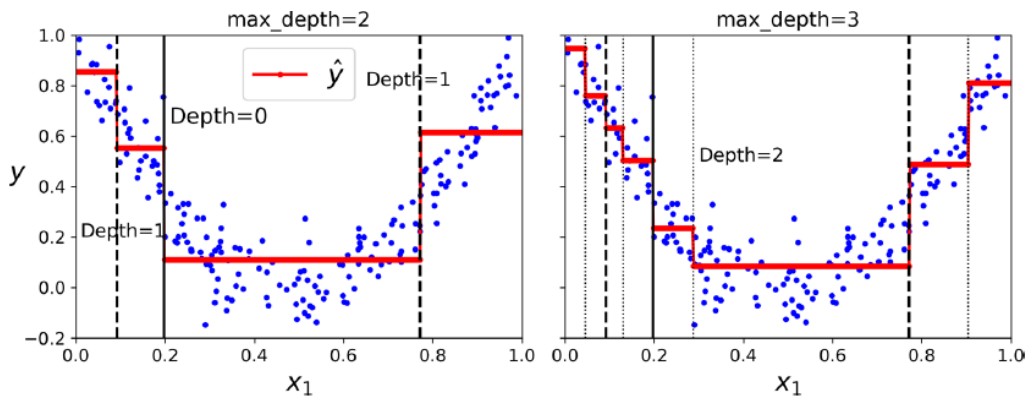


Figura 7-5: Dois exemplos de regressão realizadas nos dados de treinamento com uma árvore de decisão usando dois valores diferentes para o hiperparâmetros “profundidade máxima” (`max_depth`). A imagem foi retirada de [17].

7.3 REGULARIZAÇÃO

Uma questão sobre o treinamento de uma árvore de decisão que ainda não foi abordada é: quando sabemos que é hora de parar de crescer a árvore? Deixamo-la crescer até que haja apenas uma instância por nó? Até poderíamos fazer isso, porém, isso fariam com que essa árvore sofresse de *overfitting*. Uma solução para evitar esse problema é realizar a poda da árvore usando uma função custo que leva em consideração a complexidade da árvore. Esse método é usualmente chamado de *cost-complexity pruning*. Para aplica-lo, deixamos a árvore

T_0 crescer até atingir um determinado número mínimo de instâncias por nó. Após isso, procuramos subárvores $T_{sub} \subset T_0$ que minimizem a função custo

$$C_\alpha(T) = \sum_{m=1}^{|T|} N_m Q_m(T) + \alpha |T|,$$

em que $|T|$ é o número de folhas da árvore T .

Com isso, garantimos um *trade-off* entre a pureza dos nós terminais da árvore (primeira parcela) e a complexidade dela (segunda parcela). Quanto maior o valor de α , maior é o efeito da regularização e menor será o tamanho da árvore T_{sub} . Quando temos $\alpha = 0$, não há regularização. Uma maneira de procurar a melhor subárvore é utilizar o método weakest link pruning (poda do elo mais fraco) [2, p. 308].

Além da poda, existem outros artifícios que podem ser usados para controlar a complexidade de uma árvore de decisão. Os mais comuns consistem em interromper o crescimento da árvore seguindo algum critério de parada. Alguns desses critérios são listados abaixo [3]:

- profundidade máxima da árvore;
- número mínimo de instâncias que deve haver em um nó folha após a divisão;
- número mínimo de instâncias necessárias para dividir um nó folha;
- sendo $\Delta := Q_m(T) - \left(\frac{N_{mE} Q_{mE}(T) + N_{mD} Q_{mD}(T)}{N_m} \right)$ a diminuição de impureza ao se dividir o nó m , se Δ for menor do que um determinado limiar, não dividimos mais o nó;

Capítulo 8: BOOSTING

Boosting é uma estratégia de *ensemble* que constrói, iterativamente, um modelo composto a partir de vários modelos base de baixa complexidade, também chamados de modelos fracos [3]. Em cada iteração, treina-se, sequencialmente, um novo modelo fraco com o objetivo de minimizar o valor da função custo do modelo composto. Assim, no final do treinamento de cada modelo fraco, temos um modelo de *boosting* cuja saída corresponde à soma ponderada das saídas de cada modelo fraco [7].

A Figura 8-1 mostra como é feito o processo de predição em um modelo genérico de *boosting* para uma entrada x . Observar que a saída de cada bloco de *weak learner* é $\phi_k(x; \theta_k)$, que é função que representa o modelo fraco, sendo θ_k o parâmetro associado ao *weak learner* k . Também é possível observar que a saída de cada *weak learner* é ponderada por um peso a_k . A saída do modelo de *boosting*, portanto, é dada por $F(x) = a_1\phi_1(x; \theta_1) + a_2\phi_2(x; \theta_2) + a_3\phi_3(x; \theta_3)$.

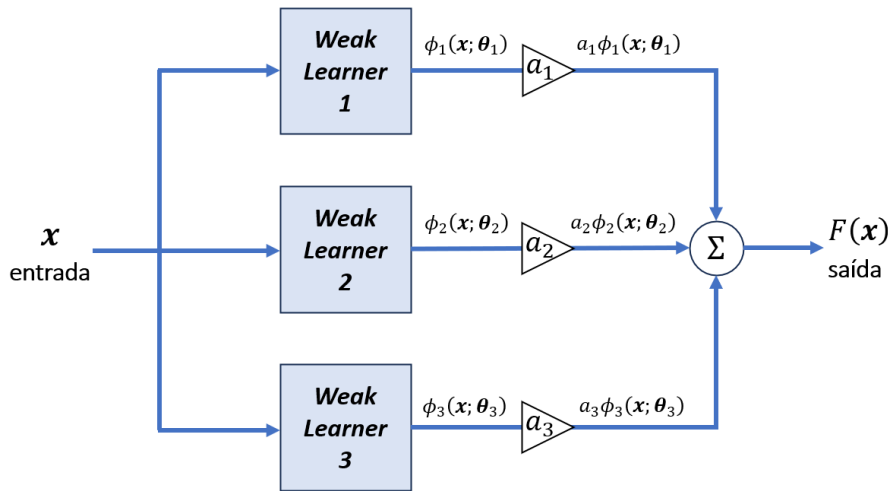


Figura 8-1: Modelo de boosting genérico com 3 weak learners sendo usado na inferência.

A seguir, são mostrados alguns dos principais modelos de *boosting*.

8.1 ADABOOST

8.1.1 Descrição do Algoritmo para Classificação

Para definir o AdaBoost (*adaptive boosting*), focamos na tarefa de classificação binária. O objetivo é construir um classificador do tipo

$$f(x) = \text{sgn}\{F(x)\}, \quad (8.1)$$

em que x é uma amostra genérica de um conjunto de dados $\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$, com $t_n \in \{-1, 1\}$, e $F(x)$ representa o modelo final de *boosting*, dado por

$$F(x) := \sum_{k=1}^K a_k \phi(x; \theta_k),$$

em que a_k são coeficientes que permitem atribuir importância diferente a modelos diferentes e $\phi(\mathbf{x}; \boldsymbol{\theta}_k) \in \{-1, 1\}$ representa o modelo base (binário) da iteração k , caracterizado em termos de um conjunto de parâmetros $\boldsymbol{\theta}_k$ (vide Figura 8-1). A cada iteração, esses parâmetros são otimizados de maneira gulosa, ou seja, para cada iteração k , encontramos os valores ótimos dos parâmetros $\boldsymbol{\theta}_k$ considerando que os parâmetros $\boldsymbol{\theta}_i, i < k$ de outras iterações são fixos.

Para começarmos a detalhar a construção desse modelo, em uma certa iteração i , podemos escrever a soma parcial dos termos, de maneira iterativa, como

$$F_i(\cdot) = \sum_{k=1}^i a_k \phi(\cdot; \boldsymbol{\theta}_k),$$

ou, de maneira recursiva, como

$$F_i(\cdot) = F_{i-1}(\cdot) + a_i \phi(\cdot; \boldsymbol{\theta}_i), \quad i = 1, 2, \dots, K.$$

Em $i = 1$, temos $F_i(\cdot) = a_i \phi(\cdot; \boldsymbol{\theta}_i)$.

Como o algoritmo é executado de maneira gulosa, os termos das iterações anteriores, como F_{i-1} , são considerados conhecidos.

Para encontrar os valores de a_i e $\boldsymbol{\theta}_i$, é realizada uma otimização empregando a **loss exponencial**, dada por $L(t, F(\mathbf{x})) = e^{-tF(\mathbf{x})}$, que é justamente um dos elementos que caracteriza o AdaBoost.

Se repararmos na Figura 8-2, tanto nos casos em que a classificação foi correta ($tF(\mathbf{x}) > 0$), quanto nos casos em que ela foi incorreta ($tF(\mathbf{x}) < 0$), a função *loss* tem valor positivo, diferentemente da *loss* 0-1, que vale 1 quando a classificação é incorreta e 0 quando for correta. Mesmo assim, na *loss* exponencial, as classificações erradas levam a uma *loss* maior do que as certas. Além disso, a *loss* exponencial é diferenciável, enquanto a *loss* 0-1 não é.

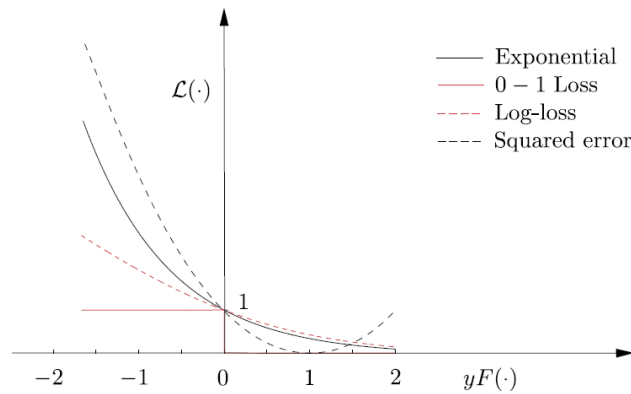


Figura 8-2: Comparação de diferentes funções de *loss* (retirado de [7]).

O problema de minimizar a função custo com a *loss* exponencial é dado por

$$(a_i, \boldsymbol{\theta}_i) = \arg \min_{a, \boldsymbol{\theta}} \sum_{n=1}^N e^{-t_n(F_{i-1}(x_n) + a\phi(x_n; \boldsymbol{\theta}))}, \quad (8.2)$$

$$(a_i, \theta_i) = \arg \min_{a, \theta} \sum_{n=1}^N w_n^{(i)} e^{-t_n a \phi(x_n; \theta)} \quad (8.3)$$

em que

$$w_n^{(i)} := \begin{cases} \frac{e^{-t_n F_{i-1}(x_n)}}{Z_{i-1}} = \frac{L(t_n, F_{i-1}(x_n))}{Z_{i-1}}, & i > 1 \\ \frac{1}{N}, & i = 1 \end{cases}, \quad (8.4)$$

com $Z_{i-1} = \sum_{n=1}^N e^{-t_n F_{i-1}(x_n)}$ sendo um fator de normalização para garantir que $w_n^{(i)} \in [0, 1]$ e

$$\sum_{n=1}^N w_n = 1. \quad (8.5)$$

Reparar que a inclusão do fator de normalização não altera o resultado da eq. (8.2), pois, como Z_{i-1} é constante para um dado i ,

$$\arg \min_{a, \theta} \sum_{n=1}^N e^{-t_n (F_{i-1}(x_n) + a \phi(x_n; \theta))} = \arg \min_{a, \theta} \frac{1}{Z_{i-1}} \sum_{n=1}^N e^{-t_n (F_{i-1}(x_n) + a \phi(x_n; \theta))}.$$

A eq. (8.3) ainda pode ser reescrita como

$$(a_i, \theta_i) = \arg \min_{a, \theta} \sum_{n=1}^N w_n^{(i)} L(t_n, a \phi(x_n; \theta)). \quad (8.6)$$

Com essa nova formulação, fica mais claro de ver que, como $w_n^{(i)}$ só depende da amostra de índice n e multiplica diretamente a *loss* do último *weak learner* quando avaliado em x_n , ele pode ser interpretado como o **peso associado à amostra n** . Para valores elevados de w_n , a parcela de erro associada à observação n torna-se mais relevante, o que faz com que seja dada mais importância a otimizar essa parcela ou, de maneira mais simplificada, ao aprendizado da observação n associada a essa parcela. Além disso, como $w_n^{(i)}$ é diretamente proporcional à *loss* do modelo até a iteração anterior, $w_n^{(i)}$ tende a ser mais elevado quanto maior o erro do modelo na iteração anterior para a observação n . Portanto, isso faz com que observações que recebem classificações muito erradas pelo modelo na iteração anterior recebam mais atenção na iteração atual. O contrário também pode ser dito: observações com erro muito baixo na iteração $i - 1$ tendem a gerar um peso que dá menos importância para essa mesma observação na iteração i . Por fim, é importante ressaltar que, como na iteração $i = 1$ não temos um modelo anterior $F_{i-1}(x_n)$, inicializamos os valores de $w_n^{(i)}$ em $1/N$, pois assim, garantimos que, inicialmente, todas as amostras tenham o mesmo peso e que a eq. (8.5) é válida.

Realizamos a otimização da eq.(8.5), primeiramente, com relação a θ , mantendo a constante:

$$\theta_i = \arg \min_{\theta} \sum_{n=1}^N w_n^{(i)} e^{-t_n a \phi(x_n; \theta)} \quad (8.7)$$

$$\theta_i = \arg \min_{\theta} P_i, \quad (8.8)$$

em que

$$P_i := \sum_{n=1}^N w_n^{(i)} \mathbb{1}_{\{\leq 0\}}(-t_n \phi(x_n; \theta)). \quad (8.9)$$

Como a é constante e $t_n \phi(x_n; \theta) \in \{-1, 1\}$, então:

- quando $t_n \phi(x_n; \theta) = 1$, temos
 - $e^{-t_n a \phi(x_n; \theta)} = e^{-a}$;
 - $\mathbb{1}_{\{\leq 0\}}(-t_n \phi(x_n; \theta)) = 0$;
- quando $t_n \phi(x_n; \theta) = -1$, temos
 - $e^{-t_n a \phi(x_n; \theta)} = e^a$;
 - $\mathbb{1}_{\{\leq 0\}}(-t_n \phi(x_n; \theta)) = 1$.

Logo, fica claro que, de fato, o valor de θ que minimiza a eq. (8.7) também minimiza a eq. (8.9). Com isso, vemos que, **apenas os pontos classificados incorretamente contribuem para o cálculo de θ_i** . Portanto,

$$P_i = \sum_{t_n \phi(x_n; \theta) = -1} w_n^{(i)}. \quad (8.10)$$

$$(8.11)$$

Reparar que $P_i \in [0, 1]$, pois, os pesos $w_n^{(i)}$ são normalizados.

Passamos para a otimização de a :

$$a_i = \arg \min_a \sum_{n=1}^N w_n^{(i)} e^{-t_n a \phi(x_n; \theta)} = \arg \min_a \sum_{t_n \phi(x_n; \theta) = -1} w_n^{(i)} e^a + \sum_{t_n \phi(x_n; \theta) = 1} w_n^{(i)} e^{-a}$$

A partir das equações (8.5) e (8.11), podemos escrever

$$a_i = \arg \min_a P_i e^a + (1 - P_i) e^{-a}.$$

Ao derivarmos $P_i e^a + (1 - P_i) e^{-a}$ com relação a a e igualar a zero, obtemos

$$a_i = \frac{1}{2} \ln \left(\frac{1 - P_i}{P_i} \right) \quad (8.12)$$

Notar que, quando $P_i < 0,5$, $a_i > 0$, o que é esperado na prática.

Conhecendo os valores de θ_i e a_i , podemos determinar $F_i(x)$ e passar para a próxima iteração.

Com isso, podemos fazer algumas observações importantes:

1. Percebemos que os pesos $w_n^{(i)}$ assumem valores maiores nas iterações seguintes para as amostras classificados incorretamente e valores menores caso contrário, pois, $w_n^{(i+1)} =$

$$\frac{e^{-t_n F_i(x_n)}}{Z_i} = \frac{e^{-t_n [F_{i-1}(x_n) + a_i \phi(x_n; \theta_i)]}}{Z_i} = \frac{Z_{i-1} w_n^{(i)} e^{-a_i \phi(x_n; \theta_i) t_n}}{Z_i} \equiv w_n^{(i)} e^{-a_i \phi(x_n; \theta_i) t_n}$$
 (lembrando que as constantes Z_i e Z_{i-1} desaparecem dentro do argmin da eq. (8.6)). Logo, vemos que $w_n^{(i+1)}$ é igual ao peso $w_n^{(i)}$ multiplicado por um fator maior que 1 quando a amostra é classificada corretamente e multiplicado por um fator entre 0 e 1 caso contrário.

2. Vemos também que o termo P_i representa a **taxa de erro ponderada** do modelo base da iteração i (o que é mais facilmente visto pela eq. (8.9)). Quando P_i aumenta, a_i diminui. Logo, modelos que têm menor taxa de erro ponderada recebem maior peso a_i durante a predição conjunta dos diferentes modelos fracos.
3. O processo de encontrar θ_i em cada iteração consiste em escolher θ_i que cria um modelo fraco que prioriza classificar corretamente as observações com maiores pesos. A soma dos pesos das amostras erradas deve ser a menor possível.

A Figura 8-3 ilustra o treinamento de um modelo AdaBoost de maneira simplificada, mostrando que o treinamento de cada modelo fraco depende dos dados de treinamento, cujas entradas estão associadas ao vetor de pesos $w^{(i)}$, que, por sua vez, depende do resultado dos modelos fracos anteriores. Já a Figura 8-4 mostra em mais detalhes o passo a passo executado no treinamento e predição do algoritmo AdaBoost.

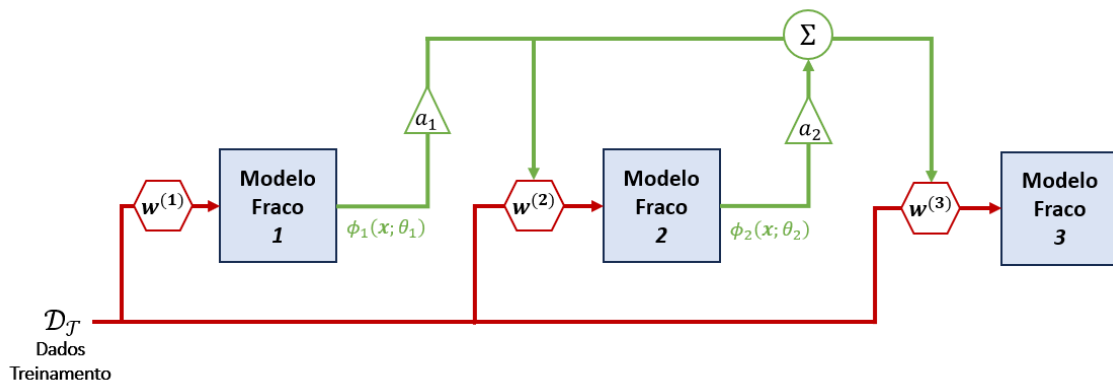
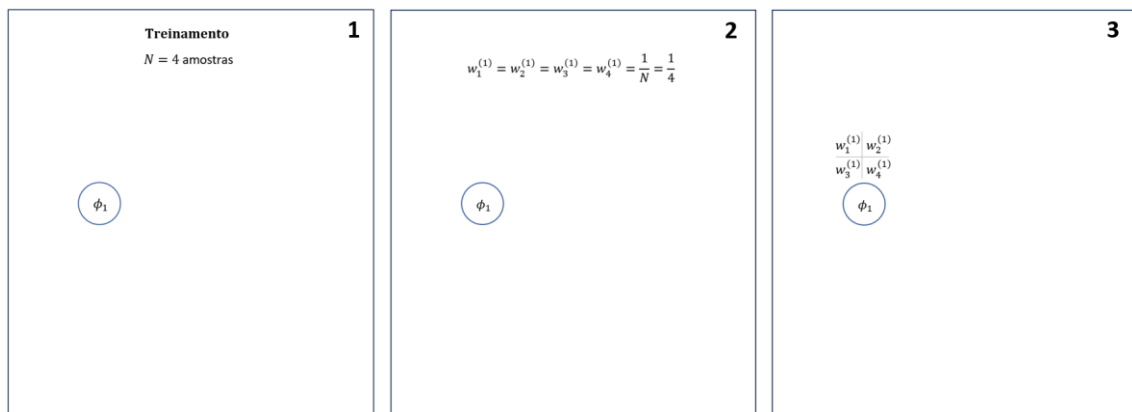


Figura 8-3: Diagrama resumindo o treinamento de um modelo AdaBoost.



4

$$(a_1, \theta_1) = \arg \min_{a, \theta} \sum_{n=1}^N w_n^{(1)} e^{-t_n a \phi(x_n; \theta)}$$

$\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1

5

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

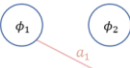
 ϕ_1

6

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1 ϕ_2

7

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1 ϕ_2
 $F_1 = a_1 \phi(x_n; \theta_1)$

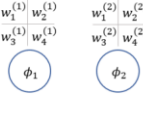
8

$$w_n^{(2)} = \frac{e^{-t_n F_1(x_n)}}{Z_1}$$

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

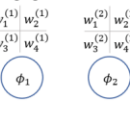
 ϕ_1 ϕ_2
 $F_1 = a_1 \phi(x_n; \theta_1)$

9

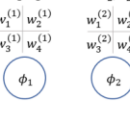
θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1 ϕ_2

10

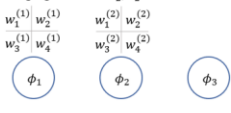
$$(a_2, \theta_2) = \arg \min_{a, \theta} \sum_{n=1}^N w_n^{(2)} e^{-t_n a \phi(x_n; \theta)}$$

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1 ϕ_2

11

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1 ϕ_2

12

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1 ϕ_2 ϕ_3

13

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1 ϕ_2 ϕ_3
 $F_2 = a_1 \phi(x_n; \theta_1) + a_2 \phi(x_n; \theta_2)$

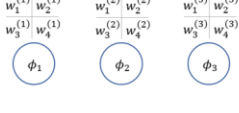
14

$$w_n^{(3)} = \frac{e^{-t_n F_2(x_n)}}{Z_1}$$

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1 ϕ_2 ϕ_3
 $F_2 = a_1 \phi(x_n; \theta_1) + a_2 \phi(x_n; \theta_2)$

15

θ_1, a_1
 $\begin{matrix} w_1^{(1)} & w_2^{(1)} \\ w_3^{(1)} & w_4^{(1)} \end{matrix}$

 ϕ_1 ϕ_2 ϕ_3

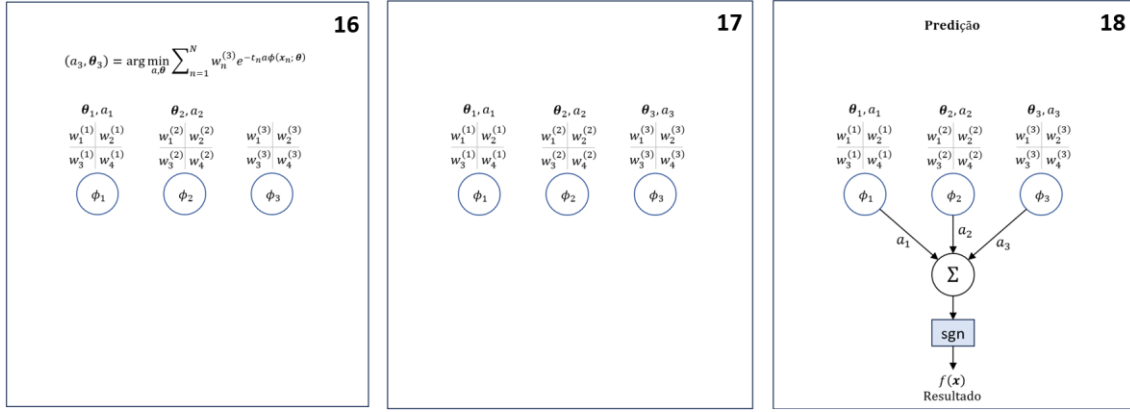


Figura 8-4: Diagrama ilustrativo do AdaBoost. Cada círculo representa um weak learner.

Uma discussão importante é que, por mais que θ_i seja determinado por

$$\theta_i = \arg \min_{\theta} P_i = \arg \min_{\theta} \sum_{t_n \phi(x_n; \theta) = -1} w_n^{(i)},$$

como encontrar na prática esse conjunto de hiperparâmetros θ_i que dá origem a um modelo que faz classificações de forma a minimizar a taxa de erro ponderada? Na prática, é inviável testar todas as combinações de hiperparâmetros e escolher a que minimiza a taxa de erro ponderada. No entanto, para alcançarmos esse objetivo, normalmente, treinamos o modelo fraco ponderando cada amostra dada a ele. Por exemplo, no caso de uma árvore de decisão, todo o modelo é baseado em contagens do número de observações nos nós. Assim, para dar pesos diferentes a cada observação, não é feita uma contagem simples, mas sim, a soma dos pesos de cada amostra. Se temos N_A observações da classe A , em vez de usarmos N_A para nossos cálculos, usamos N_{A_w} , que é a soma de todos w_n das observações da classe A . Para outros tipos de modelos fracos a lógica é parecida: utilizamos os mecanismos específicos desses modelos para atribuir pesos diferentes a cada observação.

8.1.2 Discussão sobre a *loss function*

A escolha da *loss* exponencial para o modelo *AdaBoost*, descrito na Subseção 8.1.1, e a forma como a função de classificação $f(x)$ da eq. (8.1) é definida não são arbitrárias. Para entender isso, olhemos para o valor esperado da *loss* exponencial com relação à variável aleatória T (seguindo uma distribuição qualquer) que representa a classe de um determinado ponto x :

$$\mathbb{E}_T[e^{-TF(x)}] = \mathbb{P}(T = 1|x)e^{-F(x)} + \mathbb{P}(T = -1|x)e^{F(x)}. \quad (8.13)$$

Se quisermos, em média, minimizar o valor da função *loss*, precisamos descobrir o valor $F(x) = F_*(x)$ que leva a função *loss* ao mínimo. Logo,

$$F_*(x) = \arg \min_{F(x)} \mathbb{E}[e^{-TF(x)}] = \frac{1}{2} \ln \frac{\mathbb{P}(T = 1|x)}{\mathbb{P}(T = -1|x)}$$

$$F_*(x) = \frac{1}{2} \text{logit}(\mathbb{P}(T = 1|x)) \quad (8.14)$$

Aqui, foi omitido o parâmetro θ que caracteriza a função $F(x)$, apenas por simplicidade.

Com isso, vemos que, ao utilizar a *loss* exponencial, o valor esperado dessa *loss* é minimizado quando a função $F(x)$ é proporcional a uma função logit. Isso quer dizer que, quando $\mathbb{P}(T = 1|x) > \mathbb{P}(T = -1|x)$, $F(x) > 0$ e, caso contrário, $F(x) < 0$ (vide Figura 8-5). Portanto, ao adotar a função de classificação $f(x) = \text{sgn}(F(x))$, estamos classificando o ponto x com base na classe que tem maior probabilidade a posteriori, o que está de acordo com a teoria estatística da decisão.

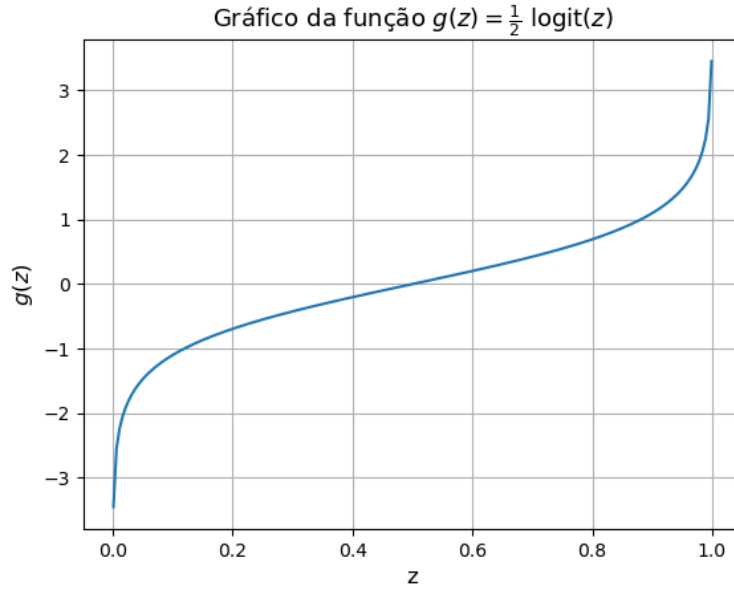


Figura 8-5: Curva da função logit multiplicada por 1/2.

Como isso se relaciona com o desenvolvimento da Subseção 8.1.1? Repare que a função custo minimizada na eq. (8.2) é justamente a média amostral $\bar{L}$ da *loss* exponencial, que é um estimador natural para $\mathbb{E}_T[e^{-TF(x)}]$:

$$\arg \min_{a, \theta} \sum_{n=1}^N e^{-t_n F(x_n; \theta)} = \arg \min_{a, \theta} \frac{1}{N} \sum_{n=1}^N e^{-t_n F(x_n; \theta)} = \arg \min_{a, \theta} \bar{L}.$$

Logo, o desenvolvimento feito na Subseção 8.1.1 está de acordo com a teoria estatística da decisão, pois estamos garantindo que os valores de a e de θ encontrados são tais que $F(x; \theta)$ é dado pela equação (8.14), o que garante que $f(x)$ será sempre igual à classe com maior probabilidade a posteriori.

No entanto, como visto anteriormente, a *loss* exponencial penaliza demais pontos errados, principalmente aqueles cuja margem $m_x := |tF(x)|$ tem valor muito elevado, ou seja, quando x está muito distante da superfície de decisão, localizada em $xF(x) = 0$. A margem costuma ter valor muito elevado quando um determinado ponto x_n é um outlier, ou seja, quando ele tem características tão diferentes de outros pontos de mesma classe que ele se torna tão difícil de acertar que muitos modelos fracos do *boosting* erram sua classificação. Isso faz com que $F(x)$ fique muito elevado, já que $F(x)$ é a combinação linear de vários modelos fracos $\phi \in \{-1, 1\}$. Por exemplo, se estivermos na iteração 5, com pesos $a_1 = 2$, $a_2 = 1$, $a_3 = 1,5$, $a_4 = 8$ (o peso a_5 ainda não foi determinado), e apenas o segundo modelo fraco da sequência acertou a classificação de x_n , com $t_n = -1$, então teríamos $F_4(x_n) = a_1 - a_2 + a_3 + a_4 = 10,5$ e $tF(x_n) = -4,5$. Logo, a *loss* seria $e^{-tF(x_n)} = e^{10,5}$, o que pode impactar significativamente a função custo final. Assim, percebemos que *outliers* tendem a ter um

impacto muito grande na função custo, já que ela tem crescimento exponencial, o que impacta o processo de otimização. Logo, na presença de *outliers* a *loss* exponencial não costuma ser a mais adequada.

Nesses casos, podemos utilizar como alternativa a **log-loss**, definida como

$$L_{\log} := - \left[\frac{1+t}{2} \log(\hat{p}_{\theta}) + \frac{1-t}{2} \log(1 - \hat{p}_{\theta}) \right], \quad (8.15)$$

em que $\hat{p}_{\theta} = \mathbb{P}(T = 1 | \mathbf{x}; \theta)$.

Se definirmos

$$\hat{p}_{\theta} = \frac{1}{1 + e^{-2F(\mathbf{x})}} = \sigma(2F(\mathbf{x})), \quad (8.16)$$

o valor de $\hat{p}_{\theta}$ que minimiza a *loss* da eq. (8.15) é tal que $F(\mathbf{x}) = \frac{1}{2} \text{logit}(\mathbb{P}(T = 1 | \mathbf{x}))$, pois, ao substituir essa expressão de $F(\mathbf{x})$ na eq. (8.16), encontramos

$\hat{p}_{\theta} = \sigma(\text{logit}(\mathbb{P}(T = 1 | \mathbf{x}))) = \mathbb{P}(T = 1 | \mathbf{x})$. Com isso, percebemos que a escolha de $\hat{p}_{\theta}$ da eq. (8.16) é feita de forma que o valor de $F(\mathbf{x})$ que minimiza a eq. (8.15) seja o mesmo que minimiza a eq. (8.13), ou seja,

$$\arg \min_{F(\mathbf{x})} L_{\log} = \arg \min_{F(\mathbf{x})} \mathbb{E}[e^{-TF(\mathbf{x})}].$$

Com isso, mesmo utilizando a *log-loss* ainda estaremos seguindo a teoria estatística da decisão ao adotarmos $f(\mathbf{x})$ como regra de decisão.

Usando a eq. (8.16), podemos reescrever a *loss* da eq. (8.15) em sua forma mais usual:

$$L_{\log} = \mathcal{L}(t, F(\mathbf{x})) = \ln(1 + e^{-2tF(\mathbf{x})}). \quad (8.17)$$

Como pode ser visto na Figura 8-2, essa *loss* apresenta valores mais balanceados para diferentes valores de $tF(\mathbf{x})$, diminuindo a atribuição de pesos excessivamente grandes para *outliers*. No então, ao utilizar essa função, a otimização fica muito complicada e, por isso, é recomendado utilizar métodos como gradiente descendente ou baseados no método de Newton [21]. Também é importante notar que, **ao utilizar a log-loss, não podemos mais chamar o algoritmo de AdaBoost**, já que o que o caracteriza é a utilização da *loss* exponencial.

8.1.3 Adaboost para Regressão

Nas subseções anteriores, foi descrito como o modelo Adaboost é construído para ser usado na tarefa de classificação. No entanto, esse modelo também pode ser usado para regressão, mas fazendo algumas pequenas modificações. A função f que define a regra de decisão do modelo é dada por

$$f(\mathbf{x}) := F(\mathbf{x}),$$

em que

$$F(\mathbf{x}) := \sum_{k=1}^K a_k \phi(\mathbf{x}; \theta_k),$$

ou seja, $F(\mathbf{x})$ para regressão tem a mesma forma que a $F(\mathbf{x})$ para classificação. A diferença é que, no caso da regressão, o modelo fraco ϕ é de regressão e assume valores contínuos e não se limita aos valores $\{-1, 1\}$. Assim, vemos que a saída do modelo é a média ponderada das saídas dos *weak learners*.

Além disso, para encontrar os parâmetros a_k e θ_k do *boosting* na regressão, usamos funções de *loss* adequadas para esse tipo de problema, como a *loss* MSE ou a MAE. A título de exemplo, para o caso da *loss* MSE, e para a iteração $i \in [1, K]$ teríamos uma função custo

$$J = \sum_{n=1}^N [t_n - F_i(\mathbf{x}_n; \theta_i)]^2.$$

8.2 GRADIENT BOOSTING

Gradient boosting [2], também conhecido como GBM (*Gradient Boosting Model*) é uma técnica utilizada para aprimorar modelos de *boosting* tendo árvores de decisão como modelos fracos. Os modelos de *boosting* baseados em árvores tendem a apresentar melhor desempenho, porém, o treinamento deles da maneira tradicional torna-se muito complexo. A partir disso, surge a técnica de *gradiente boosting*.

Como não estamos mais falando de modelos fracos de maneira genérica e sim de árvores de decisão especificamente, podemos substituir o termo $\phi(\mathbf{x}; \theta)$ por $T(\mathbf{x}; \Theta)$, em que Θ é a matriz de parâmetros da árvore, dada por $(\gamma_j, R_j)_{j=1}^J$, com R_j sendo a região do espaço de *features* representada pelo nó folha de índice j e γ_j o valor associado a esse mesmo nó.

O *Gradient boosting* pode ser resumida pelo seguinte algoritmo:

Algoritmo Gradient Boosting

1. Inicializar $F_0 = \arg \min_{\gamma} \sum_{n=1}^N L(t_n, \gamma)$
2. Para $k = 1$ a K :
 - a. Para $n = 1$ a N :
Calcular $r_{nk} = - \left[\frac{\partial L(t_n, F)}{\partial F} \right]_{F=F_{k-1}}$
 - b. Treinar uma árvore $T(\mathbf{x}; \Theta_k)$ usando como conjunto de treinamento os pares $(\mathbf{x}_n, r_{nk})_{n=1}^N$.
 - c. Atualizar $F_k(\mathbf{x}) = F_{k-1} + T(\mathbf{x}, \Theta_k)$
3. Retornar $F(\mathbf{x}) := F_K(\mathbf{x}) = F_0 + \sum_{k=1}^K T(\mathbf{x}, \Theta_k)$.

De maneira resumida, o algoritmo *gradiente boosting* consiste em, a partir de um valor inicial, dado por $F_0 = \arg \min_{\gamma} \sum_{n=1}^N L(t_n, \gamma)$, encontrar as K árvores (modelo fraco) que são treinadas para estimar os valores do gradiente da *loss* a cada iteração. No fim, teremos o valor inicial F_0 , que é corrigido incrementalmente por diversas parcelas δ_k de gradiente da *loss*, ou seja, como $F_0 + \delta_1 + \delta_2 + \dots + \delta_K$.

É importante observar que alguns algoritmos ainda multiplicam o modelo fraco de cada iteração por um passo ρ_i , que pode ser constante ou calculado a cada iteração, como é feito em algoritmos estilo *steepest descent*.

8.2.1 Exemplo com a *loss* erro quadrático

Considerando o caso em que a *loss* é o erro quadrático, teríamos $F_0(\mathbf{x}) = \arg \min_{\gamma} \frac{1}{2} \sum_{i=1}^N (t_n - \gamma)^2 = \frac{1}{N} \sum_{i=1}^N t_n = \bar{t}$, ou seja, a média amostral t_n (lembrando que essa minimização é facilmente resolvida ao fazer $\frac{\partial}{\partial \gamma} \left[\frac{1}{2} \sum_{i=1}^N (t_n - \gamma)^2 \right] = 0$). Como $F_0(\mathbf{x}) = \bar{t}$ é constante, podemos interpretá-lo como sendo uma árvore de apenas um nó, que dá como saída o escalar $\bar{t}$ independentemente da entrada.

No passo 2, na primeira iteração, em que $k = 1$, calculamos os valores de gradiente da *loss*, avaliada em $F = F_0$, ou seja, $r_{n1} = t_n - F|_{F=F_0} = t_n - F_0 = t_n - \bar{t}$. Isso é equivalente ao erro de predição do modelo F_0 .

Depois de calcular r_{n1} para todos as N instâncias, treinamos uma árvore $T(\mathbf{x}; \Theta_1)$, utilizando como entradas $\mathbf{x}_n$ e como *target* r_{n1} . Por fim, adicionamos essa árvore ao modelo de *boosting*, fazendo $F_1(\mathbf{x}) = \bar{t} + T(\mathbf{x}; \Theta_1)$.

Repetimos esses procedimentos para cada k . No final, teremos um modelo de *boosting* dado por $F(\mathbf{x}) = \bar{t} + \sum_{k=1}^K T(\mathbf{x}, \Theta_k)$. Como podemos ver, o modelo resultante corresponde à média dos *targets* mais árvores que representam pequenas correções que, para cada $\mathbf{x}_n$, levam $\bar{t}$ o mais próximo possível de t_n , por meio de pequenas correções δ_k , ou seja, $t_n \approx \bar{t} + \delta_1 + \delta_2 + \dots + \delta_K$. A cada δ_k somado, o valor de t_n se aproxima mais de $\bar{t}$.

8.2.2 Derivação Matemática

O objetivo do GBM é encontrar uma função $F^* \in \mathbb{H}$, representando o modelo, que minimize o custo, representado pelo funcional $C: \text{lin}(\mathcal{F}) \rightarrow \mathbb{R}$, $f \mapsto C(f)$, em que $\mathcal{F}$ é uma classe de funções que caracteriza a hipótese base (representando algum modelo como uma árvore de decisão, por exemplo) e $\text{lin}(\mathcal{F})$ é o conjunto de todas as combinações lineares das funções de $\mathcal{F}$. Para chegar a esse objetivo, podemos adotar um procedimento análogo ao do *steepest descent*, em que um determinado parâmetro é atualizado iterativamente da seguinte maneira:

$$\theta \leftarrow \theta + \mu \mathbf{v},$$

em que $\mathbf{v} = -\nabla C(\theta)$. Nessa formulação, estamos incrementando o parâmetro θ a cada iteração (inicializado aleatoriamente), somando a ele um vetor $\mu \mathbf{v}$ na direção da maior descida de valor de $C(\theta)$, que é uma função de θ .

No caso do GBM, desejamos fazer algo similar, porém no lugar de θ temos uma função F e no lugar da função de custo $C(\theta)$ temos o funcional de custo $C(F)$. Então, desejamos fazer a cada iteração a seguinte atualização

$$F \leftarrow F + \mu f,$$

com $f \in \mathcal{F}$. Nesse caso, como f está restrita a $\mathcal{F}$, em geral, não é possível encontrar $f = -\nabla C(F)$. Por isso, procuramos f mais similar possível a $-\nabla C(F)$. Podemos usar como métrica de similaridade o produto interno, ou seja, procuramos f tal que $\langle -\nabla C(F), f \rangle$ seja o maior possível. Além de o produto interno ser uma escolha comum para medir similaridade entre funções, podemos também ver essa escolha pelo ponto de vista de derivada direcional do funcional $C(F)$ na “direção” da função f [22], dada por

$$\langle \nabla C(F), f \rangle = \lim_{\epsilon \rightarrow 0} \frac{C(F + \epsilon f) - C(F)}{\epsilon}.$$

Reparar que essa formulação da derivada direcional para um funcional é análoga a da derivada direcional de uma função f na direção $\mathbf{v}$ em que temos $D_{\mathbf{v}}f(\mathbf{x}) = \lim_{\epsilon \rightarrow 0} \frac{f(\mathbf{x} + \epsilon \mathbf{v}) - f(\mathbf{x})}{\epsilon \|\mathbf{v}\|} = \frac{1}{\|\mathbf{v}\|} \langle \nabla f, \mathbf{v} \rangle$.

Considerando uma variação $\Delta\epsilon$ suficientemente pequena, podemos escrever

$$C(F + \Delta\epsilon f) \approx C(F) + \Delta\epsilon \langle \nabla C(F), f \rangle.$$

Com isso, vemos que a maior redução no custo $C(F)$ acontece ao minimizarmos $\langle \nabla C(F), f \rangle$ (ao tornarmos esse termo o mais negativo possível), que é equivalente a maximizar $-\langle \nabla C(F), f \rangle = \langle -\nabla C(F), f \rangle$.

Se voltarmos ao algoritmo descrito no início desta seção, veremos que ele segue a lógica descrita no desenvolvimento matemático acima:

- no passo 1, definimos uma função arbitrária F_0 ;
- no passo 2, maximizamos $\langle -\nabla C(F), f \rangle$ ao encontrar uma função $f := T(\mathbf{x}; \Theta_k)$ mais similar possível ao negativo do gradiente da seguinte função custo

$$C(D) = \frac{1}{|D|} \sum_{(\mathbf{x}_n, t_n) \in D} L(t_n, F), \quad (8.18)$$

utilizando $\{(\mathbf{x}_n, \nabla C(F_{k-1}))\}_{n=1}^N$ como conjunto de treinamento para treinar a árvore $T(\mathbf{x}; \Theta_k)$;

- passo 3, atualizamos o modelo $F_k \leftarrow F_{k-1} + T(\mathbf{x}; \Theta_k)$, de maneira a reduzir o custo.

É importante reparar que, no passo 2, apesar de não parecer, a expressão de r_{nk} é o gradiente do custo da eq. (8.18). A confusão surge porque nota-se a falta do somatório na expressão de r_{nk} , que deveria aparecer, pois a derivada de um somatório (que aparece na expressão de $C(D)$) é o somatório das derivadas. No entanto, ao realizar esse cálculo com mais cuidado, temos o seguinte:

$$\begin{aligned} \frac{\partial C(D)}{\partial F(\mathbf{x})} &= \frac{\partial}{\partial F(\mathbf{x})} \left(\frac{1}{|D|} \sum_{(\mathbf{x}_n, t_n) \in D} L(t_n, F(\mathbf{x}_n)) \right) = \frac{1}{|D|} \left[\frac{\partial}{\partial F(\mathbf{x}_1)} \sum_{(\mathbf{x}_n, t_n) \in D} L(t_n, F(\mathbf{x}_n)) \right] \\ &= \frac{1}{|D|} \left[\frac{\partial}{\partial F(\mathbf{x}_1)} L(t_1, F(\mathbf{x}_1)) \right] \\ &\quad \frac{\partial}{\partial F(\mathbf{x}_2)} L(t_2, F(\mathbf{x}_2)) \dots \end{aligned}$$

Pode parecer estranha essa forma de calcular, porém, estamos calculando o gradiente de um funcional para um espaço finito de pontos, definido pelo conjunto de dados D . Logo, não é possível calcular uma expressão “fechada” para $\frac{\partial C(D)}{\partial F}$ que generaliza para qualquer ponto $\mathbf{x}$. Assim, precisamos calcular da maneira apresentada acima, ponto a ponto, como se fosse um vetor. Nesse caso, vemos que, para cada ponto $\mathbf{x}_n$, temos uma expressão para o gradiente semelhante a r_{nk} , excluindo a constante $\frac{1}{|D|}$.

Reparar que se estivéssemos fazendo uma otimização numérica tradicional, derivando com relação a um vetor de parâmetros θ , teríamos uma lógica parecida. Vamos ver abaixo, considerando a *loss* MSE:

$$\frac{\partial \mathcal{C}(D)}{\partial \theta} = \frac{\partial}{\partial \theta} \left(\frac{1}{|D|} \sum_{(x_n, t_n) \in D} (t_n - F_{\theta}(x_n))^2 \right) = \frac{1}{|D|} \left[\frac{\partial}{\partial \theta_1} \sum_{(x_n, t_n) \in D} (t_n - F_{\theta}(x_n))^2 \right. \\ \left. \frac{\partial}{\partial \theta_2} \sum_{(x_n, t_n) \in D} (t_n - F_{\theta}(x_n))^2 \dots \right]$$

Perceba que, nesse caso, ainda temos um somatório em cada componente do vetor, pois todas as parcelas do somatório dependem da variável com relação a qual estamos derivando. Por exemplo, na primeira componente, todas as parcelas do somatório dependem de θ_1 . No entanto, quando falamos da derivada com relação a $F(x_n)$, só tem uma parcela do somatório que depende de x_n .

Depois de toda essa discussão, vemos que o GBM tem uma abordagem que consiste em reduzir incrementalmente o valor da função custo, se aproximando gradualmente do mínimo.

O GBM também pode ser generalizado para qualquer tipo de modelo fraco utilizando o desenvolvimento exposto acima. Nesse caso, o algoritmo passa a ser chamado de **AnyBoost** [23].

8.3 XGBOOST

O **XGBoost** é um modelo de *boosting*, inspirado no *gradient boosting*, que tem como sua principal marca a representação da função custo usando uma **aproximação de segunda ordem de Taylor**. Por causa dessa aproximação, em sua função custo, aparece não só um termo diferencial de primeira ordem (derivada primeira da *loss*), mas também um termo diferencial de segunda ordem (derivada segunda da *loss*). Pela semelhança com uma otimização de

Além dessa função custo diferenciada, cada árvore do *boosting* tem um algoritmo muito específico. Ele é similar ao CART, mas o critério de divisão de nós é baseado na função custo do XGBoost, que possui termos diferenciais de primeira e segunda ordens. Além disso, o valor associado a cada folha também tem uma expressão específica também associados a esses termos diferenciais.

8.3.1 Função Custo

Considere a seguinte função custo

$$C_k = \sum_{n=1}^N L(t_n, F_{k-1}(x_n)) + \sum_{k=1}^K \Omega(T_k)$$

com

$$\Omega(T_k) = \gamma |T_k| + \frac{1}{2} \lambda \|\mathbf{w}_k\|^2,$$

em que $|T_k|$ é o número de folhas da árvore, $T_k := T(\Theta_k)$ e $\mathbf{w}_k = [w_{k1} \dots w_{k|T_k|}]$, sendo w_m o valor da folha que representa a região $\mathcal{R}_m$ (vide Capítulo 7:Árvore de Decisão), com $m \in [1, |T_k|]$.

Considere uma função custo a seguir, com um *loss* L , no formato da que foi utilizada para o *gradient boosting*:

$$C_k = \sum_{n=1}^N L(t_n, F_{k-1}(\mathbf{x}_n)).$$

Observe que o subíndice de C_k indica que essa função custo está associada à iteração k do *boosting* e que o segundo argumento – o valor previsto – corresponde à saída do *boosting* até a iteração $k - 1$, exatamente o que tínhamos no *gradient boosting* tradicional. Logo, fica claro que trataremos novamente de um modelo com abordagem gulosa.

O primeiro passo que diferencia o XGBoost do *gradient boosting* está no fato de o XGBoost não minimizar C_k diretamente, mas sim, sua aproximação por série de Taylor de segunda ordem. Nesse sentido, primeiramente, vamos considerar uma *loss* genérica dada por $L(t, z)$. Considerando t constante e z variável, a aproximação nas proximidades de $z = a$ é dada por

$$L(t, z) \approx L(t, a) + (z - a) \left. \frac{\partial L(t, z)}{\partial z} \right|_{z=a} + \frac{1}{2} (z - a)^2 \left. \frac{\partial^2 L(t, z)}{\partial z^2} \right|_{z=a}$$

A expressão acima pode ser considerada uma função de t , z e a :

$$\ell(t, z, a) = L(t, a) + (z - a) \left. \frac{\partial L(t, z)}{\partial z} \right|_{z=a} + \frac{1}{2} (z - a)^2 \left. \frac{\partial^2 L(t, z)}{\partial z^2} \right|_{z=a}$$

Para $t = t_n$, $z = F_{k-1}(\mathbf{x}_n) + T_k(\mathbf{x}_n)$ e $a = F_{k-1}(\mathbf{x}_n)$, podemos escrever

$$L(t_n, F_k(\mathbf{x}_n) + T_k(\mathbf{x}_n)) \approx L(t_n, F_{k-1}(\mathbf{x}_n)) + T_k(\mathbf{x}_n)g(t, \mathbf{x}_n) + \frac{1}{2} T_k^2(\mathbf{x}_n)h(t, \mathbf{x}_n),$$

em que $g(t, \mathbf{x}_n) = \left. \frac{\partial L(t, z)}{\partial z} \right|_{z=F_{k-1}(\mathbf{x}_n)}$ e $h(t, \mathbf{x}_n) = \left. \frac{\partial^2 L(t, z)}{\partial z^2} \right|_{z=F_{k-1}(\mathbf{x}_n)}$ são chamados de estatística de gradiente de primeira e segunda ordens, respectivamente.

Portanto, com essa aproximação de segundo grau nas proximidades do ponto $F_{k-1}(\mathbf{x}_n)$, a nova função custo fica

$$C_k = \sum_{n=1}^N L(t_n, F_{k-1}(\mathbf{x}_n)) + T_k(\mathbf{x}_n)g(t, \mathbf{x}_n) + \frac{1}{2} T_k^2(\mathbf{x}_n)h(t, \mathbf{x}_n). \quad (8.19)$$

Ao voltar no desenvolvimento que foi feito até aqui, poderíamos nos perguntar se a eq. (8.19) é de fato a aproximação por série de Taylor da função custo original, uma vez que aplicamos a expansão de Taylor diretamente sobre $L(t_n, F_{k-1}(\mathbf{x}_n))$ e não sobre $\sum_{n=1}^N L(t_n, F_{k-1}(\mathbf{x}_n))$. A aplicação dessa expansão diretamente sobre a função custo nos faria depender a forma da série de Taylor para múltiplas variáveis, o que poderia ser complicado. No entanto, fazendo

diretamente sobre a função custo, chegaríamos ao mesmo resultado expresso na equação (8.19).

Dando continuidade ao desenvolvimento, ao observarmos a eq. (8.19), percebemos que o termo $L(t_n, F_k(\mathbf{x}_n))$ é constante e pode ser removido do somatório (lembre-se que otimizaremos C_k com relação a T_k). Logo, a função custo fica

$$C_k = \sum_{n=1}^N T_k(\mathbf{x}_n)g(t, \mathbf{x}_n) + \frac{1}{2}T_k^2(\mathbf{x}_n)h(t, \mathbf{x}_n). \quad (8.20)$$

Repare que, o que foi feito até aqui segue exatamente a lógica do método de otimização numérica de Newton (vide Seção D.2 do apêndice), pois tínhamos uma função objetivo e encontramos uma aproximação de Taylor de segunda ordem para ela. Se fossemos seguir exatamente o método de Newton faríamos $\min_{T_k} C_k(T_k) \Rightarrow \frac{\partial C_k(T_k)}{\partial T_k} = 0$ (observar final da seção (8.22), sobre como calcular derivada com respeito a uma função num espaço de pontos finitos) e encontraríamos $T_k^*(\mathbf{x}_n) = -\frac{g(t, \mathbf{x}_n)}{h(t, \mathbf{x}_n)}$, $\forall n \in [1, N]$, ou seja, teríamos N equações formando um sistema. A árvore T_k^* que minimiza a aproximação local de segunda ordem da função custo é aquela que satisfaz todas as N equações e representaria um passo no algoritmo de Newton.

Apesar de parecer fazer sentido aplicar o método de Newton diretamente a esse problema, não é possível resolver esse problema analiticamente, pela própria natureza de T_k . Além disso, é inviável testar todas as estruturas de árvore T_k possíveis para achar a que satisfaz todas as equações.

Como a função T_k mapeia o valor $\mathbf{x}$ para um dos possíveis valores de w_k , $k \in [1, |T_k|]$, podemos reescrever a expressão do custo em função de w_k . Para isso, primeiro, expandimos o termo de regularização:

$$C_k = \gamma|T_k| + \frac{1}{2}\lambda \sum_{m=1}^{|T_k|} w_{km}^2 + \sum_{n=1}^N T_k(\mathbf{x}_n)g(t, \mathbf{x}_n) + \frac{1}{2}T_k^2(\mathbf{x}_n)h(t, \mathbf{x}_n).$$

Além disso, agruparemos tudo em um único somatório mais externo que itera ao longo dos índices m das folhas, ou seja, a otimização é feita folha a folha. Aproveitaremos para colocar o termo w_{km}^2 em evidência, já que outro termo igual a esse aparecerá por causa de $T_k^2(\mathbf{x}_n)$:

$$C_k = \gamma|T_k| + \sum_{m=1}^{|T_k|} \left[\sum_{n \in I_m} w_{km}g(t, \mathbf{x}_n) \right] + \frac{1}{2}w_{km}^2 \left\{ \left[\sum_{n \in I_m} h(t, \mathbf{x}_n) \right] + \lambda \right\} \quad (8.21)$$

em que $I_m = \{n \mid \mathbf{x}_n \in \mathcal{R}_m\}$, ou seja, é o conjunto de todos os valores de n tais que $\mathbf{x}_n$ cai na região $\mathcal{R}_m$ (correspondente à folha de índice m).

De maneira mais sucinta, podemos escrever a função custo como

$$C_k = \gamma|T_k| + \sum_{m=1}^{|T_k|} w_{km}G_m + \frac{1}{2}w_{km}^2(H_m + \lambda) \quad (8.22)$$

com $G_m = \sum_{n \in I_m} g(t, \mathbf{x}_n)$ e $H_m = \sum_{n \in I_m} h(t, \mathbf{x}_n)$.

8.3.2 Construção das Árvores

Com a expressão final do custo, representado pela eq. (8.21), podemos calcular a árvore que minimiza o custo para a iteração k . Claro, essa árvore será expressa em termos de w_{km} , que foi como escolhemos representar a expressão do custo. Para encontrar a árvore ótima, derivamos o custo com relação a w_{km} e igualamos isso a zero:

$$\frac{\partial C_k}{\partial w_{km}} = 0$$

$$G_m + w_{km}^*(H_m + \lambda) = 0$$

Observe que o somatório sumiu, pois, só uma parcela depende de w_{km} . Todas as outras parcelas dependem de w_{km} , com $m \neq m$. Logo, o valor ótimo de uma folha genérica de índice m é dado por

$$w_{km}^* = -\frac{G_m}{H_m + \lambda} \quad (8.23)$$

Com isso, temos a expressão para cada folha da árvore que minimiza a função custo e, consequentemente, temos a estrutura da árvore.

O valor mínimo da função custo é obtido ao substituir a eq. (8.23) na eq. (8.21):

$$C_k^* = \gamma |T_k| - \frac{1}{2} \sum_{m=1}^{|T_k|} \frac{G_m^2}{H_m + \lambda} \quad (8.24)$$

Perceba que, para que os valores na folha sejam descritos pela eq. (8.23), a árvore não pode ser construída usando o algoritmo CART tradicional. Portanto, a cada iteração, para construir uma árvore que minimize o valor da função custo, é necessário um algoritmo específico de árvore para XGBoost.

Além disso, perceba que seria inviável otimizar a eq. (8.24) diretamente, pois seria necessário testar um número muito grande de árvore. Portanto, assim, como no CART, o algoritmo para construção de árvores do XGBoost também é guloso e minimiza a função custo a cada divisão de nó, sendo essa divisão feita escolhendo-se um par (X, s) de *feature* e valor dessa *feature*. O par escolhido é o que maximiza o ganho

$$\Delta = \delta_P - (\delta_D + \delta_E),$$

sendo δ_P , δ_D e δ_E as contribuições para o custo final provenientes do nó pai, do nó filho da direita e do nó filho da esquerda, respectivamente.

Para determinar a contribuição de um único nó – considerando que estamos construindo a árvore que minimiza a eq. (8.22) – podemos nos aproveitar da eq. (8.24) e considerar que a árvore é composta apenas do nó m que estamos avaliando. Nesse caso, $|T_k| = 1$ e teríamos

$$\delta_m = \gamma - \frac{1}{2} \frac{G_m^2}{H_m + \lambda}.$$

Se repararmos, de fato $C_k^* = \sum_{m=1}^{|T_k|} \delta_m$.

Logo, a expressão do ganho é

$$\Delta = \frac{1}{2} \left[\frac{G_D^2}{H_D + \lambda} + \frac{G_E^2}{H_E + \lambda} - \frac{G_P^2}{H_P + \lambda} \right] - \gamma.$$

Com isso, definimos de maneira global o princípio que rege a construção de um modelo XGBoost. Recapitulando:

1. Começamos com um modelo trivial (média dos *target*, para a regressão, e valor da classe majoritária, para a classificação);
2. Para $k = 1$ a K , construímos uma árvore T_k da seguinte forma:
 - a. Para cada nó, testamos diferentes pares (X, s) para dividi-lo em dois nós filhos;
 - b. O par escolhido é aquele que gera o maior ganho Δ .
 - c. A cada folha da árvore, é atribuído o valor da eq. (8.23).
 - d. Adicionamos a árvore T_k às outras árvores do *boosting* $F_k = \sum_{k'=0}^k T_{k'}$.

A forma como é feita a busca pelos pares (X, s) , o critério de parada de divisão de nó, entre outros detalhes, pode variar dependendo da implementação [21].

8.3.3 XGBoost é um modelo de *Gradient Boosting*?

Poderíamos nos perguntar por que o modelo tem “*extreme gradient boosting*” no nome se ele não parece ser um algoritmo que dá passos na direção do gradiente negativo, ou seja, com a forma $F = \sum_k \eta \nabla C_k$. No *gradient boosting*, garantíamos que cada nova árvore adicionada era uma aproximação do gradiente da função custo, pois treinávamos cada árvore com *target* sendo igual ao valor desse gradiente. No entanto, no XGBoost, isso não é feito. Cada árvore é treinada apenas para minimizar o custo da eq. (8.22), sem garantir que ela seja de fato uma aproximação do gradiente algo parecido.

No entanto, não dá para negar a similaridade do que é feito no XGBoost com o método de Newton (vide Seção D.2 do apêndice).

De fato, cada árvore T_k individualmente não aproxima necessariamente o “passo” do algoritmo gradiente descendente, nem do método de Newton – outra possibilidade que poderíamos cogitar, já que estamos tratando não apenas do gradiente, mas também da Hessiana $h(t, x_n)$. Por outro lado, se olharmos para a otimização de cada folha individual e considerarmos o caso em que $\lambda = 0$, teremos

$$w_{km}^* = -\frac{G_m}{H_m}, \quad (8.25)$$

que nos faz lembrar o passo do método de Newton, que, para lembrança, consiste em atualizar o parâmetro θ da seguinte maneira: $\theta_{i+1} = \theta_i - \frac{f'(\theta)}{f''(\theta)}$.

De fato, se reescrevermos a expressão de G_m e H_m , encontramos

$$G_m = \sum_{n \in I_m} \left. \frac{\partial L(t, z)}{\partial z} \right|_{z=F_{k-1}(x_n)} = \frac{\partial}{\partial z} \sum_{n \in I_m} L(t, z) \Big|_{z=F_{k-1}(x_n)} = C_m'(t, F_{k-1}(x_n))$$

$$H_m = \sum_{n \in I_m} \frac{\partial L(t, z)}{\partial z} \Big|_{z=F_{k-1}(x_n)} = \frac{\partial^2}{\partial z^2} \sum_{n \in I_m} L(t, z) \Big|_{z=F_{k-1}(x_n)} = \mathcal{C}_m''(t, F_{k-1}(x_n)),$$

em que $\mathcal{C}$ é uma espécie de função custo (ou objetivo) dentro de um subconjunto I_m do conjunto de dados completo $\mathcal{D}$. Nesse sentido, reescrevendo a equação da folha como

$$w_{km}^* = -\frac{G_m}{H_m} = -\frac{\mathcal{C}'(t, F_{k-1}(x_n))}{\mathcal{C}''(t, F_{k-1}(x_n))}$$

vemos que, de fato, a expressão da folha representa um passo do algoritmo de otimização de Newton. Se incluirmos o termo λ , continuaríamos tendo um passo de Newton, só que com uma função custo com regularização.

Claro, é necessário ressaltar que apesar da semelhança, o XGBoost não é uma otimização pelo método de Newton. Cada árvore do *boosting*, não é um passo de Newton, apenas cada uma de suas folhas olhadas individualmente, considerando uma função custo $\mathcal{C}_m$ específica para a folha m , que é diferente da função custo $\mathcal{C}$ do problema principal do XGBoost. Além disso, como já foi dito, o conjunto de dados considerado é apenas um subconjunto do total de dados que temos no problema, específico para cada folha. Portanto, apesar da semelhança, não é possível dizer que o XGBoost faz uma otimização seguindo o método de Newton.

8.3.4 Exemplos com Funções Loss Específicas

Aqui, por simplicidade, teremos $\hat{t}_n = F_{n-1}(x_n)$.

8.3.4.1 MSE

$$\begin{aligned} L(t_n, \hat{t}_n) &= \frac{1}{2} [t_n - \hat{t}_n]^2 \\ g_n &= \frac{\partial L(t, z)}{\partial z} \Big|_{z=\hat{t}_n} = \hat{t}_n - t_n \\ h_n &= \frac{\partial^2 L(t, z)}{\partial z^2} \Big|_{z=\hat{t}_n} = 1 \\ w_{km} &= \frac{\sum_{n \in I_m} (t_n - \hat{t}_n)}{\lambda + |I_m|} \end{aligned}$$

Se considerarmos o caso sem regularização, teremos

$$w_{km} = \frac{\sum_{n \in I_m} (t_n - \hat{t}_n)}{|I_m|}$$

e o valor das folhas será igual à média dos resíduos. Então, nesse caso, nas folhas das árvores treinadas pelo XGBoost, temos algo muito similar ao que temos nas folhas de uma árvore CART treinada sobre os resíduos. Logo, vemos que, de fato, cada árvore adicionada ao XGBoost representa um passo de correção similar ao que acontecia no *gradient boosting* tradicional.

O termo de regularização nesse caso, na decisão dos valores das folhas, tem um papel de redução de magnitude.

O XGBoost é parecido com o *gradient boosting*, mas apresenta algumas modificações incluindo otimizações de algumas etapas e inclusão de novas etapas.

Algumas das principais diferenças relacionadas a otimização são:

- Permite paralelização no processo de construção das árvores.
- Critério de poda das árvores é profundidade máxima, em vez do valor da *loss*.
- Uso de *cache awareness* para fazer uso mais eficiente do *hardware*.

Diferenças nas etapas do algoritmo:

- Regularização LASSO e Ridge.
- *Sparsity awareness*, que permite lidar de maneira mais eficiente com dados esparsos e *missing values*.
- Mudança na forma como o *split point* das árvores é encontrado (*quantile sketch algorithm*).
- *Built-in cross-validation*.

Capítulo 9: REDE NEURAL

Nas seções sobre regressão linear e regressão logística, vimos que os modelos tinham a forma

$$y(\mathbf{x}, \boldsymbol{\theta}) = f(\boldsymbol{\theta}^T \boldsymbol{\phi}(\mathbf{x})) = f\left(\sum_{j=1}^M \theta_j \phi_j(\mathbf{x})\right),$$

com $\boldsymbol{\theta} = [\theta_M \ \theta_{M-1} \ \dots \ \theta_0]^T$ e $\mathbf{x} = [x_M \ x_{M-1} \ \dots \ x_0]^T$. Na regressão linear, tínhamos $f(x) = x$ e, na regressão logística, tínhamos uma função não-linear (mais especificamente uma sigmoide).

Na rede neural, teremos um modelo muito parecido, mas as funções base ϕ_j , agora, também dependerão de parâmetros θ ajustáveis e terão a forma abaixo:

$$\begin{aligned} y_k(\mathbf{x}, \boldsymbol{\theta}) &= f\left(\sum_{j=1}^M \theta_{kj}^{(L)} \phi_j(\mathbf{x}, \boldsymbol{\theta}_j)\right) = f\left(\sum_{j=1}^M \theta_{kj}^{(L)} h\left(\sum_{i=1}^M \theta_{ji}^{(L-1)} \phi_i(\mathbf{x}, \boldsymbol{\theta}_i)\right)\right) \\ &= f\left(\sum_{j=1}^M \theta_{kj}^{(L)} h\left(\sum_{i=1}^M \theta_{ji}^{(L-1)} g(\dots)\right)\right), \end{aligned}$$

em que f , h e g são funções não lineares. Como a rede neural pode ter K saídas, usamos o índice k para indicar a qual saída estamos nos referindo. Além disso, percebemos que a função base ϕ_j é uma composição de várias outras funções que dependem dos parâmetros ajustáveis. Utilizamos um sobrescrito nos parâmetros para indicar a qual das L **camadas** da rede neural eles estão vinculados.

Assim, vemos que a rede neural é uma versão mais complexa de uma regressão logística (no caso da classificação), em que a função base depende dos parâmetros ajustáveis e de uma estrutura recursiva de funções não lineares. Isso nos permite criar fronteiras de decisão não lineares (no espaço das entradas $\mathbf{x}$) muito mais complexas do que as da regressão logística, graças à existência de uma função base ϕ_j bastante adaptável.

9.1 ESTRUTURA DA REDE NEURAL

O elemento básico de uma rede neural é chamado de **neurônio**, como pode ser visto na Figura 9-1.

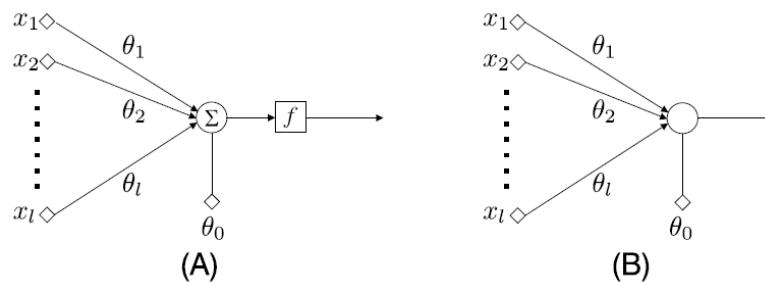


Figura 9-1: Ilustração do neurônio de uma rede neural.

Nessa imagem, podemos ver os elementos já mencionados anteriormente, nomeadamente as entradas x_i , os pesos θ_i e a função não linear f , que recebe o nome especial de **função de ativação**. Além disso, o termo θ_0 , que não está associado a nenhuma entrada recebe o nome de **bias** (ou *threshold*). A parte B dessa figura mostra que, usualmente, agrupamos o bloco de combinação linear com o da função de ativação para formar o que é chamado de **nó**. Muitas vezes, o termo nó será utilizado para se referir ao neurônio inteiro.

Juntando vários neurônios, podemos formar o que é chamado de **feed-forward network** ou **fully connected network** (FCN), como é mostrado na Figura 9-2. O nome *feed-forward network* se dá pelo fato de a informação ser unidirecional, indo das entradas x_i até camada de saída. Já o nome *fully connected network* enfatiza o fato de que cada neurônio desse tipo de rede está conectado a todos os neurônios da camada anterior.

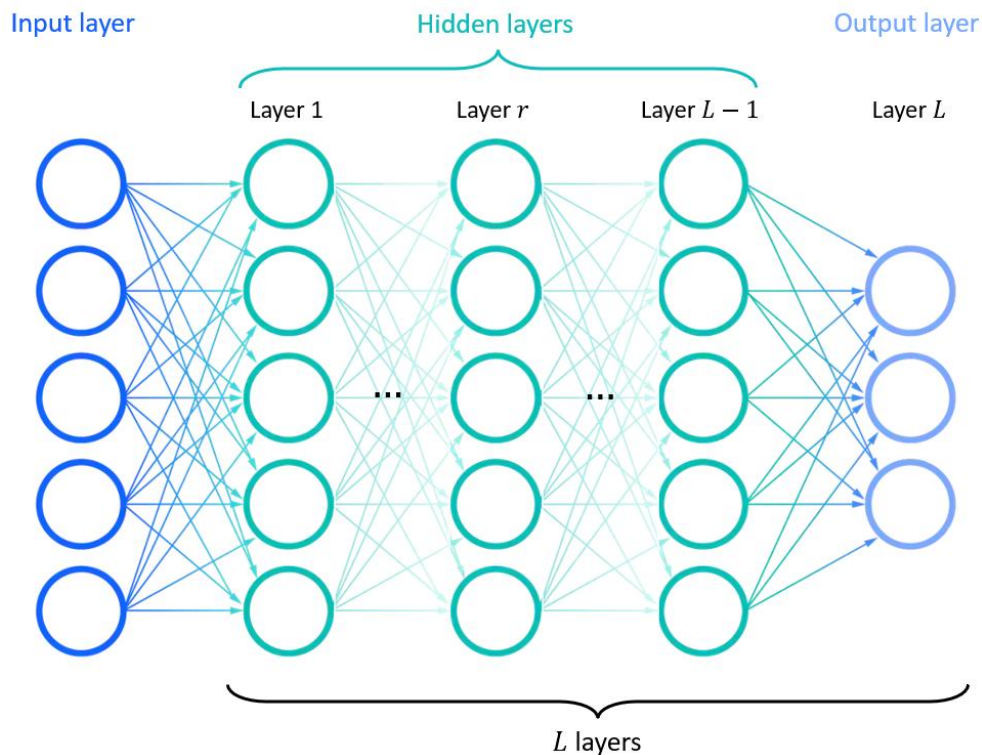


Figura 9-2: Rede neural totalmente conectada com L camadas.

Atentando-se para a questão de nomenclatura, damos os nomes de **neurônio de saída** e **neurônios escondidos** aos neurônios pertencentes às camadas de saída e camadas escondidas, respectivamente. Os nós pertencentes à camada de entrada não são neurônios, ou seja, não realizam nenhum tipo de processamento. Por causa disso, dizemos que a rede da Figura 9-2 é uma rede de L camadas, excluindo a camada de entrada da contagem. Redes com até 3 camadas (até duas camadas escondidas) são chamadas de **rasas** e redes com mais camadas são chamadas **profundas**.

Muitas pessoas também chamam essa rede de **multilayer perceptron** (MLP), por causa da origem desse modelo, que era baseado em um modelo de neurônio chamado de perceptron.

Em cada neurônio da rede neural, estamos realizando a seguinte operação:

$$y_j^r = f(z_j^r) = f(\theta_j^{rT} \mathbf{y}^{r-1}).$$

O sobrescrito r indica a qual das L camadas o neurônio pertence.

O subscrito j identifica o número do neurônio nessa camada, considerando um total de k_r neurônios.

Assim, a expressão diz que a saída y_j^r do neurônio j da camada r é igual ao produto do vetor y^{r-1} com as saídas dos neurônios da camada anterior pelo vetor de parâmetros θ_j^r , vinculado ao neurônio em questão, passado por uma função não linear $f(\cdot)$.

O vetor das saídas de todos os neurônios de uma camada r é dado por

$$\mathbf{y}^r = \begin{bmatrix} 1 \\ f(\mathbf{z}^r) \end{bmatrix} = \begin{bmatrix} 1 \\ f(\Theta^r \mathbf{y}^{r-1}) \end{bmatrix},$$

em que $\Theta^r = [\theta_1^r \quad \theta_2^r \quad \dots \quad \theta_{k_r}^r]^T$ e k_r é o número de neurônios na camada r .

Lembrando que o elemento 1 em $\mathbf{y}^r$ leva em consideração o *bias* $\theta_{j_0}^r$, contido no vetor θ_j^r .

A Figura 9-3 mostra um diagrama que resume essas equações para uma camada r .

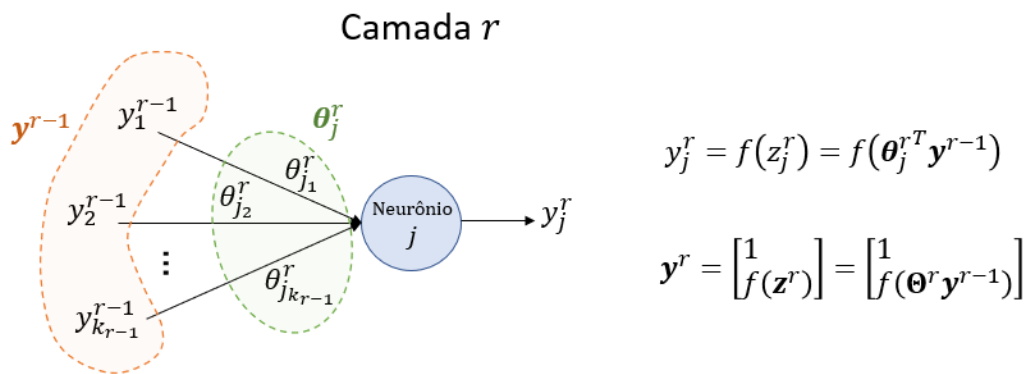


Figura 9-3: Equações da FCN.

Para a função não linear f , é comum utilizarmos funções como a sigmoide ou a tangente hiperbólica, por alguns motivos específicos, mas principalmente por elas serem diferenciáveis. Seus gráficos são mostrados na Figura 9-4.

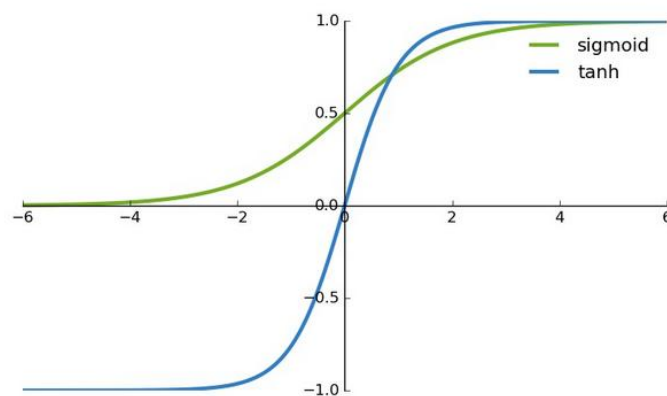


Figura 9-4: Funções sigmoide e tangente hiperbólica.

9.2 ALGORITMO DE BACKPROPAGATION

É o algoritmo utilizado pelas redes neurais para otimizar os parâmetros θ . Assim como em outros modelos de *machine learning*, nosso objetivo é otimizar uma função objetivo/custo com o seguinte formato:

$$J(\theta) = \sum_{n=1}^N \mathcal{L}(y_n, f_{\theta}(x_n)),$$

em que N é o número total de amostras de treinamento (x_n, y_n) , $n \in [1, N]$ e $\mathcal{L}(\cdot, \cdot)$ é a função *loss*.

Uma das grandes dificuldades de otimizar uma função como essa está na estrutura multicamadas da rede neural, em que cada neurônio possui um vetor de parâmetros θ único associado a cada um deles e as saídas de cada neurônio depende das saídas dos neurônios mais **rasos** (pertencentes a camadas de índice menor). Assim, temos uma estrutura em que a saída de um neurônio é uma composição de várias funções não lineares.

Outra dificuldade é que, devido à natureza altamente não linear da rede neural, a função custo tende a apresentar diversos mínimos locais, o que dificulta encontrarmos a solução ótima, principalmente quando aplicamos técnicas de otimização como o gradiente descendente. Muitas vezes, o mínimo local é tão fundo que se aproxima do valor de $J(\theta)$ ótimo. Nesses casos, podemos adotar esse mínimo local como solução, se o resultado final for satisfatório.

Feitas essas considerações, entraremos nos detalhes do algoritmo de *backpropagation*. Primeiramente, precisamos escolher uma função de perda. Nesse momento, adotaremos a função de erro quadrático para assumir esse papel. Assim, a função custo fica da seguinte forma:

$$J(\theta) = \sum_{n=1}^N J_n(\theta) \tag{9.1}$$

com

$$J_n(\theta) = \frac{1}{2} \|\hat{y}_n - y_n\|^2 = \frac{1}{2} \sum_{k=1}^{k_L} (\hat{y}_{n_k} - y_{n_k})^2.$$

Com isso, estamos avaliando o erro quadrático entre o valor estimado da saída $\hat{y}_{n_k}$ no neurônio k para a amostra de índice n e o valor real y_{n_k} que deveria estar na saída, sendo o índice k o indicador da posição no vetor y_n .

O próximo passo é escolher um algoritmo de otimização para nos auxiliar a encontrar os melhores valores de θ . Normalmente, são utilizados algoritmos baseados no cálculo do gradiente, devido à sua simplicidade. O método do gradiente descendente é um exemplo e será empregado na derivação do algoritmo do *backpropagation* por simplicidade, apesar de existirem variações desse método que são até mais utilizadas na prática (Adagrad, Adam...) [7, pp. 924-934]. De toda forma, no método do gradiente descendente básico, o valor de θ é atualizado a cada iteração da seguinte maneira:

$$\theta_j^r \leftarrow \theta_j^r - \mu \frac{\partial J(\theta)}{\partial \theta_j^r}. \quad (9.2)$$

Assim, vemos que é necessário determinar o termo da derivada. Antes de desenvolver a expressão dessa derivada, lembremos que

$$\hat{y}_{n_k} = f(z_{n_k}^L) = f(\theta_k^L y_n^L).$$

Observação: não confundir $y_{n_j}^r$, com sobrescrito, que é a saída do neurônio j da camada r , com y_{n_j} , sem sobrescrito, que é o valor do *target* na posição j correspondente à entrada x_n .

Como $J(\theta)$ é uma soma de $J_n(\theta)$, basta encontramos uma expressão para $\frac{\partial J(\theta)}{\partial \theta_j^r}$. É interessante notar também que, apesar de estarmos explicitando J_n como dependente de θ , também poderíamos escrever $J_n = J_n(z_{n_1}^r(\theta_1^r), \dots, z_{n_{k_L}}^r(\theta_{n_{k_L}}^r))$, já que J_n depende de $z_{n_j}^r$, que, por sua vez, depende de θ_j^r .

Com isso, usando a regra da cadeia, podemos escrever

$$\frac{\partial J_n(\theta)}{\partial \theta_j^r} = \frac{\partial J_n}{\partial z_{n_j}^r} \frac{\partial z_{n_j}^r}{\partial \theta_j^r} = \frac{\partial J_n}{\partial z_{n_j}^r} \frac{\partial (\theta_j^{rT} y_n^{r-1})}{\partial \theta_j^r} = \frac{\partial J_n(\theta)}{\partial z_{n_j}^r} y_n^{r-1}.$$

Devido ao fato de o termo $\frac{\partial J_n(\theta)}{\partial z_j^r}$ aparecer com frequência no desenvolvimento do algoritmo de *backpropagation*, utilizamos a seguinte definição:

$$\delta_{nj}^r := \frac{\partial J_n(\theta)}{\partial z_j^r}$$

Portanto, nosso objetivo é encontrar

$$\frac{\partial J_n(\theta)}{\partial \theta_j^r} = \delta_{nj}^r y_n^{r-1} \quad (9.3)$$

Dessa forma, partindo da eq. (9.2) podemos encontrar a expressão que resume a atualização dos pesos da rede neural:

$$\begin{aligned} \theta_j^r &\leftarrow \theta_j^r - \mu \frac{\partial J(\theta)}{\partial \theta_j^r} \\ \theta_j^r &\leftarrow \theta_j^r - \mu \sum_{n=1}^N \frac{\partial J_n(\theta)}{\partial \theta_j^r} \\ \theta_j^r &\leftarrow \theta_j^r - \mu \sum_{n=1}^N \delta_{nj}^r y_n^{r-1} \end{aligned} \quad (9.4)$$

O valor y_n^{r-1} é observável, então não precisamos calcular esse termo. Basta determinarmos o valor de δ_{nj}^r para cada camada r e para cada neurônio j nessa camada. Para a camada $r = L$,

determinar δ_{nj}^r é simples, pois $J_n(\theta) = \frac{1}{2} \sum_{k=1}^{k_L} (f(z_j^r) - y_{n_k})^2$, o que faz com que a derivada com relação a z_j^r seja simples. Estamos calculando a derivada da função custo, que está sendo avaliada na última camada, com relação a z_j^r de um neurônio que também está na última camada. Logo, não temos composições de funções por causa da passagem de uma camada à outra.

Por outro lado, quando avaliamos δ_{nj}^r para neurônios que não estão na última camada, os cálculos se tornam um pouco mais complicados por causa da composição de funções. Por isso, abaixo, vamos derivar expressões gerais para δ_{nj}^r para os dois possíveis casos de r .

Detalhes dos cálculos do algoritmo de *backpropagation*

$r = L$:

No caso mais simples, em que o neurônio está na última camada, temos

$$\delta_{nj}^r = \frac{1}{2} \frac{\partial}{\partial z_j^r} \sum_{k=1}^{k_L} \left[\hat{y}_{n_k} - y_{n_k} \right]^2 = \frac{1}{2} \frac{\partial}{\partial z_j^r} [f(z_{n_j}^r) - y_{n_j}]^2 = (\hat{y}_{n_j} - y_{n_j}) f'(z_{n_j}^r)$$

$$\delta_{nj}^r = e_{nj} f'(z_{n_j}^r),$$

com $e_{nj} = \hat{y}_{n_j} - y_{n_j}$ sendo o erro da rede neural no neurônio j da camada de saída.

$r < L$:

Quando analisamos δ_{nj}^r para as camadas anteriores à camada de saída, verificamos que existe uma distância entre $J_n(\theta)$, que está sendo avaliado na saída, e a variável $z_{n_j}^r$, com relação a qual estamos derivando e que está associada à camada r . Logo, é natural que surjam regras da cadeia.

Da mesma maneira como foi feito anteriormente, podemos explicitar a dependência de J_n com relação às variáveis da rede neural como

$$J_n = J_n(z_{n_1}^r, \dots, z_{n_{k_L}}^r) = J_n(z_{n_1}^r(z_{n_1}^{r-1}, \dots, z_{n_{k_{r-1}}}^{r-1}), \dots, z_{n_{k_L}}^r(z_{n_1}^{r-1}, \dots, z_{n_{k_{r-1}}}^{r-1})).$$

Com isso, podemos escrever,

$$\begin{aligned} \delta_{nj}^{r-1} &= \frac{\partial J_n}{\partial z_{n_1}^r} \frac{\partial z_{n_1}^r}{\partial z_{n_j}^{r-1}} + \dots + \frac{\partial J_n}{\partial z_{n_{k_r}}^r} \frac{\partial z_{n_{k_r}}^r}{\partial z_{n_j}^{r-1}} = \sum_{k=1}^{k_r} \frac{\partial J_n}{\partial z_{n_k}^r} \frac{\partial z_{n_k}^r}{\partial z_{n_j}^{r-1}} = \sum_{k=1}^{k_r} \delta_{nk}^r \frac{\partial z_{n_k}^r}{\partial z_{n_j}^{r-1}} \\ &= \sum_{k=1}^{k_r} \delta_{nk}^r \frac{\partial (\theta_{nk}^T \mathbf{y}_n^{r-1})}{\partial z_{n_j}^{r-1}} = \sum_{k=1}^{k_r} \delta_{nk}^r \frac{\partial (\theta_k^{rT} f(\mathbf{z}_n^{r-1}))}{\partial z_{n_j}^{r-1}} = \left(\sum_{k=1}^{k_r} \delta_{nk}^r \theta_{kj}^r \right) f'(z_{n_j}^{r-1}) \end{aligned}$$

$$\delta_{nj}^{r-1} = e_n^r f'(z_{n_j}^{r-1}),$$

(9.5)

com $e_n^r = \sum_{k=1}^{k_r} \delta_{nk}^r \theta_{kj}^r$. Esse termo é só para criar uma uniformidade na notação.

Lembrando que poderíamos ter escrito uma expressão para δ_{nj}^r explicitamente, mas daria no mesmo. Só teríamos que substituir o termo δ_{nj}^r na equação acima por δ_{nj}^{r+1} . Da forma como está a equação acima é mais conveniente.

Depois de todo esse desenvolvimento, vemos que para determinar δ_{nj}^r , dependemos do valor de δ_{nj}^{r+1} , da camada seguinte (uma profundidade abaixo). Por isso, para encontrarmos todos os valores de δ_{nj}^r , precisamos fazer os cálculos de trás para frente (*backwards*), começando na camada de saída indo em direção à primeira camada.

O algoritmo de *backpropagation* consiste em

1. Inicializar os pesos θ aleatoriamente;
2. Iterar variando n de 1 a N :
 - a. Realizar o *forward propagation* (calcular as saídas da rede para a entrada x_n);
 - b. Realizar o *backpropagation*, determinando os valores de δ_{nj}^r para todos os valores de j , começando na camada de saída $r = L$ e indo em direção à primeira camada.
3. Atualizar os valores de θ_j^r a partir da eq. (9.4);
4. Se o critério de parada não for satisfeito, retornar ao passo 2.

O algoritmo acima pode ser executado várias vezes no mesmo conjunto de treinamento, repetindo os pares (x_n, y_n) . Toda vez que completamos um ciclo passando por todas as amostras de treinamento (x_n, y_n) , dizemos que uma **época** foi completada.

Alguns critérios de parada que podem ser adotados são: valor da função custo baixo o suficiente; os valores dos pesos de uma iteração para outra mudam muito pouco; número máximo de épocas foi atingido.

O algoritmo de *backpropagation* descrito acima utiliza uma abordagem do tipo **batch**, em que todas as N amostras de treinamento são utilizadas de uma vez para o cálculo da função custo (observar que o somatório vai de 1 até N). Outra abordagem possível é a **stochastic gradient descent** (SGD). Nesse caso, usamos um método de otimização similar ao gradiente descendente, mas, para o cálculo da função custo, consideramos uma amostra por vez. Assim, em vez de calcular os valores δ_{nj}^r para todos os valores de n para somente no final atualizar os pesos θ_j^r com a eq. (9.1), atualizamos os pesos em cada iteração de n , de forma que, em uma época, atualizamos N vezes os pesos.

Finalmente, existe uma abordagem intermediária, chamada de **minibatch**, em que não atualizamos os pesos a cada iteração de n , mas também não deixamos para atualizá-los depois de iterarmos ao longo das N amostras. Atualizamos-nos de K em K amostras de treinamento, com $K < N$, usando uma função custo da forma

$$J(\theta) = \sum_{n=1}^K J_n(\theta),$$

em que K é o tamanho do *minibatch*.

9.3 ESCOLHENDO A FUNÇÃO CUSTO E AS NÃO-LINEARIDADES

9.3.1 Observando a Camada de Saída

Ao observarmos os neurônios da saída, podemos ter uma noção dos efeitos da combinação de determinadas funções não-lineares com determinadas funções custo. Por simplicidade, vamos considerar uma rede com um único neurônio na camada de saída. Consideraremos os casos da função custo de erro quadrático e de entropia cruzada.

9.3.1.1 Função Custo de Erro Quadrático

Nesse caso, temos

$$J = \frac{1}{2} \sum_{n=1}^N (t_n - y_n^L)^2 = \frac{1}{2} \sum_{n=1}^N (t_n - f(\mathbf{z}_n))^2 = \frac{1}{2} \sum_{n=1}^N \left(t_n - f(\boldsymbol{\theta}^L \mathbf{y}_n^{L-1}) \right)^2,$$

em que N é o total de exemplos de treinamento, t_n é o valor do *target* e y_n^L é a saída do neurônio da última camada.

Se adotarmos a função sigmoide como não-linearidade, teremos

$$\frac{\partial J}{\partial \boldsymbol{\theta}^L} = \frac{\partial J}{\partial y_n^L} \frac{\partial y_n^L}{\partial \mathbf{z}_n} \frac{\partial \mathbf{z}_n}{\partial \boldsymbol{\theta}^L} = \left[-\frac{1}{2} \sum_{n=1}^N 2y_n^L - 2t_n \right] [\sigma'(\mathbf{z}_n)] [\mathbf{y}_n^{L-1}] = - \sum_{n=1}^N (y_n^L - t_n) \sigma'(\mathbf{z}_n) \mathbf{y}_n^{L-1}.$$

Reparar que isso é exatamente o que encontraríamos se aplicássemos a eq. (9.3) diretamente.

Com isso, vemos que o gradiente da função custo depende do erro $(y_n^L - t_n)$, o que é justo, pois, à medida que o erro diminui, o gradiente também diminui, assim como o tamanho do passo do gradiente descendente. Por outro lado, a dependência de $\sigma'(\mathbf{z}_n)$ é incômoda, pois, se $\mathbf{z}_n$ for muito distante do valor zero, $\sigma'(\mathbf{z}_n)$ tende a zero, assim como o valor do gradiente.

Portanto, vemos que quando usamos a função sigmoide como não-linearidade do neurônio de saída, a função custo de erro quadrático não é muito adequada.

9.3.1.2 Função Custo de Entropia Cruzada

Nesse caso, temos

$$J = \sum_{n=1}^N H(t_n, y_n) = - \sum_{n=1}^N \sum_{k=1}^{k_L} t_{nk} \log y_{nk}^L = - \sum_{n=1}^N \sum_{k=1}^{k_L} t_{nk} \log \sigma(\mathbf{z}_{nk}).$$

Desenvolvendo a expressão para o gradiente da função custo, obtemos

$$\frac{\partial J}{\partial \boldsymbol{\theta}_j^L} = \sum_{n=1}^N y_{nj} (t_{nj} - 1) \mathbf{y}_n^{L-1}.$$

Verificamos que, diferentemente do caso da função de custo de erro quadrático, o gradiente não depende da derivada da função não-linear, eliminando o problema do gradiente que “desaparece” quando $\mathbf{z}_n$ for muito distante do valor zero. Portanto, a função custo de entropia cruzada é mais adequada quando utilizamos a função sigmoide como não-linearidade.

Em geral, quando queremos garantir que as saídas da rede neural sejam probabilidades e somem 1, utilizamos a função softmax na saída. Pode ser mostrado que, utilizar a softmax em

vez da função sigmoide gera um gradiente que também é independente da derivada da função não-linear.

9.3.2 Observando as Camadas Escondidas

Para as camadas anteriores à camada de saída, a expressão que rege o algoritmo de *backpropagation* é a eq. (9.5):

$$\delta_{nj}^{r-1} = e_n^r f' \left(z_{nj}^{r-1} \right) = \sum_{k=1}^{k_r} \delta_{nk}^r \theta_{kj}^r f' \left(z_{nj}^{r-1} \right).$$

Como podemos perceber, temos uma lógica recursiva, em que o δ da camada anterior depende do δ da camada seguinte. Podemos deixar isso mais explícito ao substituímos o termo δ_{nk}^r na equação acima da seguinte forma:

$$\delta_{nj}^{r-1} = \sum_{k=1}^{k_r} \left[\sum_{k=1}^{k_{r+1}} \delta_{nk}^{r+1} \theta_{kj}^{r+1} f' \left(z_{nj}^r \right) \right] \theta_{kj}^r f' \left(z_{nj}^{r-1} \right). \quad (9.6)$$

Com isso, percebemos que existe um produto dos pesos θ e das derivadas da função não-linear. À medida que continuamos substituindo δ recursivamente, mais termos referentes às outras camadas vão sendo adicionados a esse produto.

A derivada da função não-linear pode assumir valores menores do que 1. Com isso, dependendo do caso, pode acontecer de o gradiente da função custo com respeito aos parâmetros das camadas mais superficiais assumam valores muito pequenos e sofram desvanecimento à medida que caminhamos para a parte mais rasa da rede. Esse fenômeno é conhecido como **desvanecimento do gradiente**. O maior problema associado a esse fenômeno é que ele torna o aprendizado mais lento.

Outro fenômeno associado a esse produto recursivo é o **gradiente explosivo**, que acontece quando o valor do gradiente assume valores mais elevados à medida que caminhamos na direção das camadas mais rasas. Isso afeta o aprendizado, pois faz com que a atualização dos parâmetros dê um salto muito elevado, podendo desviar os parâmetros do valor correto. Além disso, o gradiente explosivo pode levar a problema relacionados à overflow das variáveis que armazenam os parâmetros.

Por fim, em redes neurais com muitas camadas, os parâmetros de cada camada podem assumir valores de escalas muito diferentes, o que faz com que a velocidade do aprendizado de cada uma delas seja muito diferente, causando instabilidade.

Uma maneira de lidar com esse problema é utilizar função de ativação ReLU (Rectified Linear Unit) nos neurônios das camadas escondidas. Ela é dada por

$$f(z) = \max\{0, z\} \quad (9.7)$$

Seu gráfico é mostrado na Figura 9-5.

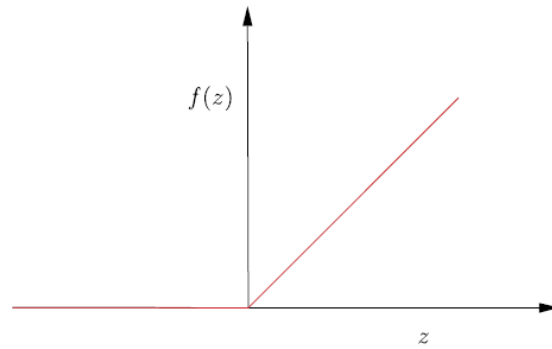


Figura 9-5: Gráfico da ReLU.

Ela ajuda a resolver os problemas do gradiente desvanecente e explosivo e, consequentemente, acelerando o treinamento porque sua derivada é sempre uma constante. Para $z > 0$, a derivada é um e os termos $f'(\cdot)$ na (9.6) desaparecem. Para $z = 0$, podemos escolher o valor 0 ou 1 para a derivada.

Como é possível perceber, para $z < 0$, a derivada se anula e o treinamento tende a ficar mais lento e a empacar. Por essa razão, ao inicializar os pesos da rede, no início do treinamento, é importante escolher pequenos valores positivos para os pesos.

9.4 REGULARIZAÇÃO

Como as redes neurais apresentam uma quantidade muito grande de parâmetros, elas ficam sujeitas ao problema de *overfitting*. Uma forma prática de lidar com esse problema é utilizar regularização. As principais técnicas de regularização que podem ser aplicadas a redes neurais são mostradas abaixo:

- **Decaimento de peso:** consiste em alterar a função custo adicionando um termo de regularização, como por exemplo

$$J'(\theta) = J(\theta) + \lambda \|\theta\|^2.$$

- **Early stopping:** no início do treinamento, há uma tendência de as curvas de erro de treinamento e de teste decaírem. A partir de determinado momento, a curva de erro de teste deixa de decair e começa a subir. A técnica de *early stopping* consiste em interromper o treinamento assim que a curva de erro de teste começa a subir. Esse é um dos métodos de regularização mais utilizados e pode ser combinado com outros métodos.
- **Dropout:** consiste em remover da rede neural, durante o treinamento, uma certa porção de nós. Quando um nó é removido, todas as suas conexões com os nós da camada anterior e da camada seguinte também são removidas. A cada iteração, cada nó tem probabilidade p de ser removido. Quando um nó é removido em uma iteração, os parâmetros associados a ele não são atualizados. Na iteração seguinte, todos os nós removidos retornam com os seus pesos conservados da iteração anterior a que eles foram removidos e é feito um novo sorteio para decidir quais são os nós que serão removidos novamente. Depois de a rede neural estar treinada e é utilizada para inferências, os pesos de cada neurônio são multiplicados por p . Esse é um dos métodos de regularização mais utilizados e pode ser combinado com outros métodos.

Capítulo 10: PRÉ-PROCESSAMENTO DE DADOS

10.1 VALORES FALTANTES

As razões para a existência de dados faltantes em um conjunto de dados podem ser divididas em três grandes categorias [24]:

- **Deficiência estrutural dos dados**

A ausência de valor tem um significado. Por exemplo, uma variável indicando o tamanho da piscina em uma casa pode ter valor faltante quando a casa não tiver piscina. Nesse caso, uma boa prática é utilizar algum outro valor para indicar a ausência de piscina em vez de não indicar nenhum valor.

- **Ocorrências aleatórias de valores faltantes**

Elas podem ser de dois tipos:

- *Missing Completely at Random* (MCAR) – valores faltantes são independentes dos dados.
- *Missing At Random* (MAR) – valor faltante é dependente do dado sendo observado. Um exemplo desse tipo de situação pode acontecer quando temos dados relacionados à idade de pessoas entrevistadas. Mulheres podem ter menor propensão de informar a idade, o que faz com que nas observações em que a pessoa entrevistada é uma mulher haja maior probabilidade de o campo “Idade” não estar preenchido.

- **Causas específicas ou *Not Missing At Random* (NMAR)**

Um exemplo é o caso em que são conduzidos exames clínicos em que paciente são examinados ao longo do tempo, mas alguns pacientes abandonam o estudo (seja por morte, seja por efeitos colaterais de algum tratamento). Portanto, os novos dados a partir da saída desses pacientes terá dados faltantes.

Capítulo 11: SELEÇÃO DE VARIÁVEIS

Seleção de variáveis é uma técnica de redução de dimensionalidade que consiste em selecionar algumas *features* do conjunto total sem modificá-las ou criar novas *features*.

Um procedimento que tem um nome parecido e pode criar confusão é *feature extraction*, que também faz parte do conjunto de técnicas de redução de dimensionalidade. No entanto, nesse caso, realizamos a projeção de todas as *features* em um novo espaço com dimensão menor. Não selecionamos um subconjunto das *features*, como é o caso da *feature selection*.

As principais vantagens da *feature selection* (e da *feature extraction* também) são [25]:

- aumento de eficiência computacional, pois são utilizadas menos variáveis durante o aprendizado do modelo;
- modelo com maior capacidade de generalização, já que, mantendo-se a quantidade de dados constante, diminuir o número de *features* deixa os dados menos esparsos e diminui o risco de *overfitting*;
- melhora de desempenho do modelo, pois a existência de *features* irrelevantes impede faz com que o modelo tente se adaptar também a variáveis que não ajudam a descrever a variável *target*. Com apenas *features* relevantes, o modelo tem curvas que se adequam apenas às variáveis importante e não precisa “tentar” se adaptar um padrão diferente introduzido por *features* irrelevante que não vão ajudar a descrever o fenômeno de interesse.

O segundo ponto está ligado à maldição da dimensionalidade, que diz que, à medida que a dimensão do problema aumenta, os dados tornam-se mais esparsos (se a quantidade de dados se mantiver constante). Uma forma de visualizar isso é imaginando o exemplo em que temos dois pontos e o posicionamos o mais distante possível dentro de um hipercubo de aresta a . Quando o número de dimensões é $d = 2$, a maior distância é $a\sqrt{2}$. Quando $d = 3$, a maior distância é $a\sqrt{3}$.

A esparsidade é um problema porque ela aumenta as chances de o modelo aprender ruídos e não um padrão generalizável. Para visualizar isso com exemplo, consideremos que o pior caso de *overfitting*, em um modelo de regressão, por exemplo, é aquela em que a curva passa exatamente em cima de todos os pontos do conjunto de treinamento. Se olharmos para a regressão linear, em que traçamos um hiperplano sobre os dados de treinamento, quanto maior a razão $\frac{\text{dimensão}}{\text{número de dados}}$, maior é a tendência de o hiperplano passar em cima dos pontos de treinamento. No limite, quando a dimensão é igual ao número de *features*, o hiperplano consegue passar exatamente em cima de cada ponto do conjunto de treinamento, gerando um modelo com *overfitting*, pois os dados estão muito esparsos.

As técnicas de *feature selection* podem ser **supervisionadas**, que é o caso em que utilizamos os *targets* para selecionar o subconjunto de *features*, ou **não-supervisionados**, que é o caso em que não são utilizados os *targets*.

Além disso, as técnicas de *features selection*, sejam elas supervisionadas ou não-supervisionadas, podem ser divididas em:

- **Wrapper methods:** o critério utilizado para avaliar as *features* selecionadas é baseado no desempenho do modelo de *machine learning*. Assim, primeiramente, selecionamos um grupo de *features* e as utilizamos para treinar o modelo. Utilizamos os resultados do teste desse modelo para comparar essa seleção com as próximas seleções. A principal desvantagem desse método é a demora relacionada aos diversos treinamentos e testes do modelo que deverão ser executadas. São 2^d possíveis combinações de *features* (cada uma delas pode ou não estar no conjunto de *features* selecionadas).
- **Filter methods:** são métodos independentes de algoritmos de aprendizado. Eles avaliam a importância de cada *feature* com base apenas nas características dos dados. Alguns critérios que podem ser utilizados são correlação entre *features*, informação mútua.
- **Embedded (ou intrinsic) methods:** são métodos em que o processo de seleção das *features* já é executada automaticamente dentro do algoritmo de aprendizado, como é o caso das árvores de decisão.

11.1 BAIXA VARIÂNCIA

Consiste em remover as variáveis de entrada que possuem variância abaixo de um determinado limiar, pois elas tendem a não contribuir muito para a explicação da variável de saída. No caso limite em a variável de entrada tem variância nula (sempre apresenta o mesmo valor), a variável de entrada não tem como ser utilizada para explicar a variável de saída.

Ao utilizar esse método, é essencial normalizar os dados para que seus valores fiquem dentro de uma faixa definida. Assim, será possível estabelecer um único limiar de variância, que pode ser aplicado a todas as variáveis, abaixo do qual eliminamos uma variável. Observar que a padronização não pode ser utilizada, já que ela levaria a variância de todas as variáveis para o valor 1.

Algumas das limitações desse método são:

- Em alguns casos, uma variável com baixa variância poder ter forte influência no valor do *target*. Nesse caso, eliminá-la seria uma decisão ruim.
- Não é possível remover variáveis redundantes com esse método, dado que ele só analisa uma variável por vez, independentemente das outras e do *target* [25].

O fato de cada variável ser analisada independentemente do *target* é uma desvantagem apenas em casos em que o conjunto de dados apresenta os *targets*. Em aplicações de **aprendizado não-supervisionado**, esse método pode ter maior utilidade.

No *scikit-learn*, pode ser utilizada a classe **VarianceThreshold** para aplicar esse método.

11.2 INFORMAÇÃO MÚTUA

A informação mútua é aplicável em seleção de variáveis porque, com ela, é possível medir o grau de dependência entre duas variáveis aleatórias. Diferentemente da correlação de

Pearson, a informação mútua não se limita a medir dependências lineares, podendo capturar graus de dependência mais genéricos entre as variáveis. Dessa forma, é possível medir de forma mais completa o grau de correlação entre as variáveis de entrada e o *target*.

Considerando a hipótese de que a otimização dos hiperparâmetros de um dado modelo pode ser feita de maneira independente do processo de otimização do conjunto de *features*, o problema de seleção de variáveis de entrada usando informação mútua se resume a minimizar a informação mútua condicional (CMI, do inglês, *Conditional Mutual Information*) $I(S^c; Y|S)$, ou seja, a informação mútua entre o conjunto S^c de variáveis de entrada que não foram selecionadas e a variável *target* Y , condicionada ao conjunto de variáveis selecionadas S [26].

No entanto, a busca pelo ótimo global é NP-difícil e, por isso, a maior parte dos algoritmos de seleção de variáveis adota abordagens heurísticas de busca sequencial [25]. Seguindo a lógica da minimização da CMI, podemos definir uma abordagem sequencial da seguinte forma:

Algoritmo Sequencial de Seleção de Features por Informação Mútua

1. Começar com um conjunto de variáveis selecionadas vazio, ou seja, $S = \emptyset$.

2. Encontrar a melhor variável $X_k \in S^c$ para adicionar a S :

$$X_k = \arg \max_{X_k \in S^c} J_{CMI}(X_k)$$

com $J_{CMI}(X_k) = I(X_k; Y|S)$.

3. Atualizar S :

$$S \leftarrow S \cup \{X_k\}$$

4. Repetir as etapas 2 e 3 até que algum critério de parada seja respeitado.

A expressão de $J_{CMI}(X_k)$ também pode ser escrita como

$$J_{CMI}(X_k) = I(X_k; Y) - I(X_k; S) + I(X_k; S|Y). \quad (11.1)$$

Cada uma das parcelas dessa expressão recebe uma interpretação:

- $I(X_k; Y)$ corresponde à **relevância** de X_k para explicar Y ;
- $I(X_k; S)$ corresponde à **redundância** entre X_k e as variáveis já selecionadas;
- $I(X_k; S|Y)$ corresponde à **complementariedade** entre X_k e as variáveis já selecionadas (caso isso pareça confuso, conferir Seção B.1.8).

Os diferentes métodos que serão apresentados a seguir utilizarão, muitas vezes, uma simplificação da função J_{CMI} , geralmente focando em uma ou duas dessas três parcelas. Mais do que isso, eles utilizam a abordagem sequencial de seleção de *features*, diferenciando-se uns dos outros pela forma como eles definem a função J a ser maximizada.

11.2.1 Maximização da Informação Mútua (MIM)

Esse método consiste em maximizar

$$J_{MIM}(X_k) = I(X_k; Y). \quad (11.2)$$

Essa expressão corresponde exatamente à parcela de relevância da eq. (11.1). No entanto, esse método ignora completamente a questão da redundância e a complementariedade entre as

variáveis. Logo, é como se estivéssemos supondo que as variáveis são todas independentes entre si, ao utilizar esse método.

11.2.2 *Mutual Information Feature Selection (MIFS)*

Esse método consiste em maximizar

$$J_{\text{MIFS}}(X_k) = I(X_k; Y) - \beta \sum_{X_j \in S} I(X_k; X_j),$$

em que $\beta \in [0,1]$ é um parâmetro definido pelo usuário. Esse método leva em consideração não só a relevância, mas também a redundância das variáveis, sendo que o peso da penalização da relevância é controlado por β .

Apesar da semelhança com a eq. (11.1), $\sum_{X_j \in S} I(X_k; X_j) \neq I(X_k; S)$. Isso fica mais claro quando olhamos para um caso mais simples em que temos $X_1 \in S^c$ e $X_2, X_3 \in S$:

$$I(X_1; (X_2, X_3)) = I(X_1; X_2) + I(X_1; X_3 | X_2)$$

$$\sum_{X_j \in S} I(X_k; X_j) = I(X_1; X_2) + I(X_1; X_3)$$

Logo, $\sum_{X_j \in S} I(X_k; X_j) \neq I(X_1; (X_2, X_3))$.

Assim, apesar de os dois termos medirem de alguma forma a redundância entre as variáveis, eles são diferentes.

Uma desvantagem desse método é que se torna necessário a escolha do parâmetro β . Outra desvantagem é que à medida que S aumenta, $\sum_{X_j \in S} I(X_k; X_j)$ aumenta, o que faz com que o método penalize de forma crescente a adição de novas variáveis.

11.2.3 *Minimum Redundancy Maximum Relevance (MRMR)*

Esse método consiste em maximizar

$$J_{\text{MRMR}}(X_k) = I(X_k; Y) - \frac{1}{|S|} \sum_{X_j \in S} I(X_k; X_j),$$

em que $|S|$ é a cardinalidade do conjunto S .

O MRMR é muito parecido com o MIFS, porém ele corrige um dos seus principais problemas, que é o aumento da penalização à medida que S cresce. Isso é atingindo ao colocar como termo de redundância a média da informação mútua entre X_k e todas as variáveis pertencentes a S .

11.2.4 *Joint Mutual Information (JMI)*

Esse método consiste em maximizar

$$J_{\text{JMI}}(X_k) = \sum_{X_j \in S} I((X_k, X_j); Y).$$

Ao maximizar essa expressão, estamos ao mesmo tempo levando em consideração a redundância, a relevância e a complementariedade. Isso fica mais claro quando reescrevemos a expressão de J_{JMI} como

$$J_{\text{JMI}}(X_k) = I(X_k; Y) - \frac{1}{|S|} \sum_{X_j \in S} I(X_j; X_k) + \frac{1}{|S|} \sum_{X_j \in S} I(X_j; X_k | Y),$$

que foi derivada em [26]. Nessa expressão, podemos ver novamente o uso da média para quantificar a relevância e a complementariedade.

11.2.5 Comparação entre Métodos

Em [26], foi feita uma comparação entre diferentes métodos baseados nos princípios da teoria da informação. O JMI foi um dos métodos que mais se destacaram, pois apresentou nos experimentos boa estabilidade e acurácia. O MRMR também obteve bons resultados, com boa acurácia e estabilidade um pouco menor do que a do JMI.

https://scikit-learn.org/stable/modules/feature_selection.html#univariate-feature-selection

Capítulo 12: PRINCIPAL COMPONENT ANALYSIS

O PCA é uma técnica comumente utilizada em aplicações como redução de dimensionalidade, compressão com perda de dados, extração de *features* e visualização de dados [3].

Essa técnica funciona encontrando a projeção linear que minimiza a distância quadrática média entre os pontos e suas projeções. Ao minimizar essa distância quadrática média também estamos maximizando a variância das projeções dos pontos no hiperplano de projeção. Isso pode ser visto de maneira intuitiva à medida que escolhemos diferentes hiperplanos de projeção, como mostrado na Figura 12-1 e na Figura 12-2.

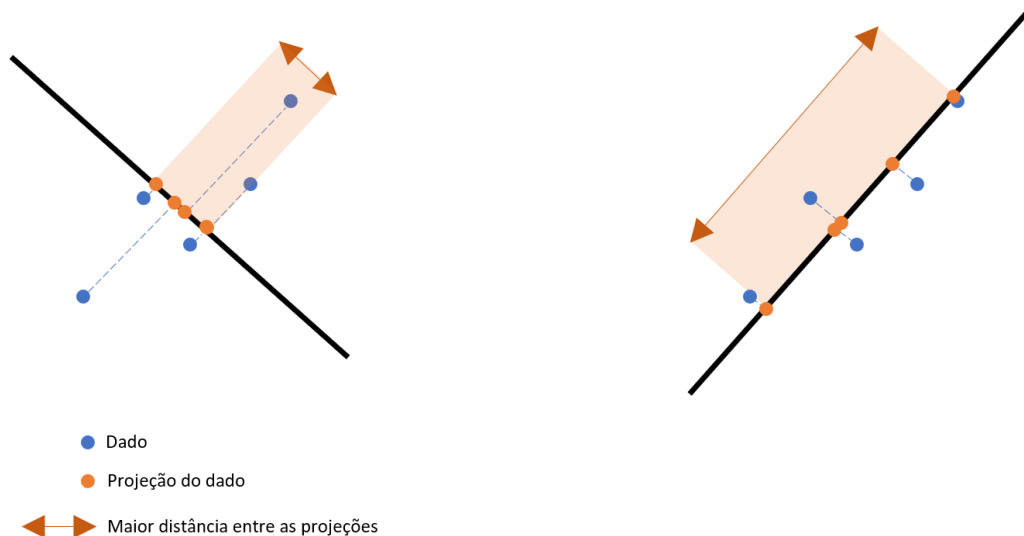


Figura 12-1: Pontos sendo ajustados por dois hiperplanos de projeção diferentes (e perpendiculares). O ajuste da direita é o ajuste que minimiza a distância quadrática média entre os pontos e o hiperplano de projeção, resultando, assim, em uma maior variância das projeções sobre o hiperplano em questão (como pode ser visto pela seta laranja, que mostra a extensão do espalhamento dos dados). No ajuste da esquerda, feita por um hiperplano ortogonal ao da direita, vemos que as projeções dos dados têm uma variância menor.

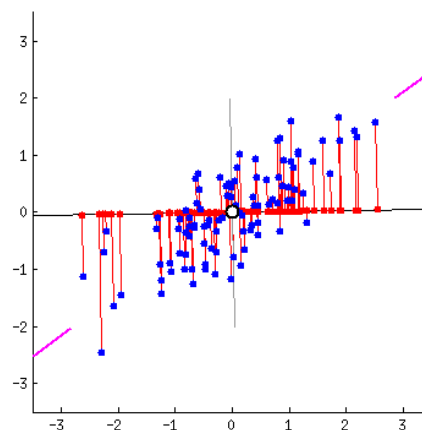


Figura 12-2: Figura animada mostrando as projeções (pontos vermelhos) dos dados (pontos azuis) à medida que giramos o hiperplano de projeção.

Ao reduzirmos a dimensionalidade maximizando a variância nessa projeção, estamos garantindo a menor perda de representatividade dos dados possível. A Figura 12-3 ilustra essa relação entre a maximização da variância e a representatividade da projeção.

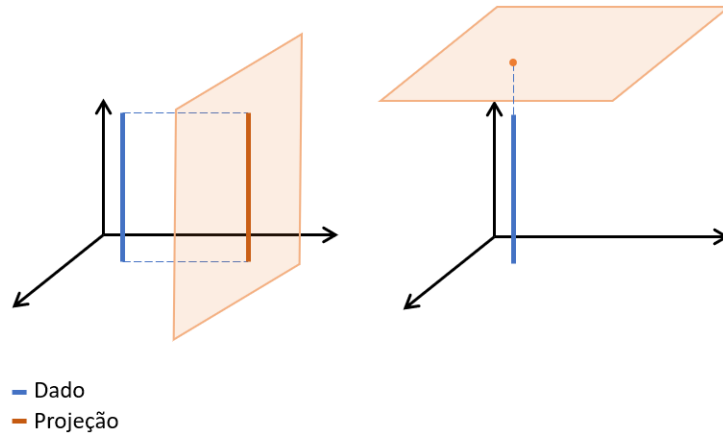


Figura 12-3: Do lado esquerdo, temos uma projeção bastante representativa dos dados, pois o plano foi escolhido de forma a maximizar a variância da projeção. Por outro lado, do lado direito, o plano escolhido minimiza a variância da projeção, resultado em uma projeção pouco representativa dos dados.

A seguir, são mostrados os cálculos envolvidos no método de PCA.

12.1 FORMULAÇÃO DA MÁXIMA VARIÂNCIA

Consideremos um espaço de dimensão D . Nosso objetivo é projetar os dados em um espaço de dimensão $M < D$ de forma que a variância dos dados projetados seja máxima.

Por simplicidade, vamos considerar, inicialmente, que $M = 1$. Podemos representar a direção desse espaço usando o vetor unitário $\mathbf{u}_1$ de D dimensões. Assim, a média das projeções dos dados é dada por

$$\bar{x}_p = \mathbf{u}_1^T \bar{\mathbf{x}},$$

em que $\bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$ e N é a quantidade total de dados.

A variância dos dados projetados é dada por

$$S_x = \frac{1}{N} \sum_{n=1}^N \left(\mathbf{u}_1^T \mathbf{x}_n - \widehat{\mathbf{u}_1^T \bar{\mathbf{x}}} \right)^2 = \mathbf{u}_1^T \left[\frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})^2 \right] \mathbf{u}_1 = \mathbf{u}_1^T \overbrace{\left[\frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T \right]}^{\mathbf{S}} \mathbf{u}_1$$

$$S_x = \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1,$$

em que $\mathbf{S}$ é a matriz de covariância dos dados.

Tendo o vetor $\mathbf{u}_1$ onde os dados serão projetados e a expressão da variância S_x , basta fazermos

$$\max_{\mathbf{u}_1} S_x = \max_{\mathbf{u}_1} \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1, \quad \|\mathbf{u}_1\| = 1.$$

O Lagrangiano desse problema é

$$L(\mathbf{u}_1) = \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 + \lambda_1 (1 - \mathbf{u}_1^T \mathbf{u}_1),$$

em que λ_1 é um multiplicador de Lagrange.

Para maximizarmos essa expressão, derivamo-la com relação a $\mathbf{u}_1$ e a igualamos a zero, encontrando como resultado a seguinte equação

$$\mathbf{S}\mathbf{u}_1 = \lambda_1 \mathbf{u}_1 \Rightarrow \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 = S_x = \lambda_1.$$

Da equação acima, notamos que $\mathbf{u}_1$ é um autovetor de $\mathbf{S}$, cujo autovalor é λ_1 . Além disso, vemos que a variância dos dados projetados é igual ao autovalor λ_1 . Como a variância está sendo maximizada, então λ_1 é o maior autovalor de $\mathbf{S}$. O vetor $\mathbf{u}_1$ recebe o nome de **primeira componente principal**.

Se considerarmos o caso genérico em que o espaço H onde os dados estão sendo projetados tem dimensão M , então $H = \langle \mathbf{u}_1, \dots, \mathbf{u}_M \rangle$ em que $\mathbf{u}_1, \dots, \mathbf{u}_M$ são os autovetores correspondentes aos maiores autovalores $\lambda_1, \dots, \lambda_M$ da matriz de covariância $\mathbf{S}$ dos dados. Além disso, como $\mathbf{S}$ é simétrica, assim como toda matriz de covariância, seus autovetores são todos ortogonais entre si [27]. Lembrando que a variância da projeção dos dados associada a cada autovetor é igual ao seu autovalor correspondente.

Por fim, para encontrar as coordenadas dos novos pontos projetados no espaço H , basta projetar os pontos originais $\mathbf{x}$ nos autovetores $\mathbf{u}_1, \dots, \mathbf{u}_M$. De forma geral,

$$[\mathbf{X}]_H = \mathbf{X}\mathbf{V},$$

em que $\mathbf{X}$ apresenta em cada linha uma instância/ um ponto $\mathbf{x}_i$, $\mathbf{V}$ é a matriz contendo os autovetores $\mathbf{u}_1, \dots, \mathbf{u}_M$ nas colunas e $[\mathbf{X}]_H$ é a matriz contendo em suas linhas os pontos $\mathbf{x}_i$ projetados na base H .

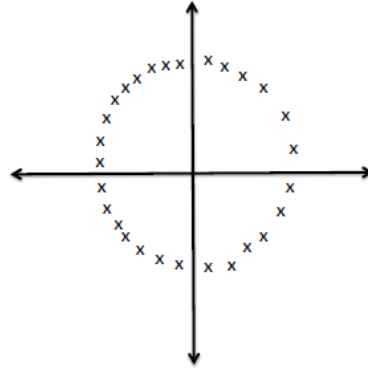
12.2 DETALHES IMPORTANTES

Como foi visto na Seção 12.1, projetamos os dados em vetores $\mathbf{u}_i$ que geram o espaço de projeção H . O problema consiste basicamente em encontrar esses vetores $\mathbf{u}_i$ cujas retas geradas minimizem a distância média quadrática aos pontos. No entanto, para fazer isso, precisamos que os dados estejam centralizados em zero, ou seja, precisamos subtrair dos dados a média deles. Observar na Figura 12-2 como os dados estão centralizados. Se, nessa figura, por exemplo, os dados estivessem centrados em $(0, -3)$, a melhor reta seria diferente, assim como o vetor $\mathbf{u}$ que a gerou. Não seria apenas diferente, como também geraria uma projeção com variância menor. Portanto, é possível ver a importância de centralizar os dados antes de aplicar o PCA.

Outros detalhes que podem estar associados a maus resultados do método PCA são listados abaixo:

- Centralização/escalamento mal realizados. A importância da centralização dos dados já foi discutida acima. O escalamento, por sua vez, também é importante, pois o resultado do PCA pode variar bruscamente com as unidades adotadas nas coordenadas dos dados [28]. Normalmente, realiza-se o escalamento utilizando o desvio padrão. Além disso, podem ser utilizadas outras técnicas de pré-processamento, como remoção de *outliers*.
- Não linearidade dos dados. Se os dados possuírem uma estrutura não linear, é bem possível que essa relação não-linear entre os dados seja perdida ao realizar a projeção

no plano H . Na imagem abaixo, a projeção dos dados em qualquer vetor $\mathbf{u}$ resultaria numa perda da estrutura não linear.



12.3 RELAÇÃO COM A DECOMPOSIÇÃO SVD

Se considerarmos $\mathbf{X}$ a matriz dos dados com dimensões $n \times m$, em que n é o número de dados e m é a dimensão desses dados, podemos escrever a matriz de covariância dos dados como [29]

$$\mathbf{S} = \frac{\mathbf{X}^T \mathbf{X}}{n-1}.$$

Ao realizar a decomposição SVD de $\frac{\mathbf{X}}{\sqrt{n-1}}$, encontramos $\frac{\mathbf{X}}{\sqrt{n-1}} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, em que $\mathbf{V}^T$ é a matriz cujas linhas são os autovetores de $\mathbf{S}$ e formam uma base ortonormal (vide Anexo C: Decomposição SVD). Além disso, na diagonal de $\mathbf{\Sigma}$ estão os valores singulares de $\frac{\mathbf{X}}{\sqrt{n-1}}$, que correspondem a $\sigma_i = \sqrt{\lambda_i}$, com λ_i os autovalores de $\mathbf{S}$.

Conhecer esses fatos é mais interessante numericamente para fazer os cálculos da PCA, pois, normalmente, é mais fácil determinar os autovalores e autovetores de $\mathbf{S}$ por meio da decomposição SVD do que por meio da multiplicação $\frac{\mathbf{X}^T \mathbf{X}}{n-1}$, que costuma ser mais custosa.

Capítulo 13: AVALIAÇÃO DE DESEMPENHO E SELEÇÃO DE MODELO

13.1 MÉTRICAS PARA CLASSIFICAÇÃO BINÁRIA

As principais métricas usadas para classificação binária são apresentadas a seguir.

13.1.1 Acurácia

$$A = \frac{TP + TN}{TP + FN + TN + FP}$$

É a taxa de acertos do modelo.

Desvantagens:

- Pode levar a conclusões errôneas caso as taxas de acertos de diferentes classes tiverem importâncias diferentes.
- Pouco adequada quando há um desbalanceamento nos dados que favorece muito uma única classe. A taxa de acerto da classe favorecida domina o resultado da acurácia, mascarando a contribuição das taxas de acerto das outras classes. Em um conjunto de dados em que apenas 1% das amostras são da classe 0 e 99% da classe 1, caso o modelo sempre dê uma predição de 1, sua acurácia será de 99%, apesar de ter errado a classificação de todos os dados da classe 0.

13.1.2 True Positive Rate (TPR) ou Sensibilidade ou *Recall*

$$TPR = \frac{TP}{TP + FN}$$

Mnemônico: *Recall* = TP / Real Positive.

Mede quão bom o modelo é com relação a não deixar de “detectar” amostras positivas (FN). O nome “sensibilidade” vem de um contexto médico [30, p. 95], em que essa métrica é utilizada para medir a capacidade do modelo de detectar a presença de uma doença (sem deixar ela passar). Em outras palavras, essa métrica indica a taxa de acerto do modelo dentro do conjunto de observações positivas, ou seja, quantas observações foram corretamente classificadas como positivas (TP) dentro do conjunto de observações positivas (TP + FN).

13.1.3 False Positive Rate (FPR)

$$FPR = \frac{FP}{TN + FP}$$

Mede a taxa de falsos positivos, ou seja, quantas das amostras negativas são classificadas como positivas do total de amostras negativas. Uma forma de lembrar amostras negativas. Uma forma de lembrar dessa expressão, é só pensar que o caso em que o modelo sempre dá uma predição positiva, teremos FPR igual a 100%, pois todas as amostras negativas serão FP e o numerador se igualará ao denominador. Se o numerador fosse a soma das amostras positivas, não teríamos FPR igual a 100% nesse caso.

A TPR e a FPR, geralmente, são reportadas em pares. Outro par equivalente, seria a FNR e a TNR, que são os complementares da TPR e FPR, respectivamente.

13.1.4 Precisão

$$P = \frac{TP}{TP + FP}$$

Mnemônico: Precisão = TP / Preditas Positivas. Além disso, Precisão = TP / TP + FP

Mede quão preciso é o modelo na sua seleção de amostras positivas, ou seja, quantas das amostras classificadas como positivas são de fato positivas. Em outras palavras, é a taxa de acerto dentro das amostras classificadas como positivas. Está relacionada à capacidade do modelo de evitar falsos positivos (observar mnemônico). Para lembrar que o *recall* está associado a falsos negativos, basta lembrar que precisão está associado a FP e deduzir por exclusão.

A precisão e a FPR fornecem informação sobre a capacidade do modelo de evitar falsos positivos. No entanto, quando o número de negativos é muito maior do que o de positivos (o que é muito comum em problemas de diagnóstico de uma doença rara), é preferível utilizar a precisão. Isso porque, nesse caso, o denominador da FPR tende a ficar muito grande, minimizar o valor de FPR e superestimar o desempenho do modelo, ou seja, dar uma falsa impressão de bom desempenho porque a métrica possui valor muito baixo. Por outro lado, a precisão não é afetada por isso, já que o denominador leva em conta apenas as amostras classificadas como positivas.

13.1.5 F1-score

$$F_1 = 2 \frac{PR}{P + R}$$

Mede o equilíbrio entre precisão e *recall* ou, de forma equivalente, o equilíbrio entre falsos positivos e falsos negativos. Varia de 0 a 1. Se o valor for baixo, a métrica indica que pelo menos *R* ou *P* são baixos. Sendo *R* e *P* altos, F_1 também será alto.

É interessante notar que o F1-score é a média harmônica de precisão e do *recall*. Reparar que $F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}}$. O motivo para se escolher a média harmônica e não a média aritmética, por

exemplo, é que a primeira penaliza os casos em que há desequilíbrio entre *P* e *R*, ou seja, *P* elevado e *R* baixo e vice-versa. Isso pode ser visto por meio dos gráficos das médias aritmética e harmônica para diferentes combinações de *P* e *R*, como mostrado na Figura 13-1. Isso penaliza casos como o do exemplo da Seção 13.1.1, em que o modelo sempre prevê a mesma classe. Nesse caso, a média harmônica penaliza mais o modelo do que a aritmética, o que é razoável, já que ele é muito ruim.

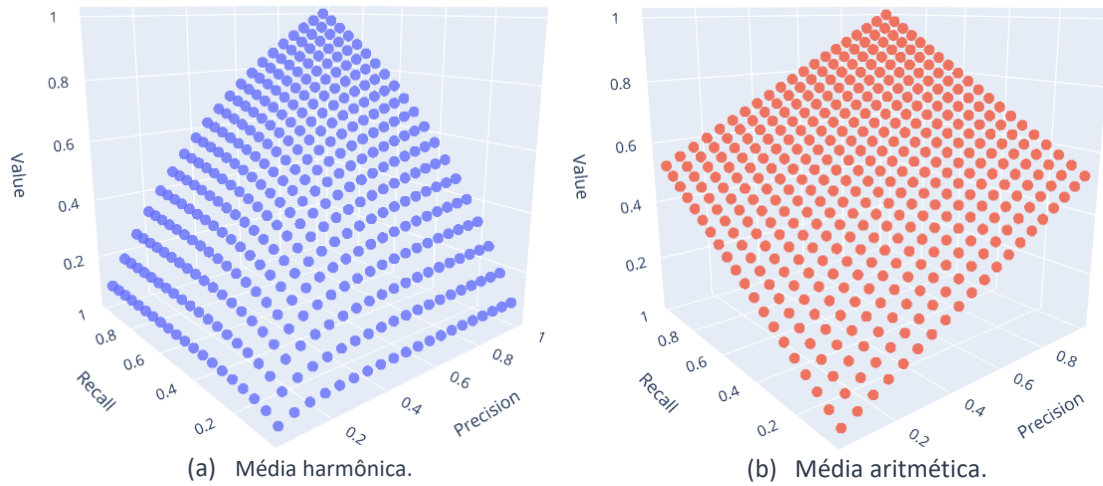


Figura 13-1: Comparação entre a média harmônica e aritmética da precisão e do recall.

13.1.6 F-score

$$F_{\alpha} = \frac{(1 + \alpha)PR}{\alpha P + R}$$

É a forma geral do F1-score, que permite atribuir graus de importância diferentes para R e P . Para $\alpha > 1$, estamos dando mais importância para R e, para $\alpha < 1$, estamos dando mais importância para P (apesar de isso não ser muito intuitivo, olhando apenas para a expressão). $\alpha = 2$ indica que estamos dando um peso duas vezes maior para R . Isso fica mais fácil de compreender quando percebemos que essa expressão corresponde à média harmônica ponderada de P e R , pois temos

$$F_{\alpha} = \frac{1 + \alpha}{\alpha R^{-1} + P^{-1}} = \left(\frac{\alpha}{1 + \alpha} R^{-1} + \frac{1}{1 + \alpha} P^{-1} \right)^{-1}.$$

Com a primeira igualdade, fica mais simples de ver que isso se trata de uma média harmônica ponderada. Com a segunda igualdade, vemos que o termo R^{-1} tem mais importância quando α maior que 1, pois nesse caso, $\frac{\alpha}{1 + \alpha} > \frac{1}{1 + \alpha}$. No caso limite em que $\alpha \rightarrow \infty$, $\frac{\alpha}{1 + \alpha} = 1$ e $\frac{1}{1 + \alpha} = 0$ e o termo associado a P tem nenhuma importância. Portanto, não importa se P é grande ou pequeno, pois ele terá uma porcentagem menor de participação no resultado final quando comparado com R . Se por exemplo, tivéssemos $\alpha = 3$, então $F_2 = (0,75R^{-1} + 0,25P^{-1})^{-1}$.

13.1.7 Curva ROC (Receiver Operating Characteristic)

É o gráfico da TPR pela FPR, como mostrado na Figura 13-2. Em problemas em que temos que decidir um limiar de probabilidade para classificar uma amostra como positiva, essa curva é útil, pois ela resume a relação da TPR e da FRP para diferentes valores de limiar. Se as duas métricas tiverem importâncias iguais, o melhor ponto dessa curva é na parte superior esquerda, já que quanto menor a FPR, melhor.

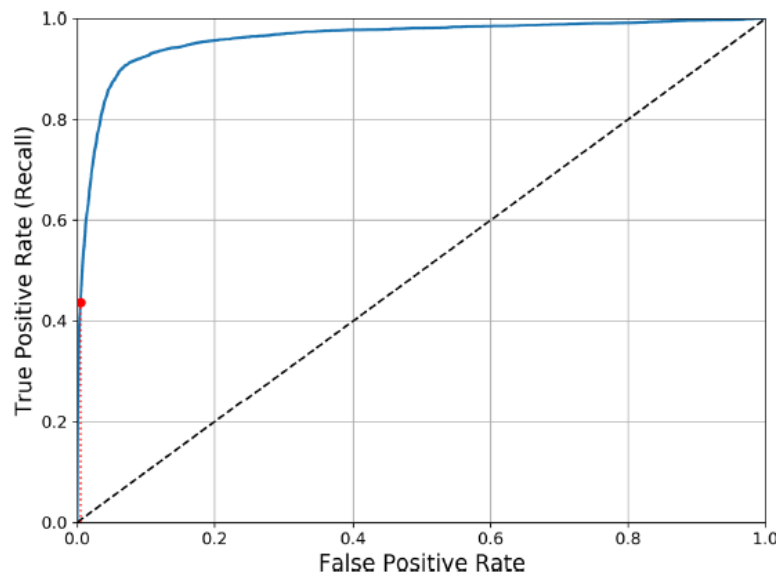


Figura 13-2: Curva ROC.

Para diferentes valores de hiperparâmetros, teremos diferentes curvas ROC. Uma forma de comparar diferentes curvas, é observar a forma de cada uma delas e determinar a que fornece a melhor combinação de TPR e FPR. Usualmente, quanto mais perto do extremo superior esquerdo estiver o ponto melhor. No entanto, isso não é muito conveniente. Uma forma mais prática de comparar essas curvas é usar a métrica **AUC** (*Area Under the Curve*). Quanto maior o AUC, melhor é o modelo. Um modelo perfeito terá AUC igual a 1.

13.1.8 Curva *Precision-Recall*

Uma alternativa à curva ROC é a curva *Precisão x Recall*, que normalmente é utilizada quando temos um número de amostras negativas muito maior do que o de amostras positivas. Reparar que, tanto na ROC quanto na *precision-recall*, o *recall* está em um dos eixos, com o detalhe de que na curva ROC, ela geralmente vem indicada com a nomenclatura *true positive rate*. Portanto, a diferença entre as duas está na métrica utilizada para contabilizar os falsos positivos: na ROC é o FPR e na *precision-recall* é a precisão.

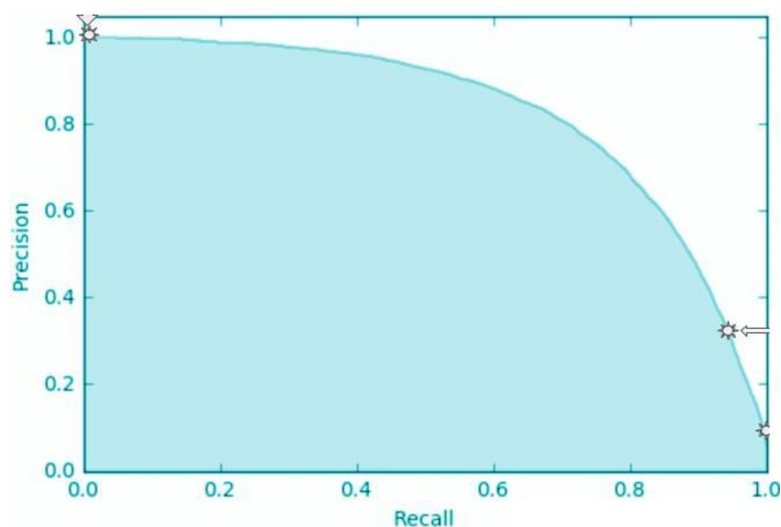


Figura 13-3: Curva Precision-Recall.

De maneira análoga, também podemos utilizar a área debaixo da curva *precision-recall* para avaliar a performance do modelo nos casos em que a classe majoritária é a negativa. Essa métrica é chamada de precisão média. Ela recebe esse nome porque a área debaixo da curva *precision-recall* é dada por

$$\frac{1}{T} \int_0^T P(R) dR,$$

com $T = 1$, que é justamente a média da precisão ao longo dos diferentes valores de *recall*. Em geral, isso não é equivalentemente a dizer que essa expressão é a média da precisão para todos os valores de limiar de probabilidade α , mesmo que $\alpha = f(R)$.

Para visualizar isso, consideremos o caso em que f é bijetora e temos $f^{-1} = g$. Podemos escrever

$$\frac{1}{T} \int_0^T P(R) dR = \frac{1}{T} \int_0^{f(T)} P(g(\alpha)) dg(\alpha) = \frac{1}{T} \int_0^{f(T)} P(\alpha) g'(\alpha) d\alpha,$$

o que não é a precisão média com respeito a α para qualquer função f . Além disso, é importante reforçar que é necessário que f seja bijetora, o que não é verdade na maior parte dos casos práticos, pois, para diferentes valores de α podemos ter o mesmo valor de *recall*.

13.1.9 Trade-off precisão-recall

Como pode ser visto a partir da curva *precisão-recall*, existe uma relação de ganha e perde entre a precisão e o *recall* para um dado modelo e um dado conjunto de hiperparâmetros. Isso acontece porque, ao variar o limiar de classificação α , estamos dando menor peso para falsos positivos ou falsos negativos. Assim, para aumentar a precisão, é necessário diminuir o *recall* e vice-versa.

No entanto, ao variarmos o valor de α para ajustar os valores de precisão e *recall*, não estamos melhorando o ajuste do modelo, ou seja, a sua capacidade de reconhecer cada classe e atribuir maiores probabilidades às classes corretas. Para melhorarmos o ajuste do modelo, é necessário melhorar a escolha de hiperparâmetros. Nesse caso, se mantivermos um valor fixo de α e mudarmos os hiperparâmetros, melhoras na precisão tendem a vir acompanhadas de melhoras no *recall*. Isso porque, ao mudarmos os hiperparâmetros, permitimos que o ajuste do modelo seja alterado, de forma que, no caso de um ajuste melhor, o modelo tende a atribuir maiores probabilidades às classes corretas e menores probabilidades às classes incorretas.

O mesmo efeito acontece quando fixamos um número máximo para observações classificadas como positivas. Por exemplo, considere o caso em que peças passam para um modelo classifica-las em defeituosas ou não. Sendo defeituosas, elas passarão por uma análise humana para identificar qual o defeito. No entanto, existe um limite máximo de 100 peças que podem ser analisadas por vez, ou seja, o número máximo de classes classificadas como positivas (defeituosas) é 100. Logo, ao usarmos esse modelo, selecionaríamos as 100 peças com maior probabilidade de serem defeituosas. Nesse caso, ao melhorarmos a precisão, também estamos melhorando o *recall*. Isso fica claro ao percebermos que o denominador das duas métricas é igual nesse caso (igual a 100 para a precisão e igual a "total de peças" – 100 para o *recall*). Assim, uma melhora na precisão tem que vir necessariamente do aumento do número de observações positivas classificadas corretamente, o que aumenta o *recall*.

13.2 MÉTRICAS PARA CLASSIFICAÇÃO MULTICLASSE

Em um problema de classificação multiclasse, ainda podemos utilizar as métricas para o problema binário, mas para cada classe. Por exemplo, se tivermos 3 classes, A, B e C, poderíamos analisar cada classe individualmente, calculando as métricas da Seção 13.1 para cada uma das classes. Para isso usamos uma estratégia *one-vs-the-rest*, ou seja, consideramos a classe em questão como positiva e todas as outras como negativa. Por exemplo, para calcular a precisão para a classe A, consideramos a classe A como positiva e as classes B e C como negativas.

Apesar de podermos usar essa estratégia, isso faz com que tenhamos várias métricas, o que torna a análise complicada, principalmente quando o número de classes é grande. Por isso, é importante definir métricas que resumam a performance global do modelo. Essas métricas são apresentadas a seguir. Na definição dessas métricas, consideraremos um conjunto de classes Ω , com cardinalidade $K \equiv |\Omega|$, e utilizaremos o termo C_{rp} para representar a contagem de casos pertencentes à classe r ('real') que foram classificados como classe p ('predito').

13.2.1 Acurácia

$$A = \frac{\sum_{r=p} C_{rp}}{\sum_{r,p \in \Omega} C_{rp}}$$

É o total de acertos sobre o total de casos.

Quando o número de exemplos de cada classe é desbalanceado, essa métrica pode mascarar o desempenho de classes com baixa contagem de exemplos. Assim, se uma classe minoritária tiver uma taxa de erros muito elevada, mas todas as outras classes tiverem bom desempenho, a acurácia terá a assumir valores mais altos, mascarando o mau desempenho da classe minoritária.

13.2.2 Acurácia Balanceada

$$A_b = \frac{1}{K} \sum_{\omega \in \Omega} R_{\omega},$$

em que $R_{\omega} = \frac{C_{\omega\omega}}{\sum_{p \in \Omega} C_{\omega p}}$ é o *recall*/TPR de uma classe ω .

Logo, a acurácia balanceada é a média dos valores de TPR para as diferentes classes.

Essa métrica atribui importâncias iguais para cada classe, em vez de atribuir importâncias iguais para cada amostra independentemente da classe, como é o caso da acurácia. Portanto, se considerarmos um exemplo em que há uma classe com poucas amostras e muitos erros de classificação e outras classes com várias amostras e boa performance, teremos $A > A_b$. Isso porque a classe minoritária terá um papel para o cálculo da acurácia balanceada tão importante quanto as outras.

Para conjuntos de dados balanceados, não faz sentido usar essa métrica, pois ela tende a se igual à acurácia. Além disso, mesmo que o conjunto de dados seja desbalanceado, se a taxa de acertos global for mais importante do que controlar a taxa de acertos em cada classe, talvez a acurácia seja uma métrica mais recomendada do que a acurácia balanceada.

13.2.3 Métricas Macro

Macro é uma maneira de calcular métricas que considera cada classe como a unidade básica para o cálculo, ou seja, é a média das métricas calculadas para cada classe no esquema *one-vs-all* (uma classe é considerada positiva e as outras negativas).

Consideremos que $P_\omega = \frac{c_{\omega\omega}}{\sum_{r \in \Omega} c_{r\omega}}$, $R_\omega = \frac{c_{\omega\omega}}{\sum_{p \in \Omega} c_{\omega p}}$ e $F_{1\omega} = \frac{2P_\omega R_\omega}{P_\omega + R_\omega}$ são, respectivamente, a precisão, o *recall* e o f1-score para uma certa classe ω , considerando que essa classe é a positiva e todas as outras negativas.

Dessa forma, a precisão, o *recall* e o f1-score macro são dados, respectivamente, por:

$$\begin{aligned} P_{macro} &= \frac{1}{K} \sum_{\omega \in \Omega} P_\omega ; \\ R_{macro} &= \frac{1}{K} \sum_{\omega \in \Omega} R_\omega ; \\ F_{1macro} &= \frac{1}{K} \sum_{\omega \in \Omega} F_{1\omega}. \end{aligned} \tag{13.1}$$

É interessante mencionar que em [31], é dada uma definição diferente para o f1-score macro, em que essa métrica é calculada a partir da média harmônica de P_{macro} e R_{macro} , ou seja, $2 \frac{P_{macro} R_{macro}}{P_{macro} + R_{macro}}$. No entanto, a definição usada na biblioteca *sklearn* do Python não é a que foi apresentada na eq. (13.1).

Além disso, é interessante observar que a acurácia balanceada é igual ao *recall* macro. No entanto, a acurácia macro não é igual à acurácia balanceada.

13.2.4 Métricas Macro Ponderadas (*weighted*)

São métricas calculadas de maneira similar às macro, porém ponderando as métricas de cada classe usando o número de amostras de cada uma delas. Dessa forma, a precisão, o *recall* e o f1-score macro ponderados são dados por

$$\begin{aligned} P_w &= \frac{\sum_{\omega \in \Omega} N_\omega P_\omega}{\sum_{\omega \in \Omega} N_\omega} ; \\ R_w &= \frac{\sum_{\omega \in \Omega} N_\omega R_\omega}{\sum_{\omega \in \Omega} N_\omega} ; \\ F_{1w} &= \frac{\sum_{\omega \in \Omega} N_\omega F_{1\omega}}{\sum_{\omega \in \Omega} N_\omega}. \end{aligned}$$

N_ω é o total de amostras pertencentes à classe ω .

Essas métricas devem ser utilizadas se for desejável ter métricas que resumam o desempenho global do modelo levando em consideração a quantidade de amostras em cada classes. As classes com poucas amostras receberão menos importância e as classes com mais amostras receberam mais importância no resultado final. Caso se queira dar a mesma importância para todas as classes, mas ainda assim ter métricas globais, é melhor utilizar as métricas macro.

13.2.5 Métricas Micro

Essas métricas são calculadas de maneira global, sem diferenciar amostras individuais pela classe a qual elas pertencem, considerando o número total de TP, FP e FN. Dessa forma, a precisão micro e o *recall* são dados por

$$P_{micro} = \sum_{\omega \in \Omega} \frac{C_{\omega\omega}}{\sum_{\gamma \in \Omega} C_{\gamma\omega}} = \frac{\sum_{\omega \in \Omega} C_{\omega\omega}}{\sum_{\omega \in \Omega} \sum_{\gamma \in \Omega} C_{\gamma\omega}};$$
$$R_{micro} = \sum_{\omega \in \Omega} \frac{C_{\omega\omega}}{\sum_{\gamma \in \Omega} C_{\omega\gamma}} = \frac{\sum_{\omega \in \Omega} C_{\omega\omega}}{\sum_{\omega \in \Omega} \sum_{\gamma \in \Omega} C_{\omega\gamma}}.$$

Reparar que $P_{micro} = R_{micro}$. Portanto, o F1-score é dado por

$$F_{1micro} = 2 \frac{P_{micro} R_{micro}}{P_{micro} + R_{micro}} = P_{micro} = R_{micro}.$$

Por fim, reparar que $P_{micro} = R_{micro} = F_{1micro}$ são iguais à acurácia, pois essas métricas consistem na razão entre os acertos e o total de amostras.

13.3 MÉTRICAS PARA REGRESSÃO

As principais métricas usadas para regressão são apresentadas abaixo.

13.3.1 Mean Squared Error (MSE)

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - t_i)^2$$

Vantagem:

- É diferenciável.

Desvantagem:

- Não é medido nas mesmas unidades que t_i e y_i .
- Como os resíduos são elevados ao quadrado, é dado um peso maior para *outliers*.

13.3.2 Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

Vantagem:

- É diferenciável.
- É medido nas mesmas unidades que t_i e y_i .

Desvantagem:

- Como os resíduos são elevados ao quadrado, é dado um peso maior para *outliers*.
- Apesar de ser medido nas mesmas unidades que t_i , o RMSE não representa a diferença média entre o *target* e o valor predito. Um RMSE igual a 10 não significa que o modelo erra em 10 na média.

13.3.3 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - t_i|$$

Vantagem:

- Como os resíduos não são elevados ao quadrado, todos os erros são pesados de maneira igual.
- É medido nas mesmas unidades que os resíduos, t_i e y_i .

Desvantagem:

- Não é facilmente diferenciável.

13.3.4 Mean Absolute Percentage Error (MAPE)

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \left| \frac{y_i - t_i}{t_i} \right|$$

Vantagem:

- É uma porcentagem. Logo, é independente de escala e pode ser comparado com MAE de modelos de diferentes conjuntos de dados.

Desvantagem:

- Não é facilmente diferenciável.

13.3.5 Coeficiente de Correlação de Pearson (PCC)

$$r_{yt} = \frac{S_{yt}}{S_y S_t}$$

S_y e S_t são as variâncias amostrais de y e t , respectivamente, e S_{yt} é a covariância amostral entre y e t .

13.3.6 Coeficiente de Determinação (R^2)

13.3.6.1 Modelos lineares

O coeficiente de determinação surge naturalmente para avaliação de modelos de regressão lineares. Considerando que $\bar{t}$ e S_t são, respectivamente, a média e a variância amostrais dos *targets*, a seguinte expressão é válida para o coeficiente de determinação para modelos de regressão linear [32, p. 68]

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - t_i)^2}{\sum_{i=1}^N (t_i - \bar{t})^2} = 1 - \frac{\text{MSE}}{S_t} = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = \frac{\text{SS}_{\text{reg}}}{\text{SS}_{\text{tot}}}, \quad (13.2)$$

em que [32, p. 44]

$$\text{SS}_{\text{res}} = \text{MSE} \times N = \sum_{i=1}^N (y_i - t_i)^2 \quad \text{:Sum of Squared Residuals (também chamada de variância não explicada pelo modelo)}$$

$$SS_{\text{tot}} = S_t \times N = \sum_{i=1}^N (t_i - \bar{t})^2 \quad \text{:Total Sum of Squared Deviations}$$

$$SS_{\text{reg}} = \sum_{i=1}^N (y_i - \bar{t})^2 \quad \text{:Sum of Squared residuals when considering Regression (também chamada de variância explicada pelo modelo)}$$

Reparar que $SS_{\text{tot}} = SS_{\text{res}} + SS_{\text{reg}}$. Intuitivamente, isso até faz sentido, já que $t_i = y_i +$ resíduo. Assim, o que esse coeficiente indica é a porcentagem da variância total $S_t = \frac{SS_{\text{tot}}}{N}$ da variável dependente t_i que é explicada pelo modelo. Se o modelo for capaz de explicar toda a variação da variável dependente, ou seja, prever perfeitamente a variável dependente, a variância não explicada SS_{res} será igual a 0, pois não há nenhuma variação de *target* que o modelo não explique. A porção, portanto, que é explicada é 1 menos a porção não explicada, que no caso específico da regressão linear se iguala a $\frac{SS_{\text{reg}}}{SS_{\text{tot}}}$.

A título de exemplo, $R^2 = 0,75$ significa que o modelo explica 75% da variação da variável dependente. Os outros 0,25 vem do *gap* entre as previsões y_i e t_i . Essa parte não explicada pode ser resultado de: ruído aleatório; variáveis independentes importantes omissas; modelo não capturar a verdadeira relação entre as variáveis independentes e a dependente.

Outra forma de interpretar essa métrica é olhando para a primeira igualdade. O termo $\frac{MSE}{S_t}$ é uma forma de reportar o MSE, porém, comparado a uma grandeza de referência, como a variância amostral S_t . O R^2 portanto, é o complementar disso.

Por fim, uma última interpretação pode ser feita. O termo SS_{tot} representa o SS'_{res} de um modelo de regressão que sempre tem como saída a média $\bar{t}$. Logo, estamos comparando o SS_{reg} do modelo com o SS'_{res} de um modelo trivial que sempre prevê a média. De forma mais intuitiva, se $\frac{SS_{\text{res}}}{SS_{\text{tot}}} = \frac{SS_{\text{res}}}{SS'_{\text{res}}}$ dá uma ideia de quanto um modelo é **pior** comparado a outro que sempre prevê a média, $R^2 = 1 - \frac{SS_{\text{res}}}{SS'_{\text{res}}}$ dá uma ideia de quanto o **modelo** é melhor que outro que sempre prevê a média.

$R^2 = 1$ quando o modelo sempre acertar as previsões, ou seja, quando $MSE = SS_{\text{reg}} = 0$.

$R^2 = 0$ quando o modelo for substituível pelo modelo que sempre prevê a média, ou seja, $SS_{\text{reg}} = SS_{\text{tot}}$.

$R^2 < 0$ quando o modelo for pior do que o modelo que sempre prevê a média, ou seja, $MSE > S_t$. Nesse caso, seria melhor simplesmente prever a média.

13.3.6.2 Modelos não-lineares

Para modelos não-lineares, a equação (13.2) não é válida [33]. Isso porque $SS_{\text{tot}} \neq SS_{\text{res}} + SS_{\text{reg}}$ e, por isso, $1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} \neq \frac{SS_{\text{reg}}}{SS_{\text{tot}}}$. Dessa forma, o coeficiente de determinação é dado por

$$R^2 = 1 - \frac{MSE}{S_t} = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

Assim como no caso linear, $R \leq 1$, sendo

$R^2 = 1$ quando o modelo acertar todas as previsões;

$R^2 = 0$ quando o modelo for equivalente a prever sempre a média;

$R^2 < 0$ quando o modelo for pior do que o modelo que só prevê a média.

13.4 MÉTRICAS PARA AGRUPAMENTO

13.4.1 Inércia

$$I = \sum_{n=0}^N \min_j (\|x_n - \mu_j\|^2)$$

É a soma das distâncias entre cada ponto e o centroide μ_j do *cluster* ao qual ele pertence, ou seja, o centroide mais próximo desse ponto, em que $j \in [1, K]$, sendo K o número de *clusters*.

Pode ser usada para escolher o número de *clusters* de algoritmos em que esse número é um hiperparâmetro. Para isso, traça-se um gráfico da inércia em função de K , como mostrado na Figura 13-4. O melhor valor de K tende a se localizar no ponto de “cotovelo”. Por esse motivo, esse método de encontrar o melhor valor de K é chamado de [método do cotovelo](#).

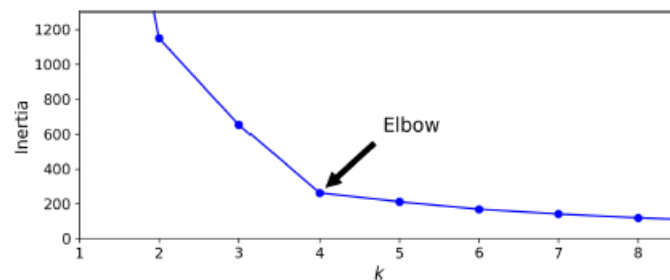


Figura 13-4: Curva de inércia em função de K .

A eficácia da inércia para medir a qualidade de agrupamentos é maior quando os *clusters* são convexos e têm formatos regulares (como hiperesferas). Por exemplo, *clusters* em formato de anel ou lua crescentes não deveriam ser avaliados, idealmente, usando essa métrica. A Figura 13-5 mostra como a inércia penaliza mais *clusters* não convexos. Apesar de nos dois casos os dados estarem muito bem agrupados, a inércia do *cluster* vermelho é mais do que o dobro da inércia do *cluster* azul.

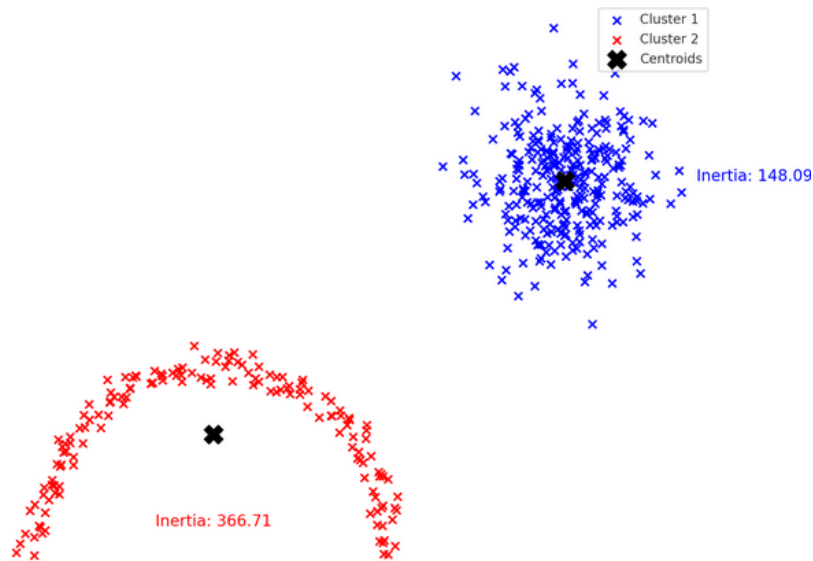


Figura 13-5: Diferença no valor da inércia para clusters de diferentes formatos.

13.4.2 Coeficiente de Silhueta

$$S(x) = \frac{b_x - a_x}{\max\{b_x, a_x\}},$$

em que

a_x : média das distâncias do ponto x para os outros pontos pertencentes ao mesmo cluster.
 b_x : média das distâncias do ponto x para os outros pontos pertencentes ao *cluster* vizinho mais próximos.

Alguns pontos importantes com relação ao significado do valor de $S(x)$:

- $S(x) \in [-1, 1]$
- $S(x) \approx 1 \rightarrow x$ está bem próximo dos pontos “irmãos”, pertencentes ao mesmo *cluster*.
- $S(x) \approx -1 \rightarrow x$ está bem próximo dos pontos do *cluster* vizinho e distante dos seus irmãos. Talvez até tenha sido alocado no *cluster errado*.
- $S(x) \approx 0 \rightarrow x$ está a uma distância mais ou menos igual dos seus irmãos e dos seus vizinhos, possivelmente na fronteira entre os dois *clusters* (mas não necessariamente).

13.4.3 Score de Silhueta

$$\bar{S} = \frac{1}{N} \sum_{i=1}^N S(x_i)$$

em que N é o número total de dados no conjunto de dados.

Portanto, o score de silhueta é a média dos coeficientes de silhueta de todos os pontos. Consequentemente,

$$\bar{S} \in [-1, 1].$$

Podemos usar o score de silhueta para determinar o número ideal de *clusters* de duas maneiras. A primeira delas, consiste em traçar a curva do score de silhueta em função do número K de *clusters* para um mesmo algoritmo de clusterização, como mostrado na Figura 13-6. O número ideal de K tende a ser aquele com maior valor de $\bar{S}$.

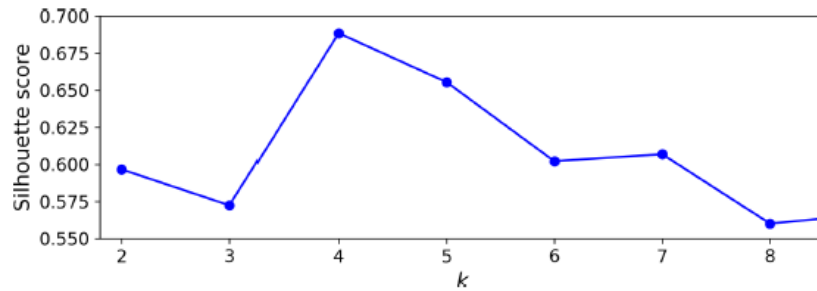


Figura 13-6: Score de silhueta em função do número de clusters.

Outra maneira de se determinar o valor de K é usar o método da silhueta, como mostrado na Figura 13-7.

A linha vermelha tracejada representa o valor de $\bar{S}$ para aquele valor de K . Cada gráfico em forma de faca é formado juntando-se os coeficientes de silhueta para cada ponto e ordenando-os. Assim, gráficos de “faca” mais altos possuem mais pontos e os mais largos apresentam pontos com maior coeficiente.

O valor de K não é adequado quando a maior parte dos valores de coeficiente estão abaixo da linha tracejada vermelha. Quanto mais para a direita da linha tracejada vermelha, melhor é o valor de K .

Esse método é mais custoso computacionalmente que o método do cotovelo.

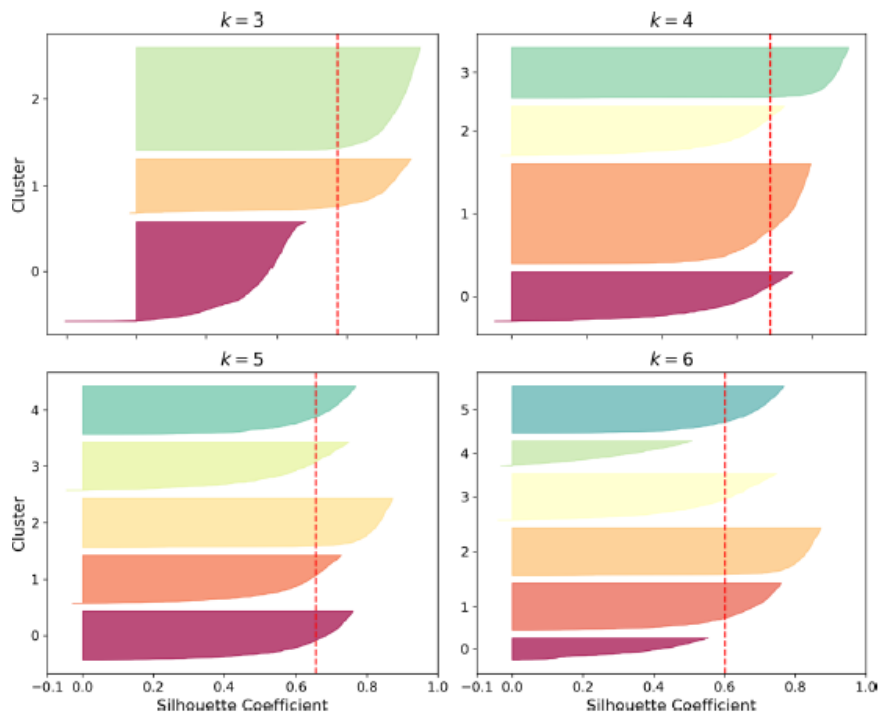


Figura 13-7: Método da silhueta para diferentes valores de K .

A eficácia do score de silhueta para medir a qualidade de agrupamentos é maior quando os *clusters* são convexos e têm formatos regulares (como hiperesferas). Por exemplo, *clusters* em formato de anel ou lua crescentes não deveriam ser avaliados, idealmente, usando essa métrica.

13.4.4 Akaike Information Criterion (AIC) e Bayesian Information Criterion (BIC)

$$\text{AIC} = 2h - 2 \log \mathcal{L}(\theta^*)$$

$$\text{BIC} = h \log N - 2 \log \mathcal{L}(\theta^*)$$

em que

h : número de parâmetros do modelo.

N : número de instâncias de dados de treinamento.

$\mathcal{L}(\theta^*)$: máxima verossimilhança (dada para um valor de θ^* específico que maximiza essa verossimilhança).

O AIC mede a qualidade de um modelo usando como critério sua complexidade e seu ajuste aos dados. Quanto menor o valor de AIC, melhor é o modelo. Assim, ao utilizar o AIC como critério para escolha de um modelo, estamos tentando encontrar o modelo que melhor explica os dados (maior verossimilhança possível) com a menor complexidade possível (menor número de parâmetros possível).

O BIC também mede a qualidade de um modelo a partir de um equilíbrio entre complexidade e explicabilidade dos dados, porém, ele também leva em consideração o tamanho da amostra de treinamento utilizada. Assim, diferentemente do AIC, podemos explorar o comportamento de um modelo ao variarmos o valor de N . Mantendo-se constantes h e $\mathcal{L}(\theta^*)$, com o BIC, podemos variar o valor de N e tentar responder a pergunta “Um aumento no número de instâncias de treinamento traz benefícios suficientes ao modelo que justifique esse aumento?”. Ao aumentar N , a tendência é que o modelo melhore o seu ajuste dos dados, o que tende a aumentar $\mathcal{L}(\theta^*)$. No entanto, nem todo aumento de $\mathcal{L}(\theta^*)$ justifica o uso de mais dados.

Por fim, para um valor fixo de N , as duas métricas mostram tendências semelhantes, como ilustrado na FIGURA, que compara a performance de um modelo de mistura de gaussianas para diferentes números de *clusters*.

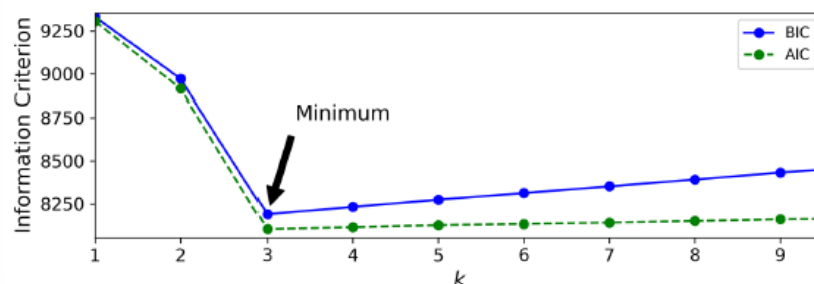


Figura 13-8: AIC e BIC em função do número de clusters de um modelo de mistura de gaussianas.

É importante ressaltar que, o AIC e o BIC não são métricas exclusivas de algoritmos de agrupamentos. Porém, eles podem ser úteis para analisar modelos em que temos fácil acesso

às expressões das verossimilhanças, como é o caso da mistura de gaussianas, e quando a penalização da complexidade é importante.

13.4.5 Índice de Calinski-Harabasz

$$CH = \frac{BGSS/(K - 1)}{WGSS/(N - K)},$$

com

$$BGSS = \sum_{k=1}^K N_k \|\boldsymbol{\mu}_k - \boldsymbol{\mu}\|^2,$$

$$WGSS = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2.$$

K : número de *clusters*.

N : número de pontos.

C_k : *cluster* de índice k .

N_k : número de pontos no *cluster* C_k

$\mathbf{x}$: ponto do conjunto de dados.

$\boldsymbol{\mu}$: centroide global (média) de todos os pontos.

$\boldsymbol{\mu}_k$: centroide do *cluster* C_k .

O índice de Calinski-Harabasz [34] é uma métrica comumente utilizada para avaliar a qualidade de soluções de agrupamentos. Assim como o score de silhueta, essa métrica também leva em consideração a compactação interna de cada *cluster* e a separação entre eles, só que de maneira diferente. Vamos analisar separadamente o numerador e o denominador de CH.

O denominador pode ser reescrito como

$$\frac{WGSS}{N - K} = \frac{\sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2}{N - K} = \frac{\sum_{k=1}^K (N_k - 1) S_k}{N - K} = \bar{S}_{ww},$$

em que S_k é a variância amostral dos pontos presentes no *cluster* C_k e $\bar{S}_{ww}$ é a média ponderada das variâncias dos K *clusters*. Reparar que podemos dizer isso porque $\frac{\sum_{k=1}^K (N_k - 1)}{N - K} = 1$.

Já o numerador pode ser reescrito como

$$\frac{BGSS}{K - 1} = \frac{\sum_{k=1}^K N_k \|\boldsymbol{\mu}_k - \boldsymbol{\mu}\|^2}{K - 1} = S_{wb},$$

em que S_{wb} é a variância amostral ponderada dos centroides dos K *clusters*, mas com pesos N_k não normalizados, já que não há um termo N no numerador. Com isso, é dado mais importância para as distâncias $\|\boldsymbol{\mu}_k - \boldsymbol{\mu}\|^2$ cujos *clusters* C_k têm mais pontos.

Portanto, podemos dizer que o coeficiente de Calinski-Harabasz é a razão entre a variância ponderada dos centroides e a média da variância interna de cada *cluster*. Assim, o primeiro termo diz respeito à separação dos *clusters* e o denominador à compactação deles.

É importante ressaltar que

$$CH \in [0, \infty),$$

em que valores pequenos de CH indicam uma *clusterização* ruim (*clusters* mais dispersos e mais juntos) e valores elevados de CH indicam uma boa *clusterização* (*clusters* mais compactos e mais separados).

13.5 MÉTODOS DE VALIDAÇÃO

Capítulo 14:

Capítulo 15: APÊNDICES

Abaixo são apresentados os apêndices.

Anexo A: COMPARAÇÕES ENTRE MODELOS

A.1. COMPLEXIDADE MÉDIA

A complexidade de teste indicada é referente a uma amostra de teste.

Modelo	Complexidade	
	Treino	Teste
<i>Naïve Bayes</i>	$O(N \times d)$	$O(d)$
KNN (K-D tree)	$O(d \times N \times \log_2 N)$	$O(K \times \log_2 N)$
KNN (<i>ball tree</i>)	$O(d \times N \times \log_2 N)$	$O(K \times \log_2 N)$
KNN (<i>brute force</i>)	$O(1)$	$O(K \times N \times d)$
Regressão logística	$O(N \times d \times i)$	$O(d)$
Regressão linear	$O(N \times d^2 + d^3)$ (forma analítica) $O(N \times d \times i)$ (Gradiente Descendente)	$O(d)$
Árvore de decisão	$O(N \times d \times \log_2 N)$	$O(\log_2 N)$
<i>Random forest</i>	$O(T \times N \times d \times \log_2 N)$	$O(T \times \log_2 N)$
<i>Gradient Boosting</i>	$O(T \times N \times d \times \log_2 N)$	$O(T \times \log_2 N)$
SVM (kernel linear)	de $O(N \times d)$ a $O(N^2 \times d)$	$O(d)$
SVM (kernel não-linear)	de $O(N^2 \times d)$ a $O(N^3)$	$O(s \times d)$
Rede Neural	$O\left(e \times N \times \sum_{\ell=1}^L k_{\ell} k_{\ell-1}\right)$	$O\left(\sum_{\ell=1}^L k_{\ell} k_{\ell-1}\right)$

N : número de observações

d : dimensão da variável de entrada

K : número vizinhos nos KNN

T : número de árvores de decisão

i : número de iterações do algoritmo de otimização

s : número de vetores de suporte

k_{ℓ} : número de neurônios na camada ℓ da rede neural

L : número de camadas da rede neural

e : número de épocas

Com relação ao treinamento os modelos mais complexos são a rede neural e o SVM. Com relação ao teste os mais complexos são a rede neural, o KNN e o SVM.

Já os mais simples no treinamento são o *naïve Bayes* e o KNN (*brute force*) e no teste são o *naïve Bayes*, a regressão logística e a regressão linear.

A.2. SENSIBILIDADE A *OUTLIERS*

Na tabela abaixo, se nenhuma consideração for feita entre parênteses, deve-se considerar a construção tradicional (*vanilla*) do modelo.

Modelo	Robusto a outlier
<i>Naïve Bayes</i>	Não
KNN	Não
KNN (<i>weighted</i>)	Sim
Regressão logística	Não
Regressão linear	Não
Regressão <i>lasso</i>	Sim
Regressão <i>ridge</i>	Sim
Árvore de classificação	Sim
Árvore de regressão	Sim (-)
<i>Random forest</i>	Sim
<i>Gradient Boosintg</i>	Sim
SVM (<i>hard margin</i>)	Não
SVM (<i>soft margin</i>)	Sim
Rede Neural	Não

Anexo B: TEORIA DA INFORMAÇÃO

B.1. CASO DISCRETO

B.1.1. QUANTIDADE DE INFORMAÇÃO

A **quantidade de informação** (ou surpresa ou informação de Shannon) é uma grandeza que quantifica a raridade (ou a improbabilidade) de observar um valor x da variável aleatória $X: \Omega \rightarrow \mathcal{X}$, com p.m.f $p(x) = \mathbb{P}(X = x)$. Muitas vezes, a quantidade de informação também é interpretada como a “surpresa” por observar o valor de x .

A expressão da quantidade de informação é dada por

$$I(x) = -\log p(x). \tag{B.1}$$

Quando a base do logaritmo é 2, a informação é medida em **bits**. Quando o logaritmo é natural, a informação é medida em **nats**. Em *machine learning*, usualmente utilizamos o logaritmo natural.

Reparar que, como $p(x) \in [0,1]$, $I(x) > 0$.

Com isso, vemos que quanto mais raro for um evento, maior é a informação associada a ele. Então, a informação de que “hoje o sol nasceu”, que tem probabilidade praticamente 1, teria informação nula, já que não é nenhuma surpresa de que o sol nasce todos os dias. Se por acaso, algum dia o sol não nascesse, a informação teria valor infinito, pois é algo com probabilidade praticamente 0 de acontecer.

Tentando dar outra interpretação intuitiva para essa grandeza, é como se ela medisse o quanto uma informação é informativa. Assim, “o sol nasceu” é pouco informativo e, por isso, tem informação nula.

Às vezes, a quantidade de informação é interpretada como a medida de incerteza do evento. No entanto, é preciso ter cuidado com essa interpretação, pois a incerteza não é do observador com relação a um evento sendo observado, mas sim, do próprio evento. Abaixo são mostradas as duas situações e seus antônimos:

- incerteza do observador – é o grau de dúvida do observador se o evento vai acontecer ou não;
- certeza do observador – é o grau de confiança do observador se o evento vai acontecer ou não;
- incerteza do evento – é o grau de improbabilidade do evento;
- certeza do evento – é o grau de plausibilidade do evento (quão certo ele é).

B.1.2. QUANTIDADE DE INFORMAÇÃO CONDICIONAL

A quantidade de informação condicional é a quantidade de informação associada ao valor observado x por conhecer o valor observado y da v.a. Y . Ela é dada por

$$I(x|y) = -\log p(x|y). \quad (\text{B.2})$$

Continuamos tendo a mesma interpretação da quantidade de informação. Só trocamos a p.m.f $p(x)$ por outra p.m.f $g(x) = p(x|y)$.

É importante ressaltar que, a quantidade de informação condicional NÃO é a parcela de quantidade de informação (ou “surpresa”) de x que se esvai por conhecer y , ou seja, não é a parcela da incerteza de x compartilhada com y . Isso ficará claro mais para frente.

B.1.3. ENTROPIA

A entropia é a quantidade de informação média (ou raridade média) de uma variável aleatória, ou seja, é a soma da quantidade de informação para cada valor observado x ponderado pela sua probabilidade (ou “frequência”, sob uma ótica frequentista). Sua expressão é dada por

$$H(X) = \mathbb{E}[I(X)] = - \sum_{x \in \mathcal{X}} p(x) \log p(x). \quad (\text{B.3})$$

Observar que $H(X) \geq 0$, pois $\ln p(x_i) \leq 0$, já que $p(x) \in [0,1]$.

A entropia é mínima quando $X \sim \delta_c$, ou seja, $\mathbb{P}(X = c) = p(c) = 1$. Nesse caso, a entropia é $H(X) = 0$.

A entropia é máxima quando $X \sim \mathcal{U}(a, b)$. Nesse caso, a entropia é dada por $H(X) = \ln M$, com $M = b - a + 1$.

De maneira intuitiva, podemos dizer que a entropia mede a incerteza do observador com relação a um valor amostrado aleatoriamente de X . No caso de uma distribuição de massa pontual, a incerteza do observador é nula, já que temos um único evento possível. Já no caso de uma distribuição uniforme, a incerteza do observador é máxima, já que todos os eventos tem a mesma probabilidade.

Pode parecer estranho pensar que estamos calculando o valor esperado de uma função $I(X) = g(X)$ que depende da própria densidade de X . No entanto, $p(x)$ é só mais uma função que depende dos parâmetros dessa distribuição. Por exemplo, se $X \sim \text{Poi}(\lambda)$, então $p(x) = \frac{\lambda^x e^{-\lambda}}{x!}$ e $I(X) = \log\left(\frac{\lambda^x e^{-\lambda}}{x!}\right)$, ou seja, só mais uma função qualquer.

$I(X) = Y$ é uma variável aleatória, de forma que $\mathbb{P}(Y = y)$ responde a pergunta “qual a probabilidade de eu sortear um valor de X e ele ter informação igual a y ?” (lembrando que diferentes valores de x podem estar associados a um mesmo valor de informação). Para calcular essa probabilidade, consideramos todos os valores de x associados ao mesmo valor de $I(x)$, multiplicamos por suas respectivas probabilidades $p(x)$ de serem amostrados e depois somamos-los, ou seja, $\mathbb{P}(Y = y) = p(y) = \sum_{x_i: I(x_i)=y} p(x_i) I(x_i)$. Logo, $\mathbb{E}(Y) = \sum_{y \in \mathcal{Y}} p(y) y = \sum_{x \in \mathcal{X}} p(x) I(x)$.

Como isso, um valor esperado $\mathbb{E}[I(X)]$ baixo indica que, em média, a raridade associada aos valores amostrados de X é baixa. Isso só pode acontecer se X tiver uma distribuição de probabilidades muito concentrada, pois, assim, teremos poucos valores de x que serão amostrados com muita frequência de X e que apresentam, consequentemente, $I(x)$ baixo (pois não são raros), e teremos alguns valores com $I(x)$ elevado (bem mais raros) que serão

amostrados com frequência muito baixa e que, portanto, contribuirão pouco para a média. Logo, $\mathbb{E}[I(X)]$ **baixo indica que a distribuição de X é muito concentrada** e que temos, consequentemente, menos incerteza com relação ao valor que será amostrado.

A lógica para $\mathbb{E}[I(X)]$ elevado é a inversa. Nesse caso, para que $\mathbb{E}[I(X)]$ tivesse valor alto, seria ideal que todos os valores de x fossem igualmente raros, pois assim, qualquer valor de x , por mais raro que seja, teria a mesma probabilidade de ser amostrado e contribuiria com um valor de $I(X)$ elevado para a média. Logo, $\mathbb{E}[I(X)]$ **indica que a distribuição de X é bem distribuída** e que a nossa incerteza com relação ao valor que será amostrado de X é muito alta, pois qualquer um poderia ser amostrado.

Obviamente, se X está limitado a apenas dois valores, por exemplo, ao tentar fazer com que todos seus valores sejam igualmente raros, teríamos que fazer com que seus valores tivessem 50 % de probabilidade de serem amostrados, que não é bem o que chamaríamos de eventos raros. Mesmo assim, seria a distribuição que leva a maior incerteza possível, levando a $H(X) = 0,69$. Mas, quanto maior a gama de valores que X pode assumir, maior é a incerteza (se a imagem de X tiver cardinalidade 100, por exemplo, cada $x \in Im(X(\omega))$ tem apenas 1 % de chance de ser amostrado, no caso de máxima incerteza, que levaria a $H(X) = 34,66$).

B.1.4. ENTROPIA CONDICIONAL

É a quantidade de informação condicional média. Dessa forma, nos baseando na eq. (B.3) e na *Law of The Unconscious Statistician* (LOTUS) [5, p. 170], também conhecida como *Rule of the Lazy Statistician* [4, p. 48], podemos escrever

$$H(X|Y) = \mathbb{E}[I(X|Y)] = - \sum_{y \in \mathcal{Y}} \sum_{x \in \mathcal{X}} p(x, y) \log p(x|y). \quad (B.4)$$

Lembrando que estamos encarando $I(X|Y) = g(X, Y)$ como uma função das variáveis aleatórias X e Y .

B.1.5. DIVERGÊNCIA KULLBACK-LEIBLER

A **divergência Kullback-Leibler** ou **entropia relativa** é uma medida utilizada para quantificar a diferença entre uma distribuição $p(x)$ e uma distribuição de referência $q(x)$. Ela é dada pela expressão

$$KL(p||q) = \mathbb{E}[I_q(x) - I(x)] = \mathbb{E}\left[\log \frac{p(x)}{q(x)}\right] = \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)}, \quad (B.5)$$

que se lê como “divergência KL de p para q ” e com isso estamos calculando o quanto a distribuição p diverge de q . $I(x)$ é a informação de X , que tem distribuição $p(x)$, e $I_q(x)$ é a informação de X quando aproximamos sua distribuição por q . Vemos que p é tido como referência, até mesmo porque multiplicamos $\log \frac{p(x)}{q(x)}$ por $p(x)$ no somatório, pois estamos considerando que a distribuição de X é descrita por $p(x)$ nesse referencial. É importante notar que $KL(p||q) \neq KL(q||p)$, ou seja, ao mudarmos o referencial, a divergência muda. Logo, não podemos dizer que a divergência KL representa uma métrica de distância.

Observar que, como $KL(p||q) = \mathbb{E}[I_q(x) - I(x)]$, temos que

$$KL(p||q) = H_q(X) - H(X),$$

em que $H_q(X)$ é a entropia de X quando aproximamos sua distribuição por q , também chamada de **entropia cruzada**, como veremos adiante.

Em *machine learning*, muitas vezes utilizamos a expressão $KL(p||q)$ para quantificar o quanto uma distribuição desconhecida p é divergente de uma distribuição q que tenta aproximá-la.

É importante reparar que $KL(p||q) \geq 0$ [3, p. 55].

Finalmente, reparamos que existe uma relação entre a informação mútua e a divergência KL, que é ilustrada na equação:

$$I(X; Y) = KL(p(x, y) || p(x)p(y)) \quad (B.6)$$

B.1.6. ENTROPIA CRUZADA

Às vezes, em problemas de *machine learning*, podemos nos deparar com a expressão da **entropia cruzada**, que é dada por

$$H_q(X) = - \sum_{x \in \mathcal{X}} p(x) \log q(x). \quad (B.7)$$

Ela é a “surpresa” média que temos ao codificarmos uma v.a. X que tem distribuição $p(x)$ com uma distribuição $q(x)$.

Se repararmos bem, ela é muito parecida com a equação da divergência KL. De fato, temos que

$$KL(p||q) = H_q(X) - H(X).$$

Considerando que $p(x)$ é a distribuição que dá a codificação ótima para X (que tem menor informação média necessária associada), então a divergência KL é a “surpresa” média por tentarmos utilizar uma distribuição q , que não é ótima (vai exigir mais informação para codificar X), menos a “surpresa” média ao utilizarmos a distribuição ótima p (entropia).

A equação da entropia cruzada surge quando temos uma variável que tem uma distribuição p e queremos aproximá-la com outra distribuição q_θ , parametrizada pelo vetor de parâmetros θ a ser otimizado. Para isso, o mais óbvio a se fazer é minimizar a divergência $KL(p||q)$, o que resulta em

$$\min_{\theta} KL(p||q) = \min_{\theta} H_q(X) - H(X) = \min_{\theta} H_q(X),$$

pois $H(X)$ não depende do vetor de parâmetros θ , já que $p(x)$ também não depende.

Geralmente, a entropia cruzada surge no contexto de classificação, pois ela é usualmente utilizada no cálculo da função custo. Se considerarmos o caso da classificação com K classes $\omega_1, \omega_2, \dots, \omega_K$, teríamos um problema modelado da seguinte forma: $\Omega = \{\omega_1, \omega_2, \dots, \omega_K\}$, representando o espaço amostral; $\mathcal{X} = [1, K]$ representando o espaço imagem da variável

aleatória $X(\omega)$; $q_x = q(x) = \mathbb{P}_q(X = x)$ sendo a p.m.f estimada de X ; $p_x = p(x) = \mathbb{P}(X = x)$ sendo a p.m.f real de X .

Considerando que estejamos avaliando um ponto v , que representa um dado, pertencente a uma classe ω_j , teremos a seguinte expressão para a entropia cruzada:

$$H_q(X) = - \sum_{x=1}^K p_x \log q_x$$

Como a classe do ponto v é conhecida, a distribuição X das classes para esse ponto é $\mathbb{P}(X = j) = 1$, ou seja, $\mathbb{P}(X = x) = p_x = 0$ para $x \neq j$. Logo,

$$H_q(X) = - \sum_{x=1}^K \mathbb{1}_{\{v \in \omega_x\}} \log q_x = - \log q_j.$$

Observar que, no desenvolvimento acima, x não representa os dados! Para evitar confusões, podemos reescrever a equação da entropia cruzada para um determinado ponto v trocando as nomenclaturas da seguinte forma:

$$H_q(W_v) = - \sum_{k=1}^K p_k \log q_k$$

em que W_v é a variável aleatória que representa o número da classe a qual pertence o ponto v .

B.1.7. INFORMAÇÃO MÚTUA

A informação mútua (também chamada de ganho de informação) entre duas v.a. X e Y quantifica a incerteza de um observador com relação a X que pode ser explicada pela variável Y e vice-versa. Em outras palavras, ela quantifica a redução da incerteza do observador com relação a X devido ao conhecimento de Y e vice-versa.

$$I(X; Y) = H(X) - H(X|Y). \quad (\text{B.8})$$

Ela muitas vezes também é definida como a quantidade de informação compartilhada entre as variáveis X e Y . No entanto, é preciso prestar atenção ao fato de que a palavra “informação” neste contexto tem um significado diferente da “informação” que foi apresentada anteriormente. Aqui, “informação” é sinônimo de “conhecimento”, enquanto a “informação” definida anteriormente, dada por $-\log p(x)$, é sinônimo de “raridade”. Portanto, a informação mútua mede o nível de conhecimento compartilhado entre as variáveis X e Y e não tem nada a ver com raridade de eventos. Para evitar ambiguidades será utilizada a expressão “quantidade de conhecimento”.

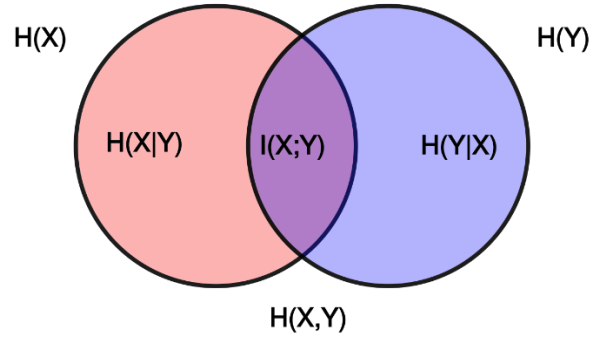


Figura 13-9: Diagrama de Venn que ilustra a informação mútua.

A Figura 13-9 ilustra a relação entre a informação mútua e as outras grandezas da teoria da informação. Cada conjunto representa a quantidade de conhecimento carregada por cada variável aleatória. A interseção dos conjuntos representa a porção de conhecimento que é comum às duas variáveis aleatórias. Cada uma das porções de conhecimento desses conjuntos pode ser quantificada pelas grandezas da teoria da informação. Então, quanto maior a entropia de uma variável aleatória, maior a incerteza associada a ela e maior a quantidade de conhecimento existente para ser descoberto sobre ela. A informação mútua é a quantidade de conhecimento compartilhada pelas duas variáveis, quantificada pela incerteza associada a essa porção de conhecimento.

Uma propriedade da informação mútua é que $I(X; Y) = I(Y; X)$. Além disso, $I(X; Y) \geq 0$, sendo igual a zero somente quando uma v.a. não traz informação (conhecimento) sobre a outra. Para duas variáveis aleatórias X e Y o valor máximo da informação mútua é $\min\{H(X), H(Y)\}$. Por exemplo, no caso em que $Y = 2X$, conhecer X nos permite conhecer completamente Y . Portanto, $H(X|Y) = 0$ e $I(X; Y) = H(X) = H(Y)$, ou seja, a informação mútua é máxima.

No exemplo anterior, a informação mútua era máxima $H(X) = H(Y)$, o que significa que os conjuntos no diagrama de Venn eram de mesmo tamanho e estavam completamente sobrepostos. Outro exemplo de informação mútua máxima, mas com $H(X) \neq H(Y)$ é o caso em que $Y = X \bmod 2$, com $\text{Im}(X) = \{1, 2, 3, 4\}$. Nesse caso, Y não é uma função injetiva, o que faz com que o conhecimento de Y não elimine toda a incerteza sobre X , pois saber que $Y = 1$ não permite dizer qual o valor de X correspondente (pode ser igualmente 1 ou 3). No entanto, conhecer X elimina toda a incerteza com respeito a Y . Logo, $H(X) > H(Y)$, o que faz com que o diagrama de Venn seja um conjunto maior – $H(X)$ – que engloba completamente um conjunto menor – $H(Y)$. Nesse caso, a informação mútua é máxima, com $I(X, Y) = \min\{H(X), H(Y)\} = H(Y) < H(X)$. Portanto, o caso de máxima informação mútua não precisa ser necessariamente o caso em que $X = Y$, basta que haja interseção completa entre os conjuntos representando $H(X)$ e $H(Y)$, ou seja, basta que um conjunto contenha o outro.

Partindo da eq. (B.8), também podemos escrever a informação mútua como

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}. \quad (\text{B.9})$$

Com um pouco de atenção, podemos notar que $I(X; Y) = \mathbb{E}[g(X, Y)]$, em que $g(x, y) = \log \frac{p(x, y)}{p(x)p(y)}$. Além disso, utilizando o teorema de Bayes, podemos escrever

$$g(x, y) = I(x) - I(x|y) = I(y) - I(y|x).$$

Em [7, p. 56], essa quantidade é chamada de informação mútua, recebendo a notação $I(x; y)$, e a quantidade definida na eq. (B.10) de informação mútua média. No entanto, essa não será a nomenclatura adotada aqui, pois tampouco é a convenção adotada pela maior parte dos autores.

Analisando com mais detalhe a expressão de g , podemos notar que é possível escrever, novamente, usando o teorema de Bayes,

$$g(x, y) = \log \frac{p(x|y)}{p(x)}.$$

Essa nova formulação, nos permite visualizar três possíveis cenários:

- A ocorrência de y torna o evento x menos provável, o que implica $p(x|y) < p(x)$ e $g(x, y) < 0$;
- A ocorrência de y torna o evento x mais provável, o que implica $p(x|y) > p(x)$ e $g(x, y) > 0$;
- A ocorrência de y não diz nada sobre x , o que implica $p(x|y) = p(x)$ e $g(x, y) = 0$.

Logo, vemos que, apesar de $I(X; Y) \geq 0$, $g(x, y)$ pode assumir valores negativos.

Uma pergunta que poderia ser feita é: “se $g(x, y)$ pode assumir valores negativos, o que impede $I(X; Y)$ de ter valor negativo?” A resposta para essa pergunta será desenvolvida abaixo.

Uma primeira possibilidade que poderia levar a $I(X; Y) < 0$ é se $g(x, y) < 0 \forall (x, y) \in \mathcal{X} \times \mathcal{Y}$, em que $\mathcal{X}$ e $\mathcal{Y}$ são, respectivamente, as imagens de X e Y , o que implica $p(x|y) < p(x) \forall (x, y) \in \mathcal{X} \times \mathcal{Y}$. Isso é equivalente a dizer que a ocorrência de qualquer evento $E_Y \in \Omega_Y$ do domínio de Y , reduz a probabilidade de qualquer evento $E_X \in \Omega_X$ do domínio de X . No entanto, se $\mathbb{P}(E_X|E_Y) < \mathbb{P}(E_X)$, então $\mathbb{P}(E_X^c|E_Y) = 1 - \mathbb{P}(E_X|E_Y) > 1 - \mathbb{P}(E_X) = \mathbb{P}(E_X^c)$. Assim, por mais que exista a relação inversa de dependência entre um evento E_Y e E_X , a ocorrência de E_Y implica um aumento de probabilidade da não ocorrência do evento E_X , o que resulta em $g(x, y) > 0$. Portanto, $g(x, y) < 0 \forall (x, y) \in \mathcal{X} \times \mathcal{Y}$ é um absurdo.

A segunda possibilidade que poderia levar a $I(X; Y) < 0$ é se, no somatório da eq. (B.9), as parcelas negativas forem predominantes. Intuitivamente, já é possível visualizar que os termos em que $g(x, y)$ é negativo receberão menor peso, pois serão multiplicados por valores menores de $p(x, y)$. Isso acontece porque, se E_x e E_y tem uma dependência inversa de ocorrência, então a ocorrência dos dois eventos simultaneamente, ou seja, $E_x \cap E_y$, terá menor probabilidade conjunta $p(x, y)$. Por outro lado, se a relação de dependência entre os eventos é direta e, conseqüentemente, $g(x, y)$ tem valor positivo, então $E_x \cap E_y$ tem maior probabilidade de ocorrer, ou seja, o valor de $p(x, y)$ tende a ser maior. Logo, de maneira não tão formal, é possível entender o porquê de $I(X; Y) \geq 0$.

Com isso, fica evidente mais uma interpretação bastante importante da informação mútua: **ela mede o grau de dependência entre duas variáveis**. Quando as variáveis são completamente independentes, $p(x, y) = p(x)p(y)$ e $I(X, Y) = 0$. Além disso, quanto maior o grau de

dependência das variáveis, maior é o valor informação mútua, até o limite em que $I(X, Y) = \min\{H(X), H(Y)\}$.

Para deixar isso ainda mais claro, podemos reparar que a informação mútua pode ser escrita por meio da divergência KL:

$$I(X; Y) = \text{KL}(p_{X,Y} \| p_X p_Y), \quad (\text{B.11})$$

em que $p_{X,Y}(x, y) = \mathbb{P}(X = x, Y = y)$, $p_X(x) = \mathbb{P}(X = x)$ e $p_Y(y) = \mathbb{P}(Y = y)$. Ou seja, a informação mútua mede quão divergente é a distribuição conjunta do produto das distribuições marginais.

B.1.8. INFORMAÇÃO MÚTUA CONDICIONAL

A informação mútua condicional [35] é dada por

$$I(X; Y|Z) = H(X|Z) - H(X|Y, Z), \quad (\text{B.12})$$

sendo que $I(X; Y|Z) = I(Y; X|Z)$ e $I(X; Y|Z) > 0$.

Em uma situação em que temos uma v.a. Y e uma v.a. Z que já é plenamente conhecida, $I(X; Y|Z)$ indica a quantidade de incerteza que é reduzida de Y por conhecer X . Em um diagrama de Venn, isso corresponde à interção entre os conjuntos que representam a incerteza de X e Y , excluído toda a incerteza dessas variáveis que são explicadas por Z , como mostra a Figura 13-10.

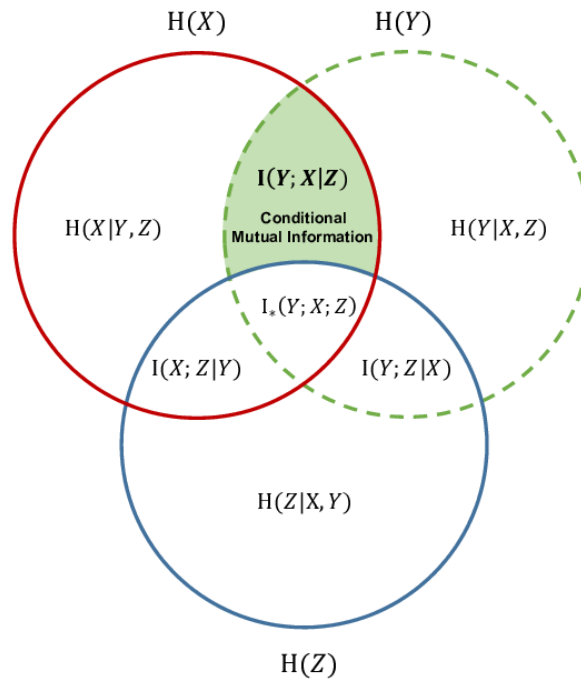


Figura 13-10: Informação mútua condicional.

Essa figura ilustra a configuração dos conhecimentos das variáveis X , Y e Z quando as três são levadas em consideração. No entanto, essa configuração pode não ser a mesma quando

deixamos de considerar alguma dessas variáveis. Por exemplo, considerando apenas as variáveis X e Y , é possível que os conjuntos que representam as incertezas dessas variáveis sejam os da Figura 13-11. Como podemos observar, quando não temos conhecimento da variável Z , não há interseção de informação/conhecimento entre X e Y . Logo, elas não interagem quando consideradas sozinhas e, portanto, são independentes, com $I(X; Y) = 0$. No entanto, com o conhecimento da variável Z , as variáveis X e Y passam a ter um grau de dependência. Além disso, vemos que, nesse caso, $I(Y; X|Z) > I(X; Y)$, o que parece contraintuitivo.

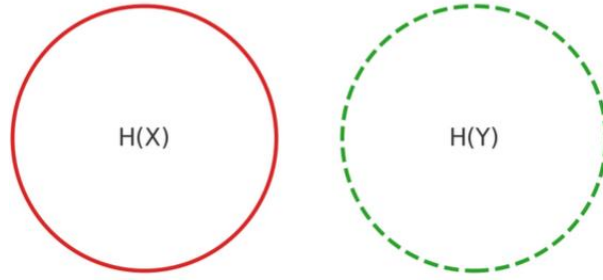


Figura 13-11: Diagrama de Venn das incertezas das variáveis X e Y .

Apesar de isso parecer não fazer sentido, é possível ver que essas situações acontecem na realidade. Se considerarmos uma variável aleatória X que represente o resultado do lançamento de um dado e uma variável aleatória Y que represente o lançamento de outro dado, fica muito evidente que X e Y são independentes e que $I(X, Y) = 0$. No entanto, ao adicionar uma variável aleatória Z no sistema, que representa a soma dos resultados dos dois dados, as variáveis X e Y tornam-se dependentes, ou seja, $I(X; Y|Z) > 0$. Por exemplo, se sabemos $Z = 7$, as combinações (x, y) tais que $x + y = 7$ são muito mais prováveis que todas as outras combinações de valores, o que não acontecia quando Z era desconhecido e todas as combinações (x, y) eram equiprováveis. Nesse cenário, conhecer o valor do primeiro dado aumenta a probabilidade de determinado valores ter saído no segundo dado para 100%.

A informação mútua condicional também pode ser escrita como

$$I(X; Y|Z) = \mathbb{E}_Z[\text{DL}(p_{X,Y|Z} \| p_{X|Z} p_{Y|Z})] = \sum_z p(z) \sum_x \sum_y p(x, y|z) \log \frac{p(x, y|z)}{p(x|z)p(y|z)} \quad (\text{B.13})$$

ou

$$I(X; Y|Z) = \mathbb{E}_{X,Y,Z} \left[\log \frac{p(X, Y|Z)}{p(X|Z)p(Y|Z)} \right] = \sum_z \sum_x \sum_y p(x, y, z) \log \frac{p(x, y|z)}{p(x|z)p(y|z)}. \quad (\text{B.14})$$

Podemos reparar que a informação mútua tem uma expressão que lembra o processo de marginalização de uma variável aleatória, ou seja, $p_X(x) = \sum_y p(x, y) = \sum_y p(y) p(x|y) = \mathbb{E}_y[p(x|y)]$. Além disso, a notação $I(X; Y|Z)$ lembra a notação do valor esperado condicional $E[X|Y]$, que é uma variável aleatória dependente de Y . No entanto, $I(X; Y|Z)$ não é uma variável aleatória, pois como vemos a partir da eq. (B.14), a média é tomada com relação à

todas as variáveis. Já em $E[X|Y]$, o valor esperado é calculado ao longo da variável X apenas, restando Y como variável aleatória.

Abaixo, se encontram algumas outras identidades:

$$I(X; Y|Z) = I(X; (Y, Z)) - I(X; Z);$$

$$I(X; Y|Z) = I(X; Y) - I(X; Z) + I(X; Z|Y).$$

Observar que em $I(X; (Y, Z))$, a notação indica a informação mútua entre X e o vetor aleatório (Y, Z) , de forma que

$$I(X; (Y, Z)) = \text{KL}(p_{X,Y,Z} \| p_X p_{Y,Z}).$$

B.1.9. INFORMAÇÃO MÚTUA MÚLTIPLA

Também chamada de informação de interação, é o ganho de informação que se tem na informação mútua entre duas variáveis X e Y por se conhecer uma terceira variável Z [36].

Matematicamente, isso é equivalente a

$$I(X; Y; Z) = I(X; Y|Z) - I(X; Y), \quad (\text{B.15})$$

com $I(X; Y; Z) = I(Z; X; Y) = I(Y; Z; X)$.

Logo, a informação mútua múltipla $I(X; Y; Z)$ é o quanto a inclusão da variável Z melhora a informação compartilhada entre X e Y .

Diferentemente da informação mútua, a informação de interação pode assumir valores negativos. Isso é resultado de algumas situações diferentes:

- $I(X; Y; Z) > 0$ indica que existe sinergia entre as variáveis, ou seja, a inclusão de uma das variáveis como contexto faz com que a interação entre as outras duas variáveis aumente;
- $I(X; Y; Z) = 0$ indica que a inclusão de uma variável não tem efeito na interação entre as outras duas;
- $I(X; Y; Z) < 0$ indica a existência de redundância de informação, ou seja, a inclusão de uma terceira variável não aumenta a interação entre as outras duas, mas, na realidade, revela que a interação que parecia ser exclusiva entre essas duas primeiras variáveis é, na verdade, explicada em parte pela terceira. Por exemplo, duas variáveis podem parecer correlacionadas inicialmente até que uma terceira variável seja conhecida, revelando que a correlação inicial era, na verdade, resultado de uma dependência comum dessa terceira variável (região de interseção entre os três conjuntos da Figura 13-10).

Nessa imagem, a propósito, a informação mútua múltipla não é representada diretamente. No entanto, reparamos a presença do termo $I_*(X; Y; Z)$ na interseção dos três conjuntos. Esse termo é oposto da informação mútua múltipla, ou seja, $I_*(X; Y; Z) = -I(X; Y; Z)$. Ele não recebe um nome específico e só foi criado neste material para dar um significado para essa região de interseção, em vez de representa-la com o termo $-I(X; Y; Z)$, que pode ser um pouco confuso.

Reparamos também que, essa região só existe quando $I_*(X; Y; Z) > 0$ ou $I(X; Y; Z) < 0$ e deixa de existir quando $I_*(X; Y; Z) \leq 0$ ou $I(X; Y; Z) \geq 0$, representando a falta de interseção e a falta de interação entre as três variáveis simultaneamente. No entanto, isso não significa que não exista interação entre os pares de variáveis. Se repararmos, $I(X; Y) = I_*(X; Y; Z) + I(X; Y|Z)$. Logo, quando $I_*(X; Y; Z) < 0$, enquanto $|I_*(X; Y; Z)| < I(X; Y|Z)$, há interação entre X e Y individualmente. Apenas quando a informação mútua múltipla é mínima que não temos interação por pares entre as variáveis, ou seja, $|I_*(X; Y; Z)| < I(X; Y|Z)$, com $I_*(X; Y; Z) < 0$, pois $I(X; Y) = 0$.

Voltando ao exemplo dos dados apresentado na Seção B.1.8, vimos que não existe interação entre os lançamentos de dados, ou seja, $I(X; Y) = 0$, mas a inclusão de uma terceira variável Z , soma dos dados, faz com que haja complementariedade entre X e Y , pois $I(X; Y|Z) > I(X; Y)$. De fato, X e Y juntas podem ser utilizadas para inferir Z , pois $I((X, Y); Z) = I(X; Z) + I(Y; Z|X) = I(Y; Z|X) > I(X; Y) = 0$.

B.2. CASO CONTÍNUO

As equações para o caso discreto podem ser generalizadas para o caso contínuo. O desenvolvimento necessário para isso está explicado em [7, p. 59].

B.2.1. ENTROPIA

Também chamada de **entropia diferencial** é dada pela expressão

$$H(X) = - \int_{-\infty}^{\infty} p(x) \log p(x) dx \quad (\text{B.16})$$

A entropia é máxima quando X segue uma distribuição normal.

Observar que, no caso contínuo, $p(x)$ pode assumir valores maiores do que 1, o que pode fazer com que possamos obter valores negativos de entropia.

B.2.2. ENTROPIA CONDICIONAL

A entropia condicional é dada pela expressão

$$H(X|Y) = - \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} p(x, y) \log p(x|y) dx dy \quad (\text{B.17})$$

B.2.3. INFORMAÇÃO MÚTUA

A informação mútua é dada pela expressão

$$I(X; Y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} dx dy \quad (\text{B.18})$$

B.2.4. DIVERGÊNCIA KULLBACK-LEIBLER

A divergência Kullback-Leibler é dada pela expressão

$$\text{KL}(p\|q) = \int_{-\infty}^{\infty} p(x) \log \frac{p(x)}{q(x)} dx$$

(B.19)

Anexo C: DECOMPOSIÇÃO SVD

Qualquer matriz A de $m \times n$ pode ser decomposta da seguinte maneira

$$A = U\Sigma V^T \quad (C.1)$$

em que

U é uma matriz ortogonal $m \times m$ cujas colunas são os autovetores de AA^T e são chamadas de *left singular vectors de A* [37],

V^T é uma matriz ortogonal $n \times n$ cujas colunas são os autovetores de $A^T A$ e são chamadas de *right singular vectors de A* e

Σ é uma matriz diagonal (possivelmente retangular) $m \times n$ cujos valores σ_i da diagonal principal são as raízes quadradas dos autovalores de AA^T e de $A^T A$ e são chamados de *valores singulares*.

Essa decomposição é chamada de SVD [38]. A Figura 13-12 ilustra essa decomposição.

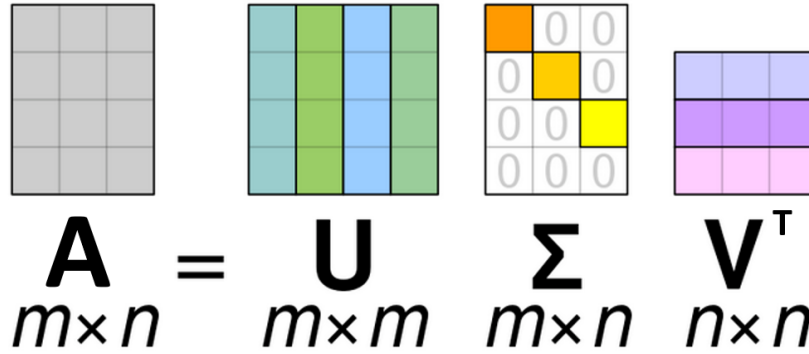


Figura 13-12: Representação gráfica da decomposição SVD.

Lembrando que AA^T e de $A^T A$ são matrizes diferentes (normalmente com dimensões diferentes) que apresentam autovetores diferentes, mas os mesmos autovalores. Lembrando também que, o número de autovalores é igual ao posto (*rank*) da matriz.

A partir da eq. (C.1), podemos escrever

$$AA^T = U\Sigma V^T A^T$$

$$AA^T = U\Sigma V^T V \Sigma U^T = U\Sigma^2 U^T = U\Lambda U^T,$$

lembrando que $\Sigma = \Sigma^T$ e que $VV^T = I$, pois V é ortogonal. A expressão acima é a diagonalização da matriz AA^T (decomposição espectral). Logo, fica claro que as colunas de U são de fato os autovetores de AA^T . Além disso, vemos que os valores σ_i da diagonal de Σ correspondem a

$$\sigma_i = \sqrt{\lambda_i},$$

em que λ_i são os valores da diagonal da matriz Λ de autovalores de AA^T .

É possível provar que toda matriz A $m \times n$ de posto 1 pode ser escrita como produto externo de um vetor de tamanho m por outro de tamanho n [37]:

$$A = \mathbf{u}\mathbf{v}^T = \begin{bmatrix} - & u_1\mathbf{v}^T & - \\ \vdots & \vdots & \vdots \\ - & u_m\mathbf{v}^T & - \end{bmatrix} = \begin{bmatrix} | & \cdots & | \\ v_1\mathbf{u} & \cdots & v_n\mathbf{u} \\ | & \cdots & | \end{bmatrix}.$$

Além disso, toda matriz de posto k pode ser escrita como a soma de k matrizes de posto 1 [37]. A título de ilustração, uma matriz A $m \times n$ de posto 2 ficaria

$$A = \mathbf{u}\mathbf{v}^T + \mathbf{w}\mathbf{z}^T = \begin{bmatrix} - & u_1\mathbf{v}^T + w_1\mathbf{z}^T & - \\ \vdots & \vdots & \vdots \\ - & u_m\mathbf{v}^T + w_m\mathbf{z}^T & - \end{bmatrix} = \begin{bmatrix} | & | \\ \mathbf{u} & \mathbf{w} \\ | & | \end{bmatrix} \begin{bmatrix} - & \mathbf{v}^T & - \\ \mathbf{z}^T & - \end{bmatrix}. \quad (\text{C.2})$$

Com isso, podemos escrever a decomposição SVD como

$$A = \sum_{i=1}^{\min(m,n)} \sigma_i \mathbf{u}_i \mathbf{v}_i^T. \quad (\text{C.3})$$

Logo, podemos escrever a decomposição SVD como a soma dos produtos externos da coluna i da matriz U com a linha i da matriz V^T escalados pelo valor singular i . Isso faz sentido quando olhamos para o termo da direita da eq. (C.2).

C.1. LOW-RANK APROXIMATION POR SVD

A *low-rank approximation* de uma matriz usando SVD consiste em aproximar a matriz A por uma matriz $\tilde{A}$ de posto $k < \text{rank}(A)$. Isso é feito ao organizar a decomposição da eq. (C.1) de forma que os valores singulares estejam dispostos em Σ em ordem decrescente (o primeiro valor da diagonal é o maior) e zerando os menores valores singulares (e as colunas/linhas correspondentes nas matrizes U e V^T) até restarem k valores singulares não-nulos. Isso é equivalente, na eq. (C.3), a eliminar as parcelas com os menores valores de σ_i até restarem apenas k parcelas.

Isso faz sentido porque, como $\mathbf{u}_i$ e $\mathbf{v}_i^T$ são unitários, a norma de Frobenius do seu produto externo é $\|\mathbf{u}_i \mathbf{v}_i^T\|_F = 1$.

Lembrando que $\|A\|_F$ é o equivalente a colocar todos os elementos de A em um único vetor $\mathbf{w}$ e calcular $\|\mathbf{w}\|_2$. Assim, a norma de Frobenius transmite uma ideia da magnitude dos elementos armazenados na matriz.

Logo, a contribuição da parcela $\sigma_i \mathbf{u}_i \mathbf{v}_i^T$ na matriz final $\tilde{A}$ tende a ser menor quanto menor for σ_i . Portanto, eliminamos as parcelas da eq. (C.3) em ordem crescente de σ_i .

Ao fazer isso, encontramos uma matriz $\tilde{A}$ aproximada tal que $\|A - \tilde{A}\|_F \leq \|A - B\|_F$, sendo B uma matriz de dimensões $m \times n$, assim como A . Isso quer dizer que $\tilde{A}$ é a matriz mais próxima de A se usarmos a norma de Frobenius como medida de distância [37].

Exemplo:

Decomposição da matriz

$$A = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 & 9 \\ 10 & 11 & 12 & 13 & 14 \\ 15 & 16 & 17 & 18 & 19 \\ 20 & 21 & 22 & 23 & 24 \end{bmatrix}.$$

Ao realizarmos a decomposição SVD de A encontramos uma matriz de valores singulares

$$\Sigma = \begin{bmatrix} 69,91 & 0 & 0 & 0 & 0 \\ 0 & 3,58 & 0 & 0 & 0 \\ 0 & 0 & 6,38 \times 10^{-15} & 0 & 0 \\ 0 & 0 & 0 & 1,24 \times 10^{-15} & 0 \\ 0 & 0 & 0 & 0 & 2,39 \times 10^{-16} \end{bmatrix}.$$

A matriz aproximada $\tilde{A}_2$ de posto 2 é

$$\tilde{A}_2 = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 & 9 \\ 10 & 11 & 12 & 13 & 14 \\ 15 & 16 & 17 & 18 & 19 \\ 20 & 21 & 22 & 23 & 24 \end{bmatrix} + \epsilon$$

$$\epsilon = \begin{bmatrix} -0,2 & -5,4 & -9,1 & -9,8 & -5,3 \\ 0,9 & -4 & -5 & -6 & -2 \\ -2 & -7 & -7 & -9 & -5 \\ -4 & -9 & -10,7 & -10,7 & -7,1 \\ -3,6 & -10,7 & -7,1 & -14,2 & -14,2 \end{bmatrix} \times 10^{-15}.$$

Assim, em vez de ter que armazenar os 25 valores da matriz A original, bastaria armazenar os 2 valores singulares de $\tilde{A}$ mais 20 valores das linhas e colunas de U e V^T correspondentes, totalizando 22 valores. O ganho parece pouco nesse exemplo, mas em aplicações em que a matriz A é muito grande, ao guardarmos apenas alguns valores singulares, temos um ganho muito significativo.

Anexo D: OTIMIZAÇÃO

D.1. CONCEITOS GERAIS E DUALIDADE

Um problema de otimização (não necessariamente convexo) na forma padrão tem a seguinte forma:

$$\begin{array}{ll} \text{minimizar} & f_0(\mathbf{x}) \\ \text{sujeito a} & f_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \\ & h_i(\mathbf{x}) = 0, \quad i = 1, \dots, p \end{array} \quad (\text{D.1})$$

em que $f_0(\mathbf{x})$ é chamada de **função objetivo** e as funções $f_i(\mathbf{x})$ e $h_i(\mathbf{x})$ são chamadas de **funções de restrição**. Denotamos o valor ótimo desse problema p^* . O valor $\mathbf{x}^*$, valor de $\mathbf{x}$ que leva a p^* , é chamado de **ótimo primal**. Além disso, todo $\tilde{\mathbf{x}}$ que pertence ao domínio $\mathcal{D}$ de f_0 e respeita as restrições do problema (D.1) é chamado de **ponto viável**. Por fim, muitas vezes chamamos o problema (D.1) de **problema primal** (essa nomenclatura fará sentido mais para frente).

A ideia básica da dualidade de Lagrange é levar em conta as restrições em uma única equação estendendo a expressão da função objetivo [39]. Essa expressão estendida é chamada de **Lagrangiano** (ou função de Lagrange) e é dada por:

$$L(\mathbf{x}, \boldsymbol{\lambda}, \mathbf{v}) = f_0(\mathbf{x}) + \sum_{i=1}^m \lambda_i f_i(\mathbf{x}) + \sum_{i=1}^p v_i h_i(\mathbf{x}). \quad (\text{D.2})$$

Os termos λ_i e v_i são chamados de **multiplicadores de Lagrange**. Os primeiros associados às restrições de desigualdade e os segundos às restrições de igualdade. Observar que, quando $\lambda_i \geq 0$, estamos premiando os valores de $\mathbf{x}$ que tornam f_i muito negativa, pois, quanto mais negativa f_i menos vale $L(\mathbf{x}, \boldsymbol{\lambda}, \mathbf{v})$.

Dando prosseguimento, ao tentar minimizar $L(\mathbf{x}, \boldsymbol{\lambda}, \mathbf{v})$, encontramos a **função de Lagrange dual**, que é definida como o mínimo do Lagrangiano com relação a $\mathbf{x}$. Ela é dada por

$$g(\boldsymbol{\lambda}, \mathbf{v}) = \inf_{\mathbf{x} \in \mathcal{D}} L(\mathbf{x}, \boldsymbol{\lambda}, \mathbf{v}) = \inf_{\mathbf{x} \in \mathcal{D}} \left(f_0(\mathbf{x}) + \sum_{i=1}^m \lambda_i f_i(\mathbf{x}) + \sum_{i=1}^p v_i h_i(\mathbf{x}) \right). \quad (\text{D.3})$$

A função de Lagrange dual é côncava, mesmo que o problema definido em (D.1) não seja (checar as condições necessárias para a convexidade de um problema de otimização em [39, p. 136]).

Para entender a afirmação acima, precisamos reparar que estamos falando da concavidade de $g(\boldsymbol{\lambda}, \mathbf{v})$ com relação a $(\boldsymbol{\lambda}, \mathbf{v})$, e não a $\mathbf{x}$. Levando isso em conta, podemos escrever o Lagrangiano como $L(\mathbf{x}, \boldsymbol{\lambda}, \mathbf{v}) = v_{\mathbf{x}}(\boldsymbol{\lambda}, \mathbf{v})$, colocando $\mathbf{x}$ como um parâmetro e $(\boldsymbol{\lambda}, \mathbf{v})$ como as únicas variáveis. Nesse caso, ao fixarmos um $\mathbf{x}$, podemos dizer que $v_{\mathbf{x}}(\boldsymbol{\lambda}, \mathbf{v})$ é uma soma de funções afins, pois com $f_0(\mathbf{x})$, $f_i(\mathbf{x})$ e $h_i(\mathbf{x})$ tornam-se constantes. Com isso, podemos dizer que $\{v_{\mathbf{x}}(\boldsymbol{\lambda}, \mathbf{v}) : \mathbf{x} \in \mathcal{D}\}$ é uma família de funções afins. Logo, $g(\boldsymbol{\lambda}, \mathbf{v}) = \inf_{\mathbf{x} \in \mathcal{D}} \{v_{\mathbf{x}}(\boldsymbol{\lambda}, \mathbf{v}) : \mathbf{x} \in \mathcal{D}\}$, que é uma função côncava, pois para cada valor de $\mathbf{x}$, $v_{\mathbf{x}}(\boldsymbol{\lambda}, \mathbf{v})$ é côncava [39, pp. 80, 87].

A função de Lagrange dual representa um limite inferior para o valor ótimo, ou seja, $g(\lambda, \nu) \leq p^*$. Isso porque, para um ponto viável $\tilde{x}$ e para $\lambda \geq 0$ ($\geq$ é igual a $\geq$ elemento a elemento),

$L(\tilde{x}, \lambda, \nu) = f_0(\tilde{x}) + \overbrace{\sum_{i=1}^m \lambda_i f_i(\tilde{x}) + \sum_{i=1}^p \nu_i h_i(\tilde{x})}^{\leq 0} \leq f_0(\tilde{x})$, já que $f_i(\tilde{x}) \leq 0$ e $h_i(\tilde{x}) = 0$. Logo, $g(\lambda, \nu) = \inf_{x \in \mathcal{D}} L(x, \lambda, \nu) \leq L(x, \lambda, \nu) \leq f_0(x)$. Assim, para cada par (λ, ν) com $\lambda \geq 0$, $g(\lambda, \nu)$ representa um limite inferior diferente para p^* .

Poderíamos nos perguntar qual é o melhor limite inferior que poderíamos encontrar a partir de $g(\lambda, \nu)$. Essa pergunta dá origem ao **problema dual de Lagrange** que é definido como:

$$\begin{aligned} & \text{maximizar } g(\lambda, \nu) \\ & \text{sujeito a } \lambda \geq 0 \end{aligned} \tag{D.4}$$

Nos referimos a (λ^*, ν^*) como **ótimo dual** e denotamos seu valor ótimo correspondente d^* . Reparar que o problema (D.4) é convexo, pois a função objetivo é côncava e a restrição é convexa.

Podemos afirmar que $d^* \leq p^*$, já que $g(\lambda, \nu)$ é um limite inferior para p^* . Essa propriedade é chamada de **dualidade fraca**. Quando temos uma igualdade, ou seja, $d^* \leq p^*$, falamos em **dualidade forte**. Além disso, chamamos de **gap de dualidade** a diferença $p^* - d^*$. Em geral, a dualidade forte não é válida. No entanto, quando o problema primal é convexo, normalmente temos dualidade forte [39, p. 226].

Quando a dualidade forte é válida, podemos escrever

$$\begin{aligned} f_0(x^*) &= g(\lambda^*, \nu^*) \\ &= \inf_{x \in \mathcal{D}} \left(f_0(x) + \sum_{i=1}^m \lambda_i^* f_i(x) + \sum_{i=1}^p \nu_i^* h_i(x) \right) \\ &\leq f_0(x^*) + \sum_{i=1}^m \lambda_i^* f_i(x^*) + \sum_{i=1}^p \nu_i^* h_i(x^*) \\ &\leq f_0(x^*) \end{aligned}$$

Com isso, podemos concluir que, na verdade, as desigualdades são igualdades. Logo, quando o gap de dualidade é zero, temos $\sum_{i=1}^m \lambda_i^* f_i(x^*) = 0$. Mas, como todos os termos dessa soma são não-positivos, temos que

$$\lambda_i^* f_i(x^*) = 0.$$

Essa propriedade é chamada de **complementary slackness**. Além disso, podemos ver que x^* minimiza $L(x, \lambda^*, \nu^*)$ com relação a x .

Por fim, todo problema de otimização na forma (D.1) e com restrições e função objetivo diferenciáveis devem respeitar as chamadas **condições de KKT** (Karush-Kuhn-Tucker), que são as seguintes:

1. $f_i(x^*) \leq 0$
2. $h_i(x^*) = 0$
3. $\lambda_i^* \geq 0$
4. $\lambda_i^* f_i(x^*) = 0$

$$5. (\nabla_x L)(\mathbf{x}^*, \boldsymbol{\lambda}^*, \mathbf{v}^*) = 0 \Rightarrow (\nabla_x f_0)(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* (\nabla_x f_i)(\mathbf{x}^*) + \sum_{i=1}^p v_i^* (\nabla_x h_i)(\mathbf{x}^*) = 0$$

As duas primeiras condições são as restrições definidas inicialmente em (D.1). A condição 3 é necessária para que a função de Lagrange dual seja um limite inferior para p^* , como visto anteriormente. A condição 4 vem da *complementary slackness*. A condição 5 vem do fato de que $\inf_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}^*, \mathbf{v}^*) = L(\mathbf{x}^*, \boldsymbol{\lambda}^*, \mathbf{v}^*)$, como visto anteriormente.

Se o problema primal for convexo, a validade das condições de KKT já são suficientes para afirmar que existe dualidade forte.

Portanto, vemos que o problema de otimização apresentando em (D.1) se resume a $\max_{(\boldsymbol{\lambda}, \mathbf{v})} \inf_{\mathbf{x}} L(\boldsymbol{\lambda}, \mathbf{v}, \mathbf{x})$ com $\boldsymbol{\lambda} \geq 0$ e, sob condição de gap de dualidade nulo, respeitando as condições de KKT.

D.2. MÉTODO DE NEWTON

É um método de otimização numérica não linear e iterativo, que tenta encontrar o ponto ótimo da função objetivo $f(x)$ utilizando uma aproximação de Taylor de segunda ordem $f_{x_k}(x)$ nas proximidades de x_k . Assim, se a função objetivo é $f(x)$, no método de Newton, teríamos a seguinte aproximação:

$$f(x) \approx f_{x_k}(x) = f(x_k) + (x - x_k)f'(x_k) + \frac{(x - x_k)^2}{2}f''(x_k).$$

Para minimizar $f_{x_k}(x)$, basta derivar sua expressão com relação a x e igualar a zero:

$$\frac{\partial f_{x_k}(x)}{\partial x} = 0$$

$$f'(x_k) + (x - x_k)f''(x_k) = 0$$

$$x^* = -\frac{f'(x_k)}{f''(x_k)} + x_k$$

Com isso, encontramos o ponto ótimo da aproximação de $f(x)$ nas proximidades de x_k , o que não é equivalente a encontrar o ponto ótimo de $f(x)$, pois a aproximação $f_{x_k}(x)$ é local. Dessa forma, é necessário repetir múltiplas vezes esse passo e atualizar gradualmente o valor de x , igualando-o ao valor ótimo encontrado em cada iteração:

$$x_{k+1} = x^* = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

Usualmente, $-\frac{f'(x_k)}{f''(x_k)}$ é interpretado como um passo dado na direção do valor ótimo, que tem direção contrária ao gradiente (no caso unidimensional, à derivada primeira).

Uma diferença notável com relação ao gradiente descendente, que tem um passo $-\eta f'(x_k)$, sendo η uma constante para limitar a magnitude do gradiente, é que, no método de Newton, temos o termo $f''(x_k)$, que ajuda controlar o tamanho do passo de maneira adaptativa. Isso acontece porque o termo de segunda ordem transmite uma noção de curvatura da curva localmente (nas proximidades de x_k). Assim, se estamos procurando um ponto de mínimo, por

exemplo, se estamos em uma vizinhança de curvatura acentuada, o termo $f''(x_k)$ diminui a magnitude do passo, comparado a outra vizinhança de menor curvatura. Portanto, em regiões onde a inclinação varia menos, o passo tende a ser maior, acelerando o algoritmo. Por outro lado, em regiões de maior variação da inclinação (maior concavidade), possivelmente próximo ao ponto ótimo, os passos tendem a ser menores, fazendo uma atualização mais refinada de x_{k+1} .

É importante lembrar que o conceito de curvatura aqui é diferente do conceito geométrico de curvatura. Essa diferença é percebida facilmente ao notarmos que a função $f(x) = x^2$ tem curvatura constante $f''(x) = 1$, mas geometricamente, a curvatura ao redor do mínimo não é a mesma para pontos afastados do mínimo. Logo, a segunda derivada transmite noção de taxa de variação da inclinação e não de curvatura geométrica.

Também é importante dizer que a frase “O método de Newton é uma aproximação de segunda ordem, enquanto o gradiente descendente é uma aproximação de primeira ordem” está errada. Isso porque o gradiente descendente não faz uma aproximação linear local e, a partir disso, gera uma regra iterativa de atualização. Até mesmo porque, não seria possível gerar tal regra uma vez que teríamos

$$f_{x_k}(x) = f(x_k) + (x - x_k)f'(x_k),$$

mas $\frac{\partial f_{x_k}(x)}{\partial x} = 0$ seria um absurdo.

A abordagem do gradiente descendente ainda é local, mas não no sentido de uma aproximação de Taylor. Damos passos na direção de maior descida, ou seja, de um mínimo local.

Anexo E: REPRODUCING KERNEL HILBERT SPACES

Para entender o conceito de *Reproducing Kernel Hilbert Spaces* (RKHS), é necessário entender outros conceitos mais básicos, como o conceito de espaço de Hilbert.

Um espaço de Hilbert $\mathbb{H}$ é um espaço com um produto interno $\langle \cdot, \cdot \rangle_{\mathbb{H}}$ que é completo com respeito à norma $\|\cdot\|_{\mathbb{H}}$ definida pelo produto interno. Um espaço completo é aquele em toda sequência de Cauchy converge para um membro desse espaço, sendo uma sequência de Cauchy é uma sequência cujos elementos se tornam arbitrariamente muito próximos uns dos outros à medida que a sequência progride [40].

O espaço de Hilbert é uma generalização do espaço Euclidiano para um espaço de dimensão finita ou infinita (o espaço Euclidiano tem dimensão finita) [40]. Aparentemente, um espaço de Hilbert finito é bem parecido com o espaço Euclidiano. Prece que as diferenças começam a surgir quando falamos de um espaço de Hilbert de dimensão infinita.

Por simplicidade, o produto interno e a norma do espaço de Hilbert serão escritos sem o subscrito $\mathbb{H}$.

Um *reproducing kernel Hilbert space* (RKHS) é um espaço de Hilbert $\mathbb{H}$ de funções $f : \mathcal{X} \rightarrow \mathbb{R}$ com um *reproducing kernel* (ou *kernel*) $\kappa : \mathcal{X}^2 \mapsto \mathbb{R}$ em que $\kappa(x, \cdot) \in \mathbb{H}$ e $f(x) = \langle \kappa(x, \cdot), f \rangle$. Essa igualdade é chamada *reproducing property*.

Considere uma função kernel $\kappa(x, y)$ de duas variáveis. Suponha que, para n pontos, fixemos a variável y para ter $\kappa(x_1, y), \kappa(x_2, y), \dots, \kappa(x_n, y)$, que são todas funções da variável y . RKHS é um espaço de funções que é o conjunto de todas as possíveis combinações lineares dessas funções, ou seja,

$$\mathbb{H} := \left\{ f(\cdot) = \sum_{i=1}^n \alpha_i \kappa(x_i, \cdot) \right\} = \left\{ f(\cdot) = \sum_{i=1}^n \alpha_i \kappa_{x_i}(\cdot) \right\}.$$

Trazendo para algo concreto, poderíamos ter o kernel $\kappa(x, y) = xe^{2y}$. As funções $g(y) = 3e^{2y}$ e $h(y) = 5e^{2y}$ seriam exemplo de funções pertencentes ao RKHS $\mathbb{H} = \{f(y) = \sum_{i=1}^n \alpha_i x_i e^{2y}\}$.

Então, vemos que o kernel é a base para gerar o RKHS.

Dado um kernel, o RKHS é único e, dado um RKHS, o kernel correspondente é único [40].

A partir da *reproducing property*, se $f(\cdot) = \kappa(\cdot, x_1), x_1 \in \mathcal{X}$, então

$$\langle \kappa(\cdot, x_1), \kappa(\cdot, x_2) \rangle = \kappa(x_2, x_1) = \kappa(x_1, x_2), \quad (\text{E.1})$$

ou seja, a ordem dos argumentos não importa.

Na primeira igualdade, temos o produto interno entre $f(\cdot)$ e $\kappa(\cdot, x_2)$, ou seja, uma função de um único parâmetro e outra com dois parâmetros, mas o segundo parâmetro está fixado em x_2 . Logo, o resultado é $f(x_2) = \kappa(x_2, x_1)$. A segunda igualdade vem do fato que $\langle \kappa(\cdot, x_1), \kappa(\cdot, x_2) \rangle = \overline{\langle \kappa(\cdot, x_2), \kappa(\cdot, x_1) \rangle} = \langle \kappa(\cdot, x_2), \kappa(\cdot, x_1) \rangle = \kappa(x_2, x_1)$, em que $\overline{\langle \kappa(\cdot, x_2), \kappa(\cdot, x_1) \rangle} = \langle \kappa(\cdot, x_2), \kappa(\cdot, x_1) \rangle$ vem do fato de esse espaço de Hilbert ser composto apenas de funções reais.

Seja $\mathbb{H}$ um RKHS, associado a uma função kernel $\kappa(\cdot, \cdot)$ e com $\mathcal{X}$ sendo um conjunto de elementos. Então, o mapeamento

$$\begin{aligned}\phi: \mathcal{X} &\rightarrow \mathbb{H} \\ x &\mapsto \phi(x)\end{aligned}$$

é chamado de **feature map** e o espaço $\mathbb{H}$ é chamado de **espaço das features**. $\mathcal{X}$ é chamado de **espaço das entradas** (*input space*).

O mapa das *features*, que pode ser finito ou infinito, é um vetor $\phi(x) = [\phi_1(x), \phi_2(x), \dots]$. Um exemplo de *feature map* finito de dimensão 3 é $\phi(x) = [\phi_1(x), \phi_2(x), \phi_3(x)] = [x_1^2, x_1 x_2 \sqrt{2}, x_2^2]$, com $x = [x_1, x_2]^T$. Nesse espaço, temos apenas três *features*: x_1^2 , $x_1 x_2 \sqrt{2}$ e x_2^2 .

O seguinte produto interno é chamado de **kernel trick**:

$$\langle \phi(x), \phi(y) \rangle = \kappa(x, y).$$

A prova para isso pode ser encontrada em [40].

A partir da eq. (E.1), vemos que $\phi(x) = \kappa(x, \cdot) = \kappa(\cdot, x)$. Com isso, fica claro que o *feature map* é uma função $\kappa(x, \cdot)$ em que fixamos um dos argumentos em x . Logo, o *feature map* leva os dados $x \in \mathbb{R}^p$ do espaço das entradas para o espaço das *features* $\mathbb{H}$ composto por funções, que costuma ter um número de dimensões muito mais elevado, possivelmente infinito.

No exemplo do espaço finito apresentado acima, o kernel é dado por $\kappa(x, y) = \langle \phi(x), \phi(y) \rangle = \phi^T(x) \phi(y) = (x^T y)^2$, com $y = [y_1, y_2]^T$. No caso específico desse exemplo, vemos que o *features map* $\phi(x_n) = \kappa(x_n, z)$ avaliado no ponto x_n não é função da variável z , apesar de isso não ser sempre verdade (aparentemente).

Com isso, vemos que, podemos realizar produtos internos de maneira bastante eficiente quando empregamos esse mapeamento, pois basta avaliar o kernel $\kappa(\cdot, \cdot)$ nos pontos x e y .

Abaixo, são listados alguns kernel mais conhecidos.

Kernel linear

$$\kappa(x, y) := x^T y$$

Nesse tipo de kernel, conhecemos o *feature map*, que é $\phi(x) = x$.

Kernel Gaussiano ou Radial Basis Function (RBF)

$$\kappa(x, y) := e^{-\gamma \|x-y\|_2^2} = e^{-\frac{\|x-y\|_2^2}{\sigma^2}}$$

Um valor típico para γ é $1/d$, em que d é a dimensionalidade dos dados.

Kernel polinomial

$$\kappa(x, y) := (\gamma x^T y + c)^d$$

Com $\gamma > 0$ sendo o coeficiente angular, c o coeficiente linear e d a dimensionalidade dos dados. Valores típicos para os parâmetros são $\gamma = 1/d$ e $c = 1$.

Anexo F: CODIFICAÇÃO DE VARIÁVEIS

F.1. CODIFICAÇÃO CÍCLICA

Usamos para codificar variáveis que foram naturalmente um ciclo, também chamadas de variáveis cíclicas. Exemplos de variáveis desse tipo são:

- dia;
- mês;
- hora;
- minuto;
- dia da semana;
- etc.

Seja x o valor observado de uma variável cíclica codificado de maneira ordinal. Se, por exemplo, x for os meses do ano, então $x \in [1, 12]$. A codificação cíclica dessa variável fica

$$x_{sin} = \sin\left(\frac{2\pi x}{T}\right)$$
$$x_{cos} = \cos\left(\frac{2\pi x}{T}\right).$$

No exemplo dos meses, percebemos que, com essa codificação, os meses adjacentes sempre têm a mesma distância entre si, inclusive os meses de janeiro e dezembro que, na codificação ordinal, estariam distantes por 11 unidades.

Esse tipo de codificação é especialmente útil quando utilizamos modelos lineares ou que dependam de distância como:

- regressão linear;
- SVM;
- KNN;
- K-Means.

Capítulo 16: REFERÊNCIAS

- [1] G. Casella e R. L. Berger, *Statistical Inference*, 2 ed., Thomson Learning, 2002.
- [2] T. Hastie, R. Tibshirani e J. Friedman, *The Elements of Statistical Learning*, 2nd ed., New York: Springer, 2009.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*, New York: Springer, 2006.
- [4] L. Wasserman, *All of Statistics*, Pittsburgh: Springer, 2003.
- [5] J. K. Blitzstein e J. Hwang, *Introduction to Probability*, 2 ed., Boca Raton: CRC Press, 2019.
- [6] "1.6 Nearest Neighbors," scikit-learn, [Online]. Available: <https://scikit-learn.org/stable/modules/neighbors.html#classification>. [Acesso em 12 06 2024].
- [7] S. Theodoridis, *Machine Learning: a bayesian and optimization perspective*, 2nd ed., London: Elsevier, 2020.
- [8] N. El Gayar, F. Schwenker e G. Palm, "A Study of the Robustness of KNN Classifiers Trained Using Soft Labels," *Artificial Neural Networks in Pattern Recognition: Second IAPR Workshop, ANNPR 2006*, pp. 67-80, 2006.
- [9] J. L. Bentley, "Multidimensional Binary Search Trees Used fo Associative Searching," *Communications of the ACM*, vol. 18, nº 9, pp. 509-517, 01 09 1975.
- [10] K. Weinberger, "16: KD Trees," Cornell University, [Online]. Available: <http://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote16.html>. [Acesso em 15 06 2024].
- [11] "KDTree," Scikit-learn, [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KDTree.html>. [Acesso em 16 06 2024].
- [12] H. Hristov, "Introduction to K-D Trees," Baeldung, 2023 03 29. [Online]. Available: <https://www.baeldung.com/cs/k-d-trees>. [Acesso em 15 06 2024].
- [13] S. M. Omohundro, *Five balltree construction algorithms*, Berkeley: International Computer Science Institute, 1989.
- [14] J. Adamczyk, "k nearest neighbors computational complexity," Medium, 06 08 2020. [Online]. Available: <https://towardsdatascience.com/k-nearest-neighbors-computational-complexity-502d2c440d5>. [Acesso em 20 06 2024].
- [15] K. Weinberger, "KD Trees," [Online]. Available: <http://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote16.html>. [Acesso em 18 06 2024].

- [16] K. Weinberger, "Lecture 2: k-nearest neighbors," Cornell University, [Online]. Available: https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote02_kNN.html. [Acesso em 20 06 2024].
- [17] A. Géron, Hands-on Machine Learning with Scikit-learn, Keras & Tensorflow: Concepts, Tools and Techniques to Build Intelligent Systems, Sebastopol: O'Reilly, 2019.
- [18] "1.4. Support Vector Machines," scikit learn, [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>. [Acesso em 17 07 2024].
- [19] T. Kanamori, S. Fujiwara e A. Takeda, "Breakdown Point of Robust Support Vector Machine," *arXiv:1409.0934v1*, pp. 1-27, 03 09 2014.
- [20] shenflow, "Cross Entropy vs Entropy (Decision Tree)," Stack Exchange Data Science, 12 03 2019. [Online]. Available: <https://datascience.stackexchange.com/questions/47168/cross-entropy-vs-entropy-decision-tree>. [Acesso em 01 07 2024].
- [21] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *Annals of statistics*, pp. 1189-1232, 2001.
- [22] M. N. Bernstein, "Functionals and functional derivatives," 10 04 2022. [Online]. Available: <https://mbernste.github.io/posts/functionals/>. [Acesso em 26 06 2024].
- [23] L. Mason, J. Baxter, P. Bartlett e M. Frean, "Boosting algorithms as gradient descent," *Advances in neural information processing systems*, vol. 12, 1999.
- [24] M. Kuhn e K. Johnson, Feature engineering and selection: A practical approach for predictive models, Chapman and Hall/CRC, 2019.
- [25] J. Li, K. Cheng, S. Wang, F. Morsttater, R. P. Trevino, J. Tang e H. Liu, "Feature Selection: A Data Perspective," *ACM Computing Surveys*, vol. 50, 2017.
- [26] G. Brown, A. Pocock, M.-J. Zhao e M. Luján, "Conditional Likelihood Maximisation: A Unifying Framework for Information Theoretic Feature Selection," *Journal of Machine Learning Research*, pp. 27-66, 01 12 2012.
- [27] M. Cabral e P. Goldfeld, Curso de Álgebra Linear: Fundamentos e Aplicações, 3 ed., Rio de Janeiro: Instituto de Matemática, 2012.
- [28] T. Roughgarden e G. Valiant, "Lecture #7: Understanding and using Principal Component Analysis (PCA)," 2023.
- [29] "Covariance Matrix," Wikipédia, [Online]. Available: https://en.wikipedia.org/wiki/Covariance_matrix. [Acesso em 14 10 2023].
- [30] N. Japkowicz e M. Shah, Evaluating Learning Algorithms: A Classification Perspective, Cambridge University Press, 2011.
- [31] M. Grandini, E. Bagli e G. Visani, "Metrics for Multi-Class Classification: An Overview," *arXiv preprint arXiv:2008.05756*, 2020.

- [32] S. Chatterjee e A. S. Hadi, *Regression Analysis by Example*, 5 ed., Cairo: Wiley, 2012.
- [33] T. O. Kvalseth, "Note on the R^2 measure of goodness of fit for nonlinear models," *Bulletin of the Psychonomic Society*, vol. 21, nº 1, pp. 79-80, 1983.
- [34] T. Calinski e J. Harabasz, "A dendrite method for cluster analysis," *Communications in Statistics*, pp. 1-27, 1974.
- [35] T. M. Cover e J. A. Thomas, *Elements of Information Theory*, 2 ed., Hoboken: Wiley-Interscience, 2006.
- [36] A. Jakulin e I. Bratko, "Quantifying and Visualizing Attribute Interactions: An Approach Based on Entropy," *arXiv preprint cs/0308002*, 2003.
- [37] T. Roughgarden e G. Valiant, "The Singular Value Decomposition (SVD) and Low-Rank Matrix Approximations," 2023.
- [38] G. Strang, *Linear Algebra and Its Applications*, 4 ed., Cengage Learning, 2005.
- [39] S. Boyd e L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- [40] B. Ghogho, A. Ghodsi, F. Karray e M. Crowley, "Reproducing Kernel Hilbert Space, Mercer's Theorem, Eigenfunctions, Nyström Method, and Use of Kernels in Machine Learning: Tutorial and Survey," *arXiv preprint arXiv:2106.08443*, 2021.
- [41] J. H. Friedman, "Greedy function approximation: a gradient boosting machine.," *Annals of statistics*, pp. 1189-1232, 2001.
- [42] W. Banerjee, "Train/Test Complexity and Space Complexity of Logistic Regression," Medium, 23 08 2020. [Online]. Available: <https://levelup.gitconnected.com/train-test-complexity-and-space-complexity-of-logistic-regression-2cb3de762054>. [Acesso em 17 07 2024].